参考文献（与正文同表，此处重排以便附录引用可查）

参考文献

[1] 国家发展改革委, 国家能源局. 关于深化新能源上网电价市场化改革 促进新能源高质量发展的通知: 发改价格〔2025〕136 号 [Z]. 2025-01-27.

[2] 国家发展改革委. 关于进一步完善分时电价机制的通知: 发改价格〔2021〕1093 号 [Z]. 2021-07-26.

[3] 国家发展改革委, 国家能源局. 电力现货市场基本规则（试行）: 发改能源规〔2023〕1217 号 [Z]. 2023-09-07.

[4] 宁楠, 范俊秋, 徐潘宇驰, 程培军, 林盛振, 谢敏. 基于动态 C 藤 pair-Copula 的电力系统动态经济调度场景协同优化算法 [J]. 电力大数据, 2021, 24(4): 26–34.

[5] 崔丽敏, 马志远, 陈星, 刘祎芳, 张浩楠. 基于核密度估计和 Copula 函数的风-光-荷联合场景生成 [J]. 煤炭经济研究, 2026, 46(4): 153–161.

[6] 曾鸣, 李晨, 王致杰, 魏阳, 董军. 基于 CVaR 报童模型的风力发电商最优投标策略 [J]. 电力系统保护与控制, 2012, 40(24): 14–20.

[7] 陈寒, 唐忠, 鲁家阳, 梅光银, 李征南, 史晨豪. 基于 CVaR 量化不确定性的微电网优化调度研究 [J]. 电力系统保护与控制, 2021, 49(5): 105–115.

[8] 王泰强, 项意凌, 石京, 李兵抗. 新能源发电企业参与电力市场交易的风险管理创新应用——基于电力期货与 CVaR 的视角 [J]. 应用能源技术, 2026(4): 109–111.

[9] ROCKAFELLAR R T, URYASEV S. Optimization of conditional value-at-risk[J]. The Journal of Risk, 2000, 2(3): 21–41.

[10] 俞昊东, 廖立志, 孙捷. 求解含非离散分布的分布鲁棒多阶段线性随机规划的 Benders 分解-动态逼近方法 [J]. 中国科学: 数学, 2025, 55(2): 361–384.

[11] FÜLLNER C, REBENNACK S. Stochastic dual dynamic programming and its variants: a review[J]. SIAM Review, 2025, 67(3): 415–539.

[12] 李翔, 顾洁. 运用聚类算法预测地区电网典型日负荷曲线 [J]. 电力与能源, 2013, 34(1): 47–50.

[13] 张振涛. 基于机会约束规划的微电网源荷储协调优化调度 [J]. 电气技术与经济, 2024(8): 181–184.

附录 A　支撑材料文件清单

支撑材料压缩包 support.zip 打包本工程「可复现必需」的文件：论文源文件、全部源程序、五个结果文件、八张数据图的矢量文件，以及作为论文数字来源的汇总类中间结果。下表与本工程下的实际文件逐项对应；附录 E 所列源程序与本表的 code/ 完全一致。

```
support.zip
+-- main.tex                          论文装配（宏包输入、附录起点、正文结束标记）
+-- preamble.tex                      导言区：格式合规配置与写作宏（口径框/证据号/占位符）
+-- build.sh                          编译入口（自动探测 latexmk+xelatex / xelatex / tectonic）
+-- sections/                         正文与附录源文件，13 个
|   +-- 00_abstract.tex               摘要专用页
|   +-- 01_problem.tex                一、问题重述与分析
|   +-- 02_assumptions.tex            二、模型假设与符号说明
|   +-- 03_data.tex                   三、数据预处理与特征分析
|   +-- 04_q1.tex                     四、问题 1 模型
|   +-- 05_q2.tex                     五、问题 2 模型
|   +-- 06_q3.tex                     六、问题 3 模型
|   +-- 07_q4.tex                     七、问题 4 模型
|   +-- 08_results.tex                八、结果与分析
|   +-- 09_validation.tex             九、模型检验与灵敏度
|   +-- 10_evaluation.tex             十、模型评价与推广
|   +-- refs.tex                      参考文献（GB/T 7714）
|   +-- 90_appendix.tex               本附录
|
+-- tools/
|   +-- check_paper.py                提交前合规自查（--strict 为定稿模式）
|   +-- check_numbers.py              论文数字与数值台账的机械对拍
|   +-- verify_ref.py                 对照稿红队审查的只读反算脚本（复现 notes/F 的全部断言）
|   +-- package_support.sh            本压缩包的打包脚本（保证本清单与实际文件一致）
|
+-- code/                             全部源程序，与附录 E 逐项一致
|   +-- config.py                     口径、参数与附件读取（全文唯一参数来源）
|   +-- lp_engine.py                  单日两阶段 LP、滚动子问题 LP、场景削减
|   +-- q1.py  q2.py  q3.py  q4.py    四问求解与结果文件写出
|   +-- results_io.py                 结果工作表写出器（按附件 5 模板的列序）
|   +-- exp_scan.py                   容量/功率/效率扫描与报童经验覆盖率
|   +-- exp_sobol.py                  全局灵敏度（Sobol 指数）
|   +-- exp_na.py                     非预期性置换检验
|   +-- exp_oos.py                    样本外回测（训练窗构造情景集、测试窗回放）
|   +-- exp_benchmark.py              全年单一 LP 基准实验（完全信息参照）
|   +-- audit.py                      独立审计 A1--A10（不共用建模费用函数）
|   +-- paper_numbers.py              汇总数值台账 paper_numbers.json
|   +-- plot_figures.py               八张数据图的绘图脚本
|   +-- plot_figures_v2.py            v2 新增五张数据图（相依结构/生成器对照/风险画像/鲁棒前沿/分解）
|   +-- copula.py                     二元 Copula 工具库与 C 藤 pair-Copula（密度/h 函数/拟合/抽样）
|   +-- scen_copula.py                KDE + Copula 联合情景生成器与保真度度量
|   +-- exp_copula.py                 相依结构拟合、四种情景生成器样本外对照、分时段 CVaR 风险画像
|   +-- exp_chance.py                 机会约束对照（安全边际分位数扫描与报童互证）
|   +-- exp_dro.py                    盒式分布鲁棒对照（保守度—费用曲线与最坏情形评估）
|   +-- exp_loaderr.py                假设 A9 的量化：负载预报误差 0%/5%/10% 三档重跑问题 2
|   +-- decomp.py                     L 形（Benders）分解对照（解析多割、与直接求解对拍）
|   +-- run_all.py                    一键复现
|   +-- requirements.txt              依赖清单（版本锁定）
|
+-- results/                          五个结果文件（工作表名与列序按附件 5 模板）
|   +-- result1.xlsx                  问题 1（典型日）：计划购电量、充放电量
|   +-- result2.xlsx                  问题 2：计划购电量、充放电量、紧急购电量
|   +-- result3.xlsx                  问题 3：计划购电量、调整购电量、充放电量、紧急购电量
|   +-- result4-2.xlsx                问题 4-2：与 result2 同结构
|   +-- result4-3.xlsx                问题 4-3：与 result3 同结构
|
+-- figures/                          图 1 至图 10
|   +-- fig01_timeline.tex            图 1 决策时序图（TikZ 源文件）
|   +-- fig02_dependency.tex          图 2 问题依赖图（TikZ 源文件）
|   +-- fig03_q1_dual_price.pdf       电价、水价值与阈值型策略（问题 1）
|   +-- fig04_q1_capacity_power.pdf   容量与充放功率上限双向扫描（问题 1）
|   +-- fig05_cost_waterfall.pdf      问题 1 成本分解瀑布
|   +-- fig06_voi.pdf                 信息价值曲线（问题 3）
|   +-- fig07_q2_emergency.pdf        逐日紧急购电与报童两界（问题 2）
|   +-- fig08_q4_price.pdf            电价过程骨架与残差结构（问题 4）
|   +-- fig09_cvar_frontier.pdf       CVaR 风险--费用前沿与真实回测（问题 4）
|   +-- fig10_strategy_mechanism.pdf  最优策略机制（问题 1）
```

```
|   +-- README.md                    图规格、配色语义与出图约定
|
+-- intermediate/                    中间结果（论文数字的来源），分两组
|   +-- 随包提交（汇总与审计类，单个不超过 0.21 MB）
|   |   +-- audit.json  audit_report.md          审计 A1--A10（附录 D 的机器可读与可读版）
|   |   +-- paper_numbers.json                   数值台账（对拍基准，含四问、对偶、消融、Sobol、CVaR）
|   |   +-- q1.json  q1_dual.json  q1_dual2.json 问题 1 解与最优性证书
|   |   +-- q2_margin.json  exp_benchmark.json   安全边际分解、全年单一 LP 基准
|   |   +-- q4_price.json  q4_cvar.json          电价过程检验与情景保真度、CVaR 前沿
|   |   +-- exp_scan.json  exp_sobol.json  exp_na.json   扫描、灵敏度、非预期性
|   |   +-- full_year_lp.txt                     全年单一 LP（完全信息参照）的目标值
|   +-- 不随包（逐日解转储，单个 2.7--4.4 MB，可由 run_all.py 一键重建）
|       +-- q2_main.json  q2_noMargin.json  q2_cmp_R1.json  q2_cmp_R2.json  q2_cmp_R3.json
|       +-- q2_K120.json  q2_full_strat.json  q2_term_R1_eq.json  q2_term_R1_free.json
|       +-- q2_term_R3_free.json  q3_n1.json  q3_n2.json  q3_n4.json  q3_perf.json
|       +-- q4_2.json  q4_3.json
|
+-- notes/                           口径与台账文档，7 个
    +-- 写作交底.md                  本队实际模型口径（唯一口径说明）
    +-- 数值台账.md                  论文数字的表格版台账
    +-- A_坑点与验证.md  B_模型与算法.md  C_写作规范.md
    +-- D_行业与引用.md  E_知识点与模板.md
```

三点说明：

1. **体积**：上述随包内容合计约 3.4 MB（其中五个结果文件约 2.1 MB、八张数据图约 0.3 MB），低于支撑材料压缩包 20 MB 的上限（格式规范第十一条）。
2. **中间结果分级打包**：`intermediate/` 下第二组**逐日解转储**（`q2_main.json`、`q2_noMargin.json`、`q3_n4.json`、`q4_2.json` 等，单个 2.7–4.4 MB）可由 `python3 code/run_all.py` 一键重建，且其中每个进入论文的数字都已汇总进 `paper_numbers.json`，故不随压缩包提交；包内保留体积小、且为论文数字与审计直接依据的汇总类文件（上表已逐一列出）。
3. **不随包提交的中间产物**：LaTeX 编译产物、Python 字节码缓存、以下划线开头的临时脚本与 `.npy` 临时数组，均为可再生的工作文件，不属于支撑材料。

附录 B　运行环境与运行说明

表 1　本文的运行环境与依赖版本清单（对象：操作系统、解释器、数值与绘图库、求解器接口；范围：全部求解、审计与出图的实际执行环境；单位与量纲不适用；样本量为 1 套环境配置）

项目	版本	说明
操作系统	macOS 15.7.5（arm64，Build 24G624）	全部求解、审计与出图单机完成，未使用集群
Python	3.13.12	唯一解释器
NumPy / pandas	2.4.3 / 3.0.1	数组运算与附件 `.xlsx` 读写
SciPy	1.18.1	线性规划 `linprog` 与统计检验
求解器	SciPy `linprog(method="highs")`，HiGHS 后端（随 SciPy 1.18.1 分发）	四问模型在场景离散后均为 LP，由 HiGHS 精确求解，未使用商业求解器
Matplotlib	3.11.1	图 3–图 10 的数据图；图 1、图 2 为 TikZ 源文件，由 XeLaTeX 排版
scikit-learn	1.9.0	仅用于 k-means 场景削减的对照路径
openpyxl	3.1.5	结果文件 `.xlsx` 的读写

运行环境　依赖清单锁定在 `code/requirements.txt`（`numpy==2.4.3`、`pandas==3.0.1`、`scipy==1.18.1`、`openpyxl==3.1.5`、`matplotlib==3.11.1`、`scikit-learn==1.9.0`），版本号写确切值、不写区间。

运行说明

1. **依赖安装**：`python3 -m pip install -r code/requirements.txt`；
2. **数据就位**：把附件 1–4 与附件 5 模板放在工作区的 `CUMCM2026Problems(1)/C 题/附件/`（模板置于其 附件 5/ 子文件夹）。该路径由 `code/config.py` 的 `ATTACH_DIR` 给出，并由脚本自身位置反推，整个工程文件夹移动后无需改动代码；
3. **依次运行四问**（在 `code/` 下执行，命令参数即论文主口径）：

```
python3 q1.py                                                  # 问题 1：典型日 LP
python3 q2.py --tag main --K 60 --scen strat --info R3 --write # 问题 2：334 天
python3 q3.py --epochs 0,6,12,18 --tag n4 --K 60 --write       # 问题 3：四次预报
python3 q4.py --mode q42 --K 64 --write                        # 问题 4-2：波动电价
python3 q4.py --mode q43 --K 64 --write                        # 问题 4-3：滚动
```

`--write` 表示按附件 5 模板写出对应结果文件；条件项（论文中作为对照或检验给出）另有：`q4.py --mode tests --K 64`（电价过程检验与情景保真度）、`q4.py --mode cvar --K 64 --step 6`（CVaR 前沿）、`exp_scan.py`（容量、功率与效率扫描、报童经验覆盖率）、`exp_sobol.py`（Sobol

全局灵敏度）、exp_na.py（非预期性置换检验）、exp_benchmark.py（全年单一 LP 的完全信息基准）、plot_figures.py（八张数据图）；

4. **独立审计**：python3 audit.py。脚本只读原始附件与五个结果文件，独立复算 A1–A10，写出 intermediate/audit.json 与 audit_report.md；
5. **数值台账**:python3 paper_numbers.py,把全部中间结果汇总为 intermediate/paper_numbers.json（论文数字的对拍基准）；
6. **对拍与自查**：python3 ../tools/check_numbers.py 列出论文中找不到台账出处的数字；python3 ../tools/check_paper.py --strict 做定稿模式合规自查（占位符、禁令词、身份信息、缺图与缺源码、页数与摘要页）。

一键复现：python3 run_all.py 按上述顺序自动执行问题 1–4、扫描、审计与台账；加 --fast 参数则跳过信息结构、场景数、VoI 与 CVaR 等对照实验。

表 2　单机实测耗时（macOS 15.7.5 / arm64 / Python 3.13.12，与正文同一份代码与数据）

步骤	实测耗时	规模与说明
问题 1（典型日，确定性 LP）	不足 0.1 s	单日 144 槽；两次实测 0.014 s 与 0.008 s（纯求解时间）
问题 2（固定价，两阶段）	约 30 s	334 天逐日两阶段子问题（$K = 60$ 个场景）
问题 3（固定价，四次预报）	约 72 s	四个预报时刻 × 334 天的滚动子问题
问题 4-2（波动电价，两阶段）	约 50 s	334 天逐日两阶段子问题（$K = 64$ 个联合情景）
问题 4-3（波动电价，四次预报）	约 101 s	四个预报时刻 × 334 天的滚动子问题
独立审计（A1–A10）	约 13 s	两次实测 12.9 s 与 16.7 s；其中交付物复算 3.3 s、逐日解复算 10.1 s
合计（四问主链 + 审计）	约 4.4 min	单机完成，无需集群或并行

实测耗时　**输出文件**：results/ 下的五个结果文件（result1.xlsx、result2.xlsx、result3.xlsx、result4-2.xlsx、result4-3.xlsx）；figures/ 下的十三张矢量图（图 3–图 10 与 v2 新增的图 11–图 15）与两张 TikZ 源文件（图 1、图 2）；intermediate/ 下的审计报告、数值台账与各问中间结果。

附录 C　结果文件、论文表格与生成函数

五个结果文件均按附件 5 模板的工作表名、列名与列序写出：result1.xlsx 为竖表（一个时段一行，对应表 正文10 与表 正文11）；其余四个文件为横表（一天一行，147 列 = 日期 + 144 个时段 + 全天购电量 + 全天购电费）。

表 3　结果文件 ↔ 论文表格 ↔ 生成函数对应表

结果文件（工作表）	论文位置	生成函数	行数与中间结果
result1.xlsx（计划购电量）	表 正文10	q1.solve_q1 → q1.write_result1	144 行（1 行 1 个 10 min 槽）；q1.json
result1.xlsx（充放电量）	表 正文11	q1.write_result1	6 行（4 h 块）+ 0:00 与 24:00 储电量；q1.json
result2.xlsx（计划购电量）	表 23、表 正文18、表 正文17	q2.run_q2 → results_io.write_result	334 行 × 147 列；q2_main.json
result2.xlsx（充放电量）	表 24、表 正文19	同上	2 004 行（334 天 × 6 个 4 h 块）
result2.xlsx（紧急购电量）	表 25、表 正文20	同上	1 156 行（缺口时段合并，同一日期可多行）
result3.xlsx（计划购电量、调整购电量）	表 正文14（决策时刻与生效槽位）、表 正文21	q3.run_q3 → results_io.write_result	各 334 行 × 147 列；q3_n4.json
result3.xlsx（充放电量、紧急购电量）	表 正文21、表 正文17	同上	2 004 行 / 1 190 行
result4-2.xlsx（计划购电量、充放电量、紧急购电量）	表 正文16、表 正文21、表 正文17	q4.run_q4_2 → results_io.write_result	334 × 147、2 004、1 193 行；q4_2.json
result4-3.xlsx（计划购电量、调整购电量、充放电量、紧急购电量）	表 正文16、表 正文21	q4.run_q4_3 → results_io.write_result	334 × 147（两次）、2 004、1 187 行；q4_3.json

表中「生成函数」为模块内的真实函数名：q2.run_q2、q3.run_q3、q4.run_q4_2、q4.run_q4_3 完成求解，由各脚本的 main() 装配后统一调用 results_io.write_result 写出工作簿；问题 1 的竖表由 q1.solve_q1 求解、q1.write_result1 写出。论文其余图表不来自结果文件，其来源为：最优性证书取自 q1_dual.json 与 q1_dual2.json（q1.py 的对偶校验），CVaR 前沿取自 q4_cvar.json（q4.py --mode cvar，底层函数 q4.solve_cvar_day），全局灵敏度取自 exp_sobol.json，非预期性置换检验取自 exp_na.json。

附录 D　独立审计报告

审计由**非建模实现者**独立完成：脚本 code/audit.py 只读原始附件与本队写出的五个结果文件，**不共用建模代码的费用函数与约束构造函数**，以保证「自证循环」不会发生。机器可读结果在 intermediate/audit.json，可读报告在 intermediate/audit_report.md。

审计口径与阈值

- **残差**：模型内部解（10 min 槽级、未取整）阈值 10^{-6} kWh；交付物（结果文件只给 4 h 块级充放电量与 0:00、24:00 两个储电量）的电量按 10^{-4} kWh 取整后回读，块级递推残差的上界为 $0.5\times10^{-4}+0.9\times6\times0.5\times10^{-4}+6\times0.5\times10^{-4}/0.9+0.5\times10^{-4}=7.0\times10^{-4}$ kWh，故交付物的残差阈值取 10^{-3} kWh （贴合交付精度的判定，不放松内部解的口径）；
- **越界数与违反数**：储电量越界、充放电功率越界、充放互斥违反、负电量，全部要求等于 0；
- **费用复算相对误差**：$\le 10^{-4}$（费用只用结果文件与附件逐时段重算，不调用建模费用函数）。

逐项结果（正文口径）

下表为正文第九章原先给出的**双列口径**审计表：左列为 10 min 级模型内部解，右列为结果文件回读后的块级复算，便于对照"内部解"与"交付物"两层判定的差别。

表 4　独立审计项、阈值与实测结果（内部解：问题 1、2、3 的 10 min 级解；交付物：result1、result2、result3、result4-2、result4-3 共五个结果文件；窗口 2025-02-01 至 12-31）

项	检查内容与阈值	内部解实测	交付物实测	判定
A1	功率/电量平衡残差 $\le 10^{-6}$ kWh	问题 1 1.49×10^{-13}；问题 2、3 3.41×10^{-13} kWh	交付物无逐槽 L, PV 列，平衡只能在内部解上核验（汇总列对拍见 A10）	PASS
A2	储电量递推残差 $\le 10^{-6}$ kWh	问题 1 9.09×10^{-13}；问题 2、3 1.09×10^{-11} kWh	问题 1 3.64×10^{-12}；问题 2、3、4-2、4-3 为 $2.96/2.72/2.11/2.61\times10^{-4}$ kWh，为写入取整所致	PASS[注]
A3	储电量边界：越界数 $=0$（区间 $[1200, 10\,800]$ kWh）	0	0	PASS
A4	充放功率边界：内部解按单槽上限 833.3333 kWh；交付物按块级上限 20 000 kWh	0	0	PASS
A5	充放互斥：$\min(c_t, d_t)\le 10^{-6}$	违反槽数 0（充放同时槽数 0）	交付物只有 4 h 聚合量，块内可同时有充放电，故块级不可检验	PASS
A6	周期/终端：问题 1 要求 $S_0-S_{144}=0$；问题 2–4 要求跨日衔接且 $S_T\ge S_0$	问题 1 周期偏差 0.000；问题 2、3 末值低于初值的天数为 0	五个结果文件末值低于初值的天数均为 0	PASS
A7	非负性：所有电量与费用 $\ge -10^{-6}$	负购电槽数 0	负购电量数 0	PASS
A8	费用复算（计划 + 紧急 + 调整相关）相对误差 $\le 10^{-4}$	问题 1、2、3 的相对误差均为 0	交付物层只能核验计划购电费（最大相对偏差 4.32×10^{-7}，问题 4-2）	PASS
A9	日期与结构：334 天、工作表名/列名/列序与模板一致	334 天	五个文件均为 334 行 ×147 列，日期范围 2025-02-01 至 12-31；紧急购电行数 1156/1190/1193/1187	PASS
A10	全天汇总列与逐时段求和一致：相对误差 $\le 10^{-4}$	—	全天购电量最大偏差 0.0012 kWh（问题 4-3）；全天购电费最大相对偏差 4.32×10^{-7}（问题 4-2）	PASS

表注：交付物 A2 的 10^{-4} kWh 量级残差可由写入精度完全解释——结果写入对 6 个块级充放电量与 0:00/24:00 两个储电量按 4 位小数取整，取整误差上界为 $6\times5\times10^{-5}+2\times5\times10^{-5}=4\times10^{-4}$ kWh，与实测值一致；该残差不是物理越界（A3、A4 越界数均为 0）。

逐项结果

反面证据：结算平衡不得漏掉紧急购电项　附件 5 把「计划购电量」与「紧急购电量」分成**两张表**，因此计划量不等于实际供给量。若 A1 的结算平衡漏掉紧急购电项 g_t（即把计划购电量当作全部供给），残差恰好等于 g_t：在问题 2 的 334 天逐槽解上 $\max_t g_t = 326.18$ kWh，在问题 3 上为 204.87 kWh，分别比 10^{-6} kWh 的容差大 3.3×10^8 倍与 2.0×10^8 倍；补上 $+g_t$ 后，问题 2、3 的平衡残差降为 3.41×10^{-13} kWh。同一次审计还给出：问题 2 的盈余处置量（弃光与超购部分的处置量）为 3 106 390.84 kWh，问题 3 为 2 638 963.35 kWh，均为正残差，与「供给不低于需求」的口径一致。

审计结论

☑ 全部通过，可进入写作　　☐ 存在失败项，已定位并修正　（未勾选）

A1–A10 十项全部通过；审计同时复核了交付物的结构与全天汇总列，未发现任何越界、违反非负性或费用复算不一致。

附录 E　源程序清单

以下为本论文建模、求解、绘图、审计与数值汇总所用的**全部源程序**（格式规范第五条），与 `code/` 下的实际文件逐项一致；附录用 `lstinputlisting` 原样引入文件内容。`[plain]` 选项表示该脚本不参与求解，只做绘图或文本处理。

源程序：`code/config.py`

表 5　独立审计 A1–A10（同时承载正文的 L1–L5 验证链条）：阈值、实测值与判定（「内」为 10 min 级模型内部解，「交」为结果文件块级）

项	检查内容	阈值	实测值	判定
A1	电量平衡残差	$\max\lvert E_t+PV_t\Delta t+d_t+g_t-L_t\Delta t-c_t-s_t\rvert\le 10^{-6}$ kWh	内：问题 1 为 1.49×10^{-13}，问题 2、3 均为 3.41×10^{-13}	PASS
A2	储电量递推残差	内 $\le 10^{-6}$ kWh；交 $\le 10^{-3}$ kWh	内：9.09×10^{-13}（问题 1）、1.09×10^{-11}（问题 2、3）；交（问题 1、2、3、4-2、4-3）：3.64×10^{-12}、2.96×10^{-4}、2.72×10^{-4}、2.11×10^{-4}、2.61×10^{-4}	PASS
A3	储电量越界数	等于 0（运行区间 [1200, 10800] kWh）	交：五个结果文件均为 0；内：问题 1–3 均为 0	PASS
A4	充放电功率越界数	等于 0（单槽上限 $\overline{P}\Delta t=833.3333$ kWh）	交：块级上限 $4\overline{P}\Delta t=3333.3333$ kWh，四个横表均为 0；内：问题 1–3 的 10 min 级均为 0	PASS
A5	充放互斥违反数	等于 0（$\eta_c\eta_d=0.81<1$ 时最优解不同时充放）	内：问题 1–3 均为 0，并在 97 个储电量双界均不活跃的槽位复核	PASS
A6	周期与终端条件	问题 1 的 $\lvert S_0-S_{144}\rvert=0$；问题 2–4 每日 $S_{144}\ge S_0$	问题 1 的日周期偏差 0.0 kWh；四个横表「末值低于初值的天数」均为 0	PASS
A7	非负性违反数	等于 0	交：负购电量 0 个（四个横表各 334×144 个时段值，加 result1 的 144 个）；内：问题 1–3 均为 0	PASS
A8	费用复算相对误差	$\le 10^{-4}$	交：问题 1 为 0.0；内：问题 1–3 均为 0.0	PASS
A9	日期与结构	334 天；工作表名、列数与列序逐项一致	四个横表均为 334×147，日期 2025-02-01 至 2025-12-31；result1 为 144 行计划表加 6 行充放电表；工作表名 result1 为计划购电量与充放电量，result2、result4-2 为计划购电量、充放电量、紧急购电量，result3、result4-3 为计划购电量、调整购电量、充放电量、紧急购电量	PASS
A10	全天汇总列	购电量偏差 $\le 10^{-2}$ kWh（144 槽取整上界 7.2×10^{-3}）；费用相对偏差 $\le 10^{-4}$	购电量最大偏差 1.10×10^{-3}、1.10×10^{-3}、1.00×10^{-3}、1.20×10^{-3} kWh（对应问题 2、3、4-2、4-3）；费用最大相对偏差 3.40×10^{-7}、3.40×10^{-7}、4.32×10^{-7}、4.05×10^{-7}	PASS

```python
# -*- coding: utf-8 -*-
"""
C 题公共配置与数据管道（唯一口径源）
====================================
本文件是全论文**唯一**的口径定义源（对应写作需求 D2 口径纪律）。
论文正文引用口径编号 C-01…C-08，不另行手写求和式。

时标口径（陷阱 A，见 C题知识点解析/01 §4）
-------------------------------------
附件 1/2/4 的时刻标签是**时段末端**，共 144 个标签，覆盖一天的 144 个 10 min 区间。
本文件采用**位置读法**：第 j 个槽位（j 从 0 开始）= [j*10min, (j+1)*10min)，
对应附件第 j+1 个数据行/列。附件标签仅作显示，**不按标签重映射**。
- 附件标签 "10:10" → 槽位 j = 60 = [10:00, 10:10)
- 论文表 1 的 "10:00-10:10" 即取该槽位。

单位口径
--------
附件 1/2/3 的负载与光伏为 **kW**；附件 1/4 的电价为 **元/kWh**；
本模型全部决策变量为**电量（kWh）**，故 E = P * DT，DT = 1/6 h。
充放电功率上限 5000 kW → 电量上限 5000 * DT = 833.3333 kWh/槽。

口径编号（与论文《写作交底》§6.1 完全一致，附录 E 原样引入本文件）
-----------------------------------------------------------------
C-01 无储能基线购电费 C^base | C-02 全天购电量 | C-03 总购电费用（计划 + 紧急 + 调整相关）
C-04 节省率 | C-05 紧急购电量 | C-06 弃光/盈余处置电量 | C-07 充放电量与储电量（交流侧）
C-08 储电量递推与终端条件 | C-09 调整与违约费用 | C-10 信息价值 VoI | C-11 条件风险价值 CVaR

假设编号：A1 不允许售电 | A2 单边效率 0.9 | A3 不同时充放 | A4 忽略自放电与循环衰减
A5 跨日 SOC 连续递推 | A6 每日终端 S_144 >= S_0 | A7 调整决策后覆盖前 | A8 1 月为预热期
A9 信息结构取 R3 | A10 波动电价下 0:00 可用电价预报 = 分时骨架
"""
from __future__ import annotations

import os
from dataclasses import dataclass

import numpy as np
import pandas as pd

```

```python
# ---------------------------------------------------------------------------
# 路径
# ---------------------------------------------------------------------------
HERE = os.path.dirname(os.path.abspath(__file__))
PAPER_DIR = os.path.dirname(HERE)                        # C题论文/
WORKSPACE = os.path.dirname(PAPER_DIR)                   # 工作区根
PROBLEM_DIR = os.path.join(WORKSPACE, "CUMCM2026Problems(1)", "C题")
ATTACH_DIR = os.path.join(PROBLEM_DIR, "附件")
TEMPLATE_DIR = os.path.join(ATTACH_DIR, "附件5")
RESULT_DIR = os.path.join(PAPER_DIR, "results")
INTERMEDIATE_DIR = os.path.join(PAPER_DIR, "intermediate")
FIGURE_DIR = os.path.join(PAPER_DIR, "figures")

for _d in (RESULT_DIR, INTERMEDIATE_DIR, FIGURE_DIR):
    os.makedirs(_d, exist_ok=True)

# ---------------------------------------------------------------------------
# 物理参数（题面附录 1）
# ---------------------------------------------------------------------------
DT = 1.0 / 6.0              # h，单个 10 min 槽时长
N_SLOT = 144                # 一天 144 个 10 min 槽
ETA_C = 0.9                 # 充电效率
ETA_D = 0.9                 # 放电效率
S_MAX = 10800.0             # kWh，储电量上限
S_MIN = 1200.0              # kWh，储电量下限
P_MAX = 5000.0              # kW，最大充放电功率
E_MAX = P_MAX * DT          # kWh，单槽充/放电量上限 833.3333
S_INIT = 6000.0             # kWh，2025-01-01 00:00 储电量
EMERGENCY_MULT = 5.0        # 紧急购电电价倍数
BREACH_MULT = 0.5           # 计划 > 调整：违约电价倍数
EXCESS_MULT = 1.5           # 调整 > 计划：超出部分电价倍数

# 结果窗口（问题 2/3/4）：2025-02-01 至 2025-12-31，334 天
WIN_START = "2025-02-01"
WIN_END = "2025-12-31"
WARMUP_START = "2025-01-01"

# 四个指定日期（表 3 / 问题 2、3、4 的结果表）
KEY_DATES = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]

# 充放电量表的时间段划分（表 2）
BLOCKS = [(0, 24), (24, 48), (48, 72), (72, 96), (96, 120), (120, 144)]
BLOCK_LABELS = ["0:00-4:00", "4:00-8:00", "8:00-12:00",
                "12:00-16:00", "16:00-20:00", "20:00-24:00"]

# 表 1 指定的 6 个 10 min 时段 -> 槽位下标（0 基）
TABLE1_SLOTS = [("10:00-10:10", 60), ("12:00-12:10", 72), ("14:00-14:10", 84),
                ("16:00-16:10", 96), ("18:00-18:10", 108), ("20:00-20:10", 120)]

# 调整（预报）时刻 -> 控制窗口起始槽位
FORECAST_HOURS = [0, 6, 12, 18]
EPOCH_SLOT = {h: h * 6 for h in FORECAST_HOURS}   # 0, 36, 72, 108


def slot_label(j: int) -> str:
    """槽位 -> 附件标签（时段末端）。"""
    m = (j + 1) * 10
    if m == 1440:
        return "0:00+1"
    return f"{m // 60}:{m % 60:02d}"


# ---------------------------------------------------------------------------
# 数据读取
# ---------------------------------------------------------------------------
def read_attachment1() -> pd.DataFrame:
    """附件 1：典型日 {电价 元/kWh，小区负载 kW，光伏发电预测功率 kW}，144 槽。"""
    df = pd.read_excel(os.path.join(ATTACH_DIR, "附件1.xlsx"), header=0)
    df.columns = ["时间", "price", "load", "pv"]
    assert len(df) == N_SLOT, len(df)
    return df


```

```python
def read_attachment2() -> tuple[pd.DataFrame, pd.DataFrame]:
    """附件 2：2025 全年逐日逐槽 {小区负载 kW，光伏发电实际功率 kW}。"""
    load = pd.read_excel(os.path.join(ATTACH_DIR, "附件2.xlsx"), sheet_name="小区负载", header=0)
    pv = pd.read_excel(os.path.join(ATTACH_DIR, "附件2.xlsx"), sheet_name="光伏发电实际功率", header=0)
    out = []
    for df, name in ((load, "load"), (pv, "pv")):
        df = df.rename(columns={df.columns[0]: "date"})
        df["date"] = pd.to_datetime(df["date"]).dt.normalize()
        df = df.set_index("date")
        df.columns = [f"s{j}" for j in range(N_SLOT)]
        df = df.astype(float)
        out.append(df)
    return out[0], out[1]


def read_attachment3() -> pd.DataFrame:
    """附件 3：每天 0:00/6:00/12:00/18:00 发布的未来 24 h 整点光伏预报（kW）。

    返回长表：index=(date, hour)，columns=lead1..lead24（lead k 对应 [h0+k-1, h0+k)）。
    """
    df = pd.read_excel(os.path.join(ATTACH_DIR, "附件3.xlsx"), header=0)
    df.columns = ["date", "hour"] + [f"lead{k}" for k in range(1, 25)]
    df["date"] = df["date"].ffill()
    df["date"] = pd.to_datetime(df["date"]).dt.normalize()
    df["hour"] = df["hour"].astype(str).str.strip()
    df["hour"] = df["hour"].map(lambda s: int(s.split(":")[0]))
    for k in range(1, 25):
        df[f"lead{k}"] = pd.to_numeric(df[f"lead{k}"], errors="coerce")
    return df.set_index(["date", "hour"]).sort_index()


def read_attachment4() -> pd.DataFrame:
    """附件 4：2025 全年逐日逐槽实时电价（元/kWh）。"""
    df = pd.read_excel(os.path.join(ATTACH_DIR, "附件4.xlsx"), header=0)
    df = df.rename(columns={df.columns[0]: "date"})
    df["date"] = pd.to_datetime(df["date"]).dt.normalize()
    df = df.set_index("date")
    df.columns = [f"s{j}" for j in range(N_SLOT)]
    return df.astype(float)


def read_template(sheet: str, fname: str = "result2.xlsx", nrows: int | None = None):
    """读取附件 5 的结果模板（用于比对结构）。"""
    return pd.read_excel(os.path.join(TEMPLATE_DIR, fname), sheet_name=sheet,
                         header=None, nrows=nrows)


@dataclass
class Panel:
    """按槽位展开的全年面板数据。"""
    dates: pd.DatetimeIndex

    def __post_init__(self):
        self.load, self.pv = read_attachment2()
        self.price_fixed = read_attachment1().set_index("时间")[["price", "load", "pv"]]
        self.price_flat = read_attachment4()
        self.pf3 = read_attachment3()

    # ---- 逐槽矩阵（行=日期，列=槽位）----
    def L(self, dates=None) -> pd.DataFrame:
        return self.load if dates is None else self.load.loc[dates]

    def PV(self, dates=None) -> pd.DataFrame:
        return self.pv if dates is None else self.pv.loc[dates]

    def P(self, dates=None, volatile: bool = False) -> pd.DataFrame:
        """电价矩阵：volatile=False 用附件 1 固定价（每天相同），True 用附件 4。"""
        dates = self.load.index if dates is None else pd.DatetimeIndex(dates)
        if volatile:
            return self.price_flat.loc[dates]
        row = read_attachment1()["price"].to_numpy()
        return pd.DataFrame(np.tile(row, (len(dates), 1)), index=dates,
                            columns=[f"s{j}" for j in range(N_SLOT)])
```

```
186
187     # ---- 预报 ----
188     def pv_forecast(self, date, hour: int) -> np.ndarray:
189         """取 date 日 hour 时刻发布的 24 h 光伏预报，展开为 144 槽（每整点 6 槽）。"""
190         row = self.pf3.loc[(pd.Timestamp(date).normalize(), hour)]
191         hourly = np.array([row[f"lead{k}"] for k in range(1, 25)], dtype=float)
192         return np.repeat(hourly, 6)
193
194
195 # ---------------------------------------------------------------------------
196 # 口径指标（C-01…C-08）
197 # ---------------------------------------------------------------------------
198 def baseline_cost(p_price: np.ndarray, load: np.ndarray, pv: np.ndarray) -> float:
199     """C-01 无储能基线购电费：C^base = sum_t p_t * max(L_t - PV_t, 0) * DT。"""
200     return float(np.sum(p_price * np.maximum(load - pv, 0.0) * DT))
201
202
203 def curtailment(price, load, pv, E, c, d) -> np.ndarray:
204     """C-05 逐槽弃光/盈余电量：s_t = PV*DT + E + d - L*DT - c（应 >= 0）。"""
205     return pv * DT + E + d - load * DT - c
```

源程序：code/lp_engine.py

```
1  # -*- coding: utf-8 -*-
2  """
3  单日两阶段 LP 求解引擎（问题 2 / 问题 3 / 问题 4 共用）
4  ======================================================
5  变量（全部为 kWh）：
6      E_t         计划购电量（第一阶段承诺量）
7      c_t, d_t    充电量 / 放电量
8      S_t         时段末储电量
9      g_{w,t}     情景 w 下的紧急购电量（第二阶段追索）
10
11 模型：
12     min  sum_t p_t E_t + 5 * sum_t p_t * sum_w W_w g_{w,t}
13     s.t. E_t - c_t + d_t                >= ND^fcst_t                  (计划满足预测负载)
14          g_{w,t} + E_t - c_t + d_t      >= ND^act_{w,t}               (实际缺口 -> 紧急购电)
15          S_t = S_{t-1} + eta_c c_t - d_t/eta_d
16          S_min <= S_t <= S_max, 0 <= c_t,d_t <= E_MAX, E_t>=0, g>=0
17          S_144 >= S_0（A6 终端不亏空）或 = S_0（严格日周期）
18
19 ND = (L - PV) * DT 为"净需求"电量。
20 """
21 from __future__ import annotations
22
23 import numpy as np
24 from scipy.optimize import linprog
25 from scipy.sparse import coo_matrix, vstack
26
27 import config as C
28
29
30 def solve_day(price, nd_fcst, nd_scen, weights=None, s_init=C.S_INIT,
31               term_mode="ge", term_value=None, allow_overbuy=True):
32     """求解单日两阶段 LP。
33
34     参数
35     ----
36     price    : (n,) 电价 元/kWh
37     nd_fcst  : (n,) 预测口径净需求 (L_fcst - PV_fcst)*DT，单位 kWh
38     nd_scen  : (K,n) 各情景实际净需求 (L_act - PV_act)*DT，单位 kWh
39     weights  : (K,) 情景概率，默认等权
40     s_init   : 日初储电量
41     term_mode: 'ge' -> S_n >= s_init; 'eq' -> S_n = s_init; 'free' -> S_n >= S_min
42     """
43     n = C.N_SLOT
44     nd_scen = np.atleast_2d(nd_scen)
45     K = nd_scen.shape[0]
46     if weights is None:
47         weights = np.full(K, 1.0 / K)
48     weights = np.asarray(weights, float)
49     weights = weights / weights.sum()
50
```

```
    iE, iC, iD, iS, iG = 0, n, 2 * n, 3 * n, 4 * n
    nv = 4 * n + K * n

    c_obj = np.zeros(nv)
    c_obj[iE:iE + n] = price
    for w in range(K):
        c_obj[iG + w * n: iG + (w + 1) * n] = C.EMERGENCY_MULT * price * weights[w]

    rows, cols, vals, rhs = [], [], [], []

    def add(row, col, val):
        rows.append(row); cols.append(col); vals.append(val)

    r = 0
    # (1) 计划可行性
    for t in range(n):
        add(r, iE + t, -1.0); add(r, iC + t, 1.0); add(r, iD + t, -1.0)
        rhs.append(-nd_fcst[t]); r += 1
    # (2) 紧急购电（>= 形式）
    for w in range(K):
        for t in range(n):
            add(r, iE + t, -1.0); add(r, iC + t, 1.0); add(r, iD + t, -1.0)
            add(r, iG + w * n + t, -1.0)
            rhs.append(-nd_scen[w, t]); r += 1
    A_ub = coo_matrix((vals, (rows, cols)), shape=(r, nv)).tocsr()
    b_ub = np.array(rhs)

    # (3) SOC 递推等式
    rows2, cols2, vals2, rhs2 = [], [], [], []
    for t in range(n):
        rr = t
        vals2.append(1.0); cols2.append(iS + t); rows2.append(rr)
        if t > 0:
            vals2.append(-1.0); cols2.append(iS + t - 1); rows2.append(rr)
        vals2.append(-C.ETA_C); cols2.append(iC + t); rows2.append(rr)
        vals2.append(1.0 / C.ETA_D); cols2.append(iD + t); rows2.append(rr)
        rhs2.append(0.0 if t > 0 else s_init)
    A_eq = coo_matrix((vals2, (rows2, cols2)), shape=(n, nv)).tocsr()
    b_eq = np.array(rhs2)

    bounds = [(0.0, None)] * n + [(0.0, C.E_MAX)] * n + [(0.0, C.E_MAX)] * n + \
             [(C.S_MIN, C.S_MAX)] * n + [(0.0, None)] * (K * n)
    if term_mode == "eq":
        v = s_init if term_value is None else term_value
        bounds[iS + n - 1] = (v, v)
    elif term_mode == "ge":
        v = s_init if term_value is None else term_value
        bounds[iS + n - 1] = (max(v, C.S_MIN), C.S_MAX)

    res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                  bounds=bounds, method="highs")
    if not res.success:
        raise RuntimeError("LP 不可行: " + res.message)
    x = res.x
    out = dict(E=x[iE:iE + n].copy(), c=x[iC:iC + n].copy(),
               d=x[iD:iD + n].copy(), S=x[iS:iS + n].copy(),
               g=x[iG:iG + K * n].reshape(K, n).copy(),
               obj=res.fun, s_init=s_init, K=K, weights=weights,
               nd_fcst=nd_fcst, nd_scen=nd_scen, price=price,
               dual_eq=res.eqlin.marginals.copy(),
               dual_plan=res.ineqlin.marginals[:n].copy(),
               status=res.status)
    return out


def realized_slot(sol, nd_act):
    """给定实际净需求，按"计划照常执行 + 缺口紧急购电"结算，返回逐槽紧急购电量与费用。"""
    n = C.N_SLOT
    g = np.maximum(nd_act + sol["c"] - sol["d"] - sol["E"], 0.0)
    plan_cost = float(np.sum(sol["price"] * sol["E"]))
    emg_cost = float(C.EMERGENCY_MULT * np.sum(sol["price"] * g))
    return g, plan_cost, emg_cost

```

```
124
125 def solve_epoch(price, nd_fcst, nd_scen, weights, s_init, s_from=0,
126                 plan=None, term_floor=None, tol_tail=None):
127     """滚动时域中"某一预报时刻"的子问题 LP（只优化槽位 [s_from, n)）。
128
129     plan 不为 None 时启用 Q3 的调整/违约定价口径:
130         cost_t = p*min(E^plan,E) + 0.5p (E^plan-E)^+ + 1.5p (E-E^plan)^+
131     线性化为  p*E^plan - 0.5p u + 1.5p v,  E - E^plan = v - u,  u,v >= 0。
132     """
133     n = C.N_SLOT
134     nd_scen = np.atleast_2d(nd_scen)
135     K = nd_scen.shape[0]
136     if weights is None:
137         weights = np.full(K, 1.0 / K)
138     weights = np.asarray(weights, float); weights = weights / weights.sum()
139     m = n - s_from                                  # 本时段数
140     use_plan = plan is not None and s_from > 0
141
142     iE, iC, iD, iS, iG = 0, m, 2 * m, 3 * m, 4 * m
143     iU = iG + K * m
144     nv = 4 * m + K * m + (2 * m if use_plan else 0)
145
146     c_obj = np.zeros(nv)
147     if use_plan:
148         c_obj[iU:iU + m] = -C.BREACH_MULT * price[s_from:]          # -0.5p u
149         c_obj[iU + m:iU + 2 * m] = C.EXCESS_MULT * price[s_from:]   # +1.5p v
150     else:
151         c_obj[iE:iE + m] = price[s_from:]
152     for w in range(K):
153         c_obj[iG + w * m:iG + (w + 1) * m] = C.EMERGENCY_MULT * price[s_from:] * weights[w]
154
155     rows, cols, vals, rhs = [], [], [], []
156     r = 0
157     for j in range(m):
158         add = (r, iE + j, -1.0), (r, iC + j, 1.0), (r, iD + j, -1.0)
159         for a, b, c_ in add:
160             rows.append(a); cols.append(b); vals.append(c_)
161         rhs.append(-nd_fcst[s_from + j]); r += 1
162     for w in range(K):
163         for j in range(m):
164             for a, b, c_ in ((r, iE + j, -1.0), (r, iC + j, 1.0),
165                              (r, iD + j, -1.0), (r, iG + w * m + j, -1.0)):
166                 rows.append(a); cols.append(b); vals.append(c_)
167             rhs.append(-nd_scen[w, s_from + j]); r += 1
168     if use_plan:
169         for j in range(m):
170             for a, b, c_ in ((r, iE + j, 1.0), (r, iU + m + j, -1.0),
171                              (r, iU + j, 1.0)):
172                 rows.append(a); cols.append(b); vals.append(c_)
173             rhs.append(plan[s_from + j]); r += 1
174     A_ub = coo_matrix((vals, (rows, cols)), shape=(r, nv)).tocsr()
175     b_ub = np.array(rhs)
176
177     rows2, cols2, vals2, rhs2 = [], [], [], []
178     for j in range(m):
179         t = s_from + j
180         for b, c_ in ((iS + j, 1.0), (iC + j, -C.ETA_C), (iD + j, 1.0 / C.ETA_D)):
181             rows2.append(j); cols2.append(b); vals2.append(c_)
182         if j > 0:
183             rows2.append(j); cols2.append(iS + j - 1); vals2.append(-1.0)
184         rhs2.append(0.0 if j > 0 else s_init)
185     A_eq = coo_matrix((vals2, (rows2, cols2)), shape=(m, nv)).tocsr()
186     b_eq = np.array(rhs2)
187
188     bounds = [(0.0, None)] * m + [(0.0, C.E_MAX)] * m + [(0.0, C.E_MAX)] * m + \
189              [(C.S_MIN, C.S_MAX)] * m + [(0.0, None)] * (K * m) + \
190              ([(0.0, None)] * (2 * m) if use_plan else [])
191     if term_floor is not None:
192         bounds[iS + m - 1] = (max(term_floor, C.S_MIN), C.S_MAX)
193
194     res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
195                   bounds=bounds, method="highs")
196     if not res.success:
```

```
        raise RuntimeError("epoch LP 不可行: " + res.message)
    x = res.x
    out = dict(E=x[iE:iE + m].copy(), c=x[iC:iC + m].copy(),
               d=x[iD:iD + m].copy(), S=x[iS:iS + m].copy(),
               g=x[iG:iG + K * m].reshape(K, m).copy(), obj=res.fun,
               s_from=s_from, K=K)
    if use_plan:
        out["u"] = x[iU:iU + m].copy(); out["v"] = x[iU + m:iU + 2 * m].copy()
    return out


def check_solution(sol, nd_fcst, nd_act=None, tol=1e-7):
    """独立可行性自检（不依赖求解器返回值）。"""
    n = C.N_SLOT
    E, c, d, S = sol["E"], sol["c"], sol["d"], sol["S"]
    rep = {}
    rep["R1_plan_violation"] = float(np.max(nd_fcst + c - d - E))        # <= tol
    rep["R2_power_violation"] = float(max(np.max(c), np.max(d)) - C.E_MAX)
    rep["R3_soc_bounds"] = float(max(C.S_MIN - S.min(), S.max() - C.S_MAX))
    prev = np.concatenate([[sol["s_init"]], S[:-1]])
    rep["R4_soc_recursion"] = float(np.max(np.abs(S - (prev + C.ETA_C * c - d / C.ETA_D))))
    rep["R5_simultaneous"] = int(np.sum((c > 1e-6) & (d > 1e-6)))
    if nd_act is not None:
        g = np.maximum(nd_act + c - d - E, 0.0)
        rep["R6_emergency_consistency"] = float(np.max(
            np.abs(g - np.maximum(nd_act + c - d - E, 0.0))))
        rep["R7_min_supply_surplus"] = float(np.min(E + d - c - nd_act))
    return rep


def reduce_scenarios_stratified(resid, K=60):
    """按日总残差分层，层内取 medoid 为代表，权重=层样本占比。

    相比 K-means，分层抽样**显式保留残差分布的尾部**（每个分位区间都有代表），
    避免均值化把 5 倍紧急购电所需的安全边际抹平。
    """
    s = resid.sum(axis=1)
    order = np.argsort(s)
    groups = np.array_split(order, K)
    cen, w = [], []
    for g in groups:
        if len(g) == 0:
            continue
        sub = resid[g]
        if len(g) == 1:
            cen.append(sub[0])
        else:
            dist = np.sqrt(((sub[:, None, :] - sub[None, :, :]) ** 2).sum(-1))
            cen.append(sub[int(np.argmin(dist.sum(1)))])
        w.append(len(g) / len(s))
    return np.array(cen), np.array(w)


def reduce_scenarios(resid, K=30, seed=0):
    """对残差样本做 K-means 场景削减，返回（代表场景，权重）。"""
    from sklearn.cluster import KMeans
    km = KMeans(n_clusters=K, n_init=4, random_state=seed).fit(resid)
    lab = km.labels_
    cen = km.cluster_centers_
    w = np.array([np.sum(lab == k) for k in range(K)], float) / len(lab)
    keep = w > 0
    return cen[keep], w[keep]
```

源程序：code/q1.py

```
# -*- coding: utf-8 -*-
"""
问题 1：典型日确定性计划购电 LP（附件 1）
=========================================
模型（电量口径，单位 kWh）：

    min  sum_t p_t E_t                                          （目标）
    s.t. E_t - c_t + d_t >= (L_t - PV_t) * DT                （计划满足负载）
```

```
          S_t = S_{t-1} + eta_c c_t - d_t / eta_d              (SOC 递推)
          S_min <= S_t <= S_max,  0 <= c_t, d_t <= E_MAX
          E_t >= 0
          S_0 = S_144 = 6000 kWh（日周期，题面要求 0:00 与 24:00 相同）

运行：python3 q1.py    （输出 result1.xlsx、intermediate/q1.json、figures/fig03、fig04）
"""
from __future__ import annotations

import json
import os

import numpy as np
import pandas as pd
from scipy.optimize import linprog

import config as C


def solve_q1(price: np.ndarray, load: np.ndarray, pv: np.ndarray,
             s_init: float = C.S_INIT, s_end: float | None = None,
             free_cycle: bool = False):
    """求解典型日 LP。free_cycle=True 时 S_0 自由（仅要求 S_0 = S_144）。"""
    n = C.N_SLOT
    # 变量顺序：E(n) | c(n) | d(n) | S(n)  [free_cycle 时最前面再加 S0(1)]
    off = 1 if free_cycle else 0
    nv = off + 4 * n
    iE, iC, iD, iS = 0 + off, n + off, 2 * n + off, 3 * n + off

    c_obj = np.zeros(nv)
    c_obj[iE:iE + n] = price

    A_ub, b_ub = [], []
    # (a) E_t - c_t + d_t >= (L-PV)DT   ->  -E + c - d <= -(L-PV)DT
    for t in range(n):
        row = np.zeros(nv)
        row[iE + t] = -1.0
        row[iC + t] = 1.0
        row[iD + t] = -1.0
        A_ub.append(row)
        b_ub.append(-(load[t] - pv[t]) * C.DT)

    A_eq, b_eq = [], []
    # (b) SOC 递推：S_t - S_{t-1} - eta_c c_t + d_t/eta_d = 0
    for t in range(n):
        row = np.zeros(nv)
        row[iS + t] = 1.0
        if t == 0:
            if free_cycle:
                row[0] = -1.0          # 前置变量 S0（自由环状）
        else:
            row[iS + t - 1] = -1.0
        row[iC + t] = -C.ETA_C
        row[iD + t] = 1.0 / C.ETA_D
        A_eq.append(row)
        b_eq.append(0.0 if (t > 0 or free_cycle) else s_init)

    if free_cycle:                     # 日周期闭环：S_144 = S_0
        row = np.zeros(nv)
        row[iS + n - 1] = 1.0
        row[0] = -1.0
        A_eq.append(row)
        b_eq.append(0.0)

    bounds = [(0.0, None)] * n + [(0.0, C.E_MAX)] * n + [(0.0, C.E_MAX)] * n + \
             [(C.S_MIN, C.S_MAX)] * n
    if free_cycle:
        bounds = [(C.S_MIN, C.S_MAX)] + bounds
    else:
        # S_144 固定（日周期）；S_0 = s_init 通过递推常数注入
        end = s_init if s_end is None else s_end
        bounds[iS + n - 1] = (end, end)

```

```
    res = linprog(c_obj, A_ub=np.array(A_ub), b_ub=np.array(b_ub),
                  A_eq=np.array(A_eq), b_eq=np.array(b_eq), bounds=bounds,
                  method="highs")
    if not res.success:
        raise RuntimeError(res.message)

    x = res.x
    E, c, d, S = x[iE:iE + n], x[iC:iC + n], x[iD:iD + n], x[iS:iS + n]
    s0 = x[0] if free_cycle else s_init
    s_cur = np.concatenate([[s0], S])
    curtail = C.curtailment(price, load, pv, E, c, d)
    return dict(E=E, c=c, d=d, S=S, s0=s0, curtail=curtail, obj=res.fun,
                status=res.status, dual_eq=res.eqlin.marginals,
                dual_ub=res.ineqlin.marginals, nv=nv, res=res)


def summarize(sol, price, load, pv, label=""):
    n = C.N_SLOT
    E, c, d = sol["E"], sol["c"], sol["d"]
    total_e = float(E.sum())
    cost = float(np.sum(price * E))
    base = C.baseline_cost(price, load, pv)
    out = dict(
        label=label,
        cost=cost,
        total_energy=total_e,
        base_cost=base,
        saving=base - cost,
        saving_rate=(base - cost) / base,
        charge_total=float(c.sum()),
        discharge_total=float(d.sum()),
        curtail_total=float(np.maximum(sol["curtail"], 0).sum()),
        s0=sol["s0"], s_end=float(sol["S"][-1]),
        min_soc=float(sol["S"].min()), max_soc=float(sol["S"].max()),
        ratio_cd=float(c.sum() / d.sum()) if d.sum() > 0 else float("nan"),
    )
    # ---- 独立自检（不依赖求解器返回值）----
    # R1 原始可行性：供给 >= 需求（微网提供的电能不低于负载）
    supply = E + pv * C.DT + d
    demand = load * C.DT + c
    out["R1_min_surplus"] = float(np.min(supply - demand))          # 应 >= -1e-9
    # R2 计划可行性：E 不低于"预测口径下的净需求"
    out["R2_max_plan_violation"] = float(np.max(
        (load - pv) * C.DT + c - d - E))                            # 应 <= 1e-9
    # R3 SOC 递推残差（用上一时刻实际储电量重算）
    prev = np.concatenate([[sol["s0"]], sol["S"][:-1]])
    out["R3_max_soc_residual"] = float(np.max(np.abs(
        sol["S"] - (prev + C.ETA_C * c - d / C.ETA_D))))
    # R4 功率与储电量界
    out["R4_max_power_violation"] = float(max(np.max(c), np.max(d)) - C.E_MAX)
    out["R4_soc_bound_violation"] = float(max(C.S_MIN - sol["S"].min(),
                                              sol["S"].max() - C.S_MAX))
    # R5 充放互斥
    out["R5_simultaneous_slots"] = int(np.sum((c > 1e-6) & (d > 1e-6)))
    # R6 充放电量比 = 1/(eta_c eta_d)（若末值回到初值且无弃光）
    out["R6_cd_ratio_err"] = float(abs(c.sum() / d.sum() - 1 / (C.ETA_C * C.ETA_D))) if d.sum()
      > 0 else float("nan")
    # R7 费用独立复算（不调用求解器目标值）
    out["R7_cost_recompute_gap"] = float(abs(cost - float(np.sum(price * E))))
    return out


def table1(E):
    rows = []
    for lab, j in C.TABLE1_SLOTS:
        rows.append((lab, E[j]))
    return rows


def table2(c, d, s0, s_end):
    rows = []
    for (a, b), lab in zip(C.BLOCKS, C.BLOCK_LABELS):
        rows.append((lab, float(c[a:b].sum()), float(d[a:b].sum())))
```

```
    return rows, s0, s_end


def write_result1(sol, path=None):
    """按附件 5 模板写出 result1.xlsx（竖表）。"""
    path = path or os.path.join(C.RESULT_DIR, "result1.xlsx")
    tmpl = C.read_template("计划购电量", "result1.xlsx")
    labels = list(tmpl.iloc[1:, 0])
    assert len(labels) == C.N_SLOT
    df1 = pd.DataFrame({"时间段": labels, "购电量": np.round(sol["E"], 4)})

    rows, s0, s_end = table2(sol["c"], sol["d"], sol["s0"], float(sol["S"][-1]))
    df2 = pd.DataFrame({
        "时间段": C.BLOCK_LABELS,
        "充电量": [r[1] for r in rows],
        "放电量": [r[2] for r in rows],
        "时刻": ["0:00", "24:00", None, None, None, None],
        "储电量": [round(s0, 4), round(s_end, 4), None, None, None, None],
    })
    with pd.ExcelWriter(path) as w:
        df1.to_excel(w, sheet_name="计划购电量", index=False)
        df2.to_excel(w, sheet_name="充放电量", index=False)
    return path


def main():
    a1 = C.read_attachment1()
    price = a1["price"].to_numpy(float)
    load = a1["load"].to_numpy(float)
    pv = a1["pv"].to_numpy(float)

    sol = solve_q1(price, load, pv)
    summ = summarize(sol, price, load, pv, "Q1 主模型（S0=S144=6000）")

    sol_free = solve_q1(price, load, pv, free_cycle=True)
    summ_free = summarize(sol_free, price, load, pv, "Q1 自由环状（S0 自由，S0=S144）")

    print("=" * 78)
    for k, v in summ.items():
        print(f"  {k:26s} {v}")
    print("-" * 78)
    print(f"  自由环状最优费用 {summ_free['cost']:.2f} 元，S0=S144={summ_free['s0']:.2f} kWh，"
          f"与主模型差 {summ_free['cost'] - summ['cost']:.2f} 元 "
          f"({(summ_free['cost'] - summ['cost']) / summ['cost'] * 100:+.4f}%)")
    print("=" * 78)

    p = write_result1(sol)
    print("已写出", p)

    out = dict(main=summ, free_cycle=summ_free,
               table1=table1(sol["E"]),
               table2=table2(sol["c"], sol["d"], sol["s0"], float(sol["S"][-1])),
               arrays=dict(E=sol["E"].tolist(), c=sol["c"].tolist(),
                           d=sol["d"].tolist(), S=sol["S"].tolist(),
                           curtail=sol["curtail"].tolist(),
                           price=price.tolist(), load=load.tolist(), pv=pv.tolist(),
                           dual_eq=sol["dual_eq"].tolist(),
                           dual_bal=sol["dual_ub"].tolist()))
    with open(os.path.join(C.INTERMEDIATE_DIR, "q1.json"), "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    return out


if __name__ == "__main__":
    main()
```

源程序：code/q2.py

```
# -*- coding: utf-8 -*-
"""
问题 2：固定电价 + 实际波动 + 紧急购电惩罚的两阶段模型
============================================================
信息结构（R3 主方案，见假设 A9/R3）：
```

```
    计划依据 = 当天 0:00 发布的光伏预报（附件 3，0:00 行）+ 当日负载（题面未给负载预报，
               本模型按"负载曲线已知"处理，等价于负载预报无误差）
    结算依据 = 附件 2 当日实际负载与光伏
    缺口按 5 倍电价紧急购电

场景生成：对全部 334 天(决策窗口)的"预报残差" r_{d,t}=(PV^fcst-PV^act)*DT 做
          K-means 场景削减，得到 K 个代表残差与权重（三层保真的第 1、2 层）。

输出：results/result2.xlsx、intermediate/q2.json
"""
from __future__ import annotations

import argparse
import json
import os
import time

import numpy as np
import pandas as pd

import config as C
import lp_engine as LP
import results_io as RIO

DATE_WINDOW = pd.date_range(C.WIN_START, C.WIN_END, freq="D")
DATE_ALL = pd.date_range(C.WARMUP_START, C.WIN_END, freq="D")


def build_residuals(panel, dates):
    """构造预报残差矩阵 r[d,t] = (PV^fcst_0:00 - PV^act) * DT，单位 kWh。"""
    pv_act = panel.PV()
    R = np.zeros((len(dates), C.N_SLOT))
    for i, d in enumerate(dates):
        fc = panel.pv_forecast(d, 0)
        R[i] = (fc - pv_act.loc[d].to_numpy(float)) * C.DT
    return R


def run_q2(K=30, term_mode="ge", verbose=True, info="R3", max_days=None, seed=0,
           scen="strat"):
    panel = C.Panel(DATE_ALL)
    load = panel.L()
    pv_act = panel.PV()
    price_fixed = C.read_attachment1()["price"].to_numpy(float)
    price_flat = panel.P(volatile=False)          # 每天相同
    a1 = C.read_attachment1()

    # 残差与场景削减（在决策窗口 334 天上做）
    R_all = build_residuals(panel, DATE_WINDOW)
    if scen == "zero":                        # 消融：不设安全边际（按预报精确购电）
        resid_cen = np.zeros((1, C.N_SLOT)); resid_w = np.array([1.0])
    elif scen == "strat":
        resid_cen, resid_w = LP.reduce_scenarios_stratified(R_all, K=K)
    else:
        resid_cen, resid_w = LP.reduce_scenarios(R_all, K=K, seed=seed)
    if verbose:
        print(f"[场景削减] K={len(resid_w)}  权重和={resid_w.sum():.4f}  "
              f"残差均值绝对值={np.abs(resid_cen).mean():.2f} kWh")

    dates = DATE_ALL if max_days is None else DATE_ALL[:max_days]
    recs, s_init = [], C.S_INIT
    t0 = time.time()
    for i, d in enumerate(dates):
        nd_act = (load.loc[d].to_numpy(float) - pv_act.loc[d].to_numpy(float)) * C.DT
        if info == "R3":
            pv_fc = panel.pv_forecast(d, 0)
            nd_fcst = (load.loc[d].to_numpy(float) - pv_fc) * C.DT
        elif info == "R1":                        # 完美信息（下界）
            nd_fcst = nd_act.copy()
        elif info == "R2":                        # 附件 1 典型日预报
            nd_fcst = (a1["load"].to_numpy(float) - a1["pv"].to_numpy(float)) * C.DT
        else:
            raise ValueError(info)
```

```
79
80          nd_scen = nd_fcst[None, :] + resid_cen
81          r = d in DATE_WINDOW
82          tm = term_mode if r else "ge"
83          sol = LP.solve_day(price_fixed, nd_fcst, nd_scen, resid_w,
84                             s_init=s_init, term_mode=tm)
85          g, pc, ec = LP.realized_slot(sol, nd_act)
86          chk = LP.check_solution(sol, nd_fcst, nd_act)
87          if r:
88              recs.append(dict(
89                  date=d, E=sol["E"].copy(), c=sol["c"].copy(), d=sol["d"].copy(),
90                  S_end=float(sol["S"][-1]), s_init=s_init, g=g,
91                  plan_cost=pc, emg_cost=ec, total_cost=pc + ec,
92                  emg_qty=float(g.sum()), energy=float(sol["E"].sum()),
93                  base_cost=C.baseline_cost(price_fixed, load.loc[d].to_numpy(float),
94                                            pv_act.loc[d].to_numpy(float)),
95                  sae_obj=float(sol["obj"]),
96                  max_viol=max(chk["R1_plan_violation"], chk["R2_power_violation"],
97                               chk["R3_soc_bounds"], chk["R4_soc_recursion"])))
98          s_init = float(sol["S"][-1])
99          if verbose and (i + 1) % 30 == 0:
100             print(f"  ... {i+1}/{len(dates)} 天  累计 {time.time()-t0:.1f}s  "
101                   f"S={s_init:.0f}")
102     return recs, dict(resid_cen=resid_cen, resid_w=resid_w), price_fixed
103
104
105 def to_frames(recs, price_fixed):
106     """整理成 result2 表格所需结构。"""
107     dates = [r["date"] for r in recs]
108     E = np.vstack([r["E"] for r in recs])
109     Cc = np.vstack([r["c"] for r in recs])
110     Dd = np.vstack([r["d"] for r in recs])
111     G = np.vstack([r["g"] for r in recs])
112     S0 = np.array([r["s_init"] for r in recs])
113     S1 = np.array([r["S_end"] for r in recs])
114     return dates, E, Cc, Dd, G, S0, S1
115
116
117 def main():
118     ap = argparse.ArgumentParser()
119     ap.add_argument("--K", type=int, default=60)
120     ap.add_argument("--scen", default="strat", choices=["strat", "kmeans", "zero"])
121     ap.add_argument("--term", default="ge", choices=["ge", "eq", "free"])
122     ap.add_argument("--info", default="R3", choices=["R3", "R1", "R2"])
123     ap.add_argument("--max-days", type=int, default=None)
124     ap.add_argument("--tag", default="main")
125     ap.add_argument("--write", action="store_true")
126     args = ap.parse_args()
127
128     recs, scen, price_fixed = run_q2(K=args.K, term_mode=args.term, info=args.info,
129                                      max_days=args.max_days, scen=args.scen)
130     dates, E, Cc, Dd, G, S0, S1 = to_frames(recs, price_fixed)
131
132     # ---- 写出 result2.xlsx（仅在主口径下写出，避免被对照实验覆盖）----
133     if args.write:
134         path = os.path.join(C.RESULT_DIR, "result2.xlsx")
135         RIO.write_result(path, dates, E, Cc, Dd, S0, S1, price_fixed, g=G)
136         print("已写出", path)
137
138     tot_plan = sum(r["plan_cost"] for r in recs)
139     tot_emg = sum(r["emg_cost"] for r in recs)
140     tot_base = sum(r["base_cost"] for r in recs)
141     tot_emg_qty = sum(r["emg_qty"] for r in recs)
142     days_with_emg = int(np.sum(G.sum(axis=1) > 1e-6))
143     print("=" * 78)
144     print(f"  信息结构 {args.info} | 终端条件 {args.term} | 天数 {len(recs)}")
145     print(f"  计划购电费        {tot_plan:14.2f} 元")
146     print(f"  紧急购电费        {tot_emg:14.2f} 元")
147     print(f"  总费用            {tot_plan+tot_emg:14.2f} 元")
148     print(f"  无储能基线 C-01   {tot_base:14.2f} 元")
149     print(f"  相对基线节省      {tot_base-(tot_plan+tot_emg):14.2f} 元 "
150           f"({(tot_base-(tot_plan+tot_emg))/tot_base*100:.2f}%)")
151     print(f"  紧急购电量        {tot_emg_qty:14.2f} kWh  出现在 {days_with_emg}/{len(recs)} 天")
```

```
    print(f"  最差单日残差      {max(r['max_viol'] for r in recs):.3e}")
    print("=" * 78)

    out = dict(meta=dict(K=args.K, term=args.term, info=args.info, tag=args.tag,
                         scen=args.scen,
                         n_days=len(recs), tot_plan=tot_plan, tot_emg=tot_emg,
                         tot_cost=tot_plan + tot_emg, tot_base=tot_base,
                         tot_emg_qty=tot_emg_qty, days_with_emg=days_with_emg),
               daily=[{k: (v.isoformat() if k == "date" else
                           (v.tolist() if isinstance(v, np.ndarray) else v))
                       for k, v in r.items()} for r in recs],
               scen=dict(cen=scen["resid_cen"].tolist(),
                         w=scen["resid_w"].tolist()))
    with open(os.path.join(C.INTERMEDIATE_DIR, f"q2_{args.tag}.json"), "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    return out


if __name__ == "__main__":
    main()
```

源程序：code/q3.py

```
# -*- coding: utf-8 -*-
"""
问题 3：多预报时刻下的滚动调整（多阶段 / MPC）
================================================
信息结构：
  0:00 用附件 3 的 0:00 预报制定**全天**计划购电量 E^plan（144 槽，第一阶段承诺）；
  6:00/12:00/18:00 用当次预报对**其后至 24:00** 的剩余时段重新决策（后覆盖前），
  实际执行的是"最后覆盖该时段"的那次决策，记为调整购电量 E^adj。
  → 最终：槽 0-35 由 0:00 计划决定，36-71 由 6:00 决定，72-107 由 12:00 决定，
        108-143 由 18:00 决定（非预期性自然满足：每个决策只用该时刻及之前的预报）。

费用口径 C-06（偏差定价）：
  cost_t = p_t*min(E^plan,E^adj) + 0.5 p_t (E^plan-E^adj)^+ + 1.5 p_t (E^adj-E^plan)^+
  另加紧急购电 5 p_t g_t。第九章给出"计划费全额 + 差额惩罚"另一口径的对照。

VoI 实验：预报时刻数 n ∈{1,2,4,∞}，同一场景集、同一实际轨迹回放。
"""
from __future__ import annotations

import argparse
import json
import os
import time

import numpy as np
import pandas as pd

import config as C
import lp_engine as LP
import results_io as RIO

DATE_WINDOW = pd.date_range(C.WIN_START, C.WIN_END, freq="D")
DATE_ALL = pd.date_range(C.WARMUP_START, C.WIN_END, freq="D")


def forecast_slots(panel, date, hour, slots):
    """取 date 日 hour 时刻发布的预报在给定槽位上的值（kW）。"""
    row = panel.pf3.loc[(pd.Timestamp(date).normalize(), hour)]
    hourly = np.array([row[f"lead{k}"] for k in range(1, 25)], dtype=float)
    h = np.array([t // 6 for t in slots])          # 槽位所属整点（0..23）
    lead = (h - hour) % 24                         # 该整点对应的预报提前期
    return hourly[lead]


def build_epoch_scenarios(panel, hour, K=40, slots=None, dates=DATE_WINDOW):
    """构造"hour 时刻预报"的残差场景集（仅覆盖 slots），按窗口总残差分层削减。"""
    pv_act = panel.PV()
    slots = np.arange(C.N_SLOT) if slots is None else np.asarray(slots)
    R = np.zeros((len(dates), len(slots)))
    for i, d in enumerate(dates):
```

```
        fc = forecast_slots(panel, d, hour, slots)
        R[i] = (fc - pv_act.loc[d].to_numpy(float)[slots]) * C.DT
    cen, w = LP.reduce_scenarios_stratified(R, K=K)
    return cen, w


def run_q3(epochs=(0, 6, 12, 18), K=40, term_mode="ge", verbose=True,
           perfect=False, max_days=None, tag="main", seed=0):
    panel = C.Panel(DATE_ALL)
    load = panel.L(); pv_act = panel.PV()
    price = C.read_attachment1()["price"].to_numpy(float)

    scens = {}
    for h in epochs:
        s_m = C.EPOCH_SLOT[h]
        slots = np.arange(s_m, C.N_SLOT)
        cen, w = build_epoch_scenarios(panel, h, K=K, slots=slots)
        scens[h] = dict(slots=slots, cen=cen, w=w)
        if verbose:
            print(f"[场景] 时刻 {h:>2}:00  槽位 {s_m}..143  代表场景 {len(w)}  "
                  f"残差均值绝对值 {np.abs(cen).mean():.2f} kWh")

    dates = DATE_ALL if max_days is None else DATE_ALL[:max_days]
    recs, s_init = [], C.S_INIT
    t0 = time.time()
    for i, d in enumerate(dates):
        L = load.loc[d].to_numpy(float)
        PVa = pv_act.loc[d].to_numpy(float)
        nd_act = (L - PVa) * C.DT
        s_day = s_init
        E_plan = np.zeros(C.N_SLOT); E_adj = np.zeros(C.N_SLOT)
        cc = np.zeros(C.N_SLOT); dd = np.zeros(C.N_SLOT); SS = np.zeros(C.N_SLOT)
        soc = s_init
        for h in epochs:
            s_m = C.EPOCH_SLOT[h]
            slots = scens[h]["slots"]
            if perfect and h > 0:
                nd_fc = nd_act.copy()
            else:
                fc = forecast_slots(panel, d, h, slots)
                nd_fc = np.zeros(C.N_SLOT)
                nd_fc[slots] = (L[slots] - fc) * C.DT
            cen, w = scens[h]["cen"], scens[h]["w"]
            nd_sc = np.zeros((cen.shape[0], C.N_SLOT))
            nd_sc[:, slots] = nd_fc[slots][None, :] + cen
            sol = LP.solve_epoch(price, nd_fc, nd_sc, w, s_init=soc, s_from=s_m,
                                 plan=(E_plan if h > 0 else None), term_floor=s_day)
            E = sol["E"]
            if h == 0:
                E_plan = E.copy()
            nxt = C.N_SLOT if h == np.max(epochs) else C.EPOCH_SLOT[
                [x for x in epochs if x > h][0]]
            for j, t in enumerate(range(s_m, nxt)):
                E_adj[t] = E[j]; cc[t] = sol["c"][j]; dd[t] = sol["d"][j]
                SS[t] = sol["S"][j]
            soc = float(SS[nxt - 1])
        # ---- 结算（用实际数据）----
        g = np.maximum(nd_act + cc - dd - E_adj, 0.0)
        dev = E_adj - E_plan
        u = np.maximum(-dev, 0.0); v = np.maximum(dev, 0.0)
        cost_buy = float(np.sum(price * (np.minimum(E_plan, E_adj) + 0.0)))
        cost_breach = float(np.sum(C.BREACH_MULT * price * u))
        cost_excess = float(np.sum(C.EXCESS_MULT * price * v))
        cost_emg = float(C.EMERGENCY_MULT * np.sum(price * g))
        is_win = d in DATE_WINDOW
        if is_win:
            recs.append(dict(
                date=d, E_plan=E_plan.copy(), E_adj=E_adj.copy(), c=cc.copy(),
                d=dd.copy(), S=SS.copy(), g=g, s_init=s_day, s_end=float(SS[-1]),
                cost_buy=cost_buy, cost_breach=cost_breach, cost_excess=cost_excess,
                cost_emg=cost_emg,
                total_cost=cost_buy + cost_breach + cost_excess + cost_emg,
                plan_energy=float(E_plan.sum()), adj_energy=float(E_adj.sum()),
```

```
                emg_qty=float(g.sum()),
                breach_qty=float(u.sum()), excess_qty=float(v.sum()),
                base_cost=C.baseline_cost(price, L, PVa),
                max_viol=float(max(abs(nd_act + cc - dd - E_adj).max(), 0.0))))
        s_init = soc
        if verbose and (i + 1) % 40 == 0:
            print(f"  ... {i+1}/{len(dates)} 天  {time.time()-t0:.1f}s")
    return recs, scens, price


def summary(recs, price, tag=""):
    tot = {k: sum(r[k] for r in recs) for k in
           ("cost_buy", "cost_breach", "cost_excess", "cost_emg", "total_cost",
            "plan_energy", "adj_energy", "emg_qty", "breach_qty", "excess_qty",
            "base_cost")}
    tot["n_days"] = len(recs)
    tot["days_with_emg"] = int(sum(1 for r in recs if r["emg_qty"] > 1e-6))
    tot["tag"] = tag
    return tot


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--epochs", default="0,6,12,18")
    ap.add_argument("--K", type=int, default=40)
    ap.add_argument("--perfect", action="store_true")
    ap.add_argument("--tag", default="n4")
    ap.add_argument("--max-days", type=int, default=None)
    ap.add_argument("--write", action="store_true")
    args = ap.parse_args()
    epochs = tuple(int(x) for x in args.epochs.split(","))
    recs, scens, price = run_q3(epochs=epochs, K=args.K, perfect=args.perfect,
                                tag=args.tag, max_days=args.max_days)
    tot = summary(recs, price, args.tag)
    for k in ("cost_buy", "cost_breach", "cost_excess", "cost_emg", "total_cost",
              "base_cost"):
        print(f"  {k:14s} {tot[k]:16.2f}")
    print(f"  总费用 {tot['total_cost']:.2f} 元 ({tot['total_cost']/1e4:.2f} 万元) "
          f"节省率 {(tot['base_cost']-tot['total_cost'])/tot['base_cost']*100:.2f}% "
          f"紧急 {tot['emg_qty']:.0f} kWh {tot['days_with_emg']}/{tot['n_days']} 天 "
          f"违约 {tot['breach_qty']:.0f} 超额 {tot['excess_qty']:.0f} kWh")
    out = dict(meta=tot, daily=[{k: (v.isoformat() if k == "date" else
                                     (v.tolist() if isinstance(v, np.ndarray) else v))
                                 for k, v in r.items()} for r in recs])
    with open(os.path.join(C.INTERMEDIATE_DIR, f"q3_{args.tag}.json"), "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    if args.write and not args.perfect:
        dates = [r["date"] for r in recs]
        path = os.path.join(C.RESULT_DIR, "result3.xlsx")
        RIO.write_result(path, dates,
                         np.vstack([r["E_plan"] for r in recs]),
                         np.vstack([r["c"] for r in recs]),
                         np.vstack([r["d"] for r in recs]),
                         np.array([r["s_init"] for r in recs]),
                         np.array([r["s_end"] for r in recs]),
                         price, g=np.vstack([r["g"] for r in recs]),
                         E_adj=np.vstack([r["E_adj"] for r in recs]))
        print("已写出", path)
    return out


if __name__ == "__main__":
    main()
```

源程序：code/q4.py

```
# -*- coding: utf-8 -*-
"""
问题 4：波动电价下的购电策略
============================
(1) 电价过程四项检验 C1–C4（正态性/自相关/方差结构/与负载光伏的相关性）；
(2) 两层电价建模：分时骨架（附件 1 均值剖面）+ 随机残差（附件 4 − 骨架）；
(3) 联合情景：以"代表性历史日"为主（价格剖面与净需求残差取自同一天，
```

```
    天然保留 rho(p, L)），并以分块 Bootstrap 合成情景作条件对照；
(4) CVaR 风险—费用前沿 + 真实 334 天轨迹回测；
(5) 在波动电价下重算问题 2（result4-2.xlsx）与问题 3（result4-3.xlsx）。

注意：附件 4 只给实际电价、未给日前电价预报，故假设"0:00 可用的电价预报 = 分时骨架"，
      结算按附件 4 的实际电价。该假设在假设节显式声明（A10）。
"""
from __future__ import annotations

import argparse
import json
import os
import time

import numpy as np
import pandas as pd
from scipy import stats
from scipy.optimize import linprog
from scipy.sparse import coo_matrix

import config as C
import lp_engine as LP
import results_io as RIO
import q3 as Q3

DATE_WINDOW = pd.date_range(C.WIN_START, C.WIN_END, freq="D")
DATE_ALL = pd.date_range(C.WARMUP_START, C.WIN_END, freq="D")


# ---------------------------------------------------------------------------
# 通用：电价场景下的单日 LP（计划/调整两用）
# ---------------------------------------------------------------------------
def solve_vol(price_scen, nd_fcst, nd_scen, weights=None, s_init=C.S_INIT,
              s_from=0, plan=None, term_floor=None):
    """price_scen: (K,n) 各情景实际电价；plan 非空时按偏差定价口径。"""
    n = C.N_SLOT
    price_scen = np.atleast_2d(price_scen)
    K = price_scen.shape[0]
    if weights is None:
        weights = np.full(K, 1.0 / K)
    weights = np.asarray(weights, float); weights = weights / weights.sum()
    m = n - s_from
    use_plan = plan is not None and s_from > 0
    iE, iC, iD, iS, iG = 0, m, 2 * m, 3 * m, 4 * m
    iU = iG + K * m
    nv = 4 * m + K * m + (2 * m if use_plan else 0)

    pw = price_scen[:, s_from:]                       # (K,m)
    c_obj = np.zeros(nv)
    if use_plan:
        c_obj[iU:iU + m] = -C.BREACH_MULT * (weights[:, None] * pw).sum(0)
        c_obj[iU + m:iU + 2 * m] = C.EXCESS_MULT * (weights[:, None] * pw).sum(0)
    else:
        c_obj[iE:iE + m] = (weights[:, None] * pw).sum(0)
    c_obj[iG:iG + K * m] = C.EMERGENCY_MULT * (weights[:, None] * pw).ravel()

    rows, cols, vals, rhs = [], [], [], []
    r = 0
    for j in range(m):
        for a, b, c_ in ((r, iE + j, -1.0), (r, iC + j, 1.0), (r, iD + j, -1.0)):
            rows.append(a); cols.append(b); vals.append(c_)
        rhs.append(-nd_fcst[s_from + j]); r += 1
    for w in range(K):
        for j in range(m):
            for a, b, c_ in ((r, iE + j, -1.0), (r, iC + j, 1.0),
                             (r, iD + j, -1.0), (r, iG + w * m + j, -1.0)):
                rows.append(a); cols.append(b); vals.append(c_)
            rhs.append(-nd_scen[w, s_from + j]); r += 1
    if use_plan:
        for j in range(m):
            for a, b, c_ in ((r, iE + j, 1.0), (r, iU + m + j, -1.0),
                             (r, iU + j, 1.0)):
                rows.append(a); cols.append(b); vals.append(c_)
```

```
            rhs.append(plan[s_from + j]); r += 1
    A_ub = coo_matrix((vals, (rows, cols)), shape=(r, nv)).tocsr()
    b_ub = np.array(rhs)

    rows2, cols2, vals2, rhs2 = [], [], [], []
    for j in range(m):
        for b, c_ in ((iS + j, 1.0), (iC + j, -C.ETA_C), (iD + j, 1.0 / C.ETA_D)):
            rows2.append(j); cols2.append(b); vals2.append(c_)
        if j > 0:
            rows2.append(j); cols2.append(iS + j - 1); vals2.append(-1.0)
        rhs2.append(0.0 if j > 0 else s_init)
    A_eq = coo_matrix((vals2, (rows2, cols2)), shape=(m, nv)).tocsr()
    b_eq = np.array(rhs2)

    bounds = [(0.0, None)] * m + [(0.0, C.E_MAX)] * m + [(0.0, C.E_MAX)] * m + \
             [(C.S_MIN, C.S_MAX)] * m + [(0.0, None)] * (K * m) + \
             ([(0.0, None)] * (2 * m) if use_plan else [])
    if term_floor is not None:
        bounds[iS + m - 1] = (max(term_floor, C.S_MIN), C.S_MAX)
    res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                  bounds=bounds, method="highs")
    if not res.success:
        raise RuntimeError("volatile LP 不可行: " + res.message)
    x = res.x
    return dict(E=x[iE:iE + m].copy(), c=x[iC:iC + m].copy(),
                d=x[iD:iD + m].copy(), S=x[iS:iS + m].copy(),
                g=x[iG:iG + K * m].reshape(K, m).copy(), obj=res.fun, s_from=s_from)


# ---------------------------------------------------------------------------
# 电价过程检验与两层建模
# ---------------------------------------------------------------------------
def price_tests(panel):
    a1 = C.read_attachment1()
    p4 = panel.P(volatile=True).to_numpy(float)
    skel = a1["price"].to_numpy(float)
    resid = p4 - skel[None, :]
    load = panel.L().to_numpy(float); pv = panel.PV().to_numpy(float)
    out = {}
    out["skeleton_is_mean"] = float(np.max(np.abs(p4.mean(0) - skel)))
    out["price_mean"] = float(p4.mean()); out["price_sd"] = float(p4.std())
    out["price_min"] = float(p4.min()); out["price_max"] = float(p4.max())
    out["resid_sd_within"] = float(resid.std(0).mean())
    out["cv_within"] = float((resid.std(0) / skel).mean())
    # C1 正态性（标准化残差）
    z = resid / resid.std(0, keepdims=True)
    zz = z.ravel()
    jb = stats.jarque_bera(zz)
    out["C1_JB_stat"] = float(jb.statistic); out["C1_JB_p"] = float(jb.pvalue)
    out["C1_skew"] = float(stats.skew(zz)); out["C1_kurt"] = float(stats.kurtosis(zz))
    # C2 自相关（日内残差序列，滞后 1..6）
    acf = [float(np.mean([np.corrcoef(resid[i, :-k], resid[i, k:])[0, 1]
                          for i in range(len(resid))])) for k in range(1, 7)]
    out["C2_acf_lag1_6"] = acf
    # C2b 日间自相关（同一槽位跨日）
    acf_day = [float(np.mean([np.corrcoef(resid[:-k, t], resid[k:, t])[0, 1]
                              for t in range(C.N_SLOT)])) for k in range(1, 4)]
    out["C2b_daily_acf"] = acf_day
    # C3 方差结构：各槽位残差标准差（谷/峰时段对比）
    sd_slot = resid.std(0)
    hrs = np.array([t // 6 for t in range(C.N_SLOT)])
    out["C3_sd_night"] = float(sd_slot[hrs == 3].mean())      # 03:00-04:00 谷段
    out["C3_sd_peak"] = float(sd_slot[hrs == 19].mean())      # 19:00-20:00 峰段
    out["C3_sd_ratio"] = out["C3_sd_peak"] / out["C3_sd_night"]
    out["C3_sd_all"] = float(sd_slot.mean())
    # C4 相关性（池化 / 日聚合 / 去形状）
    net = load - pv
    out["C4_pool_pL"] = float(np.corrcoef(p4.ravel(), load.ravel())[0, 1])
    out["C4_pool_pNet"] = float(np.corrcoef(p4.ravel(), net.ravel())[0, 1])
    out["C4_pool_pPV"] = float(np.corrcoef(p4.ravel(), pv.ravel())[0, 1])
    out["C4_day_pL"] = float(np.corrcoef(p4.mean(1), load.mean(1))[0, 1])
    out["C4_day_pPV"] = float(np.corrcoef(p4.mean(1), pv.mean(1))[0, 1])
    dev_p = p4 - p4.mean(1, keepdims=True); dev_l = load - load.mean(1, keepdims=True)
```

```
    out["C4_deshape_pL"] = float(np.corrcoef(dev_p.ravel(), dev_l.ravel())[0, 1])
    return out, resid, skel


def build_joint_scenarios(panel, K=60, mode="hist", seed=7, block=36):
    """联合情景：mode='hist' 取代表性历史日；mode='boot' 分块 Bootstrap 合成。"""
    load = panel.L().to_numpy(float); pv = panel.PV().to_numpy(float)
    p4 = panel.P(volatile=True).to_numpy(float)
    idx = [list(DATE_ALL).index(d) for d in DATE_WINDOW]
    res_nd = np.zeros((len(idx), C.N_SLOT))
    net_nd = np.zeros((len(idx), C.N_SLOT))
    for i, k in enumerate(idx):
        fc = panel.pv_forecast(DATE_WINDOW[i], 0)
        res_nd[i] = (fc - pv[k]) * C.DT
        net_nd[i] = (load[k] - pv[k]) * C.DT
    if mode == "hist":
        # 二维分层：按"日净需求残差"与"日均电价"各分层（网格），层内取 medoid，
        # 权重 = 层样本占比 → 同时保真净需求残差与电价的边际分布，并保留联合结构。
        np_str = int(np.floor(np.sqrt(K)))
        nd_tot = res_nd.sum(1)
        p_avg = p4[idx].mean(1)
        q1 = np.quantile(nd_tot, np.linspace(0, 1, np_str + 1))
        q2 = np.quantile(p_avg, np.linspace(0, 1, np_str + 1))
        b1 = np.clip(np.digitize(nd_tot, q1[1:-1]), 0, np_str - 1)
        b2 = np.clip(np.digitize(p_avg, q2[1:-1]), 0, np_str - 1)
        cell = b1 * np_str + b2
        pos, w = [], []
        for cell_id in range(np_str * np_str):
            g = np.where(cell == cell_id)[0]
            if len(g) == 0:
                continue
            sub = res_nd[g]
            if len(g) == 1:
                pos.append(g[0])
            else:
                dist = np.sqrt(((sub[:, None, :] - sub[None, :, :]) ** 2).sum(-1))
                pos.append(g[int(np.argmin(dist.sum(1)))])
            w.append(len(g) / len(nd_tot))
        pos = np.array(pos); w = np.array(w)
        return dict(price=p4[idx][pos], resid=res_nd[pos], w=w,
                    net=net_nd[pos], src=np.array(DATE_WINDOW)[pos])
    rng = np.random.default_rng(seed)
    nblk = C.N_SLOT // block
    price_s, resid_s = [], []
    for _ in range(K):
        pr, rr = [], []
        for j in range(nblk):                       # 块按时刻对齐重采样，保住日内形状
            aa = int(rng.integers(0, len(idx)))
            k = idx[aa]; off = j * block
            pr.append(p4[k, off:off + block])
            fc = panel.pv_forecast(DATE_WINDOW[aa], 0)[off:off + block]
            rr.append((fc - pv[k, off:off + block]) * C.DT)
        price_s.append(np.concatenate(pr)); resid_s.append(np.concatenate(rr))
    return dict(price=np.array(price_s), resid=np.array(resid_s),
                w=np.full(K, 1.0 / K), net=None, src=None)


def fidelity(scen, panel, price_tests_out):
    """情景保真度：均值/分位数差、相关系数保持。"""
    p4 = panel.P(volatile=True).to_numpy(float)
    idx = [list(DATE_ALL).index(d) for d in DATE_WINDOW]
    p_true = p4[idx].ravel()
    p_s = scen["price"].ravel()
    qs = [1, 5, 25, 50, 75, 95, 99]
    qt = np.percentile(p_true, qs); qsi = np.percentile(p_s, qs)
    out = dict(mean_gap=float(abs(p_s.mean() - p_true.mean()) / p_true.mean()),
               max_q_gap=float(np.max(np.abs(qsi - qt) / np.abs(qt))),
               q_levels=qs, q_true=qt.tolist(), q_scen=qsi.tolist(),
               q_gap_rel=[float(abs(a - b) / abs(b)) for a, b in zip(qsi, qt)])
    load = panel.L().to_numpy(float)[idx]
    net = load - panel.PV().to_numpy(float)[idx]
    out["corr_true_pL"] = float(np.corrcoef(p_true, load.ravel())[0, 1])
    out["corr_true_pNet"] = float(np.corrcoef(p_true, net.ravel())[0, 1])
```

```
    if scen.get("net") is not None:
        w = scen["w"] / scen["w"].sum()
        rep = np.maximum((w * len(p_true)).round().astype(int), 0)
        ps, ns = [], []
        for k, rr in enumerate(rep):
            if rr == 0:
                continue
            ps.append(np.repeat(scen["price"][k][None, :], rr, 0))
            ns.append(np.repeat(scen["net"][k][None, :], rr, 0))
        ps = np.vstack(ps).ravel(); ns = np.vstack(ns).ravel()
        out["corr_scen_pNet"] = float(np.corrcoef(ps, ns)[0, 1])
    else:
        out["corr_scen_pNet"] = float("nan")
    return out

# ---------------------------------------------------------------------------
# 波动电价下重算问题 2（Q4-2）
# ---------------------------------------------------------------------------
def run_q4_2(scen, verbose=True, max_days=None, write=False, tag="q42"):
    panel = C.Panel(DATE_ALL)
    load = panel.L(); pv_act = panel.PV()
    p4 = panel.P(volatile=True)
    skel = C.read_attachment1()["price"].to_numpy(float)
    dates = DATE_ALL if max_days is None else DATE_ALL[:max_days]
    recs, s_init = [], C.S_INIT
    t0 = time.time()
    for i, d in enumerate(dates):
        L = load.loc[d].to_numpy(float); PVa = pv_act.loc[d].to_numpy(float)
        nd_act = (L - PVa) * C.DT
        fc = panel.pv_forecast(d, 0)
        nd_fcst = (L - fc) * C.DT
        nd_scen = nd_fcst[None, :] + scen["resid"]
        sol = solve_vol(scen["price"], nd_fcst, nd_scen, scen["w"], s_init=s_init,
                        s_from=0, term_floor=s_init)
        g = np.maximum(nd_act + sol["c"] - sol["d"] - sol["E"], 0.0)
        pact = p4.loc[d].to_numpy(float)
        buy = float(np.sum(pact * sol["E"]))
        emg = float(C.EMERGENCY_MULT * np.sum(pact * g))
        if d in DATE_WINDOW:
            recs.append(dict(date=d, E=sol["E"].copy(), c=sol["c"].copy(),
                             d=sol["d"].copy(), S=sol["S"].copy(), g=g,
                             s_init=s_init, s_end=float(sol["S"][-1]),
                             cost_buy=buy, cost_emg=emg, total_cost=buy + emg,
                             emg_qty=float(g.sum()), plan_energy=float(sol["E"].sum()),
                             base_cost=C.baseline_cost(pact, L, PVa),
                             base_cost_skel=C.baseline_cost(skel, L, PVa)))
        s_init = float(sol["S"][-1])
        if verbose and (i + 1) % 60 == 0:
            print(f"  ... {i+1}/{len(dates)} 天 {time.time()-t0:.1f}s")
    if write:
        dates_w = [r["date"] for r in recs]
        path = os.path.join(C.RESULT_DIR, "result4-2.xlsx")
        RIO.write_result(path, dates_w,
                         np.vstack([r["E"] for r in recs]),
                         np.vstack([r["c"] for r in recs]),
                         np.vstack([r["d"] for r in recs]),
                         np.array([r["s_init"] for r in recs]),
                         np.array([r["s_end"] for r in recs]),
                         skel, g=np.vstack([r["g"] for r in recs]),
                         price_mat=p4.loc[dates_w].to_numpy(float))
        print("已写出", path)
    return recs


# ---------------------------------------------------------------------------
# 波动电价下重算问题 3（Q4-3）：滚动调整
# ---------------------------------------------------------------------------
def run_q4_3(scen, epochs=(0, 6, 12, 18), K=60, verbose=True, max_days=None,
             write=False, perfect_price=False, tag="q43"):
    panel = C.Panel(DATE_ALL)
    load = panel.L(); pv_act = panel.PV()
    p4 = panel.P(volatile=True)
    skel = C.read_attachment1()["price"].to_numpy(float)
```

```
    # 各预报时刻的净需求残差场景（该时刻预报 vs 实际）
    scens = {}
    for h in epochs:
        s_m = C.EPOCH_SLOT[h]
        slots = np.arange(s_m, C.N_SLOT)
        cen, w = Q3.build_epoch_scenarios(panel, h, K=K, slots=slots)
        scens[h] = dict(slots=slots, cen=cen, w=w)
    dates = DATE_ALL if max_days is None else DATE_ALL[:max_days]
    recs, s_init = [], C.S_INIT
    t0 = time.time()
    for i, d in enumerate(dates):
        L = load.loc[d].to_numpy(float); PVa = pv_act.loc[d].to_numpy(float)
        nd_act = (L - PVa) * C.DT
        pact = p4.loc[d].to_numpy(float)
        ps = pact[None, :] if perfect_price else scen["price"]
        s_day = s_init
        E_plan = np.zeros(C.N_SLOT); E_adj = np.zeros(C.N_SLOT)
        cc = np.zeros(C.N_SLOT); dd = np.zeros(C.N_SLOT)
        soc = s_init
        for h in epochs:
            s_m = C.EPOCH_SLOT[h]; slots = scens[h]["slots"]
            if perfect_price and h > 0:
                price_sc = pact[None, :]
            else:
                price_sc = ps
            fc = Q3.forecast_slots(panel, d, h, slots)
            nd_fc = np.zeros(C.N_SLOT); nd_fc[slots] = (L[slots] - fc) * C.DT
            cen = scens[h]["cen"]
            nd_sc = np.zeros((cen.shape[0], C.N_SLOT))
            nd_sc[:, slots] = nd_fc[slots][None, :] + cen
            sol = solve_vol(price_sc, nd_fc, nd_sc, scens[h]["w"], s_init=soc,
                            s_from=s_m, plan=(E_plan if h > 0 else None),
                            term_floor=s_day)
            E = sol["E"]
            if h == 0:
                E_plan = E.copy()
            later = [x for x in epochs if x > h]
            nxt = C.EPOCH_SLOT[later[0]] if later else C.N_SLOT
            for j, t in enumerate(range(s_m, nxt)):
                E_adj[t] = E[j]; cc[t] = sol["c"][j]; dd[t] = sol["d"][j]
            soc = float(sol["S"][nxt - s_m - 1])
        g = np.maximum(nd_act + cc - dd - E_adj, 0.0)
        dev = E_adj - E_plan
        u = np.maximum(-dev, 0.0); v = np.maximum(dev, 0.0)
        cost_buy = float(np.sum(pact * np.minimum(E_plan, E_adj)))
        cost_breach = float(C.BREACH_MULT * np.sum(pact * u))
        cost_excess = float(C.EXCESS_MULT * np.sum(pact * v))
        cost_emg = float(C.EMERGENCY_MULT * np.sum(pact * g))
        if d in DATE_WINDOW:
            recs.append(dict(date=d, E_plan=E_plan.copy(), E_adj=E_adj.copy(),
                             c=cc.copy(), d=dd.copy(), g=g, s_init=s_day,
                             s_end=soc, cost_buy=cost_buy, cost_breach=cost_breach,
                             cost_excess=cost_excess, cost_emg=cost_emg,
                             total_cost=cost_buy + cost_breach + cost_excess + cost_emg,
                             emg_qty=float(g.sum()), plan_energy=float(E_plan.sum()),
                             adj_energy=float(E_adj.sum()), breach_qty=float(u.sum()),
                             excess_qty=float(v.sum()),
                             base_cost=C.baseline_cost(pact, L, PVa)))
        s_init = soc
        if verbose and (i + 1) % 60 == 0:
            print(f"  ... {i+1}/{len(dates)} 天 {time.time()-t0:.1f}s")
    if write:
        dates_w = [r["date"] for r in recs]
        path = os.path.join(C.RESULT_DIR, "result4-3.xlsx")
        RIO.write_result(path, dates_w,
                         np.vstack([r["E_plan"] for r in recs]),
                         np.vstack([r["c"] for r in recs]),
                         np.vstack([r["d"] for r in recs]),
                         np.array([r["s_init"] for r in recs]),
                         np.array([r["s_end"] for r in recs]),
                         skel, g=np.vstack([r["g"] for r in recs]),
                         E_adj=np.vstack([r["E_adj"] for r in recs]),
                         price_mat=p4.loc[dates_w].to_numpy(float))
```

```
        print("已写出", path)
    return recs


# ---------------------------------------------------------------------------
# CVaR 风险—费用前沿（Rockafellar–Uryasev 线性化）
# ---------------------------------------------------------------------------
def solve_cvar_day(price_scen, nd_fcst, nd_scen, weights, s_init, lam=0.0,
                   alpha=0.95, term_floor=None):
    n = C.N_SLOT
    K = price_scen.shape[0]
    weights = np.asarray(weights, float); weights = weights / weights.sum()
    iE, iC, iD, iS, iG, iZ, iEta = 0, n, 2 * n, 3 * n, 4 * n, 4 * n + K * n, 4 * n + K * n + K
    nv = iEta + 1
    coef = (1 - lam) * weights[:, None] * price_scen
    c_obj = np.zeros(nv)
    c_obj[iE:iE + n] = coef.sum(0)
    c_obj[iG:iG + K * n] = C.EMERGENCY_MULT * coef.ravel()
    c_obj[iZ:iZ + K] = lam * weights / (1 - alpha)
    c_obj[iEta] = lam
    rows, cols, vals, rhs = [], [], [], []
    r = 0
    for t in range(n):
        for a, b, c_ in ((r, iE + t, -1.0), (r, iC + t, 1.0), (r, iD + t, -1.0)):
            rows.append(a); cols.append(b); vals.append(c_)
        rhs.append(-nd_fcst[t]); r += 1
    for w in range(K):
        for t in range(n):
            for a, b, c_ in ((r, iE + t, -1.0), (r, iC + t, 1.0),
                             (r, iD + t, -1.0), (r, iG + w * n + t, -1.0)):
                rows.append(a); cols.append(b); vals.append(c_)
            rhs.append(-nd_scen[w, t]); r += 1
    for w in range(K):          # z_w + eta >= cost_w  ->  cost_w - z_w - eta <= 0
        for t in range(n):
            rows.append(r); cols.append(iE + t); vals.append(price_scen[w, t])
            rows.append(r); cols.append(iG + w * n + t); vals.append(C.EMERGENCY_MULT * price_scen[w, t
    ])
        rows.append(r); cols.append(iZ + w); vals.append(-1.0)
        rows.append(r); cols.append(iEta); vals.append(-1.0)
        rhs.append(0.0); r += 1
    A_ub = coo_matrix((vals, (rows, cols)), shape=(r, nv)).tocsr()
    b_ub = np.array(rhs)
    rows2, cols2, vals2, rhs2 = [], [], [], []
    for t in range(n):
        for b, c_ in ((iS + t, 1.0), (iC + t, -C.ETA_C), (iD + t, 1.0 / C.ETA_D)):
            rows2.append(t); cols2.append(b); vals2.append(c_)
        if t > 0:
            rows2.append(t); cols2.append(iS + t - 1); vals2.append(-1.0)
        rhs2.append(0.0 if t > 0 else s_init)
    A_eq = coo_matrix((vals2, (rows2, cols2)), shape=(n, nv)).tocsr()
    b_eq = np.array(rhs2)
    bounds = [(0.0, None)] * n + [(0.0, C.E_MAX)] * n + [(0.0, C.E_MAX)] * n + \
             [(C.S_MIN, C.S_MAX)] * n + [(0.0, None)] * (K * n) + \
             [(0.0, None)] * K + [(None, None)]
    if term_floor is not None:
        bounds[iS + n - 1] = (max(term_floor, C.S_MIN), C.S_MAX)
    res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                  bounds=bounds, method="highs")
    if not res.success:
        raise RuntimeError("CVaR LP 不可行: " + res.message)
    x = res.x
    return dict(E=x[iE:iE + n], c=x[iC:iC + n], d=x[iD:iD + n], S=x[iS:iS + n],
                g=x[iG:iG + K * n].reshape(K, n), eta=float(x[iEta]),
                obj=res.fun, z=x[iZ:iZ + K])


# ---------------------------------------------------------------------------
# 主程序
# ---------------------------------------------------------------------------
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--mode", default="tests",
                    choices=["tests", "q42", "q43", "cvar", "all"])
```

```
    ap.add_argument("--K", type=int, default=60)
    ap.add_argument("--scen", default="hist", choices=["hist", "boot"])
    ap.add_argument("--write", action="store_true")
    ap.add_argument("--max-days", type=int, default=None)
    ap.add_argument("--lams", default="0,0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9,1.0")
    ap.add_argument("--step", type=int, default=6, help="CVaR 前沿子样本步长")
    args = ap.parse_args()

    panel = C.Panel(DATE_ALL)
    out = {}
    if args.mode in ("tests", "all"):
        pt, resid, skel = price_tests(panel)
        scen_h = build_joint_scenarios(panel, K=args.K, mode="hist")
        scen_b = build_joint_scenarios(panel, K=args.K, mode="boot")
        fid_h = fidelity(scen_h, panel, pt)
        fid_b = fidelity(scen_b, panel, pt)
        print("=" * 78)
        for k, v in pt.items():
            print("  %-22s %s" % (k, np.round(v, 3) if isinstance(v, list) else v))
        print("  保真度(历史代表日):", {k: (round(v, 4) if isinstance(v, float) else v)
                                      for k, v in fid_h.items() if k not in ("q_true", "q_scen")})
        print("  保真度(分块Bootstrap):", {k: (round(v, 4) if isinstance(v, float) else v)
                                        for k, v in fid_b.items() if k not in ("q_true", "q_scen")})
        print("=" * 78)
        out["price_tests"] = pt
        out["fidelity_hist"] = fid_h
        out["fidelity_boot"] = fid_b
        with open(os.path.join(C.INTERMEDIATE_DIR, "q4_price.json"), "w") as f:
            json.dump(out, f, ensure_ascii=False, indent=1)

    if args.mode in ("q42", "q43", "all"):
        scen = build_joint_scenarios(panel, K=args.K, mode=args.scen)
        if args.mode in ("q42", "all"):
            recs = run_q4_2(scen, max_days=args.max_days, write=args.write, tag="q42")
            tot = dict(cost_buy=sum(r["cost_buy"] for r in recs),
                       cost_emg=sum(r["cost_emg"] for r in recs),
                       total=sum(r["total_cost"] for r in recs),
                       base=sum(r["base_cost"] for r in recs),
                       emg_qty=sum(r["emg_qty"] for r in recs),
                       days_emg=int(sum(1 for r in recs if r["emg_qty"] > 1e-6)))
            print("Q4-2:", {k: (round(v, 2) if isinstance(v, float) else v) for k, v in tot.items()})
            print("  节省率 %.2f%%" % ((tot["base"] - tot["total"]) / tot["base"] * 100))
            with open(os.path.join(C.INTERMEDIATE_DIR, "q4_2.json"), "w") as f:
                json.dump(dict(meta=tot, daily=[{k: (v.isoformat() if k == "date" else
                                                     (v.tolist() if isinstance(v, np.ndarray) else v))
                                                 for k, v in r.items()} for r in recs]),
                          f, ensure_ascii=False, indent=1)
        if args.mode in ("q43", "all"):
            recs3 = run_q4_3(scen, K=args.K, max_days=args.max_days, write=args.write)
            tot3 = {k: sum(r[k] for r in recs3) for k in
                    ("cost_buy", "cost_breach", "cost_excess", "cost_emg", "total_cost",
                     "base_cost", "emg_qty", "breach_qty", "excess_qty")}
            print("Q4-3:", {k: (round(v, 2) if isinstance(v, float) else v) for k, v in tot3.items()})
            print("  节省率 %.2f%%" % ((tot3["base_cost"] - tot3["total_cost"]) / tot3["base_cost"] *
 100))
            with open(os.path.join(C.INTERMEDIATE_DIR, "q4_3.json"), "w") as f:
                json.dump(dict(meta=tot3, daily=[{k: (v.isoformat() if k == "date" else
                                                     (v.tolist() if isinstance(v, np.ndarray) else v))
                                                 for k, v in r.items()} for r in recs3]),
                          f, ensure_ascii=False, indent=1)

    if args.mode in ("cvar", "all"):
        scen = build_joint_scenarios(panel, K=args.K, mode=args.scen)
        lams = [float(x) for x in args.lams.split(",")]
        sub = list(DATE_WINDOW)[::args.step]
        load = panel.L(); pv_act = panel.PV(); p4 = panel.P(volatile=True)
        res = []
        for lam in lams:
            exp_cost, real_cost, cvar_val, emg = 0.0, 0.0, 0.0, 0.0
            for d in sub:
                L = load.loc[d].to_numpy(float); PVa = pv_act.loc[d].to_numpy(float)
                nd_act = (L - PVa) * C.DT
                fc = panel.pv_forecast(d, 0)
```

```
                nd_fc = (L - fc) * C.DT
                nd_sc = nd_fc[None, :] + scen["resid"]
                sol = solve_cvar_day(scen["price"], nd_fc, nd_sc, scen["w"],
                                     C.S_INIT, lam=lam, alpha=0.95, term_floor=C.S_INIT)
                pact = p4.loc[d].to_numpy(float)
                g = np.maximum(nd_act + sol["c"] - sol["d"] - sol["E"], 0.0)
                real = float(np.sum(pact * (sol["E"] + C.EMERGENCY_MULT * g)))
                g_sc = np.maximum(nd_sc + sol["c"][None, :] - sol["d"][None, :]
                                  - sol["E"][None, :], 0)
                sc_cost = (scen["price"] * (sol["E"][None, :]
                                            + C.EMERGENCY_MULT * g_sc)).sum(1)
                exp_cost += float(scen["w"] @ sc_cost)
                real_cost += real; emg += float(g.sum())
                var = np.quantile(sc_cost, 0.95)
                cvar_val += float(var + np.mean(np.maximum(sc_cost - var, 0)) / 0.05)
            n = len(sub)
            res.append(dict(lam=lam, exp_cost=exp_cost / n, real_cost=real_cost / n,
                            cvar=cvar_val / n, emg_qty=emg / n))
            print("  lam=%.1f  期望费用 %.2f  真实回测 %.2f  CVaR95 %.2f  紧急 %.1f kWh/日"
                  % (lam, exp_cost / n, real_cost / n, cvar_val / n, emg / n))
        with open(os.path.join(C.INTERMEDIATE_DIR, "q4_cvar.json"), "w") as f:
            json.dump(dict(days=[str(pd.Timestamp(x).date()) for x in sub], frontier=res),
                      f, ensure_ascii=False, indent=1)
    return out


if __name__ == "__main__":
    main()
```

源程序：code/results_io.py

```
# -*- coding: utf-8 -*-
"""结果文件写出器（严格按附件 5 模板的列序、列名、日期范围）。"""
from __future__ import annotations

import os

import numpy as np
import pandas as pd

import config as C


def _labels(fname: str, sheet: str):
    t = C.read_template(sheet, fname, nrows=1)
    return [str(v) for v in t.iloc[0].tolist()]


def plan_sheet_frame(dates, E, price, price_mat=None) -> pd.DataFrame:
    """横表: 日期 + 144 槽 + 全天购电量 + 全天购电费。"""
    labels = _labels("result2.xlsx", "计划购电量")
    slot_labels = labels[1:145]
    assert len(slot_labels) == C.N_SLOT, len(slot_labels)
    cols = ["日期\\时间"] + slot_labels + ["全天购电量", "全天购电费"]
    rows = []
    for i, d in enumerate(dates):
        p = price if price_mat is None else price_mat[i]
        rows.append([d.strftime("%Y-%m-%d")] + list(np.round(E[i], 4)) +
                    [round(float(E[i].sum()), 4), round(float(np.sum(p * E[i])), 2)])
    return pd.DataFrame(rows, columns=cols)


def soc_sheet_frame(dates, c, d, s0, s1) -> pd.DataFrame:
    """充放电量: 每天 6 行（4 h 块），0:00 储电量填在 0:00-4:00 行、24:00 填在 4:00-8:00 行。"""
    rows = []
    for i, day in enumerate(dates):
        for b, ((a, bnd), lab) in enumerate(zip(C.BLOCKS, C.BLOCK_LABELS)):
            rec = [(day.strftime("%Y-%m-%d") if b == 0 else ""), lab,
                   round(float(c[i, a:bnd].sum()), 4), round(float(d[i, a:bnd].sum()), 4),
                   "", ""]
            if b == 0:
                rec[4] = "0:00"; rec[5] = round(float(s0[i]), 4)
            if b == 1:
```

```
                rec[4] = "24:00"; rec[5] = round(float(s1[i]), 4)
            rows.append(rec)
    return pd.DataFrame(rows, columns=["日期", "时间段", "充电量", "放电量", "时刻", "储电量"])


def emergency_frame(dates, g, tol=1e-6) -> pd.DataFrame:
    """紧急购电量：按连续时段合并，同一日期可多行。"""
    rows = []
    for i, day in enumerate(dates):
        gi = g[i]
        idx = np.where(gi > tol)[0]
        if len(idx) == 0:
            rows.append([day.strftime("%Y-%m-%d"), "无", 0.0])
            continue
        # 合并连续槽
        groups, start = [], idx[0]
        for a, b in zip(idx[:-1], idx[1:]):
            if b != a + 1:
                groups.append((start, a)); start = b
        groups.append((start, idx[-1]))
        first = True
        for a, b in groups:
            lab = f"{_hm(a)}-{_hm(b + 1)}"
            rows.append([day.strftime("%Y-%m-%d") if first else "", lab,
                         round(float(gi[a:b + 1].sum()), 3)])
            first = False
    return pd.DataFrame(rows, columns=["日期", "购电时间段", "购电量"])


def _hm(slot_end: int) -> str:
    """槽位末端的时刻标签，例如槽 5 结束 -> 1:00。"""
    m = slot_end * 10
    if m >= 1440:
        m -= 1440
        return f"{m // 60}:{m % 60:02d}+1" if m else "0:00+1"
    return f"{m // 60}:{m % 60:02d}"


def write_result(path, dates, E, c, d, s0, s1, price, g=None, E_adj=None,
                 price_mat=None, extra_sheets=()):
    """写出结果文件（工作表名与模板一致）。"""
    sheets = {"计划购电量": plan_sheet_frame(dates, E, price, price_mat)}
    if E_adj is not None:
        sheets["调整购电量"] = plan_sheet_frame(dates, E_adj, price, price_mat)
    sheets["充放电量"] = soc_sheet_frame(dates, c, d, s0, s1)
    sheets["紧急购电量"] = emergency_frame(dates, g if g is not None else np.zeros_like(E))
    order = ["计划购电量"] + (["调整购电量"] if E_adj is not None else []) + \
            ["充放电量", "紧急购电量"]
    with pd.ExcelWriter(path) as w:
        for k in order:
            sheets[k].to_excel(w, sheet_name=k, index=False)
    return path
```

源程序：code/exp_scan.py

```
# -*- coding: utf-8 -*-
"""
补充实验：Q1 容量-功率双向扫描、报童经验覆盖率、单参数灵敏度。
输出 intermediate/exp_scan.json / exp_newsboy.json / exp_sens.json
"""
from __future__ import annotations

import json
import os

import numpy as np
import pandas as pd
from scipy.optimize import linprog

import config as C
import lp_engine as LP
import q1 as Q1
import q2 as Q2
```

```python


def q1_with(cap=None, pmax=None, eta_c=C.ETA_C, eta_d=C.ETA_D,
            s_min=None, s_max=None, s_init=C.S_INIT):
    """在给定储能参数下重解 Q1（用于容量/功率/效率扫描）。"""
    a1 = C.read_attachment1()
    price = a1["price"].to_numpy(float); load = a1["load"].to_numpy(float)
    pv = a1["pv"].to_numpy(float)
    n = C.N_SLOT
    e_max = (pmax * C.DT) if pmax is not None else C.E_MAX
    smin = C.S_MIN if s_min is None else s_min
    smax = C.S_MAX if s_max is None else s_max
    if cap is not None:                          # 保持 10%–90% DOD 同步移动上下界
        smin, smax = 0.10 * cap, 0.90 * cap
    nv = 4 * n
    iE, iC, iD, iS = 0, n, 2 * n, 3 * n
    obj = np.zeros(nv); obj[iE:iE + n] = price
    A_ub, b_ub = [], []
    for t in range(n):
        row = np.zeros(nv); row[iE + t] = -1; row[iC + t] = 1; row[iD + t] = -1
        A_ub.append(row); b_ub.append(-(load[t] - pv[t]) * C.DT)
    A_eq, b_eq = [], []
    for t in range(n):
        row = np.zeros(nv); row[iS + t] = 1
        if t > 0:
            row[iS + t - 1] = -1
        row[iC + t] = -eta_c; row[iD + t] = 1 / eta_d
        A_eq.append(row); b_eq.append(0.0 if t > 0 else s_init)
    bounds = [(0, None)] * n + [(0, e_max)] * n + [(0, e_max)] * n + [(smin, smax)] * n
    bounds[iS + n - 1] = (s_init, s_init)
    res = linprog(obj, A_ub=np.array(A_ub), b_ub=np.array(b_ub), A_eq=np.array(A_eq),
                  b_eq=np.array(b_eq), bounds=bounds, method="highs")
    return float(res.fun), res.x[iE:iE + n], res.x[iC:iC + n], res.x[iD:iD + n]


def capacity_scan():
    base = q1_with()[0]
    caps = [7200, 9600, 12000, 14400, 19200, 28800, 48000]
    out = []
    for cap in caps:
        c, E, cc, dd = q1_with(cap=cap)
        out.append(dict(cap=cap, cost=c, saving=base - c,
                        dcost=(base - c) / base * 100, charge=float(cc.sum()),
                        discharge=float(dd.sum())))
    powers = [1250, 2500, 5000, 7500, 10000]
    out2 = []
    for pm in powers:
        c, E, cc, dd = q1_with(pmax=pm)
        out2.append(dict(pmax=pm, cost=c, saving=base - c,
                         dcost=(base - c) / base * 100, charge=float(cc.sum())))
    # 单一自由度放开
    c_nocap, _, _, _ = q1_with(s_min=C.S_MIN, s_max=10 ** 7)   # 仅解除容量上限
    c_nopow, _, _, _ = q1_with(pmax=10 ** 6)                   # 仅解除功率上限
    return dict(base=base, cap_scan=out, power_scan=out2,
                no_capacity_limit=c_nocap, no_power_limit=c_nopow,
                gain_no_cap=(base - c_nocap) / base * 100,
                gain_no_pow=(base - c_nopow) / base * 100)


def eta_scan():
    out = []
    for ec, ed, name in [(0.9, 0.9, "单边各 0.9（往返 81%）"),
                         (0.9487, 0.9487, "往返 0.9 等效单边"),
                         (0.9, 1.0, "仅充电侧 0.9"),
                         (1.0, 0.9, "仅放电侧 0.9")]:
        c, *_ = q1_with(eta_c=ec, eta_d=ed)
        out.append(dict(eta_c=ec, eta_d=ed, name=name, cost=c,
                        saving=C.baseline_cost(C.read_attachment1()["price"].to_numpy(float),
                                               C.read_attachment1()["load"].to_numpy(float),
                                               C.read_attachment1()["pv"].to_numpy(float)) - c))
    return out


```

```python
def newsboy_check(K=60):
    """报童经验覆盖率：计划口径下逐槽缺货频率与临界分位数理论的对比。"""
    j = json.load(open(os.path.join(C.INTERMEDIATE_DIR, "q2_main.json")))
    G = np.vstack([np.array(r["g"]) for r in j["daily"]])
    E = np.vstack([np.array(r["E"]) for r in j["daily"]])
    c = np.vstack([np.array(r["c"]) for r in j["daily"]])
    d = np.vstack([np.array(r["d"]) for r in j["daily"]])
    panel = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D"))
    load = panel.L().loc[[pd.Timestamp(r["date"]) for r in j["daily"]]].to_numpy(float)
    pv = panel.PV().loc[[pd.Timestamp(r["date"]) for r in j["daily"]]].to_numpy(float)
    nd_act = (load - pv) * C.DT
    short = nd_act + c - d - E
    margin = E - np.maximum(nd_act * 0 + (load - pv) * C.DT, 0) * 0  # 占位
    # 计划边际：E 相对"预测口径净需求 + 充放电"的超出量
    fc = np.zeros_like(nd_act)
    for i, r in enumerate(j["daily"]):
        fc[i] = (load[i] - panel.pv_forecast(pd.Timestamp(r["date"]), 0)) * C.DT
    req = fc + c - d
    m = E - np.maximum(req, 0)
    return dict(short_slot_rate=float((G > 1e-6).mean()),
                short_energy=float(G.sum()),
                margin_total=float(np.maximum(m, 0).sum()),
                margin_per_slot=float(np.maximum(m, 0).mean()),
                short_q99=float(np.percentile(short, 99)),
                short_q80=float(np.percentile(short, 80)),
                short_q833=float(np.percentile(short, 83.33)),
                fill_rate=float(1 - G.sum() / (nd_act.sum())))


def sens_penalty():
    """紧急购电倍数 k 的灵敏度（用 Q1 日 + 单一场景集重算典型日口径）。"""
    out = []
    for k in [2, 3, 5, 8, 10]:
        C.EMERGENCY_MULT = float(k)
        # 在固定残差场景集上求典型日的最优安全边际
        a1 = C.read_attachment1()
        price = a1["price"].to_numpy(float)
        panel = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D"))
        R = Q2.build_residuals(panel, Q2.DATE_WINDOW)
        cen, w = LP.reduce_scenarios_stratified(R, K=60)
        nd_fcst = (a1["load"].to_numpy(float) - a1["pv"].to_numpy(float)) * C.DT
        sol = LP.solve_day(price, nd_fcst, nd_fcst[None, :] + cen, w, s_init=C.S_INIT)
        out.append(dict(k=k, margin=float(np.maximum(
            sol["E"] - np.maximum(nd_fcst + sol["c"] - sol["d"], 0), 0).sum()),
            exp_emg=float((sol["g"] * w[:, None]).sum()),
            obj=float(sol["obj"])))
    C.EMERGENCY_MULT = 5.0
    return out


def main():
    res = {}
    res["capacity"] = capacity_scan()
    res["eta"] = eta_scan()
    res["newsboy"] = newsboy_check()
    res["penalty"] = sens_penalty()
    with open(os.path.join(C.INTERMEDIATE_DIR, "exp_scan.json"), "w") as f:
        json.dump(res, f, ensure_ascii=False, indent=1)
    print(json.dumps(res, ensure_ascii=False, indent=1)[:4000])


if __name__ == "__main__":
    main()
```

源程序：code/exp_sobol.py

```python
# -*- coding: utf-8 -*-
"""
全局灵敏度分析（Sobol 指数，Saltelli/Jansen 估计）——在典型日（问题 1）模型上做。
参数 8 个：往返效率、SOC 上限、SOC 下限、充放功率上限、电价水平、峰谷价差、光伏出力、负载水平。
输出：一阶指数 S_i、总效应指数 S_Ti、以及各参数对"全天购电费"与"相对基线节省率"的影响。

用法：python3 exp_sobol.py [N]    （默认 N=256 → 2560 次 LP 求解）
```

```
"""
from __future__ import annotations

import json
import os
import sys

import numpy as np
from scipy.stats import qmc

import config as C
import exp_scan as ES

PARAMS = [
    ("eta_rt", "往返效率", 0.75, 0.95),
    ("s_max", "储电量上限 kWh", 8400.0, 14400.0),
    ("s_min", "储电量下限 kWh", 600.0, 2400.0),
    ("p_max", "充放功率上限 kW", 2500.0, 7500.0),
    ("price_lv", "电价水平倍数", 0.8, 1.2),
    ("price_pv", "峰谷价差倍数", 0.7, 1.3),
    ("pv_scale", "光伏出力倍数", 0.8, 1.2),
    ("load_scale", "负载水平倍数", 0.9, 1.1),
]


def model(x, ret_series=False):
    """给定参数向量，返回（全天购电费 元，相对无储能基线节省率）。"""
    d = dict(zip([p[0] for p in PARAMS], x))
    a1 = C.read_attachment1()
    price = a1["price"].to_numpy(float).copy()
    load = a1["load"].to_numpy(float).copy() * d["load_scale"]
    pv = a1["pv"].to_numpy(float).copy() * d["pv_scale"]
    # 电价：水平缩放 + 峰谷价差缩放（围绕均值）
    pm = price.mean()
    price = pm * d["price_lv"] + (price - pm) * d["price_pv"]
    eta = float(np.sqrt(d["eta_rt"]))
    smin = float(d["s_min"]); smax = float(d["s_max"])
    if smin >= smax - 100:                  # 保证可行
        smin, smax = 0.1 * smax, smax
    cost, E, c, dd = ES.q1_with(pmax=float(d["p_max"]), eta_c=eta, eta_d=eta,
                                s_min=smin, s_max=smax,
                                s_init=float(np.clip(6000.0, smin, smax)))
    base = C.baseline_cost(price, load, pv)
    save = (base - cost) / base
    return (cost, save) if ret_series else cost


def sobol_indices(N=256, seed=42, target=0):
    d = len(PARAMS)
    rng = np.random.default_rng(seed)
    A = qmc.Sobol(d=d, scramble=True, seed=seed).random(N).T
    B = qmc.Sobol(d=d, scramble=True, seed=seed + 1000).random(N).T
    lo = np.array([p[2] for p in PARAMS]); hi = np.array([p[3] for p in PARAMS])
    scale = lambda U: lo[:, None] + (hi - lo)[:, None] * U
    A, B = scale(A), scale(B)
    fA = np.array([model(A[:, j], ret_series=True)[target] for j in range(N)])
    fB = np.array([model(B[:, j], ret_series=True)[target] for j in range(N)])
    var = np.var(np.concatenate([fA, fB]), ddof=1)
    Si, STi = [], []
    for i in range(d):
        AB = A.copy(); AB[i] = B[i]
        fAB = np.array([model(AB[:, j], ret_series=True)[target] for j in range(N)])
        Si.append(float(np.mean(fB * (fAB - fA)) / var))
        STi.append(float(0.5 * np.mean((fA - fAB) ** 2) / var))
    return dict(N=N, names=[p[1] for p in PARAMS], keys=[p[0] for p in PARAMS],
                bounds=[[p[2], p[3]] for p in PARAMS],
                var=float(var), mean=float(np.mean(np.concatenate([fA, fB]))),
                S1=Si, ST=STi)


def main():
    N = int(sys.argv[1]) if len(sys.argv) > 1 else 256
    out = dict(cost=sobol_indices(N=N, target=0),
```

```
                saving=sobol_indices(N=N, target=1, seed=43))
    with open(os.path.join(C.INTERMEDIATE_DIR, "exp_sobol.json"), "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    for key, tag in (("cost", "全天购电费（元）"), ("saving", "相对基线节省率")):
        r = out[key]
        print(f"--- {tag}  N={r['N']}  均值={r['mean']:.4f}  方差={r['var']:.4g} ---")
        print(f"{'参数':<16}{'S1':>10}{'ST':>10}")
        for n, s, st in zip(r["names"], r["S1"], r["ST"]):
            print(f"{n:<16}{s:>10.4f}{st:>10.4f}")
        print("sum(S1) = %.4f" % sum(r["S1"]))


if __name__ == "__main__":
    main()
```

源程序：code/exp_na.py

```
# -*- coding: utf-8 -*-
"""
非预期性（non-anticipativity）置换检验
=====================================
做法：把某一次预报（6:00 / 12:00 / 18:00）在日与日之间**随机置换**后重新求解，
      检查"仅由该次预报控制的时段"是否变化，以及"早于该预报时刻的时段"是否**完全不变**。
- 若置换 6:00 预报后，槽 36-71 的决策发生变化、而槽 0-35 完全不变 → 该次预报只影响它之后的时段，
  说明实现中没有"用未来信息决策"的泄漏（非预期性成立）。
- 同时给出对照：置换 0:00 预报应使槽 0-35 也变化（证明检验本身有效力）。

输出 intermediate/exp_na.json
"""
from __future__ import annotations

import json
import os

import numpy as np
import pandas as pd

import config as C
import lp_engine as LP
import q3 as Q3

DAYS = pd.date_range("2025-02-01", "2025-02-12", freq="D")
EPOCHS = (0, 6, 12, 18)
CONTROL = {0: (0, 36), 6: (36, 72), 12: (72, 108), 18: (108, 144)}


def run_days(permuted_hour=None, seed=0, K=40):
    """运行 12 天；permuted_hour 不为 None 时，把该时刻的预报按日随机置换。"""
    panel = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D"))
    load = panel.L(); pv_act = panel.PV()
    price = C.read_attachment1()["price"].to_numpy(float)
    scens = {}
    for h in EPOCHS:
        s_m = C.EPOCH_SLOT[h]
        slots = np.arange(s_m, C.N_SLOT)
        cen, w = Q3.build_epoch_scenarios(panel, h, K=K, slots=slots)
        scens[h] = dict(slots=slots, cen=cen, w=w)
    rng = np.random.default_rng(seed)
    perm = rng.permutation(len(DAYS)) if permuted_hour is not None else None
    res = {}
    for i, d in enumerate(DAYS):
        L = load.loc[d].to_numpy(float)
        E_plan = np.zeros(C.N_SLOT); E_adj = np.zeros(C.N_SLOT)
        soc = C.S_INIT
        for h in EPOCHS:
            s_m = C.EPOCH_SLOT[h]; slots = scens[h]["slots"]
            src = d if (permuted_hour != h) else DAYS[perm[i]]
            fc = Q3.forecast_slots(panel, src, h, slots)
            nd_fc = np.zeros(C.N_SLOT); nd_fc[slots] = (L[slots] - fc) * C.DT
            cen = scens[h]["cen"]
            nd_sc = np.zeros((cen.shape[0], C.N_SLOT))
            nd_sc[:, slots] = nd_fc[slots][None, :] + cen
            sol = LP.solve_epoch(price, nd_fc, nd_sc, scens[h]["w"], s_init=soc,
```

```
                                s_from=s_m, plan=(E_plan if h > 0 else None),
                                term_floor=C.S_INIT)
            if h == 0:
                E_plan = sol["E"].copy()
            later = [x for x in EPOCHS if x > h]
            nxt = C.EPOCH_SLOT[later[0]] if later else C.N_SLOT
            for j, t in enumerate(range(s_m, nxt)):
                E_adj[t] = sol["E"][j]
            soc = float(sol["S"][nxt - s_m - 1])
        res[d] = E_adj.copy()
    return res


def main():
    base = run_days(None)
    out = {"baseline_days": len(base)}
    for h in EPOCHS:
        alt = run_days(permuted_hour=h, seed=1)
        diffs = {}
        for m, (a, b) in CONTROL.items():
            dmax = max(float(np.max(np.abs(alt[d][a:b] - base[d][a:b]))) for d in DAYS)
            diffs[f"槽{a}-{b}"] = dmax
        out[f"置换{h:02d}:00预报"] = diffs
    with open(os.path.join(C.INTERMEDIATE_DIR, "exp_na.json"), "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    print(json.dumps(out, ensure_ascii=False, indent=1))
    print()
    print("判读：置换某时刻预报后，该时刻**之前**控制区的最大变化应为 0，")
    print("      而该时刻**之后**控制区的变化应显著大于 0（否则检验没有效力）。")


if __name__ == "__main__":
    main()
```

源程序：code/exp_oos.py

```
# -*- coding: utf-8 -*-
"""
样本外回测：用 2025-02-01—06-30 的预报残差训练场景集，在 07-01—12-31 上回测
============================================================================
目的：检验"按日总残差分层 + 层内 medoid"的场景集是否只是拟合了样本内残差。
做法：场景集只由训练窗残差构造；测试窗每一天仍按其 0:00 预报制定计划，
      用当日实际值结算；与"用测试窗自身残差构造场景集"（样本内上界）比较。

输出 intermediate/exp_oos.json
"""
from __future__ import annotations

import json
import os

import numpy as np
import pandas as pd

import config as C
import lp_engine as LP
import q2 as Q2

TRAIN = pd.date_range("2025-02-01", "2025-06-30", freq="D")
TEST = pd.date_range("2025-07-01", "2025-12-31", freq="D")
ALL = pd.date_range(C.WARMUP_START, C.WIN_END, freq="D")


def run(scen_days, test_days, K=60):
    panel = C.Panel(ALL)
    load = panel.L(); pv = panel.PV()
    price = C.read_attachment1()["price"].to_numpy(float)
    R = np.zeros((len(scen_days), C.N_SLOT))
    for i, d in enumerate(scen_days):
        R[i] = (panel.pv_forecast(d, 0) - pv.loc[d].to_numpy(float)) * C.DT
    cen, w = LP.reduce_scenarios_stratified(R, K=K)
    tot_plan = tot_emg = tot_emg_qty = 0.0
    base = 0.0
```

```
    soc = C.S_INIT
    for d in test_days:
        Ld = load.loc[d].to_numpy(float); Pv = pv.loc[d].to_numpy(float)
        nd_act = (Ld - Pv) * C.DT
        fc = panel.pv_forecast(d, 0)
        nd_fc = (Ld - fc) * C.DT
        sol = LP.solve_day(price, nd_fc, nd_fc[None, :] + cen, w, s_init=soc,
                           term_mode="ge")
        g, pc, ec = LP.realized_slot(sol, nd_act)
        tot_plan += pc; tot_emg += ec; tot_emg_qty += float(g.sum())
        base += C.baseline_cost(price, Ld, Pv)
        soc = float(sol["S"][-1])
    return dict(n_days=len(test_days), plan=tot_plan, emg=tot_emg,
                total=tot_plan + tot_emg, emg_qty=tot_emg_qty, base=base,
                saving=(base - tot_plan - tot_emg) / base)


def main():
    out = dict(train=[str(d.date()) for d in TRAIN], test=[str(d.date()) for d in TEST])
    out["oos_train_scenarios"] = run(TRAIN, TEST)
    out["insample_test_scenarios"] = run(TEST, TEST)
    # 参考：全窗口主结果中测试窗部分的实际值
    j = json.load(open(os.path.join(C.INTERMEDIATE_DIR, "q2_main.json")))
    sel = [r for r in j["daily"] if pd.Timestamp(r["date"]) in set(TEST)]
    out["main_full_scenarios_test_window"] = dict(
        n_days=len(sel), plan=sum(r["plan_cost"] for r in sel),
        emg=sum(r["emg_cost"] for r in sel),
        total=sum(r["total_cost"] for r in sel),
        emg_qty=sum(r["emg_qty"] for r in sel),
        base=sum(r["base_cost"] for r in sel),
        saving=(sum(r["base_cost"] for r in sel) - sum(r["total_cost"] for r in sel))
        / sum(r["base_cost"] for r in sel))
    with open(os.path.join(C.INTERMEDIATE_DIR, "exp_oos.json"), "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    print(json.dumps(out, ensure_ascii=False, indent=1))
    a, b, c = out["oos_train_scenarios"], out["insample_test_scenarios"], out["
     main_full_scenarios_test_window"]
    print()
    print("测试窗 %d 天：样本外场景集总费用 %.2f 元；样本内场景集 %.2f 元；全窗口主结果同窗 %.2f 元"
          % (a["n_days"], a["total"], b["total"], c["total"]))
    print("样本外相对样本内劣化 %.2f 元（%.3f%%）；相对主结果 %.2f 元（%.3f%%）"
          % (a["total"] - b["total"], (a["total"] - b["total"]) / b["total"] * 100,
             a["total"] - c["total"], (a["total"] - c["total"]) / c["total"] * 100))


if __name__ == "__main__":
    main()
```

源程序：code/exp_benchmark.py

```
# -*- coding: utf-8 -*-
"""
基准实验：全年单一 LP（完美信息、单一储电量轨迹、无逐日决策解耦）
================================================================
作为"信息与协调"的上界基准：给出 2025-02-01 至 12-31 共 334 天、每日 144 槽
（合计 48 096 槽）的单一线性规划最优值，用以量化"逐日决策解耦 + 每日终端不亏空"
这两项建模选择的代价。

输出 intermediate/exp_benchmark.json
"""
from __future__ import annotations

import json
import os
import time

import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import coo_matrix

import config as C

```

```
24
25  def full_year_lp():
26      panel = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D"))
27      load = panel.L(); pv = panel.PV()
28      price = C.read_attachment1()["price"].to_numpy(float)
29      win = pd.date_range(C.WIN_START, C.WIN_END, freq="D")
30      n = C.N_SLOT; T = len(win); nv = 4 * T * n
31      iE, iC, iD, iS = 0, T * n, 2 * T * n, 3 * T * n
32      obj = np.zeros(nv); obj[iE:iE + T * n] = np.tile(price, T)
33      rows, cols, vals, rhs = [], [], [], []
34      r = 0
35      for k in range(T):
36          L = load.loc[win[k]].to_numpy(float); P = pv.loc[win[k]].to_numpy(float)
37          for t in range(n):
38              rows += [r, r, r]; cols += [iE + k * n + t, iC + k * n + t, iD + k * n + t]
39              vals += [-1.0, 1.0, -1.0]
40              rhs.append(-(L[t] - P[t]) * C.DT); r += 1
41      A_ub = coo_matrix((vals, (rows, cols)), shape=(r, nv)).tocsr()
42      rows2, cols2, vals2, rhs2 = [], [], [], []
43      for k in range(T):
44          for t in range(n):
45              g = k * n + t
46              rows2.append(g); cols2.append(iS + g); vals2.append(1.0)
47              if g > 0:
48                  rows2.append(g); cols2.append(iS + g - 1); vals2.append(-1.0)
49              rows2.append(g); cols2.append(iC + g); vals2.append(-C.ETA_C)
50              rows2.append(g); cols2.append(iD + g); vals2.append(1.0 / C.ETA_D)
51              rhs2.append(0.0 if g > 0 else C.S_INIT)
52      A_eq = coo_matrix((vals2, (rows2, cols2)), shape=(T * n, nv)).tocsr()
53      bounds = [(0, None)] * (T * n) + [(0, C.E_MAX)] * (T * n) + [(0, C.E_MAX)] * (T * n) + \
54               [(C.S_MIN, C.S_MAX)] * (T * n)
55      t0 = time.time()
56      res = linprog(obj, A_ub=A_ub, b_ub=np.array(rhs), A_eq=A_eq, b_eq=np.array(rhs2),
57                    bounds=bounds, method="highs")
58      dt = time.time() - t0
59      return dict(cost=float(res.fun), n_var=int(nv), n_slot=int(T * n),
60                  seconds=float(dt), success=bool(res.success),
61                  base_cost=sum(C.baseline_cost(price, load.loc[d].to_numpy(float),
62                                                pv.loc[d].to_numpy(float)) for d in win))
63
64
65  def main():
66      out = dict(full_year=full_year_lp())
67      out["decoupling_gap_vs_R1"] = None
68      q2 = os.path.join(C.INTERMEDIATE_DIR, "q2_cmp_R1.json")
69      if os.path.exists(q2):
70          r1 = json.load(open(q2))["meta"]
71          out["daily_decoupled_R1"] = r1
72          out["decoupling_gap"] = r1["tot_cost"] - out["full_year"]["cost"]
73          out["decoupling_gap_pct"] = (r1["tot_cost"] - out["full_year"]["cost"]) / r1["tot_cost"]
74      with open(os.path.join(C.INTERMEDIATE_DIR, "exp_benchmark.json"), "w") as f:
75          json.dump(out, f, ensure_ascii=False, indent=1)
76      print(json.dumps(out, ensure_ascii=False, indent=1))
77      print()
78      print("全年单一 LP: %.6f 元（%.2f 万元），变量 %d，耗时 %.1f s"
79            % (out["full_year"]["cost"], out["full_year"]["cost"] / 1e4,
80               out["full_year"]["n_var"], out["full_year"]["seconds"]))
81      if out.get("decoupling_gap") is not None:
82          print("逐日解耦（完美信息 R1）比全年单一 LP 贵 %.2f 元（%.2f%%）"
83                % (out["decoupling_gap"], out["decoupling_gap_pct"] * 100))
84
85
86  if __name__ == "__main__":
87      main()
```

源程序：code/audit.py

```
1  # -*- coding: utf-8 -*-
2  """
3  独立审计（A1 – A10）：从**结果文件**与**原始附件**出发独立复算，不调用建模目标函数。
4  ================================================================================
5  A1  电量平衡残差（结果文件的 4 h 块级 + 中间结果的 10 min 级）
6  A2  SOC 递推残差（用 eta_c=eta_d=0.9 独立重算）
```

```
A3  SOC 越界数（[1200,10800]）
A4  充放电功率越界数（<= 5000 kW 等价 <= 833.3333 kWh/槽）
A5  充放互斥违反数
A6  周期/终端条件（result1: S0=S144；其余: S144 >= S0）
A7  非负性
A8  费用复算相对误差（不调用建模费用函数）
A9  日期与结构（行数/列数/工作表名/标签）
A10 全天汇总列（全天购电量、全天购电费）

输出: intermediate/audit_report.md 与 intermediate/audit.json
"""
from __future__ import annotations

import json
import os

import numpy as np
import pandas as pd

import config as C

TOL_RES = 1e-6          # kWh
TOL_REL = 1e-4          # 费用复算相对误差
RES_FILES = [("result1.xlsx", "q1"), ("result2.xlsx", "q2"),
             ("result3.xlsx", "q3"), ("result4-2.xlsx", "q4-2"),
             ("result4-3.xlsx", "q4-3")]


def _load(fname):
    p = os.path.join(C.RESULT_DIR, fname)
    xl = pd.ExcelFile(p)
    return {sh: xl.parse(sh) for sh in xl.sheet_names}


def audit_result_files():
    rows, out = [], {}
    a1 = C.read_attachment1()
    price_fixed = a1["price"].to_numpy(float)
    price_flat = C.read_attachment4()
    panel = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D"))

    for fname, tag in RES_FILES:
        p = os.path.join(C.RESULT_DIR, fname)
        if not os.path.exists(p):
            rows.append(dict(file=fname, item="存在性", value="缺失", verdict="FAIL"))
            continue
        sh = _load(fname)
        rep = {}
        plan = sh["计划购电量"]
        if tag == "q1":
            # A9 结构
            rep["A9_计划行数"] = (len(plan), 144)
            rep["A9_工作表"] = (list(sh.keys()), ["计划购电量", "充放电量"])
            E = plan.iloc[:, 1].to_numpy(float)
            rep["A7_负购电量数"] = int((E < -TOL_RES).sum())
            soc = sh["充放电量"]
            rep["A9_充放电行数"] = (len(soc), 6)
            c_blk = soc["充电量"].to_numpy(float)[:6]
            d_blk = soc["放电量"].to_numpy(float)[:6]
            sv = soc["储电量"].to_numpy(float)
            s0 = float(sv[0]); s24 = float(sv[1])
            rep["A6_日周期偏差"] = abs(s0 - s24)
            rep["A2_SOC净递推残差"] = abs((s0 + C.ETA_C * c_blk.sum()
                                           - d_blk.sum() / C.ETA_D) - s24)
            rep["A8_费用复算相对误差"] = abs(E @ price_fixed
                                             - float(np.sum(price_fixed * E))) / max(1e-9, float(np.sum(
    price_fixed * E)))
            rep["A1_全天购电量"] = float(E.sum())
            rep["A1_购电费"] = float(E @ price_fixed)
            rep["A1_基线C01"] = C.baseline_cost(price_fixed, a1["load"].to_numpy(float),
                                                a1["pv"].to_numpy(float))
        else:
            dates = pd.to_datetime(plan.iloc[:, 0])
```

```
            E = plan.iloc[:, 1:145].to_numpy(float)
            tot = plan.iloc[:, 145].to_numpy(float)
            fee = plan.iloc[:, 146].to_numpy(float)
            price_mat = (price_flat.loc[dates].to_numpy(float) if tag.startswith("q4")
                         else np.tile(price_fixed, (len(dates), 1)))
            rep["A9_天数"] = (len(dates), 334)
            rep["A9_列数"] = (plan.shape[1], 147)
            rep["A9_日期范围"] = (str(dates.min().date()), str(dates.max().date()),
                                  "2025-02-01", "2025-12-31")
            rep["A10_全天购电量最大偏差"] = float(np.max(np.abs(E.sum(1) - tot)))
            rep["A10_全天购电费最大相对偏差"] = float(np.max(
                np.abs(fee - (price_mat * E).sum(1)) / np.maximum(fee, 1)))
            rep["A7_负购电量数"] = int((E < -TOL_RES).sum())
            rep["A9_工作表"] = (list(sh.keys()), sorted(sh.keys()))
            # SOC 块级独立递推
            soc = sh["充放电量"]
            c_blk = soc["充电量"].to_numpy(float).reshape(-1, 6)
            d_blk = soc["放电量"].to_numpy(float).reshape(-1, 6)
            st = soc["储电量"].to_numpy(float)
            s0 = st[0::6]; s24 = st[1::6]
            lhs = s0 + C.ETA_C * c_blk.sum(1) - d_blk.sum(1) / C.ETA_D
            rep["A2_SOC净递推最大残差"] = float(np.max(np.abs(lhs - s24)))
            rep["A6_末值低于初值天数"] = int((s24 - s0 < -1e-6).sum())
            rep["A3_SOC越界数"] = int(((s0 < C.S_MIN - 1e-6) | (s24 > C.S_MAX + 1e-6)).sum())
            rep["A4_块级功率越界数"] = int((c_blk > 24 * C.E_MAX + 1e-6).sum()
                                          + (d_blk > 24 * C.E_MAX + 1e-6).sum())
            rep["A1_全窗购电量"] = float(E.sum())
            rep["A1_全窗购电费"] = float((price_mat * E).sum())
            # 紧急购电表结构
            em = sh["紧急购电量"]
            qty = pd.to_numeric(em["购电量"], errors="coerce").fillna(0).to_numpy(float)
            rep["A9_紧急购电行数"] = len(em)
            rep["A1_紧急购电量合计"] = float(qty.sum())
        out[tag] = rep
        for k, v in rep.items():
            rows.append(dict(file=fname, item=k, value=str(v), verdict=""))
    return rows, out


def audit_internal():
    """内部解（10 min 级）的可行性审计。"""
    out = {}
    q1 = json.load(open(os.path.join(C.INTERMEDIATE_DIR, "q1.json")))
    a = q1["arrays"]
    E = np.array(a["E"]); c = np.array(a["c"]); d = np.array(a["d"]); S = np.array(a["S"])
    price = np.array(a["price"]); load = np.array(a["load"]); pv = np.array(a["pv"])
    out["q1"] = dict(
        A1_平衡残差=float(abs(E + pv * C.DT + d - load * C.DT - c).max()),
        A2_SOC递推残差=float(abs(S - (np.concatenate([[C.S_INIT], S[:-1]])
                                      + C.ETA_C * c - d / C.ETA_D)).max()),
        A3_SOC越界=int(((S < C.S_MIN - 1e-9) | (S > C.S_MAX + 1e-9)).sum()),
        A4_功率越界=int(((c > C.E_MAX + 1e-9) | (d > C.E_MAX + 1e-9)).sum()),
        A5_互斥违反=int(((c > 1e-6) & (d > 1e-6)).sum()),
        A6_周期偏差=float(abs(a["S"][-1] - 6000.0)),
        A7_负购电=int((E < -1e-9).sum()),
        A8_费用相对误差=float(abs(float(np.sum(price * E)) - q1["main"]["cost"])
                              / q1["main"]["cost"]),
        A10_全天购电量=float(E.sum()))
    for tag, f in (("q2", "q2_main.json"), ("q3", "q3_n4.json")):
        p = os.path.join(C.INTERMEDIATE_DIR, f)
        if not os.path.exists(p):
            continue
        j = json.load(open(p))
        E = np.vstack([np.array(r["E"] if tag == "q2" else r["E_adj"]) for r in j["daily"]])
        c = np.vstack([np.array(r["c"]) for r in j["daily"]])
        d = np.vstack([np.array(r["d"]) for r in j["daily"]])
        g = np.vstack([np.array(r["g"]) for r in j["daily"]])
        s0 = np.array([r["s_init"] for r in j["daily"]])
        s1 = np.array([r.get("s_end", r.get("S_end")) for r in j["daily"]])
        load_m = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D")).L().loc[[
            pd.Timestamp(r["date"]) for r in j["daily"]]].to_numpy(float)
        pv_m = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D")).PV().loc[[
            pd.Timestamp(r["date"]) for r in j["daily"]]].to_numpy(float)
```

```
        nd = (load_m - pv_m) * C.DT
        # 供给 >= 需求；正残差 = 盈余处置量（弃光或超购处置），负残差才是不可行
        bal = E + pv_m * C.DT + d + g - load_m * C.DT - c
        bal_no_g = bal - g
        out[tag] = dict(
            A1_供给缺口=float(-min(bal.min(), 0.0)),
            A1_盈余处置量=float(np.maximum(bal, 0).sum()),
            A1_漏g时最大缺口=float(-bal_no_g.min()),
            A2_SOC净递推残差=float(np.max(np.abs(s1 - (s0 + C.ETA_C * c.sum(1)
                                                  - d.sum(1) / C.ETA_D)))),
            A4_功率越界=int(((c > C.E_MAX + 1e-6) | (d > C.E_MAX + 1e-6)).sum()),
            A5_互斥违反=int(((c > 1e-6) & (d > 1e-6)).sum()),
            A6_末值低于初值天数=int((s1 - s0 < -1e-6).sum()),
            A7_负购电=int((E < -1e-9).sum()),
            A8_费用相对误差=float(abs(sum(r["total_cost"] for r in j["daily"])
                                      - j["meta"].get("tot_cost", j["meta"].get("total_cost", np.nan)))
                                  / max(1e-9, j["meta"].get("tot_cost", j["meta"].get("total_cost", 1)))),
            A9_天数=len(j["daily"]))
    return out


def main():
    rows, out = audit_result_files()
    inner = audit_internal()
    os.makedirs(C.INTERMEDIATE_DIR, exist_ok=True)
    with open(os.path.join(C.INTERMEDIATE_DIR, "audit.json"), "w") as f:
        json.dump(dict(deliverable=out, internal=inner), f, ensure_ascii=False, indent=1)
    lines = ["# 独立审计报告（A1–A10）", "",
             "> 由独立脚本 audit.py 从结果文件与原始附件复算、不调用建模目标函数。",
             f"> 阈值：残差 ≤ {TOL_RES} kWh；费用复算相对误差 ≤ {TOL_REL}；越界数 = 0。", ""]
    lines.append("## 一、交付物审计（结果文件）")
    lines.append("")
    lines.append("| 文件 | 审计项 | 数值 |")
    lines.append("|---|---|---|")
    for r in rows:
        lines.append(f"| {r['file']} | {r['item']} | {r['value']} |")
    lines.append("")
    lines.append("## 二、内部解审计（10 min 级）")
    lines.append("")
    lines.append("| 问题 | 审计项 | 数值 |")
    lines.append("|---|---|---|")
    for tag, rep in inner.items():
        for k, v in rep.items():
            lines.append(f"| {tag} | {k} | {v:.3e} |" if isinstance(v, float)
                         else f"| {tag} | {k} | {v} |")
    txt = "\n".join(lines)
    with open(os.path.join(C.INTERMEDIATE_DIR, "audit_report.md"), "w") as f:
        f.write(txt)
    print(txt[:4000])
    return out


if __name__ == "__main__":
    main()
```

源程序：code/paper_numbers.py

```
# -*- coding: utf-8 -*-
"""汇总全部论文数值 -> intermediate/paper_numbers.json 与 notes/数值台账.md（唯一数字源）。"""
from __future__ import annotations

import json
import os

import numpy as np
import pandas as pd

import config as C

INT = C.INTERMEDIATE_DIR
NOTES = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "notes")


```

```
def load(name):
    p = os.path.join(INT, name)
    return json.load(open(p)) if os.path.exists(p) else None


def f(x, n=2):
    return f"{x:,.{n}f}"


def wan(x):
    return f"{x/1e4:,.2f}"


def main():
    q1 = load("q1.json")
    q2 = load("q2_main.json")
    q2k120 = load("q2_K120.json")
    q2r1 = load("q2_cmp_R1.json")
    q2r2 = load("q2_cmp_R2.json")
    q3 = {t: load(f"q3_{t}.json") for t in ("n1", "n2", "n4", "perf")}
    q4p = load("q4_price.json")
    q42 = load("q4_2.json")
    q43 = load("q4_3.json")
    scan = load("exp_scan.json")
    audit = load("audit.json")

    a1 = C.read_attachment1()
    panel = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D"))
    load_m = panel.L(); pv_m = panel.PV(); p4 = panel.P(volatile=True)
    win = pd.date_range(C.WIN_START, C.WIN_END, freq="D")

    N = {}
    N["数据"] = dict(
        负载均值_kW=float(load_m.to_numpy().mean()), 负载最小_kW=float(load_m.to_numpy().min()),
        负载最大_kW=float(load_m.to_numpy().max()),
        负载日均电量_kWh=float(load_m.loc[win].to_numpy().sum(1).mean() * C.DT),
        光伏均值_kW=float(pv_m.to_numpy().mean()), 光伏峰值_kW=float(pv_m.to_numpy().max()),
        光伏日均电量_kWh=float(pv_m.loc[win].to_numpy().sum(1).mean() * C.DT),
        固定电价均值=float(a1["price"].mean()), 固定电价最小=float(a1["price"].min()),
        固定电价最大=float(a1["price"].max()),
        波动电价均值=float(p4.to_numpy().mean()), 波动电价最小=float(p4.to_numpy().min()),
        波动电价最大=float(p4.to_numpy().max()),
        附件1为逐时段均值最大偏差=float(np.abs(a1["price"].to_numpy()
                                        - p4.to_numpy().mean(0)).max()),
        附件1负载为逐时段均值最大偏差=float(np.abs(a1["load"].to_numpy()
                                         - load_m.to_numpy().mean(0)).max()),
        附件1光伏为逐时段均值最大偏差=float(np.abs(a1["pv"].to_numpy()
                                         - pv_m.to_numpy().mean(0)).max()),
    )
    # 预报误差
    FC = np.vstack([panel.pv_forecast(d, 0) for d in panel.PV().index])
    err = FC - pv_m.to_numpy()
    hrs = np.array([t // 6 for t in range(C.N_SLOT)])
    day = hrs >= 5
    N["预报误差"] = dict(
        MAE_全时段_kW=float(np.abs(err).mean()), RMSE_全时段_kW=float(np.sqrt((err ** 2).mean())),
        bias_全时段_kW=float(err.mean()),
        MAE_白天_kW=float(np.abs(err[:, day]).mean()),
        RMSE_白天_kW=float(np.sqrt((err[:, day] ** 2).mean())),
        bias_白天_kW=float(err[:, day].mean()),
        逐整点bias_kW=[float(x) for x in err.reshape(len(err), 24, 6).mean(axis=(0, 2))],
    )
    if q1:
        m = q1["main"]
        N["Q1"] = dict(
            无储能基线元=m["base_cost"], 最优购电费元=m["cost"], 节省元=m["saving"],
            节省率=m["saving_rate"], 全天购电量kWh=m["total_energy"],
            充电量kWh=m["charge_total"], 放电量kWh=m["discharge_total"],
            充放比=m["ratio_cd"], 弃光kWh=m["curtail_total"],
            SOC最小=m["min_soc"], SOC最大=m["max_soc"], S0=m["s0"], S144=m["s_end"],
            自由环状费用元=q1["free_cycle"]["cost"], 自由环状S=q1["free_cycle"]["s0"],
            自由环状差元=q1["free_cycle"]["cost"] - m["cost"],
            表1=q1["table1"], 表2=q1["table2"],
```

```
            残差=dict(R1=m["R1_min_surplus"], R2=m["R2_max_plan_violation"],
                      R3=m["R3_max_soc_residual"], R5=m["R5_simultaneous_slots"],
                      R6=m["R6_cd_ratio_err"]),
        )
    if scan:
        N["容量功率扫描"] = scan["capacity"]
        N["效率口径"] = scan["eta"]
        N["报童经验"] = scan["newsboy"]
        N["惩罚倍数灵敏度"] = scan["penalty"]
    if q2:
        N["Q2"] = dict(meta=q2["meta"])
        d0 = q2["daily"]
        N["Q2"]["四个指定日期"] = [r for r in d0 if r["date"][:10] in C.KEY_DATES]
    if q2k120:
        N["Q2_K120"] = q2k120["meta"]
    N["Q2_R1完美信息"] = q2r1["meta"] if q2r1 else None
    N["Q2_R2典型日预报"] = q2r2["meta"] if q2r2 else None
    N["Q3"] = {t: (v["meta"] if v else None) for t, v in q3.items()}
    if q3["n4"]:
        N["Q3"]["四个指定日期"] = [r for r in q3["n4"]["daily"]
                                   if r["date"][:10] in C.KEY_DATES]
    if q4p:
        N["电价检验"] = q4p["price_tests"]
        N["情景保真度_历史代表日"] = q4p["fidelity_hist"]
        N["情景保真度_分块Bootstrap"] = q4p["fidelity_boot"]
    N["Q4_2"] = q42["meta"] if q42 else None
    N["Q4_3"] = q43["meta"] if q43 else None
    if q42:
        N["Q4_2"]["四个指定日期"] = [r for r in q42["daily"] if r["date"][:10] in C.KEY_DATES]
    if q43:
        N["Q4_3"]["四个指定日期"] = [r for r in q43["daily"] if r["date"][:10] in C.KEY_DATES]
    dual2 = load("q1_dual2.json")
    N["Q1对偶验证"] = dual2
    N["Q2无安全边际消融"] = load("q2_noMargin.json")["meta"] if load("q2_noMargin.json") else None
    N["Q2安全边际分解"] = load("q2_margin.json")
    N["非预期性置换检验"] = load("exp_na.json")
    N["Sobol"] = load("exp_sobol.json")
    N["基准实验"] = load("exp_benchmark.json")
    N["窗口聚合"] = load("window_aggregates.json")
    N["样本外回测"] = load("exp_oos.json")
    N["终端条件对照"] = {t: (load(f"q2_term_{t}.json") or {}).get("meta")
                    for t in ("ge", "eq", "free")}
    N["审计"] = audit
    cvar = load("q4_cvar.json")
    N["CVaR前沿"] = cvar["frontier"] if cvar else None
    # ---- v2 新增实验（相依结构/生成器对照/风险画像/机会约束/盒式鲁棒/L 形分解）----
    v2cop = load("exp_copula.json")
    v2chance = load("exp_chance.json")
    v2dro = load("exp_dro.json")
    v2dec = load("decomp.json")
    N["v2相依结构与生成器对照"] = v2cop
    N["v2机会约束"] = v2chance
    N["v2盒式鲁棒"] = v2dro
    N["v2L形分解"] = v2dec

    with open(os.path.join(INT, "paper_numbers.json"), "w") as fh:
        json.dump(N, fh, ensure_ascii=False, indent=1, default=str)

    # ---- Markdown 台账（供写作引用，单位与口径写全）----
    L = ["# 论文数值台账（唯一数字源）", "",
         "> 由 code/paper_numbers.py 从各求解输出汇总；论文中每个数字都应能在本表找到。",
         "> 口径：电量 kWh，功率 kW，费用 元；窗口 2025-02-01 至 2025-12-31（334 天）；",
         "> 固定价=附件 1 电价，波动价=附件 4；储能 12000 kWh / 5000 kW / SOC 1200–10800 / 效率单边 0.9
    。", ""]
    L.append("## 一、数据与预报")
    L.append("")
    L.append("| 量 | 数值 |")
    L.append("|---|---|")
    for k, v in N["数据"].items():
        L.append(f"| {k} | {v:,.4f} |" if isinstance(v, float) else f"| {k} | {v} |")
    for k, v in N["预报误差"].items():
        L.append(f"| {k} | {np.round(v,2)} |")
    L.append("")
```

```
    if q1:
        m = q1["main"]
        L.append("## 二、问题 1（典型日）")
        L.append("")
        L.append("| 量 | 数值 |")
        L.append("|---|---|")
        for k, v in N["Q1"].items():
            if isinstance(v, float):
                L.append(f"| {k} | {v:,.4f} |")
            elif k == "残差":
                L.append(f"| 残差 | {v} |")
            elif k in ("表1", "表2"):
                L.append(f"| {k} | {v} |")
        L.append("")
        L.append("表 1 指定时段购电量（kWh）：" + "；".join(
            f"{a}={b:,.4f}" for a, b in q1["table1"]))
        L.append("")
        rows, s0, s24 = q1["table2"]
        L.append("表 2 充放电量（kWh）：" + "；".join(
            f"{a} 充{b:,.4f}/放{c:,.4f}" for a, b, c in rows) + f"；0:00={s0:,.4f}，24:00={s24:,.4f}")
        L.append("")
    L.append("## 三、容量/功率/效率与报童")
    L.append("")
    if scan:
        L.append("容量扫描（保持 10%–90% DOD）：")
        L.append("")
        L.append("| 容量 kWh | 最优费用 元 | 相对基线节省 元 | 节省率 % |")
        L.append("|---|---|---|---|")
        for r in scan["capacity"]["cap_scan"]:
            L.append(f"| {r['cap']:,} | {r['cost']:,.2f} | {r['saving']:,.2f} | {r['dcost']:.2f} |")
        L.append("")
        L.append(f"仅解除容量上限：{scan['capacity']['no_capacity_limit']:,.2f} 元（多省 "
                 f"{scan['capacity']['gain_no_cap']:.1f}%）；仅解除功率上限："
                 f"{scan['capacity']['no_power_limit']:,.2f} 元（多省 {scan['capacity']['gain_no_pow
']:.1f}%）")
        L.append("")
        L.append("| 充放功率上限 kW | 最优费用 元 | 节省率 % |")
        L.append("|---|---|---|")
        for r in scan["capacity"]["power_scan"]:
            L.append(f"| {r['pmax']:,} | {r['cost']:,.2f} | {r['dcost']:.2f} |")
        L.append("")
        L.append("效率口径对照：")
        L.append("")
        L.append("| 口径 | 最优费用 元 | 节省 元 |")
        L.append("|---|---|---|")
        for r in scan["eta"]:
            L.append(f"| {r['name']} | {r['cost']:,.2f} | {r['saving']:,.2f} |")
        L.append("")
        nb = scan["newsboy"]
        L.append(f"报童经验覆盖率：逐槽缺货频率 {nb['short_slot_rate']*100:.2f}%，"
                 f"缺口电量 {nb['short_energy']:,.2f} kWh，"
                 f"电量满足率 {nb['fill_rate']*100:.2f}%，"
                 f"安全边际合计 {nb['margin_total']:,.2f} kWh（{nb['margin_per_slot']:.2f} kWh/槽）")
        L.append("")
        L.append("紧急购电倍数灵敏度（典型日）：" + "；".join(
            f"k={r['k']} 边际 {r['margin']:.0f} kWh/期望紧急 {r['exp_emg']:.0f} kWh"
            for r in scan["penalty"]))
        L.append("")
    L.append("## 四、四问主结果")
    L.append("")
    L.append("| 模型 | 总费用 元 | 万元 | 相对基线节省率 | 紧急购电量 kWh |")
    L.append("|---|---|---|---|---|")
    if q2:
        mm = q2["meta"]
        L.append(f"| 问题 2（固定价、两阶段） | {mm['tot_cost']:,.2f} | {wan(mm['tot_cost'])} | "
                 f"{(mm['tot_base']-mm['tot_cost'])/mm['tot_base']*100:.2f}% | {mm['tot_emg_qty']:,.2f}
 |")
    if q3["n4"]:
        mm = q3["n4"]["meta"]
        L.append(f"| 问题 3（固定价、四次滚动） | {mm['total_cost']:,.2f} | {wan(mm['total_cost'])} | "
                 f"{(mm['base_cost']-mm['total_cost'])/mm['base_cost']*100:.2f}% | {mm['emg_qty']:,.2f}
 |")
    if q42:
```

```
        mm = q42["meta"]
        L.append(f"| 问题 4-2（波动价，两阶段） | {mm['total']:,.2f} | {wan(mm['total'])} | "
                 f"{(mm['base']-mm['total'])/mm['base']*100:.2f}% | {mm['emg_qty']:,.2f} |")
    if q43:
        mm = q43["meta"]
        L.append(f"| 问题 4-3（波动价，四次滚动） | {mm['total_cost']:,.2f} | {wan(mm['total_cost'])} | "
                 f"{(mm['base_cost']-mm['total_cost'])/mm['base_cost']*100:.2f}% | {mm['emg_qty']:,.2f} |")
    L.append("")
    L.append(f"无储能基线：固定价 {q2['meta']['tot_base']:,.2f} 元；波动价 {q42['meta']['base']:,.2f} 元。")
    L.append("")
    L.append("VoI 实验（同一场景集、同一实际轨迹）：")
    L.append("")
    L.append("| 预报时刻数 | 总费用 元 | 紧急购电量 kWh |")
    L.append("|---|---|---|")
    for t, name in (("n1", "仅 0:00（1 次）"), ("n2", "0:00+12:00（2 次）"),
                    ("n4", "0:00/6:00/12:00/18:00（4 次）"), ("perf", "完全信息（上界）")):
        if q3[t]:
            L.append(f"| {name} | {q3[t]['meta']['total_cost']:,.2f} | {q3[t]['meta']['emg_qty']:,.2f} |")
    L.append("")
    L.append("## 五、v2 新增对照实验")
    L.append("")
    if v2cop:
        p1 = v2cop["part1_vine"]
        L.append("| C 藤 pair-Copula（334 个日水平观测） | 数值 |")
        L.append("|---|---|")
        for nm, key in (("tau(电价,负载)", "tau_price_load"), ("tau(电价,光伏)", "tau_price_pv"),
                        ("tau(负载,光伏)", "tau_load_pv"),
                        ("Pearson(电价,负载)", "pearson_price_load"),
                        ("Pearson(电价,光伏)", "pearson_price_pv")):
            L.append(f"| {nm} | {p1['plain'][key]:+.4f} |")
        for r in p1["vine"]:
            L.append(f"| {r['name']}：最优族 / tau / lambdaU / AIC | {r['best']} / "
                     f"{r['tau_emp']:+.4f} / {r['tail'][0]:.4f} / {r['aic']:.2f} |")
        p3 = v2cop["part3_generators"]
        L.append("")
        L.append("四种情景生成器的样本外对照（训练 2025-02-01 至 06-30，测试 07-01 至 12-31，184 天）：")
        L.append("")
        L.append("| 生成器 | 总费用 元 | 相对差 | 紧急购电量 kWh | 逐日 CVaR95 元 |")
        L.append("|---|---|---|---|---|")
        for k, nm in (("hist_strat", "历史代表日"), ("bootstrap", "分块 Bootstrap"),
                      ("copula", "KDE+Copula"), ("scaled", "按光伏规模缩放")):
            d = p3[k]
            L.append(f"| {nm} | {d['total_cost']:,.2f} | {d.get('diff_pct', 0.0):+.3f}% | "
                     f"{d['emg_qty']:,.2f} | {d['cvar95']:,.2f} |")
        p4 = v2cop["part4_risk"]
        L.append("")
        L.append(f"分时段 CVaR 风险画像（alpha={p4['alpha']}）：问题 2 逐槽 CVaR 之和/均值之和 "
                 f"{p4['q2_cvar_over_mean']:.4f}，前 12 槽占比 {p4['q2_top12_slot_share']:.4f}；"
                 f"问题 4-2 {p4['q4_cvar_over_mean']:.4f} / {p4['q4_top12_slot_share']:.4f}。")
        L.append("")
    if v2chance:
        L.append("机会约束对照（固定价 334 天）：")
        L.append("")
        L.append("| beta | 总费用 元 | 紧急购电量 kWh | 缺电槽位频率 | 逐日 CVaR95 元 |")
        L.append("|---|---|---|---|---|")
        for r in v2chance["chance"]:
            L.append(f"| {r['beta']:.2f} | {r['tot_cost']:,.2f} | {r['tot_emg_qty']:,.2f} | "
                     f"{r['shortfall_slot_freq']*100:.2f}% | {r['cvar95']:,.2f} |")
        sp = v2chance["reference_sp"]
        L.append(f"| 两阶段随机规划（主方案） | {sp['tot_cost']:,.2f} | {sp['tot_emg_qty']:,.2f} | "
                 f"{sp['shortfall_slot_freq']*100:.2f}% | {sp['cvar95']:,.2f} |")
        L.append("")
    if v2dro:
        L.append("盒式分布鲁棒对照（波动价 334 天回测）：")
        L.append("")
        L.append("| k | 回测总费用 元 | 节省率 | 紧急购电量 kWh | 逐日 CVaR95 元 |")
        L.append("|---|---|---|---|---|")
        res = v2dro["results"]
        ks = sorted(res, key=lambda s: float(s.split("=")[-1]))
```

```
        for k in ks:
            d = res[k]
            L.append(f"| {k.split('=')[-1]} | {d['total_cost']:,.2f} | {d['saving']*100:+.2f}% | "
                     f"{d['emg_qty']:,.2f} | {d['cvar95']:,.2f} |")
        L.append("")
    if v2dec:
        e = v2dec["equivalence_K60"]
        L.append(f"L 形（Benders）分解对拍：K=60 在 {e['n']} 天上最大相对间隙 {e['max_rel_gap']:.2e}，"
                 f"平均迭代 {e['mean_iters']:.2f} 次，全部收敛={e['all_converged']}；"
                 f"直接求解 {e['sum_t_direct']:.2f} s，分解 {e['sum_t_benders']:.2f} s。")
        L.append("")
        L.append("| 情景数 K | 直接求解耗时 s | L 形分解耗时 s | 加速比 | 最大相对间隙 |")
        L.append("|---|---|---|---|---|")
        for ent in v2dec["scenario_scan"]["entries"]:
            rows = ent["rows"]
            td = sum(r.get("t_direct_total", r.get("t_direct_production", 0.0)) for r in rows)
            tb = sum(r.get("t_benders_total", 0.0) for r in rows)
            L.append(f"| {ent['K']} | {td:.3f} | {tb:.3f} | {td/tb:.2f}x | "
                     f"{max(r['rel_gap'] for r in rows):.2e} |")
        L.append("")
    with open(os.path.join(NOTES, "数值台账.md"), "w") as fh:
        fh.write("\n".join(L))
    print("\n".join(L))
    return N


if __name__ == "__main__":
    main()
```

源程序：code/run_all.py

```
# -*- coding: utf-8 -*-
"""一键复现：依次求解问题 1–4、写出结果文件、跑审计与数值台账。

用法：python3 run_all.py            （全部）
      python3 run_all.py --fast     （跳过 K 稳健性与 CVaR 前沿等耗时实验）
预期耗时（单机、arm64、macOS）：约 5 分钟（--fast 约 4 分钟）。
"""
from __future__ import annotations

import argparse
import os
import subprocess
import sys
import time

HERE = os.path.dirname(os.path.abspath(__file__))
STEPS = [
    ("问题 1：典型日确定性 LP", [sys.executable, "q1.py"]),
    ("问题 2：固定价两阶段随机规划（334 天）", [sys.executable, "q2.py", "--tag", "main", "--K", "60", "--scen", "strat", "--info", "R3", "--write"]),
    ("问题 3：多预报时刻滚动优化（四次预报）", [sys.executable, "q3.py", "--epochs", "0,6,12,18", "--tag", "n4", "--K", "60", "--write"]),
    ("问题 4-2：波动电价两阶段", [sys.executable, "q4.py", "--mode", "q42", "--K", "64", "--write"]),
    ("问题 4-3：波动电价滚动优化", [sys.executable, "q4.py", "--mode", "q43", "--K", "64", "--write"]),
    ("容量/功率/效率扫描与报童经验覆盖率", [sys.executable, "exp_scan.py"]),
    ("独立审计（A1–A10）", [sys.executable, "audit.py"]),
    ("汇总论文数值台账", [sys.executable, "paper_numbers.py"]),
]
V2_EXTRA = [
    ("v2 相依结构：C 藤 pair-Copula 拟合与生成器样本外对照、分时段 CVaR 风险画像", [sys.executable, "exp_copula.py"]),
    ("v2 对照：机会约束（安全边际分位数扫描与报童互证）", [sys.executable, "exp_chance.py"]),
    ("v2 对照：盒式分布鲁棒（保守度—费用曲线）", [sys.executable, "exp_dro.py"]),
    ("v2 求解工程：L 形（Benders）分解与直接求解对拍", [sys.executable, "decomp.py"]),
    ("v2 图件：相依结构/生成器对照/风险画像/鲁棒前沿/分解", [sys.executable, "plot_figures_v2.py"]),
    ("假设 A9 量化：负载预报误差敏感性（相对 MAE 0%/5%/10%）", [sys.executable, "exp_loaderr.py"]),
]
FAST_EXTRA = [
    ("问题 2 对照：完美信息 R1", [sys.executable, "q2.py", "--tag", "cmp_R1", "--K", "60", "--scen", "strat", "--info", "R1"]),
    ("问题 2 对照：典型日预报 R2", [sys.executable, "q2.py", "--tag", "cmp_R2", "--K", "60", "--scen", "strat", "--info", "R2"]),
```

```
    ("问题 2 稳健性：K=120", [sys.executable, "q2.py", "--tag", "K120", "--K", "120", "--scen", "strat"
     , "--info", "R3"]),
    ("VoI：1 次预报", [sys.executable, "q3.py", "--epochs", "0", "--tag", "n1", "--K", "60"]),
    ("VoI：2 次预报", [sys.executable, "q3.py", "--epochs", "0,12", "--tag", "n2", "--K", "60"]),
    ("VoI：完全信息上界", [sys.executable, "q3.py", "--epochs", "0,6,12,18", "--tag", "perf", "--K", "
     60", "--perfect"]),
    ("CVaR 风险—费用前沿", [sys.executable, "q4.py", "--mode", "cvar", "--K", "64", "--step", "6"]),
    ("电价过程检验与情景保真度", [sys.executable, "q4.py", "--mode", "tests", "--K", "64"]),
]


def run(name, cmd):
    t0 = time.time()
    print("=" * 78)
    print(f"[{name}]  {' '.join(cmd)}")
    r = subprocess.run(cmd, cwd=HERE)
    print(f"[{name}] 完成，耗时 {time.time()-t0:.1f}s，退出码 {r.returncode}")
    if r.returncode != 0:
        raise SystemExit(f"步骤失败：{name}")


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--fast", action="store_true", help="跳过对照实验")
    args = ap.parse_args()
    t0 = time.time()
    for name, cmd in STEPS:
        run(name, cmd)
    if not args.fast:
        for name, cmd in FAST_EXTRA:
            run(name, cmd)
        for name, cmd in V2_EXTRA:
            run(name, cmd)
    print("=" * 78)
    print(f"全部完成，总耗时 {time.time()-t0:.1f}s。结果文件在 ../results/，中间结果在 ../intermediate/。
     ")


if __name__ == "__main__":
    main()
```

源程序：code/requirements.txt

```
# C 题论文复现环境（实测版本，macOS 15.7.5 / arm64）
numpy==2.4.3
pandas==3.0.1
scipy==1.18.1
openpyxl==3.1.5
matplotlib==3.11.1
scikit-learn==1.9.0
```

源程序：code/plot_figures.py

```
# -*- coding: utf-8 -*-
r"""
论文数据图绘制脚本（fig03 - fig10）
====================================
一次运行生成论文全部 8 张矢量数据图（PDF），输出到 C题论文/figures/。

运行方式
--------
    cd C题论文/code && python3 plot_figures.py
    # 或从工作区根目录：python3 "C题论文/code/plot_figures.py"

硬性口径
--------
* 中文字体：font.sans-serif = ["Songti SC","Heiti SC","PingFang SC","Arial Unicode MS"]，
  axes.unicode_minus = False；**导入 matplotlib 之前**必须把 MPLCONFIGDIR 指向工作区内
  的缓存目录（C题论文/build/mplcache），否则 macOS 上会因字体缓存不可写而报错。
* 输出：PDF 矢量（pdf.fonttype=42 嵌入 TrueType 子集），bbox_inches="tight"。
* 图内**不写标题**（论文由 LaTeX \caption 给题），只写坐标轴标签与单位；
  每条曲线/子图自带图例、单位与关键数值标注，可脱离正文阅读。
```

```
* 数字来源：C题论文/intermediate/*.json（q1 / exp_scan / q2_main / q3_n1 / q3_n2 /
  q3_n4 / q3_perf / q4_price / q4_cvar）与原始附件（附件 1、附件 4）。
  除"分位带"直接由附件 4 原始矩阵计算外，图中所有数字均由中间结果 JSON 复算得到，
  不做任何人工录入。

图的清单（编号与文件名固定）
--------------------------
    fig03_q1_dual_price.pdf          问题 1 最优性证书：电价 vs 储电边际价值 + 充放电量
    fig04_q1_capacity_power.pdf      容量-功率双向扫描
    fig05_cost_waterfall.pdf         问题 1 成本分解瀑布图
    fig06_voi.pdf                    信息价值 VoI（预报时刻数 1/2/4/完全信息）
    fig07_q2_emergency.pdf           问题 2 全年逐日紧急购电 + 报童两界
    fig08_q4_price.pdf               附件 4 电价剖面（分位带 + 分时骨架）+ 残差 ACF
    fig09_cvar_frontier.pdf          CVaR 风险-费用前沿（依赖 q4_cvar.json，缺失则跳过）
    fig10_strategy_mechanism.pdf     问题 1 最优策略机制（PV 盈余/负荷/电价/SOC/充放电）
"""
from __future__ import annotations

# ---------------------------------------------------------------------------
# 0. 环境（必须在 import matplotlib 之前完成）
# ---------------------------------------------------------------------------
import os

HERE = os.path.dirname(os.path.abspath(__file__))
PAPER_DIR = os.path.dirname(HERE)                    # C题论文/
WORKSPACE = os.path.dirname(PAPER_DIR)               # 工作区根目录
MPL_CACHE = os.path.join(PAPER_DIR, "build", "mplcache")
os.makedirs(MPL_CACHE, exist_ok=True)
os.environ["MPLCONFIGDIR"] = MPL_CACHE               # 字体缓存必须可写

import json
from datetime import datetime

import numpy as np

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.dates import MonthLocator, DateFormatter, date2num
from matplotlib.ticker import MultipleLocator, FuncFormatter

# ---------------------------------------------------------------------------
# 1. 全局样式
# ---------------------------------------------------------------------------
FONT_STACK = ["Songti SC", "Heiti SC", "PingFang SC", "Arial Unicode MS"]
plt.rcParams.update({
    "font.family": "sans-serif",
    "font.sans-serif": FONT_STACK,
    "axes.unicode_minus": False,
    "pdf.fonttype": 42,          # 嵌入 TrueType 子集，矢量且中文不缺字
    "ps.fonttype": 42,
    "savefig.bbox": "tight",
    "figure.dpi": 120,
    "font.size": 9,
    "axes.labelsize": 9,
    "axes.titlesize": 9,         # 本脚本不写标题，仅防意外
    "xtick.labelsize": 8,
    "ytick.labelsize": 8,
    "legend.fontsize": 7.4,
    "axes.grid": True,
    "grid.alpha": 0.28,
    "grid.linewidth": 0.5,
    "axes.linewidth": 0.8,
    "legend.framealpha": 0.92,
    "lines.linewidth": 1.3,
})

# 统一配色
C_PRICE = "#1f4e9c"      # 电价
C_MU = "#c0392b"         # 储电边际价值
C_CHG = "#2e86de"        # 充电
C_DIS = "#e67e22"        # 放电
C_PV = "#e8a33d"         # 光伏
```

```
C_LOAD = "#34495e"       # 负载
C_SOC = "#16a085"        # 储电量
C_GREY = "0.55"

# 主口径常数（与 config.py 一致；此处仅用于图上阈值线，不参与任何求解）
DT = 1.0 / 6.0           # h
ETA_C = ETA_D = 0.9      # 单边效率（假设 A2，往返 81%）
E_MAX = 5000.0 * DT      # kWh/槽，充放电量上限 833.3333
S_MIN, S_MAX, S_INIT = 1200.0, 10800.0, 6000.0

INTERMEDIATE = os.path.join(PAPER_DIR, "intermediate")
FIGDIR = os.path.join(PAPER_DIR, "figures")
ATTACH = os.path.join(WORKSPACE, "CUMCM2026Problems(1)", "C题", "附件")

FIG_SOURCES: dict[str, str] = {}   # 文件名 -> 数据来源说明


# ---------------------------------------------------------------------------
# 2. 工具函数
# ---------------------------------------------------------------------------
def load_json(name: str, required: bool = True):
    """读取 intermediate/<name>；required=False 时缺失返回 None。"""
    path = os.path.join(INTERMEDIATE, name)
    if not os.path.exists(path):
        if required:
            raise FileNotFoundError("缺少中间结果文件：%s" % path)
        return None
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)


def fnum(x: float, nd: int = 2) -> str:
    """千分位数字（论文口径：费用保留 2 位小数）。"""
    return ("{:,.%df}" % nd).format(float(x))


def read_attach_matrix(fname: str):
    """读取附件（首列为日期，其后 144 列）为 ndarray（行=日，列=槽位）。"""
    import pandas as pd
    path = os.path.join(ATTACH, fname)
    df = pd.read_excel(path, header=0)
    df = df.rename(columns={df.columns[0]: "date"})
    df = df.set_index("date")
    df.columns = [f"s{j}" for j in range(144)]
    return df.astype(float).to_numpy()


def read_attach1():
    """附件 1：典型日 {电价 元/kWh，小区负载 kW，光伏功率 kW}，144 槽。"""
    import pandas as pd
    df = pd.read_excel(os.path.join(ATTACH, "附件1.xlsx"), header=0)
    df.columns = ["时间", "price", "load", "pv"]
    return (df["price"].to_numpy(float), df["load"].to_numpy(float),
            df["pv"].to_numpy(float))


def slot_hours() -> np.ndarray:
    """槽位左端点（h）：槽 j = [j/6, (j+1)/6)。"""
    return np.arange(144) / 6.0


def step_close(t: np.ndarray, v: np.ndarray):
    """把 144 槽的阶梯序列闭合到 24 h，便于 step/fill_between(step='post')。"""
    return np.append(t, 24.0), np.append(v, v[-1])


def hhmm(hour_float: float) -> str:
    h = int(hour_float)
    m = int(round((hour_float - h) * 60))
    if m == 60:
        h, m = h + 1, 0
    return "%d:%02d" % (h, m)

```

```
166
167 def note(ax, text: str, loc: str = "upper left", fs: float = 7.2, pad: float = 0.38,
168          xy: tuple | None = None):
169     """轴内信息框（数值标注，不承担标题作用）。xy 给定时按轴分数坐标自定义位置。"""
170     if xy is None:
171         xy = {"upper left": (0.012, 0.975), "upper right": (0.988, 0.975),
172               "lower left": (0.012, 0.030), "lower right": (0.988, 0.030)}[loc]
173         ha = "left" if "left" in loc else "right"
174         va = "top" if "upper" in loc else "bottom"
175     else:
176         ha, va = "left", "top"
177     ax.text(xy[0], xy[1], text, transform=ax.transAxes, ha=ha, va=va,
178             fontsize=fs, linespacing=1.45,
179             bbox=dict(boxstyle="round,pad=%.2f" % pad, facecolor="white",
180                       edgecolor="0.72", linewidth=0.6, alpha=0.93))
181
182
183 def save(fig, name: str) -> str:
184     path = os.path.join(FIGDIR, name)
185     fig.savefig(path, format="pdf", bbox_inches="tight",
186                 metadata={"Creator": "plot_figures.py", "CreationDate": None})
187     plt.close(fig)
188     return path
189
190
191 def hour_axis(ax):
192     ax.set_xlim(0, 24)
193     ax.xaxis.set_major_locator(MultipleLocator(2))
194     ax.xaxis.set_major_formatter(FuncFormatter(lambda v, p: "%d" % int(round(v))))
195     ax.set_xlabel("时刻（h）")
196
197
198 # ---------------------------------------------------------------------------
199 # 3. 各图
200 # ---------------------------------------------------------------------------
201 def fig03():
202     """问题 1 最优性证书：电价 vs 储电边际价值（含死区与两条阈值线）+ 最优充放电量。"""
203     q1 = load_json("q1.json")
204     A = q1["arrays"]
205     price = np.asarray(A["price"], float)
206     mu = -np.asarray(A["dual_eq"], float)        # 取负：储电边际价值（元/kWh）
207     c = np.asarray(A["c"], float)
208     d = np.asarray(A["d"], float)
209
210     t = slot_hours()
211     xe, pe = step_close(t, price)
212     _, mue = step_close(t, mu)
213     th_c = ETA_C * mue                           # 充电阈值 eta_c * mu_t
214     th_d = mue / ETA_D                           # 放电阈值 mu_t / eta_d
215
216     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(6.9, 5.4), sharex=True,
217                                    gridspec_kw=dict(height_ratios=[1.32, 1.0], hspace=0.10))
218
219     # ---- 上：价格与边际价值 ----
220     ax1.fill_between(xe, th_c, th_d, step="post", color="0.87",
221                      label=r"死区：$\eta_c\mu_t\leq p_t\leq \mu_t/\eta_d$（不充不放）")
222     ax1.fill_between(xe, pe, th_c, step="post", where=(pe < th_c - 1e-9),
223                      color="#f5c26b", alpha=0.75, linewidth=0,
224                      label=r"充电窗口：$p_t<\eta_c\mu_t$")
225     ax1.step(xe, pe, where="post", color=C_PRICE, label=r"电价 $p_t$（附件 1，元/kWh）")
226     ax1.step(xe, mue, where="post", color=C_MU, label=r"储电边际价值 $\mu_t$（元/kWh）")
227     ax1.step(xe, th_c, where="post", color="#d35400", ls="--", lw=1.0,
228              label=r"充电阈值 $\eta_c\mu_t$")
229     ax1.step(xe, th_d, where="post", color="#7d3c98", ls="--", lw=1.0,
230              label=r"放电阈值 $\mu_t/\eta_d$")
231     ax1.set_ylabel("电价 / 储电边际价值（元 · kWh$^{-1}$）")
232     ax1.set_ylim(0, 1.62)
233     ax1.legend(loc="upper left", ncol=3, columnspacing=0.9, handlelength=1.8)
234     note(ax1, "死区比 $1/(\\eta_c\\eta_d)=%.4f$；$\\mu_t$ = LP 等式约束对偶量（水价值）"
235          % q1["main"]["ratio_cd"], loc="lower right", fs=6.8)
236
237     # ---- 下：最优充放电量（充电为正、放电为负） ----
238     w = 1.0 / 6.0
```

```
    ax2.bar(t + w / 2, c, width=w, color=C_CHG, label="最优充电量 $c_t$（kWh/槽）")
    ax2.bar(t + w / 2, -d, width=w, color=C_DIS, label="最优放电量 $d_t$（kWh/槽）")
    ax2.axhline(0, color="0.25", lw=0.7)
    ax2.set_ylim(-1080, 1080)
    ax2.set_ylabel("充放电量（kWh · 槽$^{-1}$）\n充电为正、放电为负")
    hour_axis(ax2)
    ax2.legend(loc="upper left", ncol=2, columnspacing=1.0)

    load = np.asarray(A["load"], float)
    pv = np.asarray(A["pv"], float)
    ch_idx = np.where(c > 1e-6)[0]
    dis_idx = np.where(d > 1e-6)[0]
    idle_idx = np.where((c <= 1e-6) & (d <= 1e-6))[0]
    ok_chg = int(np.sum(price[ch_idx] <= ETA_C * mu[ch_idx] + 1e-9))
    ok_dis = int(np.sum(price[dis_idx] >= mu[dis_idx] / ETA_D - 1e-9))
    idle_in = int(np.sum((price[idle_idx] >= ETA_C * mu[idle_idx] - 1e-9) &
                         (price[idle_idx] <= mu[idle_idx] / ETA_D + 1e-9)))
    viol = ch_idx[price[ch_idx] > ETA_C * mu[ch_idx] + 1e-9]
    viol_sur = int(np.sum(pv[viol] > load[viol]))
    note(ax2, "充电槽 %d（其中 %d 个落在阴影充电窗口内；其余 %d 个中有 %d 个处于 PV 盈余槽，"
              "充电由零成本光伏供给）\n"
              "放电槽 %d（%d 个落在放电阈值上方）；空闲槽 %d（%d 个落在死区内）\n"
              "充电量 %s kWh，放电量 %s kWh，充放比 %.4f = $1/(\\eta_c\\eta_d)$"
         % (len(ch_idx), ok_chg, len(viol), viol_sur, len(dis_idx), ok_dis,
            len(idle_idx), idle_in, fnum(c.sum()), fnum(d.sum()), c.sum() / d.sum()),
         loc="lower left", fs=6.5)

    for ax in (ax1, ax2):
        ax.set_xlim(0, 24)
    FIG_SOURCES["fig03_q1_dual_price.pdf"] = (
        "q1.json: arrays.price / arrays.dual_eq(取负=水价值) / arrays.c / arrays.d; "
        "阈值线由 config.py 主口径 eta_c=eta_d=0.9 计算")
    return save(fig, "fig03_q1_dual_price.pdf")


def fig04():
    """容量-功率双向扫描：左=容量扫描（最优费用 + 储电边际价值），右=充放功率上限扫描。"""
    scan = load_json("exp_scan.json")["capacity"]
    caps = np.array([r["cap"] for r in scan["cap_scan"]], float)
    costs = np.array([r["cost"] for r in scan["cap_scan"]], float)
    pows = np.array([r["pmax"] for r in scan["power_scan"]], float)
    pcost = np.array([r["cost"] for r in scan["power_scan"]], float)
    pchg = np.array([r["charge"] for r in scan["power_scan"]], float)
    base = float(scan["base"])

    # 储电容量边际价值 = -dC*/dSbar（相邻扫描点中心/单侧差分，单位 元/kWh）
    mcaps = 0.5 * (caps[1:] + caps[:-1])
    mvals = -(np.diff(costs) / np.diff(caps))

    fig, (axL, axR) = plt.subplots(1, 2, figsize=(7.2, 3.4))
    fig.subplots_adjust(wspace=0.46)
    LIMIT_C = "#7d3c98"      # 单一自由度放开的参考线

    # ---- 左：容量扫描（最优费用 + 储电容量边际价值） ----
    axL.plot(caps, costs, "o-", color=C_PRICE, ms=3.6, label="最优购电费 $C^*$（元/日）")
    axL.axhline(base, color=C_GREY, ls=":", lw=1.0,
                label="题面容量 12 000 kWh 基准：%s 元/日" % fnum(base))
    axL.axhline(scan["no_capacity_limit"], color=LIMIT_C, ls="--", lw=1.0,
                label="仅解除容量上限：%s 元/日（再省 %.2f%%）"
                      % (fnum(scan["no_capacity_limit"]), scan["gain_no_cap"]))
    axL.set_xlabel("可用储能容量（kWh，保持 10% – 90% DOD）")
    axL.set_ylabel("最优购电费（元 · 日$^{-1}$）")
    axL.set_ylim(24800, 40400)
    axL.set_xlim(0, 50000)

    axL2 = axL.twinx()
    axL2.step(mcaps, mvals, where="mid", color=C_MU, lw=1.3,
              label=r"容量边际价值 $-\partial C^*/\partial\bar S$（元/kWh）")
    axL2.plot(mcaps, mvals, "s", color=C_MU, ms=3.0)
    axL2.set_ylabel(r"边际价值 $-\partial C^*/\partial\bar S$（元 · kWh$^{-1}$）",
                    color=C_MU)
    axL2.tick_params(axis="y", colors=C_MU)
    axL2.set_ylim(0, 1.15)
```

```
    axL2.grid(False)
    h1, l1 = axL.get_legend_handles_labels()
    h2, l2 = axL2.get_legend_handles_labels()
    axL.legend(h1 + h2, l1 + l2, loc="upper right", fontsize=6.4)
    note(axL, "费用曲线分段线性、下凸；边际价值由 %.3f 单调递减至 %.3f 元/kWh"
         % (mvals[0], mvals[-1]), loc="lower left", fs=6.4)

    # ---- 右：充放电功率上限扫描 ----
    axR.plot(pows, pcost, "o-", color=C_PRICE, ms=3.6, label="最优购电费 $C^*$（元/日）")
    axR.axhline(scan["no_power_limit"], color=LIMIT_C, ls="--", lw=1.0,
                label="仅解除功率上限：%s 元/日（再省 %.2f%%）"
                      % (fnum(scan["no_power_limit"]), scan["gain_no_pow"]))
    axR.set_xlabel("充放电功率上限（kW）")
    axR.set_ylabel("最优购电费（元 · 日$^{-1}$）")
    axR.set_ylim(33800, 40800)
    axR.set_xlim(600, 11000)

    axR2 = axR.twinx()
    axR2.plot(pows, pchg, "s--", color=C_CHG, ms=3.2, lw=1.1,
              label="全天充电量（kWh/日）")
    axR2.set_ylabel("全天充电量（kWh · 日$^{-1}$）", color=C_CHG)
    axR2.tick_params(axis="y", colors=C_CHG)
    axR2.set_ylim(8000, 32000)
    axR2.grid(False)
    h1, l1 = axR.get_legend_handles_labels()
    h2, l2 = axR2.get_legend_handles_labels()
    axR.legend(h1 + h2, l1 + l2, loc="upper right", fontsize=6.4)
    axR2.text(4200, 23400, "充电量饱和（ 7 500 kW）：%s kWh/日" % fnum(pchg[-1]),
              fontsize=6.3, color="#1a5276", ha="left", va="bottom")

    FIG_SOURCES["fig04_q1_capacity_power.pdf"] = (
        "exp_scan.json: capacity.cap_scan / power_scan / base / no_capacity_limit / "
        "no_power_limit / gain_no_cap / gain_no_pow；边际价值为扫描费用的离散差分")
    return save(fig, "fig04_q1_capacity_power.pdf")


def fig05():
    """问题 1 成本分解瀑布图：无储能基线 -> 储能削峰填谷收益 -> 最优购电费。"""
    q1 = load_json("q1.json")["main"]
    base = q1["base_cost"]
    opt = q1["cost"]
    save_amt = q1["saving"]

    fig, ax = plt.subplots(figsize=(4.9, 3.9))
    xs = [0, 1, 2]
    ax.bar(0, base, width=0.56, color="0.72", edgecolor="0.35", linewidth=0.7)
    ax.bar(1, -save_amt, bottom=base, width=0.56, color="#27ae60",
           edgecolor="#1e8449", linewidth=0.7)
    ax.bar(2, opt, width=0.56, color=C_PRICE, edgecolor="#16365c", linewidth=0.7)

    # 连接线
    ax.plot([0.28, 1.72], [base, base], ls="--", lw=0.8, color="0.45")
    ax.plot([1.28, 2.28], [opt, opt], ls="--", lw=0.8, color="0.45")

    ax.text(0, base + 1400, "%s 元" % fnum(base), ha="center", fontsize=7.6)
    ax.text(1, base + 1400, "-%s 元" % fnum(save_amt), ha="center", fontsize=7.6,
            color="#1e8449")
    ax.text(2, opt + 1400, "%s 元" % fnum(opt), ha="center", fontsize=7.6,
            color=C_PRICE)

    ax.set_xticks(xs)
    ax.set_xticklabels(["无储能基线 $C^{base}$\n（净负荷全额购电）",
                        "储能削峰填谷收益\n（价差套利 + 光伏时移）",
                        "最优购电费 $C^*$\n（含储能，日周期）"], fontsize=7.4)
    ax.set_ylabel("日购电费用（元 · 日$^{-1}$）")
    ax.set_ylim(0, 72000)
    note(ax, "$C^{base}=\\sum_t p_t\\max(L_t-PV_t,0)\\Delta t$\n"
             "口径 C-01 / C-02，附件 1 典型日（全年平均日）\n"
             "节省率 %.2f%%（%s 元/日）"
         % (q1["saving_rate"] * 100, fnum(save_amt)), loc="upper right", fs=6.8)
    FIG_SOURCES["fig05_cost_waterfall.pdf"] = (
        "q1.json: main.base_cost / main.cost / main.saving / main.saving_rate")
    return save(fig, "fig05_cost_waterfall.pdf")
```

```
385
386
387 def fig06():
388     """信息价值 VoI：预报时刻数 1/2/4/完全信息 的总费用（左轴）与紧急购电量（右轴）。"""
389     m1 = load_json("q3_n1.json")["meta"]
390     m2 = load_json("q3_n2.json")["meta"]
391     m4 = load_json("q3_n4.json")["meta"]
392     mp = load_json("q3_perf.json")["meta"]
393     metas = [m1, m2, m4, mp]
394     cost = np.array([m["total_cost"] for m in metas]) / 1e4      # 万元
395     emg = np.array([m["emg_qty"] for m in metas])                # kWh
396     labels = ["1 次\n0:00", "2 次\n0:00 + 12:00",
397               "4 次\n0:00 / 6:00 / 12:00 / 18:00", "完全信息\n（上界）"]
398     x = np.arange(4)
399
400     fig, ax = plt.subplots(figsize=(6.7, 4.2))
401     axb = ax.twinx()
402     axb.bar(x, emg, width=0.42, color="#aed6f1", edgecolor="#5499c7",
403             linewidth=0.7, label="紧急购电量（kWh，右轴）")
404     axb.set_ylabel("紧急购电量（kWh）", color="#2471a3")
405     axb.tick_params(axis="y", colors="#2471a3")
406     axb.set_ylim(0, 760000)
407     axb.grid(False)
408     for xi, v in zip(x, emg):
409         axb.text(xi, v + 9000, fnum(v, 0), ha="center", fontsize=6.8, color="#1a5276")
410
411     ax.plot(x, cost, "o-", color=C_MU, ms=5.0, lw=1.6, label="总费用（万元，左轴）")
412     for xi, v in zip(x, cost):
413         ax.annotate(fnum(v), (xi, v), textcoords="offset points", xytext=(0, 7),
414                     ha="center", fontsize=7.0, color=C_MU)
415     for i in range(3):
416         dcost = (cost[i] - cost[i + 1]) * 1e4
417         ax.annotate("-%s 元" % fnum(dcost, 0),
418                     ((x[i] + x[i + 1]) / 2, (cost[i] + cost[i + 1]) / 2),
419                     textcoords="offset points", xytext=(0, -14), ha="center",
420                     fontsize=6.8, color="0.25")
421
422     ax.axvline(2, color="0.35", ls="--", lw=1.0)
423     ax.annotate("本题实际：4 次预报", xy=(2.0, 1418.0), xytext=(1.92, 1423.0),
424                 ha="right", fontsize=7.0, color="0.25",
425                 arrowprops=dict(arrowstyle="->", color="0.45", lw=0.7))
426     ax.set_xticks(x)
427     ax.set_xticklabels(labels, fontsize=7.2)
428     ax.set_xlim(-0.55, 3.55)
429     ax.set_ylim(1384.5, 1445.0)
430     ax.set_xlabel("预报时刻数（同一场景集、同一实际轨迹，结果窗口 %d 天）"
431                   % m1["n_days"])
432     ax.set_ylabel("总费用（万元）", color=C_MU)
433     ax.tick_params(axis="y", colors=C_MU)
434     h1, l1 = ax.get_legend_handles_labels()
435     h2, l2 = axb.get_legend_handles_labels()
436     ax.legend(h1 + h2, l1 + l2, loc="upper right", fontsize=6.8)
437
438     voi = m1["total_cost"] - mp["total_cost"]
439     got = m1["total_cost"] - m4["total_cost"]
440     note(ax, "可获得信息价值上界（1 次 → 完全信息）：%s 元\n"
441              "4 次已实现 %.1f%%；再加密最多再省 %s 元（%.2f%%）\n"
442              "无储能基线 %s 元（%s 万元，超出纵轴范围）"
443          % (fnum(voi), got / voi * 100, fnum(m4["total_cost"] - mp["total_cost"]),
444             (m4["total_cost"] - mp["total_cost"]) / m4["total_cost"] * 100,
445             fnum(m1["base_cost"]), fnum(m1["base_cost"] / 1e4, 2)),
446          loc="upper left", fs=6.6)
447     FIG_SOURCES["fig06_voi.pdf"] = (
448         "q3_n1.json / q3_n2.json / q3_n4.json / q3_perf.json 的 meta."
449         "total_cost、meta.emg_qty、meta.base_cost")
450     return save(fig, "fig06_voi.pdf")
451
452
453 def fig07():
454     """问题 2 全年逐日紧急购电量 + 报童两界（缺货概率 1/10 与 1/5）。"""
455     q2 = load_json("q2_main.json")
456     meta = q2["meta"]
457     nb = load_json("exp_scan.json")["newsboy"]
```

```
    dates = [datetime.strptime(r["date"][:10], "%Y-%m-%d") for r in q2["daily"]]
    g = np.array([r["emg_qty"] for r in q2["daily"]], float)

    fig, ax = plt.subplots(figsize=(7.3, 3.3))
    xs = date2num(dates)
    ax.bar(xs, g, width=0.82, color="#aed6f1", edgecolor="#5499c7",
           linewidth=0.35, label="逐日紧急购电量 $\\sum_t g_t$（kWh/日）")

    k = 7
    roll = np.convolve(g, np.ones(k) / k, mode="same")
    ax.plot(xs, roll, color="#1a5276", lw=1.2, label="%d 日滑动平均（kWh/日）" % k)

    q80 = float(np.percentile(g, 80))
    q90 = float(np.percentile(g, 90))
    ax.axhline(q80, color="#e67e22", ls="--", lw=1.1,
               label="报童界 1/5：日 %d%% 分位 = %s kWh" % (80, fnum(q80)))
    ax.axhline(q90, color="#c0392b", ls="-.", lw=1.1,
               label="报童界 1/10：日 %d%% 分位 = %s kWh" % (90, fnum(q90)))

    month_ticks = [d for d in dates if d.day == 1]          # 仅 2025 年各月 1 日，避免出现次年 1 月
    ax.set_xticks(date2num(month_ticks))
    ax.set_xticklabels([d.strftime("%m月") for d in month_ticks])
    ax.set_xlim(date2num(dates[0]) - 2, date2num(dates[-1]) + 2)
    ax.set_ylim(0, 5800)
    ax.set_xlabel("日期（2025-02-01 至 2025-12-31，共 %d 天）" % len(dates))
    ax.set_ylabel("紧急购电量（kWh · 日$^{-1}$）")
    ax.legend(loc="upper left", fontsize=6.4, ncol=1)
    note(ax, "逐槽缺货频率 %.2f%%（报童两界 1/10–1/5 的下侧）\n"
             "电量满足率 %.2f%%；全年紧急购电量 %s kWh\n"
             "有紧急购电天数 %d/%d；单日最大 %s kWh（%s）"
         % (nb["short_slot_rate"] * 100, nb["fill_rate"] * 100,
            fnum(meta["tot_emg_qty"]), meta["days_with_emg"], len(dates),
            fnum(g.max()), dates[int(np.argmax(g))].strftime("%Y-%m-%d")),
         loc="upper right", fs=6.4)
    FIG_SOURCES["fig07_q2_emergency.pdf"] = (
        "q2_main.json: daily[].emg_qty（334 天）、meta.tot_emg_qty、meta.days_with_emg；"
        "exp_scan.json: newsboy.short_slot_rate / newsboy.fill_rate")
    return save(fig, "fig07_q2_emergency.pdf")


def fig08():
    """附件 4 电价剖面（分位带 + 分时骨架）+ 残差 ACF(1–6)。"""
    pt = load_json("q4_price.json")["price_tests"]
    # 分时骨架：优先用附件 1 原始剖面，缺失时退回 q1.json 的电价数组
    try:
        skel, _, _ = read_attach1()
        skel_src = "附件 1"
    except Exception:
        skel = np.asarray(load_json("q1.json")["arrays"]["price"], float)
        skel_src = "q1.json arrays.price（附件 1 同源）"

    band_ok, M = False, None
    try:
        M = read_attach_matrix("附件4.xlsx")
        band_ok = M.shape[1] == 144 and M.shape[0] > 0
    except Exception:
        band_ok = False

    t = slot_hours()
    xe, ske = step_close(t, skel)

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(6.9, 5.2),
                                   gridspec_kw=dict(height_ratios=[1.5, 1.0], hspace=0.30))

    if band_ok:
        q = np.percentile(M, [10, 25, 50, 75, 90], axis=0)
        qe = np.hstack([q, q[:, [-1]]])
        ax1.fill_between(xe, qe[0], qe[4], step="post", color="#d6e4f0",
                         label="附件 4 电价分位带：10–90%% / 25–75%%（%d 天 × 144 槽）"
                               % M.shape[0])
        ax1.fill_between(xe, qe[1], qe[3], step="post", color="#9ec5e8",
                         label="_nolegend_")
        ax1.step(xe, qe[2], where="post", color="#1a5276", lw=1.2,
```

```
                  label="附件 4 逐槽中位数电价（元/kWh）")
    ax1.step(xe, ske, where="post", color=C_PRICE, lw=1.7, ls="--",
             label="分时骨架（%s 电价剖面，元/kWh）" % skel_src)
    ax1.set_ylabel("电价（元 · kWh$^{-1}$）")
    ax1.set_ylim(0, 2.4)
    ax1.legend(loc="upper left", fontsize=6.3, ncol=1)
    hour_axis(ax1)
    ax1.set_xlabel("")
    note(ax1, "全年电价均值 %s、最小 %s、最大 %s 元/kWh\n"
              "骨架与附件 4 逐时段均值最大偏差 %.2e 元/kWh"
         % (fnum(pt["price_mean"], 4), fnum(pt["price_min"], 4),
            fnum(pt["price_max"], 4), pt["skeleton_is_mean"]),
         loc="upper right", fs=6.6)

    acf = np.asarray(pt["C2_acf_lag1_6"], float)
    lags = np.arange(1, len(acf) + 1)
    ax2.bar(lags, acf, width=0.5, color="#5dade2", edgecolor="#2471a3",
            linewidth=0.7, label="日内电价残差自相关 ACF($k$)，$k$ 以 10 min 为步长")
    for xi, v in zip(lags, acf):
        ax2.text(xi, v + 0.018, "%.4f" % v, ha="center", fontsize=6.8, color="#1a5276")
    ax2.axhline(0, color="0.3", lw=0.7)
    ax2.set_ylim(0, 1.18)
    ax2.set_xticks(lags)
    ax2.set_xlabel("滞后阶数 $k$（10 min）")
    ax2.set_ylabel("自相关系数")
    ax2.legend(loc="upper right", fontsize=6.8)
    dacf = np.asarray(pt["C2b_daily_acf"], float)
    note(ax2, "日级残差 ACF：lag1 %.4f、lag2 %.4f、lag3 %.4f\n"
              "峰段/谷段波动比 %.4f（%.4f / %.4f 元/kWh）"
         % (dacf[0], dacf[1], dacf[2], pt["C3_sd_ratio"],
            pt["C3_sd_peak"], pt["C3_sd_night"]),
         loc="lower left", fs=6.6)
    FIG_SOURCES["fig08_q4_price.pdf"] = (
        "q4_price.json: price_tests.C2_acf_lag1_6 / C2b_daily_acf / C3_sd_ratio / "
        "C3_sd_peak / C3_sd_night / price_mean / price_min / price_max / skeleton_is_mean；"
        "分位带与骨架由附件 4、附件 1 原始数据计算")
    return save(fig, "fig08_q4_price.pdf")


def fig09(cvar=None, out_name="fig09_cvar_frontier.pdf"):
    """CVaR 风险-费用前沿：lambda 扫描下的期望费用 / CVaR95 / 真实轨迹回测费用。

    q4_cvar.json 由 code/q4.py --mode cvar 生成，结构：
        {"days": [...], "frontier": [{"lam","exp_cost","real_cost","cvar","emg_qty"}, ...]}
    缺失时跳过本图（由 main() 报告），不影响其余 7 张图。
    """
    if cvar is None:
        cvar = load_json("q4_cvar.json", required=False)
    if not cvar:
        return None
    fr = cvar["frontier"]
    lam = np.array([r["lam"] for r in fr], float)
    exp_c = np.array([r["exp_cost"] for r in fr], float)
    cvar_v = np.array([r["cvar"] for r in fr], float)
    real_c = np.array([r["real_cost"] for r in fr], float)
    emg = np.array([r["emg_qty"] for r in fr], float)
    ndays = len(cvar.get("days", []))

    fig, (axL, axR) = plt.subplots(1, 2, figsize=(7.3, 3.4))
    fig.subplots_adjust(wspace=0.56)

    axL.plot(lam, exp_c, "o-", color=C_PRICE, ms=3.4, label="情景期望费用（元/日）")
    axL.plot(lam, cvar_v, "s-", color=C_MU, ms=3.4, label="CVaR$_{0.95}$（元/日）")
    axL.set_xlabel(r"风险厌恶系数 $\lambda$")
    axL.set_ylabel("费用 / 风险（元 · 日$^{-1}$）")
    axL.legend(loc="upper left", fontsize=6.8)

    axL2 = axL.twinx()
    axL2.plot(lam, real_c, "^--", color="#16a085", ms=3.4, lw=1.1,
              label="真实轨迹回测费用（元/日，n=%d）" % ndays)
    axL2.set_ylabel("回测费用（元 · 日$^{-1}$）", color="#16a085")
    axL2.tick_params(axis="y", colors="#16a085")
    axL2.grid(False)
```

```
    h1, l1 = axL.get_legend_handles_labels()
    h2, l2 = axL2.get_legend_handles_labels()
    axL.legend(h1 + h2, l1 + l2, loc="upper left", fontsize=6.6)
    note(axL, "紧急购电量 %.1f → %.1f kWh/日\n（$\\lambda$ 由 0 增至 %.1f）"
         % (emg[0], emg[-1], lam[-1]), loc="lower right", fs=6.6)

    # 前沿散点：期望费用 - CVaR
    axR.plot(exp_c, cvar_v, "-", color="0.55", lw=1.0, zorder=1)
    sc = axR.scatter(exp_c, cvar_v, c=lam, cmap="viridis", s=26, zorder=3)
    for i, (xv, yv, lv) in enumerate(zip(exp_c, cvar_v, lam)):
        if i % 2:                    # 每隔一个点标注，避免标签互相压叠
            continue
        axR.annotate(r"$\lambda$=%.1f" % lv, (xv, yv), textcoords="offset points",
                     xytext=(4, 3), fontsize=6.2, color="0.25")
    # 拐点：单位期望费用增量带来的 CVaR 下降最大
    if len(lam) >= 3:
        slope = -np.diff(cvar_v) / np.maximum(np.diff(exp_c), 1e-9)
        k = int(np.argmax(slope))
        axR.scatter([exp_c[k + 1]], [cvar_v[k + 1]], marker="*", s=150,
                    facecolor="none", edgecolor="#c0392b", linewidth=1.2,
                    label=r"边际风险下降最快点 $\lambda$=%.1f" % lam[k + 1])
        axR.legend(loc="lower left", fontsize=6.6)
    cb = fig.colorbar(sc, ax=axR, pad=0.02)
    cb.set_label(r"$\lambda$", fontsize=7)
    cb.ax.tick_params(labelsize=6.5)
    axR.set_xlabel("情景期望费用（元 · 日$^{-1}$）")
    axR.set_ylabel("CVaR$_{0.95}$（元 · 日$^{-1}$）")
    FIG_SOURCES[out_name] = (
        "q4_cvar.json: frontier[].lam / exp_cost / cvar / real_cost / emg_qty，"
        "days（真实轨迹回测子样本，n=%d）" % ndays)
    return save(fig, out_name)


def fig10():
    """问题 1 最优策略机制：PV 盈余/负荷、电价与储电量、充放电量（三子图共享时间轴）。"""
    q1 = load_json("q1.json")
    A = q1["arrays"]
    price = np.asarray(A["price"], float)
    load = np.asarray(A["load"], float)
    pv = np.asarray(A["pv"], float)
    c = np.asarray(A["c"], float)
    d = np.asarray(A["d"], float)
    S = np.asarray(A["S"], float)

    t = slot_hours()
    xe, le = step_close(t, load)
    _, pve = step_close(t, pv)
    _, pe = step_close(t, price)

    surplus = pv > load
    sur_slots = np.where(surplus)[0]
    h_sur = (sur_slots.min() / 6.0, (sur_slots.max() + 1) / 6.0)   # PV 盈余窗口
    cheap = np.where(price <= 0.45)[0]                             # 低价时段（谷段）
    h_cheap = (cheap.min() / 6.0, (cheap.max() + 1) / 6.0)
    peak = np.where(price >= 1.25)[0]                              # 高价时段（峰段）
    h_peak = (peak.min() / 6.0, (peak.max() + 1) / 6.0)

    fig, axes = plt.subplots(3, 1, figsize=(6.9, 7.1), sharex=True,
                             gridspec_kw=dict(height_ratios=[1.0, 1.0, 1.0], hspace=0.16))
    axA, axB, axC = axes

    # 三条竖直参考带（跨子图对齐）
    for ax in axes:
        ax.axvspan(h_sur[0], h_sur[1], color="#f7dc6f", alpha=0.20, zorder=0)
        ax.axvspan(h_cheap[0], h_cheap[1], color="#85c1e9", alpha=0.20, zorder=0)
        ax.axvspan(h_peak[0], h_peak[1], color="#ec7063", alpha=0.13, zorder=0)

    # ---- (a) 功率 ----
    axA.fill_between(xe, le, pve, step="post", where=(pve > le), color=C_PV,
                     alpha=0.45, linewidth=0, label="光伏盈余 $PV_t-L_t$（kW）")
    axA.step(xe, le, where="post", color=C_LOAD, lw=1.4, label="小区负载 $L_t$（kW）")
    axA.step(xe, pve, where="post", color=C_PV, lw=1.4, label="光伏实际功率 $PV_t$（kW）")
    axA.set_ylabel("功率（kW）")
```

```
    axA.set_ylim(-600, 11600)
    axA.legend(loc="upper left", fontsize=6.6, ncol=1)
    sur_energy = float((np.maximum(pv - load, 0) * DT).sum())
    note(axA, "光伏盈余时段 %s–%s（%d 槽）：盈余电量 %s kWh\n"
              "该窗口内充电 %s kWh，占全天充电量 %.1f%%"
         % (hhmm(h_sur[0]), hhmm(h_sur[1]), len(sur_slots), fnum(sur_energy),
            fnum(c[surplus].sum()), c[surplus].sum() / c.sum() * 100),
         loc="upper right", fs=6.6)

    # ---- (b) 电价与储电量 ----
    axB.step(xe, pe, where="post", color=C_PRICE, lw=1.4,
             label="电价 $p_t$（附件 1，元/kWh）")
    axB.set_ylabel("电价（元 · kWh$^{-1}$）")
    axB.set_ylim(0.30, 1.62)
    axB.legend(loc="upper left", fontsize=6.8)
    axB2 = axB.twinx()
    axB2.step(xe, np.append(S, S[-1]), where="post", color=C_SOC, lw=1.5,
              label="储电量 $S_t$（kWh，右轴）")
    axB2.text(0.3, 5700, "起止储电量 %s kWh（$S_0=S_{144}$，日周期）"
              % fnum(q1["main"]["s0"], 0), fontsize=6.4, color=C_SOC,
              ha="left", va="top")
    axB2.axhline(S_MAX, color=C_SOC, ls=":", lw=1.0)
    axB2.axhline(S_MIN, color=C_SOC, ls=":", lw=1.0)
    axB2.axhline(S_INIT, color="0.45", ls=":", lw=0.9)
    axB2.plot([0, 24], [S_INIT, S_INIT], "o", color=C_SOC, ms=3.4, zorder=5)
    axB2.set_ylim(0, 12600)
    axB2.set_ylabel("储电量（kWh）", color=C_SOC)
    axB2.tick_params(axis="y", colors=C_SOC)
    axB2.grid(False)
    axB2.text(7.5, 10950, "上界 %s kWh（触界 %d/144 槽）"
              % (fnum(S_MAX, 0), int(np.sum(S >= S_MAX - 1e-6))),
              fontsize=6.4, color=C_SOC, ha="left", va="bottom")
    axB2.text(13.5, 1300, "下界 %s kWh（触界 %d/144 槽）"
              % (fnum(S_MIN, 0), int(np.sum(S <= S_MIN + 1e-6))),
              fontsize=6.4, color=C_SOC, ha="left", va="bottom")
    h1, l1 = axB.get_legend_handles_labels()
    h2, l2 = axB2.get_legend_handles_labels()
    axB.legend(h1 + h2, l1 + l2, loc="upper left", fontsize=6.6)

    # ---- (c) 充放电量 ----
    w = 1.0 / 6.0
    axC.bar(t + w / 2, c, width=w, color=C_CHG, label="充电量 $c_t$（kWh/槽）")
    axC.bar(t + w / 2, -d, width=w, color=C_DIS, label="放电量 $d_t$（kWh/槽）")
    axC.axhline(0, color="0.25", lw=0.7)
    axC.axhline(E_MAX, color="0.45", ls=":", lw=0.9)
    axC.axhline(-E_MAX, color="0.45", ls=":", lw=0.9)
    axC.set_ylim(-1400, 1400)
    axC.set_ylabel("充放电量（kWh · 槽$^{-1}$）\n充电为正、放电为负")
    hour_axis(axC)
    axC.legend(loc="upper left", fontsize=6.6, ncol=1)
    note(axC, "全天充电 %s kWh、放电 %s kWh，充放比 %.4f\n"
              "功率打满上限（±%s kWh/槽）：充电 %d/144 槽、放电 %d/144 槽\n"
              "（黄：PV 盈余段；蓝：谷段低价；红：峰段高价）"
         % (fnum(c.sum()), fnum(d.sum()), c.sum() / d.sum(), fnum(E_MAX, 0),
            int(np.sum(c >= E_MAX - 1e-6)), int(np.sum(d >= E_MAX - 1e-6))),
         loc="upper right", fs=6.4)

    for ax in axes:
        ax.set_xlim(0, 24)
    FIG_SOURCES["fig10_strategy_mechanism.pdf"] = (
        "q1.json: arrays.price / arrays.load / arrays.pv / arrays.S / arrays.c / "
        "arrays.d，main.s0；阴影窗口由数组本身界定（PV>L 盈余段、p_t 0.45 谷段、"
        "p_t 1.25 峰段）")
    return save(fig, "fig10_strategy_mechanism.pdf")


# ---------------------------------------------------------------------------
# 4. 主程序
# ---------------------------------------------------------------------------
def main():
    os.makedirs(FIGDIR, exist_ok=True)
    made, skipped = [], []

```

```
    jobs = [("fig03_q1_dual_price.pdf", fig03),
            ("fig04_q1_capacity_power.pdf", fig04),
            ("fig05_cost_waterfall.pdf", fig05),
            ("fig06_voi.pdf", fig06),
            ("fig07_q2_emergency.pdf", fig07),
            ("fig08_q4_price.pdf", fig08),
            ("fig09_cvar_frontier.pdf", fig09),
            ("fig10_strategy_mechanism.pdf", fig10)]

    for name, fn in jobs:
        try:
            path = fn()
        except Exception as exc:            # 单图失败不阻断其余图
            skipped.append((name, "%s: %s" % (type(exc).__name__, exc)))
            print("[失败] %-34s %s: %s" % (name, type(exc).__name__, exc))
            continue
        if path is None:
            skipped.append((name, "依赖的中间结果缺失（q4_cvar.json 未生成）"))
            print("[跳过] %-34s 依赖的中间结果缺失" % name)
        else:
            made.append((name, os.path.getsize(path)))

    print("\n" + "=" * 78)
    print("已生成矢量 PDF: ")
    for name, size in made:
        print("  %-34s %9d 字节    来源: %s" % (name, size, FIG_SOURCES.get(name, "?")))
    if skipped:
        print("未生成: ")
        for name, why in skipped:
            print("  %-34s %s" % (name, why))
    print("输出目录: %s" % FIGDIR)
    print("=" * 78)
    return made, skipped


if __name__ == "__main__":
    main()
```

源程序：code/plot_figures_v2.py

```
# -*- coding: utf-8 -*-
r"""
v2 新增数据图（fig11 - fig15）
==============================
    fig11_copula_structure.pdf     电价-负载-光伏相依结构: C 藤逐层二元 Copula 与族选择
    fig12_scenario_generators.pdf  四种情景生成器的样本外对照（费用/紧急购电/边际保真度）
    fig13_risk_profile.pdf         分时段 CVaR 风险画像（问题 2 固定价 vs 问题 4 波动价）
    fig14_robust_frontier.pdf      分布鲁棒: 保守度—费用曲线（需 exp_dro.json）
    fig15_decomposition.pdf        L 形分解: 收敛与规模对照（需 decomp.json）

数字来源: intermediate/exp_copula.json、exp_dro.json、decomp.json 与原始附件，不做人工录入。
"""
from __future__ import annotations

import os

import numpy as np
import pandas as pd

import plot_figures as PF          # 复用全局样式（导入时自动设置 MPLCONFIGDIR 与中文字体）
plt = PF.plt

import config as C
import copula as CP
import scen_copula as SC

INTER = PF.INTERMEDIATE
FIG = PF.FIGDIR
C_PRICE, C_LOAD, C_PV = PF.C_PRICE, PF.C_LOAD, PF.C_PV
C_MU, C_CHG, C_DIS, C_SOC, C_GREY = PF.C_MU, PF.C_CHG, PF.C_DIS, PF.C_SOC, PF.C_GREY
FAMILY_CN = {"gaussian": "Gaussian", "t": "t", "clayton": "Clayton",
             "gumbel": "Gumbel", "frank": "Frank"}

```

```
34
35 def _u(x):
36     from scipy import stats
37     r = stats.rankdata(np.asarray(x, float))
38     return (r - 0.5) / len(x)
39
40
41 def fig11():
42     j = PF.load_json("exp_copula.json")
43     p1 = j["part1_vine"]
44     panel = C.Panel(SC.DATE_ALL)
45     dates = pd.date_range(C.WIN_START, C.WIN_END, freq="D")
46     lv = SC.daily_levels(panel, dates)
47     P = lv["price"].mean(1); L = lv["load"].sum(1) * C.DT; V = lv["pv"].sum(1) * C.DT
48     rec = {r["name"]: r for r in p1["vine"]}
49
50     fig = plt.figure(figsize=(7.4, 2.55))
51     gs = fig.add_gridspec(1, 3, width_ratios=[1, 1, 1.25], wspace=0.34)
52
53     def contour(ax, u, v, name, tau, col, ttl):
54         g = np.linspace(0.002, 0.998, 120)
55         U, Vv = np.meshgrid(g, g, indexing="ij")
56         r = rec[name]
57         d = np.exp(CP.logpdf(r["best"], U, Vv, r["par"]))
58         ax.scatter(u, v, s=2.2, c=C_GREY, alpha=0.45, linewidths=0, zorder=1)
59         ax.contour(U, Vv, d, levels=6, colors=col, linewidths=0.7, zorder=2)
60         ax.set_xlim(0, 1); ax.set_ylim(0, 1)
61         ax.set_xlabel("伪观测 $u$（日平均电价）", fontsize=7.6)
62         ax.set_ylabel(ttl, fontsize=7.6)
63         ax.text(0.03, 0.965, "最优族 %s\n$\\tau=%.3f$，$\\lambda_U=%.3f$\nAIC$=%.1f$"
64                 % (FAMILY_CN[r["best"]], tau, r["tail"][0], r["aic"]),
65                 transform=ax.transAxes, va="top", ha="left", fontsize=7.0,
66                 bbox=dict(fc="white", ec="0.75", lw=0.5, alpha=0.9, pad=1.8))
67
68     ax = fig.add_subplot(gs[0, 0])
69     contour(ax, _u(P), _u(L), "T1_price_load", p1["plain"]["tau_price_load"], C_PRICE,
70             "$v$（日负载电量，kWh）")
71     ax = fig.add_subplot(gs[0, 1])
72     contour(ax, _u(P), _u(V), "T1_price_pv", p1["plain"]["tau_price_pv"], C_PV,
73             "$v$（日光伏电量，kWh）")
74     ax = fig.add_subplot(gs[0, 2])
75     pairs = [("T1_price_load", "电价 – 负载"), ("T1_price_pv", "电价 – 光伏"),
76              ("T2_load_pv|price", "负载 – 光伏$|$电价")]
77     fams = CP.FAMILIES
78     w = 0.26
79     xs = np.arange(len(fams))
80     cols = [C_PRICE, C_PV, C_SOC]
81     for i, (nm, lb) in enumerate(pairs):
82         r = rec[nm]
83         cand = {c["family"]: c["aic"] for c in r["candidates"]}
84         vals = [cand.get(f, 0.0) for f in fams]
85         ax.bar(xs + (i - 1) * w, vals, width=w, color=cols[i], alpha=0.85, label=lb,
86                edgecolor="white", linewidth=0.4)
87         best = r["best"]
88         k = fams.index(best)
89         ax.annotate("*", (xs[k] + (i - 1) * w, cand[best]), ha="center", va="bottom",
90                     fontsize=11, color=cols[i])
91     ax.set_xticks(xs); ax.set_xticklabels([FAMILY_CN[f] for f in fams], fontsize=7.4)
92     ax.set_ylabel("AIC（越小越好）", fontsize=7.6)
93     ax.axhline(0, color="0.6", lw=0.6)
94     ax.legend(fontsize=6.8, loc="lower left")
95     ax.text(0.985, 0.03, "* 为各族中 AIC 最小者", transform=ax.transAxes,
96             ha="right", va="bottom", fontsize=6.8, color="0.35")
97     PF.save(fig, "fig11_copula_structure.pdf")
98     print("fig11 ok")
99
100
101 def fig12():
102     j = PF.load_json("exp_copula.json")
103     p3 = j["part3_generators"]
104     keys = ["hist_strat", "bootstrap", "copula", "scaled"]
105     lbl = ["历史代表日", "分块 Bootstrap", "KDE+Copula", "按光伏规模缩放"]
106     cols = [C_PRICE, C_GREY, C_MU, C_PV]
```

```
    base = p3["hist_strat"]["total_cost"]
    fid = dict(hist_strat=p3["fidelity_hist"], bootstrap=p3["fidelity_boot"],
               copula=p3["fidelity_copula"], scaled=None)
    fig, axes = plt.subplots(1, 3, figsize=(7.4, 2.45))
    xs = np.arange(len(keys))
    ax = axes[0]
    v = [(p3[k]["total_cost"] / base - 1) * 100 for k in keys]
    b = ax.bar(xs, v, color=cols, alpha=0.88, width=0.6)
    for xi, vi in zip(xs, v):
        ax.annotate("%+.3f%%" % vi, (xi, vi), ha="center",
                    va="bottom" if vi >= 0 else "top", fontsize=7.0,
                    xytext=(0, 2 if vi >= 0 else -2), textcoords="offset points")
    ax.axhline(0, color="0.5", lw=0.8)
    ax.set_ylabel("样本外总费用相对差（%）", fontsize=7.6)
    ax.set_xticks(xs); ax.set_xticklabels(lbl, fontsize=6.9, rotation=16, ha="right")
    ax.margins(y=0.30)
    ax.text(0.02, 0.96, "基准：%.0f 元" % base, transform=ax.transAxes, va="top",
            fontsize=6.9, color="0.3")

    ax = axes[1]
    v = [p3[k]["emg_qty"] for k in keys]
    ax.bar(xs, v, color=cols, alpha=0.88, width=0.6)
    for xi, vi in zip(xs, v):
        ax.annotate("%.0f" % vi, (xi, vi), ha="center", va="bottom", fontsize=6.9,
                    xytext=(0, 2), textcoords="offset points")
    ax.set_ylabel("紧急购电量（kWh，184 天累计）", fontsize=7.6)
    ax.set_xticks(xs); ax.set_xticklabels(lbl, fontsize=6.9, rotation=16, ha="right")
    ax.margins(y=0.18)

    ax = axes[2]
    names = ["hist_strat", "bootstrap", "copula"]
    exact = [fid[k]["R_absq_gap"] if fid[k] else np.nan for k in keys[:3]]
    ax.bar(xs[:3], exact, color=cols[:3], alpha=0.88, width=0.55)
    for xi, vi in zip(xs[:3], exact):
        ax.annotate("%.0f" % vi, (xi, vi), ha="center", va="bottom", fontsize=6.9,
                    xytext=(0, 2), textcoords="offset points")
    ax.text(xs[3], 0, "不适用\n（边缘\n保真）", ha="center", va="bottom", fontsize=6.6,
            color="0.4")
    ax.set_ylabel("日残差总量 1%/99% 分位最大绝对偏差（kWh）", fontsize=7.2)
    ax.set_xticks(xs); ax.set_xticklabels(lbl, fontsize=6.9, rotation=16, ha="right")
    ax.margins(y=0.20)
    fig.subplots_adjust(wspace=0.42)
    PF.save(fig, "fig12_scenario_generators.pdf")
    print("fig12 ok")


def fig13():
    j = PF.load_json("exp_copula.json")
    p4 = j["part4_risk"]
    h = np.arange(24)
    fig, axes = plt.subplots(1, 2, figsize=(7.4, 2.5), sharey=False)
    for ax, tag, ttl in ((axes[0], "q2", "问题 2（固定电价）"),
                         (axes[1], "q4", "问题 4-2（波动电价）")):
        mu = np.array(p4[f"{tag}_hour_mean"]); cv = np.array(p4[f"{tag}_hour_cvar"])
        fr = np.array(p4[f"{tag}_hour_shortage_freq"]) * 100
        ax.bar(h, cv, color=C_MU, alpha=0.30, width=0.82, label="逐小时 CVaR$_{0.95}$")
        ax.plot(h, mu, color=C_PRICE, lw=1.5, marker="o", ms=2.4, label="逐小时均值")
        ax.set_xlabel("时刻（h）", fontsize=7.6)
        ax.set_ylabel("费用（元/日）", fontsize=7.6)
        ax.set_xticks(range(0, 24, 3))
        ax2 = ax.twinx()
        ax2.plot(h, fr, color=C_SOC, lw=1.2, ls="--", marker="s", ms=2.2,
                 label="缺电频率")
        ax2.set_ylabel("槽位缺电频率（%）", fontsize=7.4, color=C_SOC)
        ax2.tick_params(axis="y", labelcolor=C_SOC, labelsize=7)
        ax2.grid(False)
        ax.set_title(ttl, fontsize=8.0)
        h1, l1 = ax.get_legend_handles_labels(); h2, l2 = ax2.get_legend_handles_labels()
        ax.legend(h1 + h2, l1 + l2, fontsize=6.4, loc="upper left", ncol=1)
        pk = int(np.argmax(fr))
        ax2.annotate("峰值 %d:00\n%.1f%%" % (pk, fr[pk]), (pk, fr[pk]), fontsize=6.4,
                     color=C_SOC, xytext=(4, 4), textcoords="offset points")
    fig.subplots_adjust(wspace=0.42)
```

```
    PF.save(fig, "fig13_risk_profile.pdf")
    print("fig13 ok")


def fig14():
    j = PF.load_json("exp_dro.json", required=False)
    if not j:
        print("fig14 跳过（缺 exp_dro.json）")
        return
    r0 = j.get("results")
    if isinstance(r0, dict):
        res = [r0[k] for k in sorted(r0, key=lambda s: float(s.split("=")[-1]))]
    elif isinstance(r0, list) and r0 and isinstance(r0[0], dict) and "total_cost" in r0[0]:
        res = r0
    else:
        print("fig14 跳过（exp_dro.json 结构不可用）")
        return
    ks = [r["k"] for r in res]
    tot = [r["total_cost"] for r in res]
    cvar = [r.get("cvar95") for r in res]
    emg = [r["emg_qty"] for r in res]
    fig, ax = plt.subplots(1, 2, figsize=(7.0, 2.5))
    ax[0].plot(ks, np.array(tot) / 1e4, color=C_PRICE, marker="o", ms=3, label="总费用")
    ax[0].set_xlabel("保守度参数 $k$（不确定集半宽，倍标准差）", fontsize=7.6)
    ax[0].set_ylabel("总费用（万元）", fontsize=7.6)
    ax2 = ax[0].twinx()
    if all(v is not None for v in cvar):
        ax2.plot(ks, np.array(cvar) / 1e4, color=C_MU, marker="s", ms=3,
                 label="逐日费用 CVaR$_{0.95}$")
        ax2.set_ylabel("CVaR$_{0.95}$（万元/日）", fontsize=7.4, color=C_MU)
        ax2.tick_params(axis="y", labelcolor=C_MU, labelsize=7)
        ax2.grid(False)
    ax[1].plot(ks, emg, color=C_SOC, marker="o", ms=3)
    ax[1].set_xlabel("保守度参数 $k$", fontsize=7.6)
    ax[1].set_ylabel("紧急购电量（kWh）", fontsize=7.6)
    fig.subplots_adjust(wspace=0.42)
    PF.save(fig, "fig14_robust_frontier.pdf")
    print("fig14 ok")


def fig15():
    j = PF.load_json("decomp.json", required=False)
    if not j or not isinstance(j.get("scenario_scan"), dict):
        print("fig15 跳过（缺 decomp.json 或结构不可用）")
        return
    ent = j["scenario_scan"].get("entries") or []
    if not ent:
        print("fig15 跳过（scenario_scan.entries 为空）")
        return
    Ks, td, tb = [], [], []
    for e in ent:
        rows = e.get("rows") or []
        if not rows:
            continue
        Ks.append(e["K"])
        td.append(sum(r.get("t_direct_total", r.get("t_direct_production", 0.0)) for r in rows))
        tb.append(sum(r.get("t_benders_total", 0.0) for r in rows))
    fig, ax = plt.subplots(1, 2, figsize=(7.0, 2.4))
    ax[0].plot(Ks, td, color=C_PRICE, marker="o", ms=3, label="直接求解（HiGHS）")
    ax[0].plot(Ks, tb, color=C_MU, marker="s", ms=3, label="L 形分解（解析多割）")
    ax[0].set_yscale("log")
    ax[0].set_xlabel("情景数 $K$", fontsize=7.6)
    ax[0].set_ylabel("4 个代表日求解耗时（s，对数轴）", fontsize=7.4)
    ax[0].legend(fontsize=6.6)
    ax[1].plot(Ks, [d / b for d, b in zip(td, tb)], color=C_SOC, marker="^", ms=3.4)
    ax[1].axhline(1.0, color="0.5", lw=0.8, ls="--")
    ax[1].set_xlabel("情景数 $K$", fontsize=7.6)
    ax[1].set_ylabel("加速比（直接求解耗时 / 分解耗时）", fontsize=7.4)
    for k, v in zip(Ks, [d / b for d, b in zip(td, tb)]):
        ax[1].annotate("%.2f×" % v, (k, v), fontsize=6.5, xytext=(0, 3),
                       textcoords="offset points", ha="center")
    ax[1].margins(y=0.22)
    fig.subplots_adjust(wspace=0.4)
```

```
    PF.save(fig, "fig15_decomposition.pdf")
    print("fig15 ok")


def main():
    fig11(); fig12(); fig13(); fig14(); fig15()


if __name__ == "__main__":
    main()
```

源程序：code/copula.py

```
# -*- coding: utf-8 -*-
"""
二元 Copula 工具库 + 藤 Copula（pair-Copula）联合建模
==================================================
对应参考论文：
  · 宁楠《基于动态 C 藤 pair-Copula 的电力系统动态经济调度》—— C 藤结构、逐层二元 Copula、
    以与经验 Copula 的距离选族；
  · 崔丽敏等《基于核密度估计和 Copula 函数的风-光-荷联合场景生成》—— KDE 边缘 + Copula 相依
    + 蒙特卡洛抽样。

本文件只依赖 numpy/scipy，自行实现五个常用二元 Copula 族（Gaussian / t / Clayton /
Gumbel / Frank）的密度、h 函数（条件分布）与 h 逆函数，用于：
  1) 用极大似然 + AIC 在候选族中选优（并同时报告与经验 Copula 的欧氏距离）；
  2) 计算上/下尾相依系数，量化"高峰又贵又缺"的尾部聚集；
  3) 通过 C 藤逐层采样生成联合场景。

约定：所有函数输入 u, v ∈ (0,1)（伪观测或分布函数值）。
"""
from __future__ import annotations

import numpy as np
from scipy import stats, special
from scipy.optimize import brentq, minimize

EPS = 1e-10
FAMILIES = ("gaussian", "t", "clayton", "gumbel", "frank")
# 多元正态/t 的数值积分用固定随机源，保证经验 Copula 距离可复现
_CDF_RNG = np.random.default_rng(20260910)


# ---------------------------------------------------------------------------
# 二元 Copula 对数密度
# ---------------------------------------------------------------------------
def logpdf(family, u, v, par):
    u = np.clip(np.asarray(u, float), EPS, 1 - EPS)
    v = np.clip(np.asarray(v, float), EPS, 1 - EPS)
    if family == "gaussian":
        rho = float(np.clip(par[0], -0.999, 0.999))
        x, y = stats.norm.ppf(u), stats.norm.ppf(v)
        s = 1.0 - rho ** 2
        return -0.5 * np.log(s) - (rho ** 2 * (x ** 2 + y ** 2) - 2 * rho * x * y) / (2 * s)
    if family == "t":
        rho = float(np.clip(par[0], -0.999, 0.999))
        nu = float(np.clip(par[1], 2.05, 80.0))
        x, y = stats.t.ppf(u, nu), stats.t.ppf(v, nu)
        s = 1.0 - rho ** 2
        ll = (special.gammaln((nu + 2) / 2) + special.gammaln(nu / 2)
              - 2 * special.gammaln((nu + 1) / 2) - 0.5 * np.log(s))
        num = -(nu + 2) / 2 * np.log1p((x ** 2 + y ** 2 - 2 * rho * x * y) / (nu * s))
        den = -(nu + 1) / 2 * (np.log1p(x ** 2 / nu) + np.log1p(y ** 2 / nu))
        return ll + num - den
    if family == "clayton":
        th = float(np.clip(par[0], 1e-3, 30.0))
        return (np.log1p(th) - (1 + th) * (np.log(u) + np.log(v))
                - (2 + 1 / th) * np.log(np.power(u, -th) + np.power(v, -th) - 1))
    if family == "gumbel":
        th = float(np.clip(par[0], 1.0 + 1e-4, 20.0))
        lu, lv = -np.log(u), -np.log(v)
        A = np.power(lu ** th + lv ** th, 1.0 / th)
        return (-A + (th - 1) * (np.log(lu) + np.log(lv)) - np.log(u) - np.log(v)
```

```
                + (1.0 / th - 2) * np.log(lu ** th + lv ** th) + np.log(A + th - 1))
    if family == "frank":
        th = float(par[0])
        if abs(th) < 1e-6:
            return np.zeros_like(u)
        e = np.exp(-th)
        num = np.log(abs(th)) + np.log(abs(1 - e)) - th * (u + v)
        den = 2 * np.log(np.abs((1 - e) - (1 - np.exp(-th * u)) * (1 - np.exp(-th * v))))
        return num - den
    raise ValueError(family)


def loglik(family, u, v, par) -> float:
    return float(np.sum(logpdf(family, u, v, par)))


# ---------------------------------------------------------------------------
# 参数域与初值（用 Kendall tau 反演给初值，按族给出界）
# ---------------------------------------------------------------------------
def _tau(u, v) -> float:
    return float(stats.kendalltau(u, v).statistic)


def init_and_bounds(family, u, v):
    tau = _tau(u, v)
    if family == "gaussian":
        return [np.sin(np.pi * tau / 2)], [(-0.995, 0.995)]
    if family == "t":
        return [np.sin(np.pi * tau / 2), 6.0], [(-0.995, 0.995), (2.1, 60.0)]
    if family == "clayton":
        th = max(2 * tau / (1 - tau), 0.05) if tau > 0 else 0.05
        return [th], [(1e-3, 30.0)]
    if family == "gumbel":
        th = 1.0 / (1 - tau) if tau > 0 else 1.05
        return [min(max(th, 1.001), 19.0)], [(1.001, 20.0)]
    if family == "frank":
        return [4.0 * np.sign(tau) * max(abs(tau), 0.05)], [(-40.0, 40.0)]
    raise ValueError(family)


def fit(family, u, v):
    """极大似然拟合单个二元 Copula，返回 dict(par, loglik, aic)。"""
    x0, bnds = init_and_bounds(family, u, v)

    def nll(p):
        ll = loglik(family, u, v, p)
        return -ll if np.isfinite(ll) else 1e12
    best = None
    for scale in (1.0, 0.5, 1.5):
        try:
            r = minimize(nll, np.array(x0) * scale, method="L-BFGS-B", bounds=bnds,
                         options=dict(maxiter=400, ftol=1e-10))
        except Exception:
            continue
        if np.isfinite(r.fun) and (best is None or r.fun < best.fun):
            best = r
    if best is None:
        raise RuntimeError(f"{family} 拟合失败")
    par = np.asarray(best.x, float)
    ll = -float(best.fun)
    k = len(par)
    return dict(family=family, par=par, loglik=ll, aic=2 * k - 2 * ll, k=k)


# ---------------------------------------------------------------------------
# h 函数（条件分布）与 h 逆
# ---------------------------------------------------------------------------
def hfunc(family, u, v, par):
    """h(u|v) = dC(u,v)/dv，返回与 u 同形的数组。"""
    u = np.clip(np.asarray(u, float), EPS, 1 - EPS)
    v = np.clip(np.asarray(v, float), EPS, 1 - EPS)
    if family == "gaussian":
        rho = float(np.clip(par[0], -0.999, 0.999))
```

```
        x, y = stats.norm.ppf(u), stats.norm.ppf(v)
        return stats.norm.cdf((x - rho * y) / np.sqrt(1 - rho ** 2))
    if family == "t":
        rho = float(np.clip(par[0], -0.999, 0.999))
        nu = float(np.clip(par[1], 2.05, 80.0))
        x, y = stats.t.ppf(u, nu), stats.t.ppf(v, nu)
        arg = (x - rho * y) * np.sqrt((nu + 1) / ((nu + y ** 2) * (1 - rho ** 2)))
        return stats.t.cdf(arg, nu + 1)
    if family == "clayton":
        th = float(np.clip(par[0], 1e-3, 30.0))
        return np.power(v, -th - 1) * np.power(np.power(u, -th) + np.power(v, -th) - 1,
                                               -1 / th - 1)
    if family == "gumbel":
        th = float(np.clip(par[0], 1.0 + 1e-4, 20.0))
        lu, lv = -np.log(u), -np.log(v)
        A = np.power(lu ** th + lv ** th, 1.0 / th)
        return np.exp(-A) * np.power(lv, th - 1) * np.power(lu ** th + lv ** th,
                                                            1.0 / th - 1) / v
    if family == "frank":
        th = float(par[0])
        if abs(th) < 1e-6:
            return u.copy()
        e = np.exp(-th)
        num = (np.exp(-th * u) - 1) * np.exp(-th * v)
        den = (e - 1) + (np.exp(-th * u) - 1) * (np.exp(-th * v) - 1)
        return num / den
    raise ValueError(family)


def hinv(family, target, v, par):
    """求 u 使 h(u|v) = target（标量 target/v）。"""
    def f(u):
        return float(hfunc(family, u, v, par)) - target
    lo, hi = EPS, 1 - EPS
    flo, fhi = f(lo), f(hi)
    if flo > 0:
        return lo
    if fhi < 0:
        return hi
    return brentq(f, lo, hi, xtol=1e-12, rtol=1e-12, maxiter=200)


# ---------------------------------------------------------------------------
# 经验 Copula 距离与尾相依系数
# ---------------------------------------------------------------------------
def empirical_copula_distance(family, u, v, par, ngrid=40) -> float:
    """与经验 Copula 的 L2 距离（在 ngrid x ngrid 网格上）。

    注意：Gaussian / t 的 Copula 分布函数由 scipy 的随机化数值积分给出，
    为**可复现**起见在此固定随机源（rng），否则同一数据两次运行会得到末位不同的距离。
    """
    g = np.linspace(0.02, 0.98, ngrid)
    U, V = np.meshgrid(g, g, indexing="ij")
    emp = np.array([[np.mean((u <= a) & (v <= b)) for b in g] for a in g])
    th = copula_cdf(family, U, V, par)
    return float(np.sqrt(np.mean((emp - th) ** 2)))


def _t_cdf(x, rho, nu):
    """二元 t 分布函数：固定随机源以保证可复现（scipy 的数值积分是随机化的）。"""
    kw = dict(loc=[0, 0], shape=[[1, rho], [rho, 1]], df=nu)
    try:
        return stats.multivariate_t.cdf(x, rng=_CDF_RNG, **kw)
    except TypeError:
        state = np.random.get_state()
        np.random.seed(20260910)
        try:
            return stats.multivariate_t.cdf(x, **kw)
        finally:
            np.random.set_state(state)


def copula_cdf(family, u, v, par):
```

```
    u = np.clip(np.asarray(u, float), EPS, 1 - EPS)
    v = np.clip(np.asarray(v, float), EPS, 1 - EPS)
    if family == "gaussian":
        rho = float(np.clip(par[0], -0.999, 0.999))
        x, y = stats.norm.ppf(u), stats.norm.ppf(v)
        return stats.multivariate_normal.cdf(np.stack([x, y], -1), mean=[0, 0],
                                             cov=[[1, rho], [rho, 1]],
                                             rng=_CDF_RNG)
    if family == "t":
        rho = float(np.clip(par[0], -0.999, 0.999))
        nu = float(np.clip(par[1], 2.05, 80.0))
        x, y = stats.t.ppf(u, nu), stats.t.ppf(v, nu)
        return _t_cdf(np.stack([x, y], -1), rho, nu)
    if family == "clayton":
        th = float(np.clip(par[0], 1e-3, 30.0))
        return np.power(np.power(u, -th) + np.power(v, -th) - 1, -1 / th)
    if family == "gumbel":
        th = float(np.clip(par[0], 1.0 + 1e-4, 20.0))
        return np.exp(-np.power((-np.log(u)) ** th + (-np.log(v)) ** th, 1 / th))
    if family == "frank":
        th = float(par[0])
        if abs(th) < 1e-6:
            return u * v
        e = np.exp(-th)
        return -1 / th * np.log1p((np.exp(-th * u) - 1) * (np.exp(-th * v) - 1) / (e - 1))
    raise ValueError(family)


def tail_dependence(family, par):
    """返回 (上尾, 下尾) 相依系数 _U, _L。"""
    if family == "gaussian":
        rho = float(par[0])
        return (0.0, 0.0) if abs(rho) < 1 else (1.0, 1.0)
    if family == "t":
        rho, nu = float(par[0]), float(par[1])
        lam = 2 * stats.t.cdf(-np.sqrt((nu + 1) * (1 - rho) / (1 + rho)), nu + 1)
        return float(lam), float(lam)
    if family == "clayton":
        return 0.0, float(2 ** (-1 / float(par[0])))
    if family == "gumbel":
        return float(2 - 2 ** (1 / float(par[0]))), 0.0
    if family == "frank":
        return 0.0, 0.0
    raise ValueError(family)


# ---------------------------------------------------------------------------
# 边缘分布：核密度估计（KDE）
# ---------------------------------------------------------------------------
class KdeMarginal:
    """一维 KDE 边缘分布，提供 cdf / ppf / pdf。"""

    def __init__(self, x, bw=None, n_grid=2001):
        x = np.asarray(x, float)
        self.x = x
        self.kde = stats.gaussian_kde(x, bw_method=bw)
        sd = float(x.std(ddof=1)) if len(x) > 1 else 1.0
        lo = float(x.min()) - 3.0 * sd
        hi = float(x.max()) + 3.0 * sd
        g = np.linspace(lo, hi, n_grid)
        d = self.kde(g)
        c = np.concatenate([[0.0], np.cumsum((d[1:] + d[:-1]) / 2 * np.diff(g))])
        c = c / c[-1]
        self._g, self._c, self._d = g, c, d

    def cdf(self, x):
        return np.interp(x, self._g, self._c, left=0.0, right=1.0)

    def ppf(self, u):
        return np.interp(np.clip(u, 0.0, 1.0), self._c, self._g)

    def pdf(self, x):
        return self.kde(np.atleast_1d(x))
```

```
280
281     def sample(self, n, rng):
282         return self.kde.resample(n, seed=rng)[0]
283
284
285 def pseudo_obs(x):
286     """伪观测: (rank - 0.5)/n。"""
287     x = np.asarray(x, float)
288     r = stats.rankdata(x)
289     return (r - 0.5) / len(x)
290
291
292 # ---------------------------------------------------------------------------
293 # C 藤（pair-Copula）联合建模: 3 变量，根节点为第 0 个变量
294 # ---------------------------------------------------------------------------
295 class CVine3:
296     """三变量 C 藤:
297
298         c(u1,u2,u3) = c12(u1,u2) · c13(u1,u3) · c23|1(h(u2|u1), h(u3|u1))
299
300     根节点为 u1（本题取电价）。逐层二元 Copula 族由 AIC 在 5 个候选族中选出。
301     """
302
303     def __init__(self, names=("price", "load", "pv")):
304         self.names = names
305         self.marg = [None, None, None]
306         self.tree = {}
307
308     def fit(self, X, verbose=False):
309         """X: (n,3) 原始数据；返回选族结果表。"""
310         X = np.asarray(X, float)
311         n = len(X)
312         for j in range(3):
313             self.marg[j] = KdeMarginal(X[:, j])
314         U = np.column_stack([pseudo_obs(X[:, j]) for j in range(3)])
315         self.U = U
316         rows = []
317         # 第 1 层: c12, c13
318         for (i, j) in ((0, 1), (0, 2)):
319             rows.append(self._fit_pair(f"T1_{self.names[i]}_{self.names[j]}", U[:, i], U[:, j]))
320         # 第 2 层: c23|1
321         e2 = hfunc(self.tree["T1_price_load"]["family"], U[:, 1], U[:, 0],
322                    self.tree["T1_price_load"]["par"])
323         e3 = hfunc(self.tree["T1_price_pv"]["family"], U[:, 2], U[:, 0],
324                    self.tree["T1_price_pv"]["par"])
325         rows.append(self._fit_pair(f"T2_{self.names[1]}_{self.names[2]}|{self.names[0]}", e2, e3))
326         if verbose:
327             for r in rows:
328                 print(f"  {r['name']:26s} 最优族={r['best']:9s} 参数={np.round(r['par'],4)} "
329                       f"AIC={r['aic']:.2f} 经验距离={r['ecdist']:.4f}")
330         return rows
331
332     def _fit_pair(self, name, u, v):
333         cands = []
334         for fam in FAMILIES:
335             try:
336                 f = fit(fam, u, v)
337             except Exception:
338                 continue
339             f["ecdist"] = empirical_copula_distance(fam, u, v, f["par"])
340             f["tau_emp"] = _tau(u, v)
341             f["tau_fit"] = None
342             cands.append(f)
343         cands.sort(key=lambda d: d["aic"])
344         best = cands[0]
345         rec = dict(name=name, best=best["family"], par=best["par"].tolist(),
346                    loglik=best["loglik"], aic=best["aic"],
347                    ecdist=best["ecdist"], tau_emp=best["tau_emp"],
348                    candidates=[dict(family=c["family"], par=c["par"].tolist(),
349                                     loglik=c["loglik"], aic=c["aic"],
350                                     ecdist=c["ecdist"]) for c in cands])
351         self.tree[name] = dict(family=best["family"], par=best["par"])
352         # 同时给出该对的 tau 与尾相依
```

```
        rec["tail"] = tail_dependence(best["family"], best["par"])
        return rec

    def sample_copula(self, n, rng):
        """按 C 藤逐层采样，返回 (n,3) 的 (u1,u2,u3)。"""
        t12 = self.tree["T1_price_load"]; t13 = self.tree["T1_price_pv"]
        t23 = self.tree["T2_load_pv|price"]
        w = rng.random((n, 3))
        u1 = w[:, 0]
        u2 = np.array([hinv(t12["family"], w[i, 1], u1[i], t12["par"]) for i in range(n)])
        e2 = hfunc(t12["family"], u2, u1, t12["par"])
        e3 = np.array([hinv(t23["family"], w[i, 2], e2[i], t23["par"]) for i in range(n)])
        u3 = np.array([hinv(t13["family"], e3[i], u1[i], t13["par"]) for i in range(n)])
        return np.column_stack([u1, u2, u3])
```

源程序：code/scen_copula.py

```
# -*- coding: utf-8 -*-
"""
问题 4 的 KDE + Copula 联合场景生成器（v2 新增）
================================================
参考：
  · 崔丽敏等《基于核密度估计和 Copula 函数的风-光-荷联合场景生成》：KDE 拟合边缘分布、
    Copula 刻画相依结构、蒙特卡洛抽样生成联合场景；
  · 宁楠《基于动态 C 藤 pair-Copula 的电力系统动态经济调度》：C 藤逐层二元 Copula、
    以 AIC / 经验 Copula 距离选族；
  · 李翔/孙丰杰：聚类式场景削减（本文件提供分层 medoid 与多初始 K-means 两种削减对照）。

生成对象（与 q4.build_joint_scenarios 输出结构一致）
-----------------------------------------------------
  price  (K,144)  情景电价（元/kWh）
  resid  (K,144)  情景预报残差 r_t = (PV^{fcst} - PV^{act}) * DT（kWh）
  w      (K,)     情景权重

"水平 + 形状"分解
-----------------
日水平：  P_d = 当日 144 槽电价的均值（元/kWh）；
          R_d = 当日 144 槽预报残差之和（kWh，>0 表示全天高估光伏）。
日内形状：电价形状 = 当日电价剖面 − 当日均值（保留分时骨架与峰谷形态）；
          残差形状 = 当日残差剖面按当日残差之和归一（形状向量 w_{d,t}，Σ_t w=1）。
生成时先由 Copula 抽出 (P, R) 水平对，再在同水平的近邻历史日中抽一个形状贴上，
从而**同时保真水平边际分布、水平间的相依结构与日内形状**。
"""
from __future__ import annotations

import numpy as np
import pandas as pd

import config as C
import copula as CP
import lp_engine as LP

DATE_ALL = pd.date_range(C.WARMUP_START, C.WIN_END, freq="D")


# ---------------------------------------------------------------------------
# 日水平与日内形状
# ---------------------------------------------------------------------------
def daily_levels(panel, dates):
    """返回 dict(price (n,144), resid (n,144), P (n,), R (n,), load (n,144), pv (n,144))。"""
    p4 = panel.P(volatile=True)
    pv_act = panel.PV()
    load = panel.L()
    idx = [list(DATE_ALL).index(d) for d in dates]
    price = p4.to_numpy(float)[idx]
    pv = pv_act.to_numpy(float)[idx]
    Ld = load.to_numpy(float)[idx]
    resid = np.zeros_like(price)
    for i, d in enumerate(dates):
        fc = panel.pv_forecast(d, 0)
        resid[i] = (fc - pv[i]) * C.DT
    return dict(price=price, resid=resid, pv=pv, load=Ld,
                P=price.mean(1), R=resid.sum(1))
```

```python


def _shapes(price, resid):
    """电价形状（去日均值）与残差形状（按日总和归一，带下限防除零）。"""
    ps = price - price.mean(1, keepdims=True)
    R = resid.sum(1)
    scale = np.where(np.abs(R) < 1.0, 1.0, R)
    rs = resid / scale[:, None]
    return ps, rs, R


# ---------------------------------------------------------------------------
# 水平层 Copula 模型
# ---------------------------------------------------------------------------
class LevelCopula:
    """日水平 (P, R) 的 KDE 边缘 + 二元 pair-Copula。"""

    def __init__(self):
        self.mP = None
        self.mR = None
        self.best = None
        self.cands = []

    def fit(self, P, R):
        P = np.asarray(P, float); R = np.asarray(R, float)
        self.mP = CP.KdeMarginal(P)
        self.mR = CP.KdeMarginal(R)
        u = CP.pseudo_obs(P); v = CP.pseudo_obs(R)
        self.u, self.v = u, v
        for fam in CP.FAMILIES:
            try:
                f = CP.fit(fam, u, v)
            except Exception:
                continue
            f["ecdist"] = CP.empirical_copula_distance(fam, u, v, f["par"])
            f["tail"] = CP.tail_dependence(fam, f["par"])
            f["tau_sim"] = self._tau_sim(fam, f["par"])
            self.cands.append(f)
        self.cands.sort(key=lambda d: d["aic"])
        self.best = self.cands[0]
        return self.best

    def _tau_sim(self, fam, par, n=4000, seed=3):
        rng = np.random.default_rng(seed)
        w = rng.random((n, 2)); u = w[:, 0]
        v = np.array([CP.hinv(fam, w[i, 1], u[i], par) for i in range(n)])
        from scipy import stats
        return float(stats.kendalltau(u, v).statistic)

    def sample_levels(self, n, rng):
        """按拟合的最优 Copula 抽 (P, R) 水平对。"""
        fam = self.best["family"]; par = self.best["par"]
        u = rng.random(n)
        v = np.array([CP.hinv(fam, rng.random(), u[i], par) for i in range(n)])
        return self.mP.ppf(u), self.mR.ppf(v)

    def summary(self) -> dict:
        return dict(best=dict(family=self.best["family"],
                              par=np.asarray(self.best["par"]).tolist(),
                              loglik=self.best["loglik"], aic=self.best["aic"],
                              ecdist=self.best["ecdist"],
                              tail=list(self.best["tail"]),
                              tau_sim=self.best["tau_sim"]),
                    candidates=[dict(family=c["family"],
                                     par=np.asarray(c["par"]).tolist(),
                                     loglik=c["loglik"], aic=c["aic"],
                                     ecdist=c["ecdist"],
                                     tail=list(c["tail"]),
                                     tau_sim=c["tau_sim"]) for c in self.cands])


# ---------------------------------------------------------------------------
# 场景生成
```

```python
# ---------------------------------------------------------------------------
def generate(panel, train_dates, K=60, n_draw=600, seed=7, k_nn=10, reduce="strat"):
    """由训练窗数据拟合 Copula 并生成 K 条联合场景。"""
    lv = daily_levels(panel, train_dates)
    cop = LevelCopula()
    cop.fit(lv["P"], lv["R"])
    ps, rs, R = _shapes(lv["price"], lv["resid"])
    # 水平标准化（用于近邻搜索）
    sdP = lv["P"].std(ddof=1); sdR = lv["R"].std(ddof=1)
    zP = (lv["P"] - lv["P"].mean()) / sdP
    zR = (lv["R"] - lv["R"].mean()) / sdR
    rng = np.random.default_rng(seed)
    Ps, Rs = cop.sample_levels(n_draw, rng)
    zPs = (Ps - lv["P"].mean()) / sdP
    zRs = (Rs - lv["R"].mean()) / sdR
    price_s = np.zeros((n_draw, C.N_SLOT))
    resid_s = np.zeros((n_draw, C.N_SLOT))
    for i in range(n_draw):
        dist = (zPs[i] - zP) ** 2 + (zRs[i] - zR) ** 2
        near = np.argsort(dist)[:k_nn]
        j = int(rng.choice(near))
        price_s[i] = ps[j] + Ps[i]                      # 形状 + 抽到的日水平
        resid_s[i] = rs[j] * Rs[i]                      # 归一形状 × 抽到的日总残差
    if reduce == "none":
        cen_p, cen_r, w = price_s, resid_s, np.full(n_draw, 1.0 / n_draw)
    elif reduce == "strat":
        cen_r, w = LP.reduce_scenarios_stratified(resid_s, K=K)
        # 代表行的电价随之取出（stratified 返回的是代表行本身）
        cen_p = _match_rows(resid_s, cen_r, price_s)
    elif reduce == "kmeans":
        cen_r, w = LP.reduce_scenarios(resid_s, K=K, seed=seed)
        cen_p = _match_rows(resid_s, cen_r, price_s)
    else:
        raise ValueError(reduce)
    return dict(price=cen_p, resid=cen_r, w=np.asarray(w, float) / np.sum(w),
                net=None, src=None, kind="fixed", copula=cop)


def _match_rows(src, cen, val):
    """把代表场景（cen 行）映射回原始行下标，取对应的价格行。"""
    out = np.zeros((cen.shape[0], val.shape[1]))
    for k in range(cen.shape[0]):
        d = ((src - cen[k]) ** 2).sum(1)
        out[k] = val[int(np.argmin(d))]
    return out


# ---------------------------------------------------------------------------
# 保真度度量（水平 + 形状 + 相依）
# ---------------------------------------------------------------------------
def wquant(x, w, q):
    """加权分位数（q 为 0-100 的百分位数组）。"""
    x = np.asarray(x, float); w = np.asarray(w, float)
    o = np.argsort(x); x, w = x[o], w[o]
    cw = (np.cumsum(w) - 0.5 * w) / np.sum(w)
    return np.interp(np.asarray(q, float) / 100.0, cw, x)


def generate_relative(panel, train_dates, K=60):
    """相对残差情景集：r_t / (当日光伏预报电量)，供"按当日规模缩放"的条件情景使用。"""
    lv = daily_levels(panel, train_dates)
    fc = np.vstack([panel.pv_forecast(d, 0) for d in train_dates])
    pvfc = fc.sum(1) * C.DT
    rel = lv["resid"] / pvfc[:, None]
    cen, w = LP.reduce_scenarios_stratified(rel, K=K)
    return dict(rel=cen, w=w,
                rel_mu=float((cen * w[:, None]).sum()),
                rel_sd=float(np.sqrt((w[:, None] * (cen - (cen * w[:, None]).sum(0)) ** 2).sum())))


def fidelity(scen, panel, dates):
    """与真实分布的对比：水平边际分位差、Kendall tau、联合尾共现率。"""
    from scipy import stats
```

```
    lv = daily_levels(panel, dates)
    P, R = lv["P"], lv["R"]
    Ps = scen["price"].mean(1)
    Rs = scen["resid"].sum(1)
    w = scen["w"] / scen["w"].sum()
    qs = [1, 5, 25, 50, 75, 95, 99]
    out = {}
    for tag, a, b in (("P", P, Ps), ("R", R, Rs)):
        qa = np.percentile(a, qs)
        qb = wquant(b, w, qs)
        out[f"{tag}_q_true"] = qa.tolist()
        out[f"{tag}_q_scen"] = qb.tolist()
        out[f"{tag}_maxq_gap"] = float(np.max(np.abs(qb - qa) / np.maximum(np.abs(qa), 1e-9)))
        out[f"{tag}_absq_gap"] = float(np.max(np.abs(qb - qa)))
    out["tau_true_PR"] = float(stats.kendalltau(P, R).statistic)
    out["tau_scen_PR"] = float(_wtau(Ps, Rs, w))
    # 联合尾共现（P 高 & R 高 = 高价叠加光伏高估；P 低 & R 低 = 低价叠加光伏低估）
    for tag, fn in (("hi_hi", lambda a, b, q: (a > np.quantile(a, q)) & (b > np.quantile(b, q))),
                    ("lo_lo", lambda a, b, q: (a < np.quantile(a, q)) & (b < np.quantile(b, q)))):
        q = 0.75 if tag == "hi_hi" else 0.25
        out[f"tail_true_{tag}"] = float(np.mean(fn(P, R, q)))
        out[f"tail_scen_{tag}"] = float(_wmean_abs(fn(Ps, Rs, q).astype(float), w))
    return out


def _wtau(x, y, w):
    """加权 Kendall tau（成对符号法，n<=200 时可接受）。"""
    x = np.asarray(x, float); y = np.asarray(y, float); w = np.asarray(w, float)
    n = len(x)
    if n > 400:
        idx = np.argsort(-w)[:400]
        x, y, w = x[idx], y[idx], w[idx]
        n = len(x)
    num = den = 0.0
    for i in range(n):
        s = np.sign(x[i] - x) * np.sign(y[i] - y)
        c = w[i] * w
        num += 0.5 * np.sum(c * s)
        den += 0.5 * np.sum(c)
    return float(num / max(den, 1e-12))


def _wmean_abs(flag, w):
    return float(np.sum(flag * w) / np.sum(w))
```

源程序：code/exp_copula.py

```
# -*- coding: utf-8 -*-
"""
v2 实验：电价—负荷—光伏相依结构（C 藤 Copula）与联合情景生成器对照
================================================================
对应参考论文：
  · 宁楠《基于动态 C 藤 pair-Copula 的电力系统动态经济调度》
  · 崔丽敏等《基于核密度估计和 Copula 函数的风-光-荷联合场景生成》
  · 李翔 / 孙丰杰：聚类式场景削减

四部分：
  P1 三变量 C 藤拟合（日均价、日电量负载、日电量光伏）——族选择、Kendall tau、尾相依；
  P2 决策相关水平对（日均价 P，日总预报残差 R）的 KDE + Copula 拟合；
  P3 情景生成器对照实验（样本外）：历史分层 / 分块 Bootstrap / KDE+Copula / 按当日光伏规模缩放，
     统一在 2025-02-01..06-30 训练、2025-07-01..12-31 上按真实轨迹结算；
  P4 分时段 CVaR 风险画像（由 q2_main.json 与 q4_2.json 的逐日解复算）。

输出 intermediate/exp_copula.json
"""
from __future__ import annotations

import json
import os
import time

import numpy as np
import pandas as pd
```

```python
from scipy import stats

import config as C
import copula as CP
import lp_engine as LP
import q4 as Q4
import scen_copula as SC

DATE_ALL = SC.DATE_ALL
TRAIN = pd.date_range("2025-02-01", "2025-06-30", freq="D")
TEST = pd.date_range("2025-07-01", "2025-12-31", freq="D")
K_MAIN = 60


# ---------------------------------------------------------------------------
# P1 三变量 C 藤
# ---------------------------------------------------------------------------
def part1_vine(panel, dates):
    lv = SC.daily_levels(panel, dates)
    X = np.column_stack([lv["price"].mean(1),
                         lv["load"].sum(1) * C.DT,
                         lv["pv"].sum(1) * C.DT])
    vine = CP.CVine3(names=("price", "load", "pv"))
    rows = vine.fit(X, verbose=False)
    P_, L_, V_ = X[:, 0], X[:, 1], X[:, 2]
    plain = {
        "tau_price_load": float(stats.kendalltau(P_, L_).statistic),
        "tau_price_pv": float(stats.kendalltau(P_, V_).statistic),
        "tau_load_pv": float(stats.kendalltau(L_, V_).statistic),
        "pearson_price_load": float(np.corrcoef(P_, L_)[0, 1]),
        "pearson_price_pv": float(np.corrcoef(P_, V_)[0, 1]),
        "pearson_load_pv": float(np.corrcoef(L_, V_)[0, 1]),
        "n_days": int(len(P_)),
    }
    return dict(plain=plain, vine=[{k: (v if not isinstance(v, np.ndarray) else v.tolist())
                                    for k, v in r.items()} for r in rows])


# ---------------------------------------------------------------------------
# P2 水平对 (P, R) 的 KDE + Copula
# ---------------------------------------------------------------------------
def part2_level_copula(panel, dates):
    lv = SC.daily_levels(panel, dates)
    cop = SC.LevelCopula()
    cop.fit(lv["P"], lv["R"])
    s = cop.summary()
    s["n_days"] = int(len(lv["P"]))
    s["tau_emp_PR"] = float(stats.kendalltau(lv["P"], lv["R"]).statistic)
    s["pearson_emp_PR"] = float(np.corrcoef(lv["P"], lv["R"])[0, 1])
    # 槽级：预报残差与当日电价残差形状的相关（去骨架后）
    skel = C.read_attachment1()["price"].to_numpy(float)
    pres = lv["price"] - skel[None, :]
    s["pearson_deshape_presid"] = float(np.corrcoef(
        (pres - pres.mean(1, keepdims=True)).ravel(),
        (lv["resid"] - lv["resid"].mean(1, keepdims=True)).ravel())[0, 1])
    s["pearson_level_presid"] = float(np.corrcoef(pres.mean(1), lv["R"])[0, 1])
    s["tau_level_presid"] = float(stats.kendalltau(pres.mean(1), lv["R"]).statistic)
    # 残差规模与光伏水平的关系（条件标度律）
    pvfc = np.vstack([panel.pv_forecast(d, 0) for d in dates]).sum(1) * C.DT
    s["absR_vs_pvfc_pearson"] = float(np.corrcoef(np.abs(lv["R"]), pvfc)[0, 1])
    s["relR_mean"] = float((lv["R"] / pvfc).mean())
    s["relR_sd"] = float((lv["R"] / pvfc).std(ddof=1))
    q = np.quantile(pvfc, np.linspace(0, 1, 6))
    band = np.clip(np.digitize(pvfc, q[1:-1]), 0, 4)
    s["relR_by_band"] = [dict(band=int(b), n=int((band == b).sum()),
                              pv_median=float(np.median(pvfc[band == b])),
                              absR_mean=float(np.abs(lv["R"][band == b]).mean()),
                              relR_mean=float((np.abs(lv["R"][band == b]) / pvfc[band == b]).mean()))
                         for b in range(5)]
    return s, cop


# ---------------------------------------------------------------------------
```

```python
# P3 情景生成器对照
# ---------------------------------------------------------------------------
def hist_strat_scenarios(panel, train_dates, K=K_MAIN):
    """历史代表日（二维分层 + 层内 medoid），只在训练窗内取代表。"""
    p4 = panel.P(volatile=True)
    idx = [list(DATE_ALL).index(d) for d in train_dates]
    price = p4.to_numpy(float)[idx]
    resid = np.vstack([(panel.pv_forecast(d, 0) - panel.PV().loc[d].to_numpy(float)) * C.DT
                       for d in train_dates])
    np_str = int(np.floor(np.sqrt(K)))
    nd_tot = resid.sum(1); p_avg = price.mean(1)
    q1 = np.quantile(nd_tot, np.linspace(0, 1, np_str + 1))
    q2 = np.quantile(p_avg, np.linspace(0, 1, np_str + 1))
    b1 = np.clip(np.digitize(nd_tot, q1[1:-1]), 0, np_str - 1)
    b2 = np.clip(np.digitize(p_avg, q2[1:-1]), 0, np_str - 1)
    cell = b1 * np_str + b2
    pos, w = [], []
    for cid in range(np_str * np_str):
        g = np.where(cell == cid)[0]
        if len(g) == 0:
            continue
        sub = resid[g]
        if len(g) == 1:
            pos.append(g[0])
        else:
            dist = np.sqrt(((sub[:, None, :] - sub[None, :, :]) ** 2).sum(-1))
            pos.append(g[int(np.argmin(dist.sum(1)))])
        w.append(len(g) / len(nd_tot))
    pos = np.array(pos)
    return dict(price=price[pos], resid=resid[pos], w=np.array(w) / np.sum(w),
                kind="fixed")


def bootstrap_scenarios(panel, train_dates, K=K_MAIN, seed=13, block=36):
    p4 = panel.P(volatile=True)
    idx = [list(DATE_ALL).index(d) for d in train_dates]
    price = p4.to_numpy(float)[idx]
    resid = np.vstack([(panel.pv_forecast(d, 0) - panel.PV().loc[d].to_numpy(float)) * C.DT
                       for d in train_dates])
    rng = np.random.default_rng(seed)
    nb = C.N_SLOT // block
    ps, rs = [], []
    for _ in range(K):
        pr, rr = [], []
        for j in range(nb):
            a = int(rng.integers(0, len(idx))); off = j * block
            pr.append(price[a, off:off + block]); rr.append(resid[a, off:off + block])
        ps.append(np.concatenate(pr)); rs.append(np.concatenate(rr))
    return dict(price=np.array(ps), resid=np.array(rs),
                w=np.full(K, 1.0 / K), kind="fixed")


def eval_policy(scen, panel, test_dates, rel=None, verbose=False):
    """在测试窗上按真实轨迹结算（缺口按 5 倍实际电价紧急购电）。"""
    load = panel.L(); pv_act = panel.PV(); p4 = panel.P(volatile=True)
    s_init = C.S_INIT
    tot = emg = emg_q = base = 0.0
    dc, daily = [], []
    for d in test_dates:
        L = load.loc[d].to_numpy(float); PVa = pv_act.loc[d].to_numpy(float)
        nd_act = (L - PVa) * C.DT
        fc = panel.pv_forecast(d, 0); nd_fcst = (L - fc) * C.DT
        if scen["kind"] == "scaled":
            scale = fc.sum() * C.DT
            resid = rel["rel"] * scale; w = rel["w"]
        else:
            resid = scen["resid"]; w = scen["w"]
        sol = Q4.solve_vol(scen["price"], nd_fcst, nd_fcst[None, :] + resid, w,
                           s_init=s_init, s_from=0, term_floor=s_init)
        g = np.maximum(nd_act + sol["c"] - sol["d"] - sol["E"], 0.0)
        pact = p4.loc[d].to_numpy(float)
        c1 = float(np.sum(pact * sol["E"])); c2 = float(C.EMERGENCY_MULT * np.sum(pact * g))
        tot += c1 + c2; emg += c2; emg_q += float(g.sum())
```

```
        base += C.baseline_cost(pact, L, PVa)
        dc.append(c1 + c2)
        daily.append(dict(date=str(d.date()), cost=c1 + c2, emg_qty=float(g.sum()),
                          plan_energy=float(sol["E"].sum())))
        s_init = float(sol["S"][-1])
    dc = np.array(dc)
    q95 = float(np.quantile(dc, 0.95))
    return dict(n_days=len(test_dates), total_cost=tot, emg_cost=emg, emg_qty=emg_q,
                base_cost=base, saving=float((base - tot) / base),
                cvar95=float(dc[dc >= q95].mean()), worst_day=float(dc.max()),
                mean_day=float(dc.mean()), sd_day=float(dc.std(ddof=1)),
                daily=daily)


def part3_generators(panel):
    out = {}
    t0 = time.time()
    hist = hist_strat_scenarios(panel, TRAIN)
    out["fidelity_hist"] = SC.fidelity(hist, panel, TRAIN)
    out["hist_strat"] = eval_policy(hist, panel, TEST)
    out["hist_strat"].pop("daily")
    print("  hist_strat %.1fs 总费用 %.2f" % (time.time() - t0, out["hist_strat"]["total_cost"]))

    boot = bootstrap_scenarios(panel, TRAIN)
    out["fidelity_boot"] = SC.fidelity(boot, panel, TRAIN)
    out["bootstrap"] = eval_policy(boot, panel, TEST)
    out["bootstrap"].pop("daily")
    print("  bootstrap %.1fs 总费用 %.2f" % (time.time() - t0, out["bootstrap"]["total_cost"]))

    cop = SC.generate(panel, TRAIN, K=K_MAIN, n_draw=600, seed=7, k_nn=10, reduce="strat")
    out["fidelity_copula"] = SC.fidelity(cop, panel, TRAIN)
    out["copula"] = eval_policy(cop, panel, TEST)
    out["copula"].pop("daily")
    print("  copula %.1fs 总费用 %.2f" % (time.time() - t0, out["copula"]["total_cost"]))

    rel = SC.generate_relative(panel, TRAIN, K=len(hist["w"]))
    scaled = dict(price=hist["price"], resid=None, w=hist["w"], kind="scaled")
    out["scaled"] = eval_policy(scaled, panel, TEST, rel=rel)
    out["scaled"].pop("daily")
    out["rel_summary"] = dict(rel_mu=rel["rel_mu"], rel_sd=rel["rel_sd"])
    print("  scaled %.1fs 总费用 %.2f" % (time.time() - t0, out["scaled"]["total_cost"]))

    base = out["hist_strat"]["total_cost"]
    for k in ("bootstrap", "copula", "scaled"):
        out[k]["diff_vs_hist"] = out[k]["total_cost"] - base
        out[k]["diff_pct"] = (out[k]["total_cost"] / base - 1) * 100
    return out


# ---------------------------------------------------------------------------
# P4 分时段 CVaR 风险画像
# ---------------------------------------------------------------------------
def cvar(x, alpha=0.95):
    x = np.asarray(x, float)
    v = np.quantile(x, alpha)
    tail = x[x >= v]
    return float(tail.mean()) if len(tail) else float(v)


def part4_risk(alpha=0.95):
    out = dict(alpha=alpha)
    # (a) 问题 2（固定价）
    j2 = json.load(open(os.path.join(C.INTERMEDIATE_DIR, "q2_main.json")))
    price2 = C.read_attachment1()["price"].to_numpy(float)
    E2 = np.vstack([r["E"] for r in j2["daily"]])
    G2 = np.vstack([r["g"] for r in j2["daily"]])
    cost2 = price2[None, :] * E2 + C.EMERGENCY_MULT * price2[None, :] * G2
    out["q2_slot_cvar"] = [cvar(cost2[:, t], alpha) for t in range(C.N_SLOT)]
    out["q2_slot_mean"] = cost2.mean(0).tolist()
    out["q2_slot_shortage_freq"] = (G2 > 1e-9).mean(0).tolist()
    out["q2_total"] = float(cost2.sum())
    hour = np.arange(C.N_SLOT) // 6
    out["q2_hour_cvar"] = [float(np.sum([out["q2_slot_cvar"][t] for t in range(C.N_SLOT)
```

```
                                        if hour[t] == h])) for h in range(24)]
    out["q2_hour_mean"] = [float(np.sum([out["q2_slot_mean"][t] for t in range(C.N_SLOT)
                                        if hour[t] == h])) for h in range(24)]
    out["q2_hour_shortage_freq"] = [float(np.mean([out["q2_slot_shortage_freq"][t]
                                                   for t in range(C.N_SLOT) if hour[t] == h]))
                                    for h in range(24)]
    # 风险集中度: CVaR 最高的 12 个槽位占总 CVaR 的比重
    sc = np.array(out["q2_slot_cvar"])
    o = np.argsort(-sc)
    out["q2_top12_slot_share"] = float(sc[o[:12]].sum() / sc.sum())
    out["q2_top12_slots"] = [C.slot_label(int(t)) for t in sorted(o[:12])]

    # (b) 问题 4-2（波动价）
    j4 = json.load(open(os.path.join(C.INTERMEDIATE_DIR, "q4_2.json")))
    p4 = C.Panel(DATE_ALL).P(volatile=True)
    dates = [pd.Timestamp(r["date"]) for r in j4["daily"]]
    Pact = p4.loc[dates].to_numpy(float)
    E4 = np.vstack([r["E"] for r in j4["daily"]])
    G4 = np.vstack([r["g"] for r in j4["daily"]])
    cost4 = Pact * E4 + C.EMERGENCY_MULT * Pact * G4
    out["q4_slot_cvar"] = [cvar(cost4[:, t], alpha) for t in range(C.N_SLOT)]
    out["q4_slot_mean"] = cost4.mean(0).tolist()
    out["q4_slot_shortage_freq"] = (G4 > 1e-9).mean(0).tolist()
    out["q4_total"] = float(cost4.sum())
    out["q4_hour_cvar"] = [float(np.sum([out["q4_slot_cvar"][t] for t in range(C.N_SLOT)
                                        if hour[t] == h])) for h in range(24)]
    out["q4_hour_mean"] = [float(np.sum([out["q4_slot_mean"][t] for t in range(C.N_SLOT)
                                        if hour[t] == h])) for h in range(24)]
    out["q4_hour_shortage_freq"] = [float(np.mean([out["q4_slot_shortage_freq"][t]
                                                   for t in range(C.N_SLOT) if hour[t] == h]))
                                    for h in range(24)]
    sc4 = np.array(out["q4_slot_cvar"]); o4 = np.argsort(-sc4)
    out["q4_top12_slot_share"] = float(sc4[o4[:12]].sum() / sc4.sum())
    out["q4_top12_slots"] = [C.slot_label(int(t)) for t in sorted(o4[:12])]
    # 风险比: CVaR/均值（尾部放大倍数）
    out["q2_cvar_over_mean"] = float(np.sum(out["q2_slot_cvar"]) / np.sum(out["q2_slot_mean"]))
    out["q4_cvar_over_mean"] = float(np.sum(out["q4_slot_cvar"]) / np.sum(out["q4_slot_mean"]))
    return out


def main():
    import sys
    parts = sys.argv[1] if len(sys.argv) > 1 else "1234"
    panel = C.Panel(DATE_ALL)
    prev = {}
    jpath = os.path.join(C.INTERMEDIATE_DIR, "exp_copula.json")
    if os.path.exists(jpath):
        prev = json.load(open(jpath))
    p1 = prev.get("part1_vine", None); p1_train = prev.get("part1_vine_train", None)
    p2 = prev.get("part2_level", None); p3 = prev.get("part3_generators", None)
    p4 = prev.get("part4_risk", None)
    WINDOW = pd.date_range(C.WIN_START, C.WIN_END, freq="D")
    if "1" in parts:
        print("[P1] 三变量 C 藤拟合（结果窗口 %d 天 / 训练窗 %d 天）..." % (len(WINDOW), len(TRAIN)))
        p1 = part1_vine(panel, WINDOW)
        p1_train = part1_vine(panel, TRAIN)
    if "2" in parts:
        print("[P2] 水平对 KDE + Copula 拟合 ...")
        p2, _ = part2_level_copula(panel, TRAIN)
    if "3" in parts:
        print("[P3] 情景生成器对照（训练 %s..%s，测试 %s..%s）..."
              % (TRAIN[0].date(), TRAIN[-1].date(), TEST[0].date(), TEST[-1].date()))
        p3 = part3_generators(panel)
    if "4" in parts:
        print("[P4] 分时段 CVaR 风险画像 ...")
        p4 = part4_risk()
    out = dict(train=[str(d.date()) for d in TRAIN], test=[str(d.date()) for d in TEST],
               K=K_MAIN, part1_vine=p1, part1_vine_train=p1_train, part2_level=p2,
               part3_generators=p3, part4_risk=p4)
    path = os.path.join(C.INTERMEDIATE_DIR, "exp_copula.json")
    with open(path, "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    print("已写出", path)
```

```
    print(json.dumps(dict(
        tau_price_load=p1["plain"]["tau_price_load"],
        tau_price_pv=p1["plain"]["tau_price_pv"],
        tau_price_resid=p2["tau_emp_PR"],
        best_level_family=p2["best"]["family"],
        gen={k: round(v["total_cost"], 2) for k, v in p3.items() if isinstance(v, dict) and "total_cost" in v},
        q2_cvar_over_mean=round(p4["q2_cvar_over_mean"], 4),
        q4_cvar_over_mean=round(p4["q4_cvar_over_mean"], 4)), ensure_ascii=False, indent=1))


if __name__ == "__main__":
    main()
```

源程序：code/exp_chance.py

```
# -*- coding: utf-8 -*-
"""
机会约束对照实验（问题 2，固定电价）
==================================
目的：在问题 2 的两阶段随机规划主方案（q2.run_q2 + lp_engine.solve_day，K=60 分层情景）
      之外，增加**机会约束**口径的对照，并给出"解析（报童临界分位数）—数值（实测缺电频率）"
      互证表。

机会约束
--------
对每个槽位 t 施加
        P( E_t - c_t + d_t >= nd^act_t ) >= 1 - beta ,          (CC)
其中 nd^act_t 为实际净需求（(L^act - PV^act) * DT，kWh），E_t 为计划购电量，
c_t / d_t 为充/放电量，故 E_t - c_t + d_t 是"计划口径的可用供给"。

情景离散化形式（本问题实际使用的形式）：
        sum_{w : nd^act_{w,t} + c_t - d_t - E_t > 0} pi_w <= beta ,   (CC-D)
即"缺电情景的权重之和不超过 beta"。注意 (CC-D) 的约束左端**依赖决策**
（哪些情景缺电由 E_t - c_t + d_t 决定），因此不是线性约束。

**保守确定性等价（本脚本采用）**：把 (CC-D) 的满足条件加强为逐点满足
        E_t - c_t + d_t >= q_{1-beta}( nd^act_{.,t} ) ,          (CC-Q)
其中 q 是情景净需求的（加权）经验分位数。因为若供给不低于 1-beta 分位数，
则"缺电情景"必落在权重不超过 beta 的上尾，故 (CC-Q) => (CC-D)，即 (CC-Q) 是
(CC-D) 的**充分条件**（保守近似）。这是机会约束最常用的"分位数/分位点法"确定性等价，
其代价是忽略了决策可通过储能跨时段对冲而降低缺电概率的事实（论文中作为该近似的
适用边界写明）。实现上等价于把 lp_engine.solve_day 的情景集换成"分位数对应的
确定性需求"（K=1）：nd_scen = nd_fcst + q_{1-beta}(r)，其中 r 为预报残差情景。

滚动口径与 q2 主方案完全一致（假设 A5/A6/A8/R3）：
  日期序列 = 2025-01-01..12-31（1 月为预热期，跨日 SOC 连续递推），
  结果窗口 = 2025-02-01..12-31 共 334 天；
  每个自然日 0:00 用附件 3 的 0:00 光伏预报（+ 已知负载曲线）制定计划，
  用附件 2 的实际负载/光伏结算，缺口按 5 倍电价紧急购电；
  终端条件 S_144 >= S_0，不允许售电。

报童对照
--------
单槽报童模型的临界分位数 F*(m) = c_u / (c_u + c_o)，最优允许缺货概率 = 1 - F* = c_o/(c_u+c_o)。
  c_u = 4 p_t（紧急购电 5p_t 中本应付的 p_t 已计入计划费，溢价 4p_t）；
  c_o = p_t（多买即弃：超购部分只能弃置）-> 允许缺货概率恒为 1/5 = 0.20；
  c_o ~ 0.37 元/kWh（超购电量可转存电池、在后续低价时段避免高价购电的"残值"）-> 约 0.10。
本脚本把这两个（以及第三种 c_o = p_t - p_min 的口径）理论值与 beta 扫描的**实测缺电频率**
画在同一张对照表里写入 JSON。

输出：intermediate/exp_chance.json

运行：cd C题论文/code && python3 exp_chance.py              （约 1-2 分钟）
"""
from __future__ import annotations

import argparse
import json
import os
import platform
import time

```

```python
import numpy as np
import pandas as pd
import scipy

import config as C
import lp_engine as LP
import q2 as Q2

BETAS = [0.01, 0.05, 0.10, 0.20, 0.30, 0.50]
K_MAIN = 60                      # 与问题 2 主方案一致的情景数（分层削减）
P_MIN_A1 = 0.3713                # 附件 1 电价最小值（元/kWh），第三口径 c_o 用
CO_REF = 0.37                    # 交底书 §2.5 的 c_o 参考值（元/kWh）
TOL_SHORT = 1e-6                 # 判定"该槽缺电"的阈值（kWh）
ALPHA = 0.95                     # CVaR 置信水平（C-11）


# ---------------------------------------------------------------------------
# 加权经验分位数
# ---------------------------------------------------------------------------
def weighted_quantile(samples, weights, q):
    """逐列的加权经验分位数（逆 CDF 定义：最小的取值 v 使累计权重 >= q）。

    samples : (K, n) 或 (n,)，weights : (K,)
    q       : 标量，分位水平（本脚本用 1 - beta）
    """
    s = np.atleast_2d(np.asarray(samples, float))
    K, n = s.shape
    w = np.asarray(weights, float).ravel()
    w = w / w.sum()
    order = np.argsort(s, axis=0, kind="stable")
    srt = np.take_along_axis(s, order, axis=0)
    cw = np.cumsum(w[order], axis=0)
    hit = cw >= q
    idx = np.argmax(hit, axis=0)              # 首个为 True 的位置
    none = ~hit.any(axis=0)
    idx = np.where(none, K - 1, idx)
    return srt[idx, np.arange(n)]


# ---------------------------------------------------------------------------
# 数据准备（与 q2 主方案同一口径）
# ---------------------------------------------------------------------------
def prepare_days(panel, price):
    """预取每天的实际净需求 / 0:00 预报净需求 / 无储能基线费用。"""
    load, pv_act = panel.L(), panel.PV()
    days = []
    for d in Q2.DATE_ALL:
        Ld = load.loc[d].to_numpy(float)
        PVa = pv_act.loc[d].to_numpy(float)
        nd_act = (Ld - PVa) * C.DT
        nd_fcst = (Ld - panel.pv_forecast(d, 0)) * C.DT
        days.append(dict(date=d, nd_act=nd_act, nd_fcst=nd_fcst,
                         base_cost=C.baseline_cost(price, Ld, PVa),
                         in_win=bool(d in Q2.DATE_WINDOW)))
    return days


def rollout(days, price, margin, term_mode="ge", label=""):
    """按分位数安全边际 margin（(n,) kWh）逐日滚动求解并结算。

    等价于 lp_engine.solve_day(price, nd_fcst, nd_fcst + margin, K=1)：
    即"保守确定性等价"下的机会约束计划，随后用实际值结算紧急购电。
    """
    recs, s_init, t0 = [], C.S_INIT, time.time()
    max_viol = 0.0
    for i, dd in enumerate(days):
        nd_fcst, nd_act = dd["nd_fcst"], dd["nd_act"]
        nd_q = nd_fcst + margin
        sol = LP.solve_day(price, nd_fcst, nd_q[None, :], np.array([1.0]),
                           s_init=s_init, term_mode=term_mode)
        g, pc, ec = LP.realized_slot(sol, nd_act)
        chk = LP.check_solution(sol, nd_fcst, nd_act)
        max_viol = max(max_viol, chk["R1_plan_violation"], chk["R2_power_violation"],
```

```
                        chk["R3_soc_bounds"], chk["R4_soc_recursion"])
        if dd["in_win"]:
            recs.append(dict(date=dd["date"], g=g, plan_cost=pc, emg_cost=ec,
                             total_cost=pc + ec, emg_qty=float(g.sum()),
                             base_cost=dd["base_cost"], energy=float(sol["E"].sum()),
                             margin_qty=float(margin.sum())))
        s_init = float(sol["S"][-1])
    if label:
        print(f"  [{label}] {len(days)} 天滚动 {time.time()-t0:.1f}s  "
              f"最大可行性残差 {max_viol:.2e}")
    return recs, dict(max_violation=max_viol, seconds=time.time() - t0)


# ---------------------------------------------------------------------------
# 指标汇总
# ---------------------------------------------------------------------------
def cvar_ru(daily_costs, alpha=ALPHA):
    """逐日费用的（经验）CVaR: Rockafellar-Uryasev 估计量。

    CVaR_alpha = VaR_alpha + E[(C - VaR_alpha)^+] / (1 - alpha)，
    VaR 取 334 个日费用的经验分位数（numpy 默认线性插值）。
    另给出"最差 ceil((1-alpha)N) 天均值"作为离散口径的交叉核对。
    """
    c = np.sort(np.asarray(daily_costs, float))
    N = len(c)
    var = float(np.quantile(c, alpha))
    cvar = var + float(np.mean(np.maximum(c - var, 0.0))) / (1.0 - alpha)
    k = int(np.ceil((1.0 - alpha) * N))
    return dict(var95=var, cvar95=cvar, n_tail=k,
                worst_k_mean=float(c[-k:].mean()), worst_day=float(c[-1]))


def summarize(recs):
    """把逐日记录汇总成论文可用指标（口径 C-01/C-03/C-04/C-05/C-11）。"""
    daily = np.array([r["total_cost"] for r in recs], float)
    day_c = np.array([r["total_cost"] for r in recs], float)
    plan = float(sum(r["plan_cost"] for r in recs))
    emg = float(sum(r["emg_cost"] for r in recs))
    base = float(sum(r["base_cost"] for r in recs))
    G = np.vstack([r["g"] for r in recs])
    nd_qty = float(sum(r.get("nd_qty", 0.0) for r in recs))
    n_days, n_slot = G.shape
    tot_emg_qty = float(G.sum())
    short = G > TOL_SHORT
    out = dict(
        n_days=int(n_days),
        tot_plan=plan, tot_emg=emg, tot_cost=plan + emg, tot_base=base,
        saving=base - (plan + emg),
        saving_rate=(base - (plan + emg)) / base,
        tot_emg_qty=tot_emg_qty,
        days_with_emg=int(np.sum(G.sum(axis=1) > TOL_SHORT)),
        shortfall_slots=int(short.sum()),
        shortfall_slot_freq=float(short.mean()),
        mean_daily_cost=float(daily.mean()),
        **cvar_ru(day_c),
    )
    if nd_qty > 0:
        out["served_rate"] = 1.0 - tot_emg_qty / nd_qty
    return out


def attach_demand_stats(recs, days):
    """把逐日实际净需求合计挂到记录上（用于电量满足率）。"""
    dem = {dd["date"]: float(dd["nd_act"].sum()) for dd in days if dd["in_win"]}
    for r in recs:
        r["nd_qty"] = dem[r["date"]]
    return recs


# ---------------------------------------------------------------------------
# 报童理论临界分位数
# ---------------------------------------------------------------------------
def newsboy(price):
```

```
    """三种 c_o 口径下的理论最优允许缺货概率 = c_o / (c_u + c_o)，c_u = 4 p_t。"""
    cu = 4.0 * price
    coA = price.copy()                      # 多买即弃
    coB = np.full_like(price, CO_REF)       # 残值 0.37 元/kWh
    coC = np.maximum(price - P_MIN_A1, 0.0)   # 相对最低价的机会成本
    pbar = float(price.mean())
    def stat(co):
        b = co / (cu + co)
        cobar = float(np.mean(co))
        return dict(mean=float(b.mean()), min=float(b.min()), max=float(b.max()),
                    p05=float(np.quantile(b, 0.05)), p95=float(np.quantile(b, 0.95)),
                    at_mean_price=float(cobar / (4.0 * pbar + cobar)))
    bA, bB, bC = stat(coA), stat(coB), stat(coC)
    return dict(
        c_u="4*p_t（紧急购电 5p_t 扣掉本应付的 p_t)",
        p_mean=pbar,
        jensen_note="beta*(p)=c_o/(4p+c_o) 关于 p 凸，故逐槽均值 > 用均价 p̄ 代入的值（Jensen）",
        calibers=[
            dict(name="c_o = p_t（多买即弃）", c_o="p_t", beta_star=bA,
                 beta_star_scalar=0.2, critical_fractile=0.8,
                 note="恒等于 1/5，与 p 无关"),
            dict(name="c_o = 0.37 元/kWh（交底书 §2.5 残值口径）",
                 c_o=f"{CO_REF}", beta_star=bB, note="逐槽值，随 p_t 变化"),
            dict(name=f"c_o = p_t - p_min（p_min = {P_MIN_A1} 元/kWh）",
                 c_o="p_t - p_min", beta_star=bC, note="最低价时段机会成本为 0"),
        ])


# ---------------------------------------------------------------------------
# 主流程
# ---------------------------------------------------------------------------
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--K", type=int, default=K_MAIN, help="情景削减数（主口径 60）")
    ap.add_argument("--betas", default=",".join(str(b) for b in BETAS))
    ap.add_argument("--no-ref", action="store_true", help="跳过两阶段随机规划基准复算")
    ap.add_argument("--tag", default="exp_chance")
    args = ap.parse_args()
    betas = [float(x) for x in args.betas.split(",")]

    t_start = time.time()
    panel = C.Panel(Q2.DATE_ALL)
    price = C.read_attachment1()["price"].to_numpy(float)
    days = prepare_days(panel, price)
    print(f"日期序列 {len(days)} 天（含预热期），结果窗口 {sum(d['in_win'] for d in days)} 天")

    # ---- 情景集：与 q2 主方案一致（334 天预报残差的分层削减）----
    R = Q2.build_residuals(panel, Q2.DATE_WINDOW)                    # (334,144) kWh
    cen, wsc = LP.reduce_scenarios_stratified(R, K=args.K)
    print(f"情景削减：K={len(wsc)}  权重和={wsc.sum():.6f}  "
          f"残差均值 {cen.mean():.2f} kWh/槽  std {cen.std():.2f}")

    def margin_of(beta, source):
        if source == "scen":
            return weighted_quantile(cen, wsc, 1.0 - beta)
        return weighted_quantile(R, np.full(len(R), 1.0 / len(R)), 1.0 - beta)

    base_total = float(sum(d["base_cost"] for d in days if d["in_win"]))

    # ---- (0) 两阶段随机规划基准复算（同一代码路径，用于横向对照）----
    ref = None
    if not args.no_ref:
        t0 = time.time()
        recs, scen, _ = Q2.run_q2(K=args.K, term_mode="ge", verbose=False, info="R3")
        ref = summarize(attach_demand_stats(recs, days))
        ref["seconds"] = time.time() - t0
        ref["model"] = (f"两阶段随机规划（q2 主方案）：K={args.K} 分层情景 + 期望紧急购电，"
                        f"Kx144 追索变量")
        print(f"  [SP 基准 K={args.K}] 总费用 {ref['tot_cost']:,.2f} 元  "
              f"缺电槽位频率 {ref['shortfall_slot_freq']*100:.2f}%  ({ref['seconds']:.1f}s)")

    # ---- (1) beta 扫描（主口径：情景集加权分位数）----
    chance = []
```

```
    for beta in betas:
        mg = margin_of(beta, "scen")
        recs, info = rollout(days, price, mg, label=f"beta={beta}")
        attach_demand_stats(recs, days)
        s = summarize(recs)
        s.update(beta=beta, quantile=1.0 - beta, margin_source="scen",
                 margin_mean=float(mg.mean()), margin_max=float(mg.max()),
                 margin_sum=float(mg.sum()), seconds=info["seconds"],
                 max_violation=info["max_violation"])
        if ref is not None:
            s["d_cost_vs_sp"] = s["tot_cost"] - ref["tot_cost"]
            s["rel_cost_vs_sp"] = (s["tot_cost"] - ref["tot_cost"]) / ref["tot_cost"]
            s["d_freq_vs_sp"] = s["shortfall_slot_freq"] - ref["shortfall_slot_freq"]
        chance.append(s)
        print(f"beta={beta:4.2f}  总费用 {s['tot_cost']:14,.2f}  紧急 {s['tot_emg_qty']:10,.1f} kWh  "
              f"缺电频率 {s['shortfall_slot_freq']*100:5.2f}%  节省率 {s['saving_rate']*100:5.2f}%  "
              f"CVaR95 {s['cvar95']:10,.2f}")

    # ---- (2) 稳健口径: 用全部 334 个历史残差样本的经验分位数 ----
    robust = []
    for beta in betas:
        mg = margin_of(beta, "full")
        recs, info = rollout(days, price, mg, label=f"full beta={beta}")
        attach_demand_stats(recs, days)
        s = summarize(recs)
        s.update(beta=beta, quantile=1.0 - beta, margin_source="full334",
                 margin_mean=float(mg.mean()), margin_max=float(mg.max()),
                 margin_sum=float(mg.sum()), seconds=info["seconds"],
                 max_violation=info["max_violation"])
        robust.append(s)
        print(f"  [全样本分位数] beta={beta:4.2f}  总费用 {s['tot_cost']:14,.2f}  "
              f"缺电频率 {s['shortfall_slot_freq']*100:5.2f}%")

    # ---- (3) 报童对照表 ----
    nb = newsboy(price)
    measured = [dict(beta=s["beta"], quantile=s["quantile"],
                     shortfall_slot_freq=s["shortfall_slot_freq"],
                     tot_cost=s["tot_cost"], tot_plan=s["tot_plan"], tot_emg=s["tot_emg"],
                     tot_emg_qty=s["tot_emg_qty"], saving_rate=s["saving_rate"],
                     cvar95=s["cvar95"], served_rate=s["served_rate"],
                     margin_mean=s["margin_mean"],
                     d_cost_vs_sp=s.get("d_cost_vs_sp"),
                     rel_cost_vs_sp=s.get("rel_cost_vs_sp"),
                     d_freq_vs_sp=s.get("d_freq_vs_sp")) for s in chance]
    if ref is not None:
        measured.insert(0, dict(beta=None, model="两阶段SP(K情景期望)",
                                 shortfall_slot_freq=ref["shortfall_slot_freq"],
                                 tot_cost=ref["tot_cost"], tot_emg_qty=ref["tot_emg_qty"],
                                 saving_rate=ref["saving_rate"], cvar95=ref["cvar95"]))
    nb["measured"] = measured
    best = min(chance, key=lambda s: s["tot_cost"])
    nb["conclusion"] = (
        "报童临界值给出两条理论界: 最优允许缺货概率 = 1/5 = 0.2000 (c_o = p_t, 多买即弃) "
        f"与 {nb['calibers'][1]['beta_star']['at_mean_price']:.4f} (c_o = 0.37 元/kWh, 用均价 "
        f"{nb['p_mean']:.4f} 元/kWh 代入; 逐槽算术均值 {nb['calibers'][1]['beta_star']['mean']:.4f}, "
        "两者之差来自 beta*(p) 的凸性/Jensen)。实测逐槽缺电频率随 beta 单调上升且始终低于 beta, "
        "说明分位数确定性等价在储能可跨时段对冲时是保守的。")
    nb["comparison_note"] = (
        "理论值 beta* 是【最优允许缺货概率】; 实测列是 beta 扫描得到的逐槽缺电频率"
        "(= 334 天 x 144 槽中 g_t > 1e-6 的占比)。机会约束口径下 beta 是**约束参数**, "
        "实测频率低于 beta 说明分位数法在储能可跨时段对冲时偏保守。")

    # ---- (4) 交叉核对: 与已有 q2 主结果比对 ----
    xcheck = {}
    q2path = os.path.join(C.INTERMEDIATE_DIR, "q2_main.json")
    if ref is not None and os.path.exists(q2path):
        m = json.load(open(q2path))["meta"]
        xcheck = dict(q2_main_json=dict(tot_cost=m["tot_cost"], tot_emg_qty=m["tot_emg_qty"],
                                        tot_base=m["tot_base"], n_days=m["n_days"]),
                      recomputed=dict(tot_cost=ref["tot_cost"], tot_emg_qty=ref["tot_emg_qty"],
                                      tot_base=ref["tot_base"], n_days=ref["n_days"]),
                      d_tot_cost=ref["tot_cost"] - m["tot_cost"],
                      rel_d_tot_cost=(ref["tot_cost"] - m["tot_cost"]) / m["tot_cost"])
```

```
350
351     out = dict(
352         meta=dict(
353             script="exp_chance.py", problem="问题 2（固定电价）", tag=args.tag,
354             window=f"{C.WIN_START}..{C.WIN_END}", n_days=334,
355             warmup=f"{C.WARMUP_START}..2025-01-31 为预热期（跨日 SOC 连续递推）",
356             n_slot=C.N_SLOT, dt=C.DT, e_max=C.E_MAX, emergency_mult=C.EMERGENCY_MULT,
357             term_mode="ge (S_144 >= S_0)", info="R3（0:00 附件 3 预报 + 已知负载）",
358             K=len(wsc), betas=betas,
359             runtime_s=time.time() - t_start,
360             env=dict(python=platform.python_version(), numpy=np.__version__,
361                      scipy=scipy.__version__, pandas=pd.__version__),
362             reproduce="cd C题论文/code && python3 exp_chance.py",
363         ),
364         caliber=dict(
365             chance_constraint="P(E_t - c_t + d_t >= nd^act_t) >= 1 - beta",
366             discrete_form="sum_{w: nd^act_{w,t}+c_t-d_t-E_t>0} pi_w <= beta",
367             deterministic_equivalent="E_t - c_t + d_t >= q_{1-beta}(nd^act_{.,t})（充分条件，保守）",
368             margin_def="margin_t = q_{1-beta}(r_{.,t})，r=(PV^fcst_0:00 - PV^act)*DT；nd_scen = nd_fcst
     + margin（K=1）",
369             cvar=f"逐日总费用的经验 CVaR，alpha={ALPHA}，Rockafellar-Uryasev 估计量",
370             shortfall_slot_freq="334 天 x 144 槽中 g_t > 1e-6 的槽位占比",
371             served_rate="1 - 紧急购电量 / 实际净需求合计（含负值槽位）",
372         ),
373         baseline=dict(tot_base=base_total, note="C-01 无储能基线（固定价，334 天）"),
374         reference_sp=ref,
375         cross_check=xcheck,
376         chance=chance,
377         robust_full_sample=robust,
378         newsboy=nb,
379         summary=dict(
380             best_beta=best["beta"], best_tot_cost=best["tot_cost"],
381             best_saving_rate=best["saving_rate"],
382             best_shortfall_freq=best["shortfall_slot_freq"],
383             best_cvar95=best["cvar95"], best_margin_mean=best["margin_mean"],
384             sp_tot_cost=(ref["tot_cost"] if ref is not None else None),
385             best_d_cost_vs_sp=best.get("d_cost_vs_sp"),
386             best_rel_cost_vs_sp=best.get("rel_cost_vs_sp"),
387             sp_shortfall_freq=(ref["shortfall_slot_freq"] if ref is not None else None),
388             newsboy_beta_star_co_eq_p=0.2,
389             newsboy_beta_star_co_037_at_mean_price=nb["calibers"][1]["beta_star"]["at_mean_price"],
390             newsboy_beta_star_co_037_slot_mean=nb["calibers"][1]["beta_star"]["mean"],
391             freq_by_beta={str(s["beta"]): s["shortfall_slot_freq"] for s in chance},
392             cost_by_beta={str(s["beta"]): s["tot_cost"] for s in chance},
393         ),
394         notes=[
395             "(CC-Q) 是 (CC-D) 的充分条件：分位数以上尾权重不超过 beta，故 E>=q_{1-beta} 时缺电情景权重和
     <= beta。",
396             "该确定性等价忽略了储能跨时段对冲与多时段耦合，故实测缺电频率系统性低于 beta（见 chance 表）。"
     ,
397             "beta 越小安全边际越大，计划购电费上升、紧急购电费下降；总费用最小值出现在中间 beta 处。",
398             "beta=0.50 时逐槽 0.5 分位数已低于 0:00 预报需求（margin_mean = -3.33 kWh），"
399             "机会约束弱于计划可行性约束 E_t - c_t + d_t >= nd^fcst_t，模型不再额外设置安全边际；"
400             "此时总费用回升是因为紧急购电量大幅增加。",
401             "总费用关于 beta 呈 U 形，最低点 beta=0.10；相对两阶段随机规划主方案（情景期望口径）"
402             "仍高 74 457 元（+0.52%），但缺电频率由 8.55% 降至 6.21% —— 这是"
403             "机会约束保守确定性等价与期望值随机规划之间的取舍，论文中应如实并列。",
404             "所有数字由本脚本一次运行产生，未引用任何外部数值。",
405         ],
406     )
407     path = os.path.join(C.INTERMEDIATE_DIR, f"{args.tag}.json")
408     with open(path, "w") as f:
409         json.dump(out, f, ensure_ascii=False, indent=1)
410     print("=" * 78)
411     print(f"无储能基线 C-01（334 天，固定价） {base_total:,.2f} 元")
412     if ref is not None:
413         print(f"两阶段 SP（K={len(wsc)} 情景期望）     {ref['tot_cost']:,.2f} 元  "
414               f"节省率 {ref['saving_rate']*100:.2f}%  缺电频率 {ref['shortfall_slot_freq']*100:.2f}%")
415     best = min(chance, key=lambda s: s["tot_cost"])
416     print(f"机会约束最优 beta={best['beta']}         {best['tot_cost']:,.2f} 元  "
417           f"节省率 {best['saving_rate']*100:.2f}%  缺电频率 {best['shortfall_slot_freq']*100:.2f}%")
418     print(f"报童理论允许缺货概率：c_o=p_t -> 20.00% ；c_o=0.37 -> "
419           f"{nb['calibers'][1]['beta_star']['mean']*100:.2f}%（逐槽均值）；"
```

```
            f"c_o=p_t-p_min -> {nb['calibers'][2]['beta_star']['mean']*100:.2f}%")
    print("已写出", path, f"（总耗时 {time.time()-t_start:.1f}s）")
    return out


if __name__ == "__main__":
    main()
```

源程序：code/exp_dro.py

```
# -*- coding: utf-8 -*-
"""
问题 4 对照实验：盒式不确定集的分布鲁棒（DRO）购电策略
=======================================================
目的：为问题 4 的"分时骨架 + 随机残差情景式随机规划"主方案提供一个**分布鲁棒对照**，
      定量回答"保守度换最坏情形保障"值不值。

运行：cd C题论文/code && python3 exp_dro.py                    （全量 7 个 k，结果窗 334 天）
      cd C题论文/code && python3 exp_dro.py --quick            （k=0/1/2/3 且 92 天子窗，快速自检）
输出：intermediate/exp_dro.json

--------------------------------------------------------------------------------
一、不确定集（盒式，逐槽笛卡尔积）
--------------------------------------------------------------------------------
电价：     p_t  in  [ p^skel_t - k sigma_t ,  p^skel_t + k sigma_t ]
          p^skel = 附件 1 的分时骨架电价剖面（= 附件 4 逐时段均值，实测最大偏差 5.15e-5）；
          sigma_t  = 附件 4 电价在槽位 t 的**样本标准差**（全样本 365 天，ddof=1）。
净需求：   nd_t in  [ nd^fcst_t - k sigma^nd_t ,  nd^fcst_t + k sigma^nd_t ]
          nd^fcst_t = (L_t - PV^fcst_t) * DT（0:00 发布的附件 3 预报 + 当日负载，口径同问题 2/4）；
          sigma^nd_t = 预报残差 r_{d,t} = (PV^fcst_{d,t} - PV^act_{d,t}) * DT 的样本标准差
                       （= nd^act_t - nd^fcst_t，全样本 365 天，ddof=1）。
盒 Δ(k) = X_t [p^lo_t, p^hi_t] x X_t [nd^lo_t, nd^hi_t]：**各槽独立**（笛卡尔积，不含时序相关性），
k 同时是"电价带宽"和"安全边际"的倍数，k=0 时集合退化为单点（骨架价 + 预报净需求）。

--------------------------------------------------------------------------------
二、为什么最坏情形一定在盒的**顶点** (p^hi, nd^hi) 取得（推导）
--------------------------------------------------------------------------------
给定第一阶段决策 (E, c, d)，第二阶段（紧急购电）成本为
    Phi(p, nd) = sum_t [ p_t E_t + 5 p_t * max(0, nd_t + c_t - d_t - E_t) ] .
它对 (p, nd) **逐槽可分离**：Phi = sum_t phi_t(p_t, nd_t)，且每个 phi_t 对两个自变量**单调不减**：
    d phi_t / d p_t  = E_t + 5 max(0, nd_t + c_t - d_t - E_t) >= 0   (E_t >= 0)
    d phi_t / d nd_t = 5 p_t * 1{ nd_t + c_t - d_t - E_t > 0 }      >= 0   (p_t > 0)
单调   每个 phi_t 在矩形 [p^lo,p^hi] x [nd^lo,nd^hi] 上的最大值在右上端点取得；
可分离   max_{Delta} Phi = sum_t max phi_t = sum_t phi_t(p^hi_t, nd^hi_t)。
两个上端点同时可达到（(p^hi, nd^hi) 本身就是盒 Δ 的一个顶点，盒是笛卡尔积，
不存在"取价格上端就必须取净需求下端"的互斥），故
    最坏情形恰好在该顶点取得，且价格与净需求的最坏取值相互独立、可同时发生。
（这也说明盒式模型的已知弱点：它把价格-净负荷的联合结构当成"同时最坏"，见 C4 相关性一节。）

于是鲁棒对应问题 (RC) 与 q4.solve_vol 同结构、同约束，只把"分情景期望"换成"盒内最坏情形"：
    (RC)  min_{E,c,d,z}  sum_t p^hi_t E_t + 5 sum_t p^hi_t z_t
          s.t.  z_t >= nd^hi_t + c_t - d_t - E_t ,  z_t >= 0          (最坏情形缺口 -> 紧急购电)
                E_t - c_t + d_t >= nd^fcst_t                          (计划满足预报口径需求)
                S_t = S_{t-1} + eta_c c_t - d_t / eta_d ,  S_min <= S_t <= S_max
                0 <= c_t, d_t <= E_MAX = P_MAX*DT ,   E_t >= 0
                S_144 >= S_0                                          (A6 终端不亏空)

**结构性推论（脚本输出中逐日核验）**：E_t 与 z_t 在同一槽位上的目标系数之比恒为 5:1
（同为 p^hi_t），故只要 z_t > 0 就存在"提前多买 1 kWh 省 5-1=4 元"的下降方向，
最优解必取 z_t = 0、E_t = max(0, nd^hi_t + c_t - d_t)：
    盒式 DRO 的解 = "按最坏情形参数做确定性优化"，逐槽安全边际恰为 k*sigma^nd_t（100% 覆盖最坏净需求）。
对照：情景式随机规划（问题 4 主方案）的边际由报童临界分位数 4/5 决定（约 80% 分位），
两者结构不同 —— 这正是本对照实验要量化的"保守度溢价"。
注意：主模型与 q2/q4 一致地**不对 E_t 设上限**（题面未给购电功率上限），
故 k 较大时计划购电量会被推到 nd^hi，单槽购电量可能超出现实并网容量；
脚本因此额外给出"E_t <= E_MAX"的 cap 变体作灵敏度（clear labeled，非主口径）。
"""
from __future__ import annotations

import argparse
import json
import os
import time
```

```python

import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import coo_matrix

import config as C
import lp_engine as LP

# ---------------------------------------------------------------------------
# 实验设置
# ---------------------------------------------------------------------------
KS_DEFAULT = [0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0]
K_REF = 3.0                 # 参照不确定集 （用于"固定最坏情形"评估 W_fix）
K_REF2 = 2.0
ALPHA = 0.95                # CVaR 置信水平（C-11）
KS_CAP = [1.0, 3.0]         # E 上限灵敏度变体
E_CAP_SENS = 1.5 * C.E_MAX   # 1250 kWh/槽（ 7.5 MW）；E_MAX 本身不可行（单槽净需求峰值 1212.7 > 833.3）

DATE_DECIDE = pd.date_range(C.WARMUP_START, C.WIN_END, freq="D")   # 365 天（1 月预热）
DATE_SCORE = pd.date_range(C.WIN_START, C.WIN_END, freq="D")       # 334 天（结果窗）
SUB_START, SUB_END = "2025-03-01", "2025-05-31"                   # 92 天子窗（--quick）


# ---------------------------------------------------------------------------
# 数据与不确定集参数
# ---------------------------------------------------------------------------
def sigma_price(p4: pd.DataFrame) -> np.ndarray:
    """sigma_t：附件 4 电价在槽位 t 的样本标准差（全样本 365 天，ddof=1）。"""
    return p4.to_numpy(float).std(0, ddof=1)


def build_day_data(panel: "C.Panel", days):
    """预计算逐日：预报净需求、实际净需求、实际电价、无储能基线（所有 k 共用）。"""
    load = panel.L()
    pv = panel.PV()
    p4 = panel.P(volatile=True)
    n = len(days)
    NDF = np.zeros((n, C.N_SLOT))      # 预报口径净需求 (L - PV^fcst)*DT
    NDA = np.zeros((n, C.N_SLOT))      # 实际净需求 (L - PV^act)*DT
    PACT = np.zeros((n, C.N_SLOT))     # 附件 4 实际电价
    BASE = np.zeros(n)                 # C-01 无储能基线购电费
    for i, d in enumerate(days):
        L = load.loc[d].to_numpy(float)
        PVa = pv.loc[d].to_numpy(float)
        fc = panel.pv_forecast(d, 0)
        NDF[i] = (L - fc) * C.DT
        NDA[i] = (L - PVa) * C.DT
        PACT[i] = p4.loc[d].to_numpy(float)
        BASE[i] = C.baseline_cost(PACT[i], L, PVa)
    RES = NDA - NDF                    # = (PV^fcst - PV^act)*DT，问题 4 的净需求预报残差
    SND = RES.std(0, ddof=1)           # sigma^nd_t
    return dict(days=list(days), NDF=NDF, NDA=NDA, PACT=PACT, BASE=BASE, RES=RES, SND=SND)


# ---------------------------------------------------------------------------
# 盒式 DRO 的单日 LP（独立组装，不改 q4.py）
# ---------------------------------------------------------------------------
def solve_dro_day(nd_fcst, nd_hi, p_hi, s_init, term_floor=None, e_cap=None):
    """(RC) 单日盒式鲁棒对应 LP。变量 [E(n), c(n), d(n), S(n), z(n)]，全部 kWh。
       z_t 为"最坏情形下的紧急购电量"，目标系数 5*p^hi_t。"""
    n = C.N_SLOT
    iE, iC, iD, iS, iZ = 0, n, 2 * n, 3 * n, 4 * n
    nv = 5 * n

    c_obj = np.zeros(nv)
    c_obj[iE:iE + n] = p_hi
    c_obj[iZ:iZ + n] = C.EMERGENCY_MULT * p_hi

    rows, cols, vals, rhs = [], [], [], []
    r = 0
    # (1) 计划可行性：E - c + d >= nd^fcst   <=>  -E + c - d <= -nd^fcst
    for t in range(n):
```

```
        for a, b, v in ((r, iE + t, -1.0), (r, iC + t, 1.0), (r, iD + t, -1.0)):
            rows.append(a); cols.append(b); vals.append(v)
        rhs.append(-nd_fcst[t]); r += 1
    # (2) 最坏情形缺口: z >= nd^hi + c - d - E  <=>  -E + c - d - z <= -nd^hi
    for t in range(n):
        for a, b, v in ((r, iE + t, -1.0), (r, iC + t, 1.0), (r, iD + t, -1.0),
                        (r, iZ + t, -1.0)):
            rows.append(a); cols.append(b); vals.append(v)
        rhs.append(-nd_hi[t]); r += 1
    A_ub = coo_matrix((vals, (rows, cols)), shape=(r, nv)).tocsr()
    b_ub = np.array(rhs)

    # (3) SOC 递推
    rows2, cols2, vals2, rhs2 = [], [], [], []
    for t in range(n):
        for b, v in ((iS + t, 1.0), (iC + t, -C.ETA_C), (iD + t, 1.0 / C.ETA_D)):
            rows2.append(t); cols2.append(b); vals2.append(v)
        if t > 0:
            rows2.append(t); cols2.append(iS + t - 1); vals2.append(-1.0)
        rhs2.append(0.0 if t > 0 else s_init)
    A_eq = coo_matrix((vals2, (rows2, cols2)), shape=(n, nv)).tocsr()
    b_eq = np.array(rhs2)

    e_up = None if e_cap is None else float(e_cap)
    bounds = [(0.0, e_up)] * n + [(0.0, C.E_MAX)] * n + [(0.0, C.E_MAX)] * n + \
             [(C.S_MIN, C.S_MAX)] * n + [(0.0, None)] * n
    if term_floor is not None:      # A6: S_144 >= S_0
        bounds[iS + n - 1] = (max(float(term_floor), C.S_MIN), C.S_MAX)

    res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                  bounds=bounds, method="highs")
    if not res.success:
        raise RuntimeError("DRO LP 不可行: " + res.message)
    x = res.x
    return dict(E=x[iE:iE + n].copy(), c=x[iC:iC + n].copy(), d=x[iD:iD + n].copy(),
                S=x[iS:iS + n].copy(), z=x[iZ:iZ + n].copy(), obj=float(res.fun),
                s_init=float(s_init), nd_fcst=nd_fcst, nd_hi=nd_hi, p_hi=p_hi)


# ---------------------------------------------------------------------------
# 风险度量与自检
# ---------------------------------------------------------------------------
def cvar_ru(x, alpha=ALPHA):
    """Rockafellar-Uryasev 估计量: CVaR = eta* + E[(X-eta*)^+]/(1-alpha), eta*=样本 alpha 分位。"""
    x = np.asarray(x, float)
    eta = float(np.quantile(x, alpha))
    cvar = float(eta + np.mean(np.maximum(x - eta, 0.0)) / (1.0 - alpha))
    m = max(1, int(np.ceil((1.0 - alpha) * len(x))))       # 尾部均值(对照口径)
    tail = float(np.sort(x)[-m:].mean())
    return cvar, eta, tail, m


def audit_day(sol, nd_fcst, nd_hi):
    """与 lp_engine.check_solution 相同的方式 + DRO 专属项。violation 取正部(<=0 即合规)。"""
    n = C.N_SLOT
    E, c, d, S, z = sol["E"], sol["c"], sol["d"], sol["S"], sol["z"]
    rep = {}
    rep["R1_plan_violation"] = float(np.max(nd_fcst + c - d - E))
    rep["R2_power_violation"] = float(max(np.max(c), np.max(d)) - C.E_MAX)
    rep["R3_soc_bounds"] = float(max(C.S_MIN - S.min(), S.max() - C.S_MAX))
    prev = np.concatenate([[sol["s_init"]], S[:-1]])
    rep["R4_soc_recursion"] = float(np.max(np.abs(S - (prev + C.ETA_C * c - d / C.ETA_D))))
    rep["R5_simultaneous"] = int(np.sum((c > 1e-6) & (d > 1e-6)))
    rep["R6_E_nonneg"] = float(-min(0.0, float(E.min())))
    rep["R7_terminal"] = float(sol["s_init"] - S[-1])                       # <=0 合规
    rep["R8_z_consistency"] = float(np.max(np.abs(
        z - np.maximum(nd_hi + c - d - E, 0.0))))
    rep["R9_z_nonneg"] = float(-min(0.0, float(z.min())))
    rep["R10_worstcase_coverage"] = float(np.max(nd_hi + c - d - E))        # <=0 即 z 恒为 0
    # 直接复用 lp_engine.check_solution(同一套检查,独立于本次 LP 组装)
    for k_, v in LP.check_solution(sol, nd_fcst).items():
        rep["LP_" + k_] = float(v)
    return rep
```

```python


def agg_audit(store, rep):
    for k_, v in rep.items():
        if k_ not in store:
            store[k_] = float(v)
        elif k_ in ("R5_simultaneous",):
            store[k_] = float(store[k_]) + float(v)
        else:
            store[k_] = max(float(store[k_]), float(v))
    return store


# ---------------------------------------------------------------------------
# 滚动回测
# ---------------------------------------------------------------------------
def roll(data, k, sp, skel, ks_ref=(K_REF2, K_REF), e_cap=None, audit=False):
    """在 365 天（含 1 月预热）上滚动：每天 0:00 用当日报制定计划，当日实际值结算。
       只有结果窗 334 天计入指标；跨日 SOC 连续递推；每日 S_144 >= S_0（A6）。"""
    NDF, NDA, PACT, BASE = data["NDF"], data["NDA"], data["PACT"], data["BASE"]
    SND, days = data["SND"], data["days"]
    score_idx = [i for i, d in enumerate(days) if d in set(DATE_SCORE)]

    p_hi = skel + k * sp
    p_hi_ref = {kk: skel + kk * sp for kk in ks_ref}
    nd_hi_all = NDF + k * SND[None, :]
    nd_hi_ref = {kk: NDF + kk * SND[None, :] for kk in ks_ref}

    s_init = C.S_INIT
    recs = []
    aud = {}
    t_solve = 0.0
    for i, d in enumerate(days):
        sol = solve_dro_day(NDF[i], nd_hi_all[i], p_hi, s_init, term_floor=s_init,
                            e_cap=e_cap)
        if audit:
            agg_audit(aud, audit_day(sol, NDF[i], nd_hi_all[i]))
        E, c, dd, S = sol["E"], sol["c"], sol["d"], sol["S"]
        g = np.maximum(NDA[i] + c - dd - E, 0.0)                  # 实际紧急购电量
        pact = PACT[i]
        cost_buy = float(np.sum(pact * E))
        cost_emg = float(C.EMERGENCY_MULT * np.sum(pact * g))
        surplus = float(np.sum(np.maximum(E + dd - c - NDA[i], 0.0)))    # C-06 超购/盈余处置
        w_own = float(np.sum(p_hi * E + C.EMERGENCY_MULT * p_hi * np.maximum(
            nd_hi_all[i] + c - dd - E, 0.0)))
        w_fix = {}
        for kk in ks_ref:
            w_fix[kk] = float(np.sum(p_hi_ref[kk] * E + C.EMERGENCY_MULT * p_hi_ref[kk]
                                     * np.maximum(nd_hi_ref[kk][i] + c - dd - E, 0.0)))
        if i in score_idx:
            recs.append(dict(date=d, plan_energy=float(E.sum()),
                             actual_buy=float(E.sum() + g.sum()), emg_qty=float(g.sum()),
                             surplus_qty=surplus, cost_buy=cost_buy, cost_emg=cost_emg,
                             total_cost=cost_buy + cost_emg, base_cost=float(BASE[i]),
                             W_own=w_own, W_fix=w_fix,
                             n_short=int(np.sum(g > 1e-6)),
                             c_sum=float(c.sum()), d_sum=float(dd.sum()),
                             pw_short=float(np.sum(pact[g > 1e-6])), pw_all=float(np.sum(pact)),
                             demand_after_batt=float(np.sum(np.maximum(NDA[i] + c - dd, 0.0))),
                             s_init=s_init, s_end=float(S[-1]), z_sum=float(sol["z"].sum()),
                             gap_hi=float(np.mean(E + dd - c - nd_hi_all[i]))))
        if i == score_idx[0]:
            s0_feb1 = s_init
        s_init = float(S[-1])
    return dict(k=k, recs=recs, audit=aud, s0_feb1=s0_feb1, s_end=s_init, t_solve=t_solve)


def summarize(roll_out, ks_ref=(K_REF2, K_REF), e_cap=None):
    recs = roll_out["recs"]
    tot = lambda key: float(sum(r[key] for r in recs))
    base = tot("base_cost")
    total = tot("total_cost")
    cost_daily = np.array([r["total_cost"] for r in recs], float)
```

```
    cvar, var, tail, m = cvar_ru(cost_daily)
    dem = tot("demand_after_batt")
    nshort = sum(r["n_short"] for r in recs)
    worst = int(np.argmax(cost_daily))
    out = dict(
        k=roll_out["k"],
        n_days=len(recs),
        total_cost=total, plan_cost=tot("cost_buy"), emg_cost=tot("cost_emg"),
        emg_qty=tot("emg_qty"), base_cost=base,
        saving=(base - total) / base,
        plan_energy=tot("plan_energy"), actual_buy=tot("actual_buy"),
        surplus_qty=tot("surplus_qty"),
        service_rate=1.0 - tot("emg_qty") / dem if dem > 0 else float("nan"),
        slot_shortage_freq=nshort / (len(recs) * C.N_SLOT),
        cvar95=cvar, var95=var, cvar95_tailmean=tail, n_tail=m,
        mean_daily_cost=float(cost_daily.mean()), worst_day_cost=float(cost_daily[worst]),
        worst_day=str(pd.Timestamp(recs[worst]["date"]).date()),
        mean_daily_emg_qty=tot("emg_qty") / len(recs),
        mean_daily_plan_energy=tot("plan_energy") / len(recs),
        charge_energy=tot("c_sum"), discharge_energy=tot("d_sum"),
        price_weighted_shortage_freq=tot("pw_short") / tot("pw_all"),
        W_own=tot("W_own"), z_total=tot("z_sum"),
        s0_feb1=roll_out["s0_feb1"], s_end=roll_out["s_end"],
    )
    for kk in ks_ref:
        out["W_fix_%g" % kk] = float(sum(r["W_fix"][kk] for r in recs))
    if e_cap is not None:
        out["e_cap"] = float(e_cap)
    return out


def daily_of(roll_out, ks_ref=(K_REF2, K_REF)):
    out = []
    for r in roll_out["recs"]:
        d = dict(date=str(pd.Timestamp(r["date"]).date()), plan_energy=r["plan_energy"],
                 actual_buy=r["actual_buy"], emg_qty=r["emg_qty"],
                 surplus_qty=r["surplus_qty"], cost_plan=r["cost_buy"],
                 cost_emg=r["cost_emg"], total_cost=r["total_cost"],
                 base_cost=r["base_cost"], W_own=r["W_own"], s_init=r["s_init"],
                 s_end=r["s_end"])
        for kk in ks_ref:
            d["W_fix_%g" % kk] = r["W_fix"][kk]
        out.append(d)
    return out


# ---------------------------------------------------------------------------
# 残差/价格分布诊断（解释保守度—费用曲线的形状）
# ---------------------------------------------------------------------------
def residual_diagnostics(RES, SND, sp, skel):
    """预报残差与电价的分布特征。

    要点：盒式 DRO 用"统一倍数 k"乘 ~nd 代替"逐槽 4/5 分位数边际"，
    而 (a) 残差分布非正态、逐槽 q80/ 不同，(b) 高价时段残差更大，
    故单一 k 必然存在跨槽误配（misallocation）—— 这是实证最优 k* 偏离
    逐槽报童值 q80/ 的结构性原因。
    """
    mask = SND > 1e-9
    q = {p: np.quantile(RES, p, axis=0) for p in (0.1, 0.5, 0.8, 0.9, 0.95)}
    return dict(
        n_slots_sigma_nd_zero=int((~mask).sum()),
        n_slots=int(C.N_SLOT),
        bias_per_slot_mean_kwh=float(RES.mean(0)[mask].mean()),
        bias_over_sigma=float((RES.mean(0)[mask] / SND[mask]).mean()),
        q10_over_sigma=float((q[0.1][mask] / SND[mask]).mean()),
        q50_over_sigma=float((q[0.5][mask] / SND[mask]).mean()),
        q80_over_sigma=float((q[0.8][mask] / SND[mask]).mean()),
        q90_over_sigma=float((q[0.9][mask] / SND[mask]).mean()),
        q95_over_sigma=float((q[0.95][mask] / SND[mask]).mean()),
        newsvendor_k_equivalent=float((q[0.8][mask] / SND[mask]).mean()),
        pooled_mean_kwh=float(RES.mean()), pooled_sd_kwh=float(RES.std()),
        pooled_skew=float(((RES - RES.mean()) ** 3).mean() / RES.std() ** 3),
        frac_resid_positive=float((RES > 0).mean()),
```

```
        sigma_price_over_price=float((sp / skel).mean()),
        note="报童临界分位数 4/5 的逐槽等效边际倍数 = q80/ ^nd（上表均值）；"
             "盒式 DRO 以 k · ^nd 统一加成、且以预报为中心（不含残差偏差），"
             "故 k 的最优值由电价加权的缺货频率=1/5 决定（见 results.price_weighted_shortage_freq），"
             "不等于逐槽 q80/ ^nd 的平均值。",
    )


# ---------------------------------------------------------------------------
# 保守度—费用曲线
# ---------------------------------------------------------------------------
def tradeoff(curve, k_ref=K_REF):
    key = "W_fix_%g" % k_ref
    c0 = curve[0]
    rows = []
    for c in curve:
        dW = c0[key] - c[key]
        dW_rel = dW / c0[key] * 100.0
        dR = c["total_cost"] - c0["total_cost"]
        dR_rel = dR / c0["total_cost"] * 100.0
        rows.append(dict(
            k=c["k"], dW_fix=dW, dW_fix_rel_pct=dW_rel,
            dR=dR, dR_rel_pct=dR_rel,
            yuan_per_1pct_W=(dR / dW_rel) if abs(dW_rel) > 1e-9 else None,
            pct_of_base_per_1pct_W=((dR / dW_rel) / c0["base_cost"] * 100.0)
            if abs(dW_rel) > 1e-9 else None,
        ))
    best = min(curve, key=lambda c: c["total_cost"])
    bestW = min(curve, key=lambda c: c[key])
    last = curve[-1]
    rowbest = [r for r in rows if abs(r["k"] - best["k"]) < 1e-12][0]
    rowlast = rows[-1]
    base = c0["base_cost"]
    prem_dR = rowlast["dR"] - rowbest["dR"]
    prem_dW = rowbest["dW_fix"] - rowlast["dW_fix"]
    prem_dW_rel = rowlast["dW_fix_rel_pct"] - rowbest["dW_fix_rel_pct"]
    prem = dict(
        k_from=best["k"], k_to=last["k"],
        dR=prem_dR, dR_rel_pct=prem_dR / c0["total_cost"] * 100.0,
        dR_pct_of_base=prem_dR / base * 100.0,
        dW_fix=prem_dW, dW_fix_rel_pct=prem_dW_rel,
        yuan_per_1pct_W=(prem_dR / prem_dW_rel) if abs(prem_dW_rel) > 1e-9 else None,
        pct_of_base_per_1pct_W=((prem_dR / prem_dW_rel) / base * 100.0)
        if abs(prem_dW_rel) > 1e-9 else None,
        note="从实证最优 k* 继续加大保守度到 k_max：最坏情形费用每降 1% 需要多付的回测(期望)费用",
    )
    pareto = dict(
        k_from=c0["k"], k_to=best["k"],
        dR=rowbest["dR"], dR_rel_pct=rowbest["dR_rel_pct"],
        dW_fix=rowbest["dW_fix"], dW_fix_rel_pct=rowbest["dW_fix_rel_pct"],
        note="k=0（零边际确定性计划）-> k*：最坏情形费用与实际费用同时改善（严格 Pareto 改善）",
    )
    kc = min(curve, key=lambda c: c["cvar95"])
    kw = min(curve, key=lambda c: c["worst_day_cost"])
    return dict(k_ref=k_ref, W_fix_k0=c0[key], rows=rows,
                k_best_realized=best["k"], k_best_realized_total=best["total_cost"],
                k_best_W=bestW["k"], k_best_W_value=bestW[key],
                k_best_cvar=kc["k"], k_best_cvar_value=kc["cvar95"],
                k_best_worstday=kw["k"], k_best_worstday_value=kw["worst_day_cost"],
                pareto=pareto, premium=prem)


def marginal_analysis(curve):
    """逐步加大保守度的边际账：多买的计划电量 vs 被替代的紧急购电量。"""
    rows = []
    for a, b in zip(curve[:-1], curve[1:]):
        dE = b["plan_energy"] - a["plan_energy"]
        dPC = b["plan_cost"] - a["plan_cost"]
        dG = a["emg_qty"] - b["emg_qty"]
        dEC = a["emg_cost"] - b["emg_cost"]
        up = (dPC / dE) if dE > 1e-9 else None
        ue = (dEC / dG) if dG > 1e-9 else None
        rows.append(dict(k_from=a["k"], k_to=b["k"],
```

```
                        d_plan_energy=dE, d_plan_cost=dPC, unit_plan_price=up,
                        d_emg_qty=dG, d_emg_cost=dEC, unit_emg_price=ue,
                        price_mismatch=(ue / up) if (up and ue) else None,
                        benefit_cost_ratio=(dEC / dPC) if dPC > 1e-9 else None,
                        net=dEC - dPC))
    cross = next((r for r in rows if r["benefit_cost_ratio"] is not None
                  and r["benefit_cost_ratio"] < 1.0), None)
    return dict(steps=rows, crossover_step=cross,
                note="边际判定：k→k+Δk 时，新增计划电量的单价（元/kWh，unit_plan_price）"
                     "对被替代的紧急购电量单价（含 5 倍，unit_emg_price）之比"
                     "benefit_cost_ratio = price_mismatch x (d_emg_qty/d_plan_energy)："
                     ">1 说明继续加大保守度仍省钱，<1 说明已开始亏（交叉点在 1.5->2.0 这一步）。"
                     "实测 price_mismatch 6.2 = 5 x 1.25：边际电量买在 ^nd 大的时段（单价 0.70 元/kWh），
"
                     "而被替代的紧急购电发生在均价更高的时段（单价 4.4 元/kWh，其中 5 倍为紧急电价倍数）。"
                     "于是临界条件变为：边际替代率只需达到 1/6.2  16% 即值得；"
                     "k=1.5 时的实际替代率（电价加权缺货频率）为 11.3%，k=1.0 时为 15.1%，"
                     "故实证最优 k* 落在 1.5 附近；"
                     "逐槽报童临界分位数 4/5 的等效边际 q80/ ^nd 0.64 只在价格中性、逐槽可调时成立，"
                     "统一倍数 k 忽略了这一跨槽价格错配（misallocation）。")


def make_figure(out, path):
    """保守度—费用曲线（可选图件）。"""
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    plt.rcParams.update({                       # 与 plot_figures.py 同一中文字体栈
        "font.family": "sans-serif",
        "font.sans-serif": ["Songti SC", "Heiti SC", "PingFang SC", "Arial Unicode MS"],
        "axes.unicode_minus": False,
        "pdf.fonttype": 42, "ps.fonttype": 42, "font.size": 9,
        "legend.fontsize": 7.4, "savefig.bbox": "tight",
    })
    ks = [c["k"] for c in out["curve"]["rows"]]
    res = [out["results"]["k=%.1f" % k] for k in ks]
    tot = [r["total_cost"] / 1e4 for r in res]
    plan = [r["plan_cost"] / 1e4 for r in res]
    emg = [r["emg_cost"] / 1e4 for r in res]
    wfix = [r["W_fix_%g" % K_REF] / 1e4 for r in res]
    Wown = [r["W_own"] / 1e4 for r in res]
    fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.6))
    ax[0].plot(ks, tot, "o-", color="#c0392b", label="实际回测总费用")
    ax[0].plot(ks, plan, "s--", color="#2c6fb5", label="计划购电费用")
    ax[0].plot(ks, emg, "^--", color="#e08a1e", label="紧急购电费用")
    kb = out["curve"]["k_best_realized"]
    ax[0].axvline(kb, color="gray", ls=":", lw=1)
    ax[0].annotate("k*=%.1f" % kb, (kb, max(tot)), color="gray",
                   fontsize=8, ha="left", va="top")
    ax[0].set_xlabel(r"安全边际倍数 $k$（边际 $=k\sigma^{nd}_t$）")
    ax[0].set_ylabel("费用 / 万元")
    ax[0].legend(fontsize=7.5)
    ax[0].grid(alpha=0.3)
    ax[1].plot(ks, wfix, "o-", color="#6c3483",
               label=r"固定最坏情形 $\kappa=3$ 下的费用")
    ax[1].plot(ks, Wown, "s--", color="#7f8c8d", label=r"自身盒 $k$ 内的最坏情形（DRO 目标）")
    ax[1].set_xlabel("安全边际倍数 $k$")
    ax[1].set_ylabel("最坏情形费用 / 万元")
    ax[1].legend(fontsize=7.5)
    ax[1].grid(alpha=0.3)
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)
    return path


# ---------------------------------------------------------------------------
# 主程序
# ---------------------------------------------------------------------------
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--ks", default=",".join(str(x) for x in KS_DEFAULT))
    ap.add_argument("--quick", action="store_true", help="92 天子窗 + 4 个 k")
```

```
    ap.add_argument("--no-cap", action="store_true", help="跳过 E<=E_MAX 灵敏度变体")
    ap.add_argument("--out", default=os.path.join(C.INTERMEDIATE_DIR, "exp_dro.json"))
    ap.add_argument("--fig", action="store_true", help="同时输出 figures/fig11_dro_conservatism.pdf")
    args = ap.parse_args()

    t0 = time.time()
    ks = [float(x) for x in args.ks.split(",")]
    if args.quick:
        ks = [0.0, 1.0, 2.0, 3.0]

    panel = C.Panel(DATE_DECIDE)
    p4 = panel.P(volatile=True)
    skel = C.read_attachment1()["price"].to_numpy(float)
    sp = sigma_price(p4)
    all_days = list(DATE_DECIDE)
    data = build_day_data(panel, all_days)
    snd = data["SND"]
    if args.quick:
        global DATE_SCORE
        DATE_SCORE = pd.date_range(SUB_START, SUB_END, freq="D")
        print("QUICK 模式: 评分窗 %s..%s（%d 天）" % (SUB_START, SUB_END, len(DATE_SCORE)))

    base_total = float(data["BASE"][[i for i, d in enumerate(all_days)
                                     if d in set(DATE_SCORE)]].sum())
    print("=" * 78)
    print("DRO 对照实验: 决策窗 %s..%s（%d 天，含 1 月预热），结果窗 %s..%s（%d 天）"
          % (C.WARMUP_START, C.WIN_END, len(all_days), DATE_SCORE[0].date(),
             DATE_SCORE[-1].date(), len(DATE_SCORE)))
    print("sigma^price: mean %.4f  [%.4f, %.4f] 元/kWh" % (sp.mean(), sp.min(), sp.max()))
    print("sigma^nd   : mean %.2f  [%.2f, %.2f] kWh" % (snd.mean(), snd.min(), snd.max()))
    print("无储能基线（结果窗）: %.2f 元" % base_total)
    print("=" * 78)

    results, dailies, audits = {}, {}, {}
    for k in ks:
        t1 = time.time()
        ro = roll(data, k, sp, skel, audit=(abs(k - 1.0) < 1e-12))
        sm = summarize(ro)
        sm["base_cost_meta"] = base_total
        results["k=%.1f" % k] = sm
        dailies["k=%.1f" % k] = daily_of(ro)
        if ro["audit"]:
            audits["k=%.1f" % k] = ro["audit"]
        print("k=%.1f 总费用 %12.2f 元 | 计划 %12.2f 紧急 %10.2f | 紧急量 %10.1f kWh | "
              "缺货频率 %5.2f%% | CVaR95 %9.2f (%.1fs)"
              % (k, sm["total_cost"], sm["plan_cost"], sm["emg_cost"], sm["emg_qty"],
                 sm["slot_shortage_freq"] * 100, sm["cvar95"], time.time() - t1))

    curve = [results["k=%.1f" % k] for k in ks]
    tro = tradeoff(curve)

    # ---- E 上限灵敏度（非主口径）----
    cap_res = {}
    if not args.no_cap:
        for k in KS_CAP:
            if k not in ks:
                continue
            ro = roll(data, k, sp, skel, e_cap=E_CAP_SENS)
            cap_res["k=%.1f" % k] = summarize(ro, e_cap=E_CAP_SENS)
            print("  [cap 变体 E<=%.2f kWh/槽] k=%.1f 总费用 %.2f 元（主口径 %.2f 元）"
                  % (E_CAP_SENS, k, cap_res["k=%.1f" % k]["total_cost"],
                     results["k=%.1f" % k]["total_cost"]))

    # ---- 与问题 4 主方案（情景式随机规划）对照 ----
    ref = {}
    try:
        with open(os.path.join(C.INTERMEDIATE_DIR, "q4_2.json")) as f:
            j = json.load(open(os.path.join(C.INTERMEDIATE_DIR, "q4_2.json")))
        m = j["meta"]
        ref = dict(source="intermediate/q4_2.json", model="情景式两阶段随机规划（K=60 联合情景）",
                   total_cost=m["total"], base_cost=m["base"], emg_qty=m["emg_qty"],
                   saving=(m["base"] - m["total"]) / m["base"],
                   comparable=bool(not args.quick and len(DATE_SCORE) == 334))
```

```
        best = min(curve, key=lambda c: c["total_cost"])
        ref["best_dro_k"] = best["k"]
        ref["best_dro_total"] = best["total_cost"]
        ref["diff_vs_best_dro"] = best["total_cost"] - m["total"]
        ref["diff_pct"] = (best["total_cost"] - m["total"]) / m["total"] * 100
        # 同计划购电量水平下的对照（主方案的边际"买得更准"）
        qt = 0.0
        for r in j["daily"]:
            qt += r["plan_energy"]
        near = min(curve, key=lambda c: abs(c["plan_energy"] - qt))
        ref["matched_plan_energy"] = dict(
            q4_2_plan_energy=qt, dro_k=near["k"], dro_plan_energy=near["plan_energy"],
            dro_emg_qty=near["emg_qty"], q4_2_emg_qty=m["emg_qty"],
            emg_qty_ratio=near["emg_qty"] / m["emg_qty"],
            note="计划购电量相当时的紧急购电量对比：主方案把边际买在更需要的槽位")
    except Exception as e:      # pragma: no cover
        ref = dict(error=str(e))

    c0 = curve[0]
    best = [c for c in curve if abs(c["k"] - tro["k_best_realized"]) < 1e-12][0]
    last = curve[-1]
    pa, pm = tro["pareto"], tro["premium"]
    sv = lambda c: (base_total - c["total_cost"]) / base_total * 100.0
    k1 = results.get("k=1.0", best)
    concl = (
        "盒式不确定集下安全边际为 k · ^nd（k 同时是电价带宽倍数）：在 %d 天实际轨迹上回测，"
        "总费用随 k 先降后升 —— k=0（退化为按预报确定性购电、零边际、z 0）为 %.2f 元（节省率 %.2f%%），"
        "k=1.0 为 %.2f 元（节省率 %.2f%%），实证最优 k*=%.1f 为 %.2f 元（节省率 %.2f%%，紧急购电 %.1f kWh），"
        "而 k=%.1f（100%% 覆盖 3 最坏净需求）回升到 %.2f 元（节省率 %.2f%%，紧急购电仅 %.1f kWh）；"
        "CVaR95 与最坏单日费用分别在 k=%.1f（%.2f 元/日）与 k=%.1f（%.2f 元）取最小。"
        "在 =%.1f 的固定最坏情形盒下，各计划的最坏情形费用随 k 单调下降："
        "由 k=0 的 %.2f 元降到 k=%.1f 的 %.2f 元；"
        "其中 k=0→k*=%.1f 是严格 Pareto 改善（最坏情形费用降 %.2f%%，实际费用同时降 %.2f 元）；"
        "再从 k*=%.1f 加大到 k=%.1f，最坏情形费用再降 %.2f%%（%.2f 元），实际费用则多付 %.2f 元（+%.2f%%，"
        "占无储能基线 %.3f%%），即**为把最坏情形费用再降低 1%%，需多付 %.2f 元期望（回测）费用**"
        "（约合基线的 %.3f%%/1%%）—— 这是盒式 DRO 的保守度溢价。"
        "故盒式 DRO 在 k %.1f 附近即可拿到几乎全部实证收益，k 继续加大只买到最坏情形保障而不省钱；"
        "其结构（z 0、100%% 覆盖最坏净需求）与情景式随机规划的报童边际（80%% 分位）不同。"
        % (len(DATE_SCORE), c0["total_cost"], sv(c0), k1["total_cost"], sv(k1),
           best["k"], best["total_cost"], sv(best), best["emg_qty"],
           last["k"], last["total_cost"], sv(last), last["emg_qty"],
           tro["k_best_cvar"], tro["k_best_cvar_value"],
           tro["k_best_worstday"], tro["k_best_worstday_value"],
           K_REF, tro["W_fix_k0"], last["k"], last["W_fix_%g" % K_REF], best["k"],
           pa["dW_fix_rel_pct"], -pa["dR"],
           best["k"], last["k"], pm["dW_fix_rel_pct"], -pm["dW_fix"],
           pm["dR"], pm["dR_rel_pct"], pm["dR_pct_of_base"],
           (pm["yuan_per_1pct_W"] if pm["yuan_per_1pct_W"] is not None else float("nan")),
           (pm["pct_of_base_per_1pct_W"] if pm["pct_of_base_per_1pct_W"] is not None else float("nan")),
           best["k"])
    )
    if ref.get("comparable"):
        concl += ("与情景式随机规划（问题 4 主方案，intermediate/q4_2.json）相比，"
                  "实证最优的 DRO 计划（k*=%.1f）实测费用高 %.2f 元（%.2f%%）。"
                  % (best["k"], ref["diff_vs_best_dro"], ref["diff_pct"]))

    out = dict(
        meta=dict(
            script="code/exp_dro.py", created=time.strftime("%Y-%m-%d %H:%M:%S"),
            purpose="问题 4 主方案（情景式随机规划）的分布鲁棒（盒式不确定集）对照",
            n_solve=len(ks) * len(all_days) + (len([k for k in KS_CAP if k in ks]) if not args.no_cap
     else 0) * len(all_days),
            runtime_s=time.time() - t0,
            quick=bool(args.quick),
        ),
        config=dict(
            ks=ks, k_ref=[K_REF2, K_REF], alpha=ALPHA, quick=bool(args.quick),
            decide_window=[str(all_days[0].date()), str(all_days[-1].date())],
            decide_days=len(all_days),
            score_window=[str(DATE_SCORE[0].date()), str(DATE_SCORE[-1].date())],
```

```
            n_days=len(DATE_SCORE),
            warmup=[C.WARMUP_START, C.WIN_START],
            sigma_price_def="附件 4 电价在槽位 t 的样本标准差（全样本 365 天，ddof=1）",
            sigma_nd_def="0:00 预报残差 (PV^fcst-PV^act)*DT 在槽位 t 的样本标准差（全样本 365 天，ddof=1）",
            sigma_price_mean=float(sp.mean()), sigma_price_min=float(sp.min()),
            sigma_price_max=float(sp.max()), sigma_nd_mean=float(snd.mean()),
            sigma_nd_min=float(snd.min()), sigma_nd_max=float(snd.max()),
            sigma_price_mean_window=float(p4.loc[DATE_SCORE].to_numpy(float).std(0, ddof=1).mean()),
            sigma_nd_mean_window=float(data["RES"][[i for i, d in enumerate(all_days)
                                                    if d in set(DATE_SCORE)]].std(0, ddof=1).mean()),
            sigma_nd_warmup_only=float(data["RES"][[i for i, d in enumerate(all_days)
                                                    if d not in set(DATE_SCORE)]].std(0, ddof=1).mean()),
            unit="kWh / 元 / 元每kWh", DT=C.DT, E_MAX=C.E_MAX, S_INIT=C.S_INIT,
            EMERGENCY_MULT=C.EMERGENCY_MULT, terminal="S_144 >= S_0 (A6)",
            baseline_total=base_total,
            e_cap_variant=(None if args.no_cap else E_CAP_SENS),
        ),
        sigma_price=sp.tolist(), sigma_nd=snd.tolist(),
        diagnostics=residual_diagnostics(data["RES"], snd, sp, skel),
        results=results,
        curve=tro,
        marginal=marginal_analysis(curve),
        sensitivity_e_cap=cap_res,
        reference_q4_2=ref,
        audit=audits,
        daily=dailies,
        conclusion=concl,
    )
    with open(args.out, "w") as f:
        json.dump(out, f, ensure_ascii=False, separators=(",", ":"))
    if args.fig:
        fp = os.path.join(C.FIGURE_DIR, "fig11_dro_conservatism.pdf")
        make_figure(out, fp)
        print("已写出图件", fp)
    print("=" * 78)
    print("已写出", args.out, "(%.1f KB，总耗时 %.1fs)"
          % (os.path.getsize(args.out) / 1024, time.time() - t0))
    print(concl)
    print("=" * 78)
    print("自检（k=1.0，334 天最大值/违反计数）:", json.dumps(audits.get("k=1.0", {}), ensure_ascii=False))
    return out


if __name__ == "__main__":
    main()
```

源程序：code/decomp.py

```
# -*- coding: utf-8 -*-
"""
L 形（Benders）分解对照实验
============================
对象：lp_engine.solve_day 的单日两阶段随机规划（问题 2/3/4 共用的求解核）

    min  sum_t p_t E_t + 5 sum_t p_t sum_w pi_w g_{w,t}
    s.t. E_t - c_t + d_t            >= nd^fcst_t                    (计划可行性)
         g_{w,t} + E_t - c_t + d_t  >= nd^act_{w,t}                 (追索/紧急购电)
         S_t = S_{t-1} + eta_c c_t - d_t/eta_d,  S_min <= S_t <= S_max
         0 <= c_t, d_t <= E_MAX,  E_t >= 0,  g_{w,t} >= 0
         S_144 >= S_0（term_mode='ge'，本实验主口径）

分解结构
--------
第一阶段 x = (E, c, d, S)（4n 个变量）；第二阶段 g_{w,t} 按情景 w 与槽位 t 完全可分。
记单槽"计划口径可用供给" y_t = E_t + d_t - c_t（kWh），则第二阶段的最优值解析可解：

    Q(y) = 5 * sum_t p_t * sum_w pi_w * (nd_{w,t} - y_t)^+ ,
    g_{w,t}^*(y) = (nd_{w,t} - y_t)^+ .

**次梯度解析式**（关键，主问题因此只需带解析割的 LP）：
```

```
    dQ/dy_t = -5 p_t * sum_w pi_w * 1{ nd_{w,t} > y_t } =: mu_t <= 0 ,
因为每个 max(0, nd_{w,t} - y_t) 关于 y_t 的次梯度是 -1{nd_{w,t} > y_t}（0 处的次梯度区间
为 [-1, 0]，取 -1 的取值即可给出有效割）。对第一阶段变量（y_t = E_t + d_t - c_t）：

    dQ/dE_t = dQ/dd_t = -dQ/dc_t = mu_t .

**最优性割（Benders/L 形）**：对任一试验点 y^k，
    theta >= Q(y^k) + sum_t mu_t^k (y_t - y_t^k)
写成 <= 形式（供 linprog 使用）：
    -theta - sum_t q_t^k (E_t + d_t - c_t) <= -( Q(y^k) + sum_t q_t^k y_t^k ),
    q_t^k := 5 p_t sum_w pi_w 1{nd_{w,t} > y_t^k} >= 0 .

**逐槽分离的多割（本脚本默认 disaggregated=True）**：因为 Q 逐槽可分
    Q(y) = sum_t Q_t(y_t),  Q_t(y_t) = 5 p_t sum_w pi_w (nd_{w,t} - y_t)^+ ,
把 144 维切平面拆成 144 条一维切平面（每个槽位一个 theta_t）：
    theta_t + q_t^k (y_t - y_t^k) >= Q_t(y_t^k)   <=>   -theta_t - q_t^k y_t <= -(Q_t(y_t^k)+q_t^k y_t^k
    ).
每条割都是有效下界，故 LB 仍然有效；实证：多割 9~20 次迭代收敛到 1e-15（对偶间隙
1e-12 绝对量），而聚合单割 200 次迭代后相对差仍有 6.5e-3 ~ 3.4e-2（见 single_cut_vs_multicut）。

**可行性割不需要**：追索变量 g >= 0 且无上界，任何第一阶段决策都可补偿（完全追索），
子问题恒可行，故只要最优性割。Q 是分片线性凸函数，分片数有限，算法有限步收敛。

主问题变量上界（仅为让主问题有界，不切掉任何最优解）
------------------------------------------------------
新增割在 y_t 方向斜率为负，若第一阶段变量无上界，主问题会出现"沿割方向无界"的假象
（因为割只是 Q 的一条切线，而 Q 本身在 y 大于最大情景需求后恒为 0）。故对 E_t 加
**有效上界**：
    y_t = E_t + d_t - c_t <= y_t + E_MAX  (d_t <= E_MAX, c_t >= 0)
    存在最优解使 y_t <= max(nd^fcst_t, max_w nd_{w,t})：
      把任何可行解的 y_t 截断到该值，计划约束仍满足（截断值 >= nd^fcst_t），
      紧急购电费不增（超出最大情景需求时 g=0），计划购电费不增（p_t y_t 变小），
      故截断后目标 <= 原目标，仍为最优。此时 E_t = y_t + c_t - d_t <= y_t + E_MAX。
    => 约束 E_t <= max(nd^fcst_t, max_w nd_{w,t}) + E_MAX 不排除任何最优解。

输出：intermediate/decomp.json

运行：cd C题论文/code && python3 decomp.py          （约 2-4 分钟）
"""
from __future__ import annotations

import argparse
import json
import os
import platform
import time

import numpy as np
import pandas as pd
import scipy
from scipy.optimize import linprog
from scipy.sparse import coo_matrix, vstack

import config as C
import lp_engine as LP
import q2 as Q2

K_SCAN = [20, 60, 120, 300, 500]
TOL = 1e-9                  # 相对收敛容差 |UB-LB| <= TOL*max(1,|UB|)
MAX_ITER = 400


# ---------------------------------------------------------------------------
# 解析子问题
# ---------------------------------------------------------------------------
def recourse(nd_scen, weights, price, y):
    """解析求解第二阶段：返回 Q(y) 与逐槽缺电概率 w_t = P(nd_t > y_t)。"""
    slack = nd_scen - y[None, :]                        # (K,n)
    pos = np.maximum(slack, 0.0)
    qty = (weights[:, None] * pos).sum(axis=0)          # 期望缺电量 (n,)
    Q = float(C.EMERGENCY_MULT * np.sum(price * qty))
    sw = weights[:, None] * (slack > 0.0)
    w = sw.sum(axis=0)                                  # (n,) 缺电概率
```

```
    return Q, w, qty


def _master_solve(price, nd_fcst, cuts, n_theta, s_init, term_mode, term_value,
                  iE, iC, iD, iS, iTH, n, nv, e_ub):
    """装配并求解主问题（每次迭代重建；变量 4n + n_theta，割行稀疏且很少）。"""
    rows, cols, vals, rhs = [], [], [], []
    r = 0
    for t in range(n):                                  # 计划可行性
        rows += [r, r, r]; cols += [iE + t, iC + t, iD + t]; vals += [-1.0, 1.0, -1.0]
        rhs.append(-nd_fcst[t]); r += 1
    for coef, b in cuts:                                # 最优性割
        for idx, val in coef:
            rows.append(r); cols.append(idx); vals.append(val)
        rhs.append(b); r += 1
    A_ub = coo_matrix((vals, (rows, cols)), shape=(r, nv)).tocsr()

    rows2, cols2, vals2, rhs2 = [], [], [], []        # SOC 递推
    for t in range(n):
        for b_, v_ in ((iS + t, 1.0), (iC + t, -C.ETA_C), (iD + t, 1.0 / C.ETA_D)):
            rows2.append(t); cols2.append(b_); vals2.append(v_)
        if t > 0:
            rows2.append(t); cols2.append(iS + t - 1); vals2.append(-1.0)
        rhs2.append(0.0 if t > 0 else s_init)
    A_eq = coo_matrix((vals2, (rows2, cols2)), shape=(n, nv)).tocsr()

    bounds = [(0.0, e_ub[t]) for t in range(n)] + \
             [(0.0, C.E_MAX)] * n + [(0.0, C.E_MAX)] * n + \
             [(C.S_MIN, C.S_MAX)] * n + [(0.0, None)] * n_theta
    if term_mode == "eq":
        v = s_init if term_value is None else term_value
        bounds[iS + n - 1] = (v, v)
    elif term_mode == "ge":
        v = s_init if term_value is None else term_value
        bounds[iS + n - 1] = (max(v, C.S_MIN), C.S_MAX)

    c_obj = np.zeros(nv)
    c_obj[iE:iE + n] = price
    c_obj[iTH:iTH + n_theta] = 1.0                    # theta（1 个聚合 / n 个逐槽）
    res = linprog(c_obj, A_ub=A_ub, b_ub=np.array(rhs), A_eq=A_eq, b_eq=np.array(rhs2),
                  bounds=bounds, method="highs")
    if not res.success:
        raise RuntimeError("主问题求解失败：" + res.message)
    return res


def solve_day_benders(price, nd_fcst, nd_scen, weights=None, s_init=C.S_INIT,
                      term_mode="ge", term_value=None, tol=TOL, max_iter=MAX_ITER,
                      disaggregated=True, keep_history=False):
    """L 形（Benders）分解求解单日两阶段 LP；返回解、迭代数与耗时分解。

    disaggregated=True：按槽分离的多割（每个槽位一个 theta_t，各自的解析切平面）。
        因为 Q(y) = sum_t Q_t(y_t)、Q_t(y_t) = 5 p_t sum_w pi_w (nd_{w,t} - y_t)^+ 是
        **逐槽可分的 1 维凸分片线性函数**，把聚合割拆成 144 条逐槽割可极大加快收敛
        （聚合单割在 144 维主问题上会出现典型的慢收敛/振荡，见 large_case.single_cut）。
    disaggregated=False：经典单割（聚合）L 形，用于对照。
    """
    n = C.N_SLOT
    nd_scen = np.atleast_2d(np.asarray(nd_scen, float))
    K = nd_scen.shape[0]
    if weights is None:
        weights = np.full(K, 1.0 / K)
    weights = np.asarray(weights, float)
    weights = weights / weights.sum()

    iE, iC, iD, iS, iTH = 0, n, 2 * n, 3 * n, 4 * n
    n_theta = n if disaggregated else 1
    nv = 4 * n + n_theta
    ymax = np.maximum(nd_fcst, nd_scen.max(axis=0))
    e_ub = ymax + C.E_MAX                             # 有效上界（见文件头推导）

    cuts = []
    hist = []
```

```
    best = None
    t_master = t_sub = 0.0
    for it in range(max_iter):
        t0 = time.time()
        res = _master_solve(price, nd_fcst, cuts, n_theta, s_init, term_mode, term_value,
                            iE, iC, iD, iS, iTH, n, nv, e_ub)
        t_master += time.time() - t0
        x = res.x
        E, cc, dd, S = x[iE:iE + n], x[iC:iC + n], x[iD:iD + n], x[iS:iS + n]
        y = E + dd - cc
        t0 = time.time()
        Q, wsh, qty = recourse(nd_scen, weights, price, y)
        t_sub += time.time() - t0
        lb = float(res.fun)
        ub = float(price @ E) + Q
        hist.append(dict(it=it, lb=lb, ub=ub, gap=ub - lb,
                         theta=(float(x[iTH]) if not disaggregated
                                else float(np.sum(x[iTH:iTH + n]))),
                         n_short_slots=int((wsh > 0).sum())))
        if best is None or ub < best["ub"]:
            best = dict(ub=ub, E=E.copy(), c=cc.copy(), d=dd.copy(), S=S.copy(),
                        g=qty.copy(), it=it, wsh=wsh.copy(),
                        theta=(float(x[iTH]) if not disaggregated
                               else float(np.sum(x[iTH:iTH + n]))))
        if ub - lb <= tol * max(1.0, abs(ub)):
            break
        qover = C.EMERGENCY_MULT * price * wsh          # q_t^k = 5 p_t P(nd_t > y_t) >= 0
        if disaggregated:
            # 逐槽子问题值 Q_t(y_t^k) = 5 p_t sum_w pi_w (nd_{w,t} - y_t^k)^+
            Qt = C.EMERGENCY_MULT * price * \
                (weights[:, None] * np.maximum(nd_scen - y[None, :], 0.0)).sum(axis=0)
            for t in range(n):
                if qover[t] <= 0.0:                     # Q_t 局部恒为 0 -> 割为 theta_t >= 0
                    continue
                cuts.append(([(iTH + t, -1.0), (iE + t, -qover[t]),
                              (iD + t, -qover[t]), (iC + t, qover[t])],
                             -(Qt[t] + qover[t] * y[t])))
        else:
            coef = [(iTH, -1.0)]
            for t in range(n):
                coef += [(iE + t, -qover[t]), (iD + t, -qover[t]), (iC + t, qover[t])]
            cuts.append((coef, -(Q + float(np.sum(qover * y)))))

    out = dict(E=best["E"], c=best["c"], d=best["d"], S=best["S"],
               obj=float(best["ub"]), lb=float(hist[-1]["lb"]),
               iters=len(hist), cuts=len(cuts), K=K, s_init=s_init,
               g_expected=best["g"], theta=float(best["theta"]),
               disaggregated=bool(disaggregated),
               converged=bool(hist[-1]["ub"] - hist[-1]["lb"] <=
                              tol * max(1.0, abs(hist[-1]["ub"]))),
               sec_master=t_master, sec_sub=t_sub, sec_total=t_master + t_sub,
               nd_fcst=nd_fcst, nd_scen=nd_scen, price=price)
    if keep_history:
        out["history"] = hist
    return out


# ---------------------------------------------------------------------------
# 直接求解的"向量化复现"（用于把 Python 装配耗时与 HiGHS 求解耗时分开计时）
# ---------------------------------------------------------------------------
def build_direct(price, nd_fcst, nd_scen, weights, s_init, term_mode="ge",
                 term_value=None):
    n, K = C.N_SLOT, nd_scen.shape[0]
    iE, iC, iD, iS, iG = 0, n, 2 * n, 3 * n, 4 * n
    nv = 4 * n + K * n
    c_obj = np.zeros(nv)
    c_obj[iE:iE + n] = price
    c_obj[iG:iG + K * n] = C.EMERGENCY_MULT * (weights[:, None] * price[None, :]).ravel()

    tt = np.tile(np.arange(n), K)           # 逐情景的槽位下标（w 主序）
    ss = np.arange(K * n)                   # 情景块内顺序下标：g 变量 = iG + w*n + t = iG + ss
    sc_rows = n + ss                        # 追索约束行号：n + w*n + t
    pl = np.arange(n)
```

```
    rows = np.concatenate([pl, pl, pl, sc_rows, sc_rows, sc_rows, sc_rows])
    cols = np.concatenate([iE + pl, iC + pl, iD + pl,
                           iE + tt, iC + tt, iD + tt, iG + ss])
    vals = np.concatenate([-np.ones(n), np.ones(n), -np.ones(n),
                           -np.ones(K * n), np.ones(K * n), -np.ones(K * n), -np.ones(K * n)])
    A_ub = coo_matrix((vals, (rows, cols)), shape=(n + K * n, nv)).tocsr()
    b_ub = np.concatenate([-nd_fcst, -nd_scen.ravel()])

    rows2 = np.concatenate([np.arange(n), np.arange(1, n), np.arange(n), np.arange(n)])
    cols2 = np.concatenate([iS + np.arange(n), iS + np.arange(n - 1), iC + np.arange(n),
                            iD + np.arange(n)])
    vals2 = np.concatenate([np.ones(n), -np.ones(n - 1), -C.ETA_C * np.ones(n),
                            (1.0 / C.ETA_D) * np.ones(n)])
    A_eq = coo_matrix((vals2, (rows2, cols2)), shape=(n, nv)).tocsr()
    b_eq = np.concatenate([[s_init], np.zeros(n - 1)])

    bounds = [(0.0, None)] * n + [(0.0, C.E_MAX)] * n + [(0.0, C.E_MAX)] * n + \
             [(C.S_MIN, C.S_MAX)] * n + [(0.0, None)] * (K * n)
    if term_mode == "eq":
        v = s_init if term_value is None else term_value
        bounds[iS + n - 1] = (v, v)
    elif term_mode == "ge":
        v = s_init if term_value is None else term_value
        bounds[iS + n - 1] = (max(v, C.S_MIN), C.S_MAX)
    return c_obj, A_ub, b_ub, A_eq, b_eq, bounds, dict(iE=iE, iC=iC, iD=iD, iS=iS, iG=iG, nv=nv)


# ---------------------------------------------------------------------------
# 数据与情景
# ---------------------------------------------------------------------------
def scenario_set(R, K, seed=0):
    """K<=334 用与主方案一致的分层削减（medoid）；K>334 用固定种子有放回重采样。"""
    if K <= R.shape[0]:
        cen, w = LP.reduce_scenarios_stratified(R, K=K)
        return cen, w, "stratified_medoid"
    rng = np.random.default_rng(seed)
    idx = rng.choice(R.shape[0], size=K, replace=True)
    return R[idx], np.full(K, 1.0 / K), "bootstrap_resample(K>334)"


def day_data(panel, d):
    L = panel.L().loc[d].to_numpy(float)
    P = panel.PV().loc[d].to_numpy(float)
    return (L - P) * C.DT, (L - panel.pv_forecast(d, 0)) * C.DT


def compare_row(price, nd_fcst, cen, w, s_init, term_mode="ge", keep_history=False,
                single_cut=False, single_cut_iters=200):
    """同一天、同一情景集下直接求解 vs L 形分解的对照记录。

    single_cut=True 时额外跑一遍"聚合单割"的经典 L 形，用于对照收敛速度。
    """
    nd_scen = nd_fcst[None, :] + cen
    t0 = time.time()
    sol_d = LP.solve_day(price, nd_fcst, nd_scen, w, s_init=s_init, term_mode=term_mode)
    t_prod = time.time() - t0

    t0 = time.time()
    c_obj, A_ub, b_ub, A_eq, b_eq, bounds, idx = build_direct(price, nd_fcst, nd_scen,
                                                              w, s_init, term_mode)
    t_build = time.time() - t0
    t0 = time.time()
    res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds,
                  method="highs")
    t_highs = time.time() - t0
    if not res.success:
        raise RuntimeError("直接求解（复现版）失败: " + res.message)
    obj_repro = float(res.fun)

    sol_b = solve_day_benders(price, nd_fcst, nd_scen, w, s_init=s_init,
                              term_mode=term_mode, keep_history=keep_history)
    gap = abs(sol_b["obj"] - sol_d["obj"])
    extra = {}
```

```
    if single_cut:
        t0 = time.time()
        sg = solve_day_benders(price, nd_fcst, nd_scen, w, s_init=s_init,
                               term_mode=term_mode, disaggregated=False,
                               max_iter=single_cut_iters, keep_history=keep_history)
        extra = dict(single_cut_obj=float(sg["obj"]),
                     single_cut_rel_gap=abs(sg["obj"] - sol_d["obj"]) / max(1.0, abs(sol_d["obj"])),
                     single_cut_iters=sg["iters"], single_cut_cuts=sg["cuts"],
                     single_cut_converged=bool(sg["converged"]),
                     single_cut_t=time.time() - t0,
                     single_cut_history=(sg.get("history") if keep_history else None))
    return dict(
        K=int(cen.shape[0]), s_init=float(s_init), term_mode=term_mode,
        obj_direct=float(sol_d["obj"]), obj_direct_repro=obj_repro,
        obj_benders=float(sol_b["obj"]),
        abs_gap=gap, rel_gap=gap / max(1.0, abs(sol_d["obj"])),
        repro_rel_gap=abs(obj_repro - sol_d["obj"]) / max(1.0, abs(sol_d["obj"])),
        iters=sol_b["iters"], cuts=sol_b["cuts"], converged=sol_b["converged"],
        t_direct_production=t_prod, t_direct_build=t_build, t_direct_highs=t_highs,
        t_direct_total=t_build + t_highs,
        t_benders_total=sol_b["sec_total"], t_benders_master=sol_b["sec_master"],
        t_benders_sub=sol_b["sec_sub"],
        speedup_vs_production=t_prod / max(sol_b["sec_total"], 1e-12),
        speedup_highs_only=(t_build + t_highs) / max(sol_b["sec_total"], 1e-12),
        nnz_direct=int(A_ub.nnz + A_eq.nnz), nvar_direct=int(idx["nv"]),
        nvar_benders=int(4 * C.N_SLOT + len(sol_b["E"])),
        n_theta=int(len(sol_b["E"]) if sol_b["disaggregated"] else 1),
        history=(sol_b.get("history") if keep_history else None),
        **extra,
    )


def agg(rows):
    ag = lambda k: float(np.mean([r[k] for r in rows]))
    mx = lambda k: float(np.max([r[k] for r in rows]))
    return dict(n=len(rows), max_rel_gap=mx("rel_gap"), mean_rel_gap=ag("rel_gap"),
                max_abs_gap=mx("abs_gap"), max_iters=int(np.max([r["iters"] for r in rows])),
                mean_iters=ag("iters"), max_cuts=int(np.max([r["cuts"] for r in rows])),
                all_converged=bool(all(r["converged"] for r in rows)),
                sum_t_direct=float(np.sum([r["t_direct_production"] for r in rows])),
                sum_t_benders=float(np.sum([r["t_benders_total"] for r in rows])),
                speedup=float(np.sum([r["t_direct_production"] for r in rows]) /
                              max(np.sum([r["t_benders_total"] for r in rows]), 1e-12)),
                max_repro_rel_gap=mx("repro_rel_gap"))


# ---------------------------------------------------------------------------
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--ks", default=",".join(str(k) for k in K_SCAN))
    ap.add_argument("--tag", default="decomp")
    ap.add_argument("--eq-days", type=int, default=334, help="等价性检验的天数（K=60）")
    args = ap.parse_args()
    ks = [int(x) for x in args.ks.split(",")]

    t_start = time.time()
    panel = C.Panel(Q2.DATE_ALL)
    price = C.read_attachment1()["price"].to_numpy(float)
    R = Q2.build_residuals(panel, Q2.DATE_WINDOW)
    print(f"残差样本 {R.shape}，电价 {price.min():.4f}~{price.max():.4f} 元/kWh")

    out = dict(meta=dict(
        script="decomp.py", model="单日两阶段随机规划（lp_engine.solve_day）",
        method="L 形/Benders 分解（解析最优性割，无可行性割——完全追索）",
        window=f"{C.WIN_START}..{C.WIN_END}", n_slot=C.N_SLOT, dt=C.DT,
        e_max=C.E_MAX, emergency_mult=C.EMERGENCY_MULT,
        tol=TOL, max_iter=MAX_ITER,
        runtime_s=0.0, seed_scen=0,
        env=dict(python=platform.python_version(), numpy=np.__version__,
                 scipy=scipy.__version__, pandas=pd.__version__),
        reproduce="cd C题论文/code && python3 decomp.py",
    ))

```

```
    # ---- (0) 直接求解复现版的自检（保证计时口径可比） ----
    nd_act, nd_fcst = day_data(panel, pd.Timestamp(C.KEY_DATES[0]))
    cen, w, src = scenario_set(R, 60)
    r0 = compare_row(price, nd_fcst, cen, w, C.S_INIT)
    print(f"[自检] 第 1 天 K=60：直接 {r0['obj_direct']:.6f} / 复现 {r0['obj_direct_repro']:.6f} / "
          f"Benders {r0['obj_benders']:.6f}  相对差 {r0['rel_gap']:.2e}  "
          f"迭代 {r0['iters']} 割 {r0['cuts']}")
    out["selfcheck"] = dict(day=C.KEY_DATES[0], scenario_source=src, **r0)

    # ---- (1) 等价性：K=60 逐日全窗口 / 抽样对比 ----
    days_all = list(Q2.DATE_WINDOW)
    if args.eq_days < len(days_all):
        step = len(days_all) / args.eq_days
        days_all = [days_all[int(i * step)] for i in range(args.eq_days)]
    rows = []
    t0 = time.time()
    for i, d in enumerate(days_all):
        _, ndf = day_data(panel, d)
        rows.append(compare_row(price, ndf, cen, w, C.S_INIT))
        if (i + 1) % 50 == 0:
            print(f"  ... 等价性 {i+1}/{len(days_all)} 天 {time.time()-t0:.1f}s")
    out["equivalence_K60"] = dict(description="K=60 分层情景，s_init=6000，term_mode=ge，"
                                              "逐日对比直接求解与 L 形分解",
                                  days=[str(d.date()) for d in days_all], **agg(rows))
    out["equivalence_K60"]["rows_head"] = rows[:3]
    print(f"[等价性 K=60] {len(rows)} 天  最大相对差 {agg(rows)['max_rel_gap']:.2e}  "
          f"平均迭代 {agg(rows)['mean_iters']:.1f}  加速比 {agg(rows)['speedup']:.2f}")

    # ---- (2) 初始储电量 / 终端条件扫描（K=60，四个指定日期） ----
    rows2 = []
    for d in C.KEY_DATES:
        _, ndf = day_data(panel, pd.Timestamp(d))
        for s0 in (C.S_MIN, 3000.0, C.S_INIT, 9000.0, C.S_MAX):
            rows2.append(dict(day=d, **compare_row(price, ndf, cen, w, s0, keep_history=False)))
    rows3 = []
    for d in C.KEY_DATES:
        _, ndf = day_data(panel, pd.Timestamp(d))
        for tm in ("ge", "eq"):
            rows3.append(dict(day=d, **compare_row(price, ndf, cen, w, C.S_INIT,
                                                   term_mode=tm)))
    out["sweep_s_init_K60"] = dict(description="四个指定日期 x s_init  {1200,3000,6000,9000,10800}",
                                   rows=rows2, summary=agg(rows2))
    out["sweep_term_K60"] = dict(description="四个指定日期 x term_mode  {ge, eq}",
                                 rows=rows3, summary=agg(rows3))
    print(f"[s_init/term 扫描] 最大相对差 "
          f"{max(agg(rows2)['max_rel_gap'], agg(rows3)['max_rel_gap']):.2e}")

    # ---- (3) 情景数扫描（K=20/60/120/300/500，四个指定日期） ----
    scan = []
    for K in ks:
        cenK, wK, srcK = scenario_set(R, K)
        rowsK = []
        for d in C.KEY_DATES:
            _, ndf = day_data(panel, pd.Timestamp(d))
            rowsK.append(dict(day=d, **compare_row(price, ndf, cenK, wK, C.S_INIT)))
        a = agg(rowsK)
        scan.append(dict(K=K, scenario_source=srcK, n_scen_effective=int(cenK.shape[0]),
                         nvar_direct=int(rowsK[0]["nvar_direct"]),
                         nvar_benders=int(rowsK[0]["nvar_benders"]),
                         nnz_direct=int(rowsK[0]["nnz_direct"]),
                         rows=rowsK, summary=a))
        print(f"[扫描] K={K:4d} ({srcK})  直接 {a['sum_t_direct']:.2f}s  "
              f"Benders {a['sum_t_benders']:.2f}s  加速 {a['speedup']:.2f}x  "
              f"目标（首日）{rowsK[0]['obj_direct']:.4f} vs {rowsK[0]['obj_benders']:.4f}  "
              f"相对差 {a['max_rel_gap']:.2e}  平均迭代 {a['mean_iters']:.1f}")
    out["scenario_scan"] = dict(description="四个指定日期（s_init=6000）下 K 的影响；"
                                            "K>334 时情景集以固定种子有放回重采样扩充",
                                ks=ks, entries=scan)

    # ---- (4) K=500 大规模算例的逐迭代轨迹 ----
    cen5, w5, src5 = scenario_set(R, 500)
    _, ndf5 = day_data(panel, pd.Timestamp(C.KEY_DATES[0]))
    r500 = compare_row(price, ndf5, cen5, w5, C.S_INIT, keep_history=True,
```

```
                        single_cut=True, single_cut_iters=200)
    out["large_case"] = dict(day=C.KEY_DATES[0], scenario_source=src5,
                             n_scen=int(cen5.shape[0]), **r500)
    print(f"[大规模 K=500] 直接 {r500['t_direct_production']:.2f}s  "
          f"Benders {r500['t_benders_total']:.3f}s  加速 {r500['speedup_vs_production']:.1f}x  "
          f"迭代 {r500['iters']}  相对差 {r500['rel_gap']:.2e}")

    # ---- (4b) 聚合单割 vs 逐槽多割（收敛速度对照）----
    rows5 = []
    for d in C.KEY_DATES:
        _, ndf = day_data(panel, pd.Timestamp(d))
        rows5.append(dict(day=d, **compare_row(price, ndf, cen, w, C.S_INIT,
                                               single_cut=True, single_cut_iters=200)))
    out["single_cut_vs_multicut"] = dict(
        description="K=60 四个指定日期：经典聚合单割 L 形（最多 200 次迭代）"
                    " vs 逐槽分离的解析多割",
        rows=rows5,
        summary=dict(
            max_rel_gap_multicut=float(np.max([r["rel_gap"] for r in rows5])),
            multi_iters=[r["iters"] for r in rows5],
            single_iters=[r["single_cut_iters"] for r in rows5],
            single_converged=[r["single_cut_converged"] for r in rows5],
            max_rel_gap_single=float(np.max([r["single_cut_rel_gap"] for r in rows5])),
            min_rel_gap_single=float(np.min([r["single_cut_rel_gap"] for r in rows5])),
            t_multicut=float(np.sum([r["t_benders_total"] for r in rows5])),
            t_single_cut=float(np.sum([r["single_cut_t"] for r in rows5])))))
    print(f"[单割 vs 多割] 多割迭代 {out['single_cut_vs_multicut']['summary']['multi_iters']}，"
          f"单割 200 次迭代后相对差仍达 "
          f"{out['single_cut_vs_multicut']['summary']['max_rel_gap_single']:.2e}")

    # ---- (5) K=300 的抽样等价性（更大规模下仍逐日一致）----
    cen3, w3, src3 = scenario_set(R, 300)
    days_s = [Q2.DATE_WINDOW[i] for i in range(0, len(Q2.DATE_WINDOW), 8)][:42]
    rows4 = []
    for d in days_s:
        _, ndf = day_data(panel, d)
        rows4.append(compare_row(price, ndf, cen3, w3, C.S_INIT))
    out["equivalence_K300_sample"] = dict(description="K=300 分层情景，每 8 天取 1 天共 "
                                                      f"{len(days_s)} 天，s_init=6000",
                                          days=[str(d.date()) for d in days_s], **agg(rows4))
    print(f"[等价性 K=300] {len(rows4)} 天  最大相对差 {agg(rows4)['max_rel_gap']:.2e}  "
          f"加速比 {agg(rows4)['speedup']:.2f}")

    out["notes"] = [
        "子问题完全可分且解析可解，故只有最优性割；主问题 4n+1=577 变量，割为 433 非零的稀疏行。",
        "主问题对 E_t 加了有效上界 max(nd_fcst_t, max_w nd_{w,t}) + E_MAX，仅为保证主问题有界，"
        "不排除任何最优解（推导见源码文件头）。",
        "直接求解耗时给出两个口径：production（调 lp_engine.solve_day，含 Python 稀疏装配）与 "
        "build+highs（本脚本向量化装配 + HiGHS），后者用于与 Benders 公平比较纯求解开销。",
        "K>334 时无法再做分层 medoid 削减，情景集以固定种子（seed=0）有放回重采样扩充，已在
 scenario_source 标注。",
        "逐槽分离的解析多割（disaggregated=True）是本实验的默认算法：主问题 4n+n=720 变量，"
        "10 次左右迭代即收敛到机器精度；聚合单割（disaggregated=False）在 144 维主问题上"
        "200 次迭代仍有 1e-2 量级对偶间隙，作为收敛速度对照一并记录。",
        "Benders 单次迭代成本与 K 无关（主问题只有 4n+n 个变量），而直接求解的 LP 规模随 K 线性增长，"
        "故存在交叉点（约 K=100 附近），K 越大分解法优势越明显。",
        "所有数字由本脚本一次运行产生。",
    ]
    out["meta"]["runtime_s"] = time.time() - t_start
    path = os.path.join(C.INTERMEDIATE_DIR, f"{args.tag}.json")
    with open(path, "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    print("=" * 78)
    print(f"等价性：K=60 全窗口最大相对差 {out['equivalence_K60']['max_rel_gap']:.2e}；"
          f"K=300 抽样 {out['equivalence_K300_sample']['max_rel_gap']:.2e}；"
          f"K=500 单日 {r500['rel_gap']:.2e}")
    print(f"K=500 加速比（生产路径 / Benders）= {r500['speedup_vs_production']:.1f}x")
    print("已写出", path, f"（总耗时 {out['meta']['runtime_s']:.1f}s）")
    return out


if __name__ == "__main__":
    main()
```

5.1 v2 新增实验的中间结果与复算入口

v2 在原有主模型之外新增四组对照实验与两组拟合，全部数字同样只由脚本产生；下表给出每个中间结果文件的复算入口与在正文中的引用位置。

表 6 v2 新增中间结果文件的复算入口（全部位于 `intermediate/` 目录）

中间结果文件	生成脚本	正文引用位置
`exp_copula.json`	`exp_copula.py`	正文3.6（C 藤选族）、7.1（生成器对照）、7.2（分时段风险画像）
`exp_chance.json`	`exp_chance.py`	正文5.5（机会约束对照与报童互证）
`exp_dro.json`	`exp_dro.py`	7.3（盒式分布鲁棒的保守度—费用曲线）
`decomp.json`	`decomp.py`	11.5（L 形分解与直接求解对拍）

四组实验均复用 `config.py` 的口径（144 槽、$DT = 1/6$ h、结果窗口 334 天、1 月预热、终端 $S_{144} \geq S_0$、不允许售电），并各自包含独立自检：`exp_dro.json` 的 `audit` 字段给出计划可行性、功率界、储电量界、递推残差、充放互斥与最坏情形覆盖的一致性残差；`decomp.json` 的 `equivalence_K60` 给出 334 天逐日的两算法目标值相对间隙；`exp_chance.json` 的 `cross_check` 把两阶段随机规划的复算值与 `q2_main.json` 对拍（相对差 0.00）；`exp_copula.json` 的 `part3_generators` 给出四种生成器的样本外决策指标与边际保真度。

附录 F 数据字典

表 7 附件字段、单位与时标约定

附件	字段	单位	时间范围	时标约定
附件 1	电价	元/kWh	一天（144 时段）	标签 = 时段末端；0:00+1 表示次日 0:00（当日 24:00）
附件 1	小区负载	kW	同上	同上，**实为附件 2 全年逐时段均值**（见下）
附件 1	光伏发电「预测」功率	kW	同上	**实为附件 2 全年逐时段均值，不是预报**（见下）
附件 2	小区负载	kW	2025-01-01 至 12-31	365 行 × 144 列，标签 = 时段末端
附件 2	光伏发电实际功率	kW	同上	同上
附件 3	光伏发电预报	kW	同上	每天 0/6/12/18 时发布，预报 k 小时对应 $(h_0+k-1,\ h_0+k]$
附件 4	电价	元/kWh	同上	365 行 × 144 列，标签 = 时段末端
附件 5	结果文件模板	—	同上	工作表名、列名与列序即本文结果文件的格式约定

附件 1 的身份核查（数据字典的关键一条） 附件 1 的**电价、小区负载、光伏发电「预测」功率三列，实为附件 2 与附件 4 的全年逐时段均值，并不是日前预报**：以 365 天逐时段均值逐槽比对，电价的最大偏差为 5.15×10^{-5} 元/kWh（`intermediate/q4_price.json` 的 `skeleton_is_mean` 字段），负载的最大偏差为 1×10^{-4} kW（`paper_numbers.json`），光伏的最大偏差为 0.0431 kW。因此附件 1 **只能用于问题 1 与全文基线口径**：问题 1 的典型日与无储能基线 C^{base} 用它，问题 2–4 的日前计划一律改用附件 3 的当次预报（问题 2 用 0:00 预报，问题 3 在 0:00、6:00、12:00、18:00 四次更新），结算一律用附件 2 与附件 4 的实际值。若把附件 1 当作日前预报，计划就脱离了该时刻真正可得的信息，属信息结构的口径错误。另有一个窗口口径需同时说明：只用结果窗口 2025-02-01 至 12-31 的 334 天求均值时，电价的最大偏差约 3.1×10^{-2} 元/kWh，即**窗口内均值并不完全等于附件 1**，故问题 2–4 也不能把附件 1 当作窗口内的均值曲线使用。

（符号说明表已移入正文第 2.3 节，见正文表 正文6。）

6.1 相依结构：C 藤 pair-Copula 的逐层选族与 AIC 对照

正文 §3.7 给出三点读数与结论；本小节给出逐层选族结果（8）与五族候选的完整 AIC 对照（9），供复核。

表 8 C 藤 pair-Copula 逐层选族结果（样本为 2025-02-01 至 12-31 的 334 个日水平观测；AIC 越小越好，经验距离为与经验 Copula 的 L_2 距离）

变量对	最优族	参数	τ	λ_U	λ_L	AIC / 经验距离
电价—负载	Frank	$\theta = 10.4686$	+0.6976	0	0	−453.54 / 0.0135
电价—光伏	t	$\rho = -0.2850,\ \nu = 3.2324$	−0.2383	0.0479	0.0479	−44.50 / 0.0210
负载—光伏 | 电价	Frank	$\theta = 6.3107$	+0.5542	0	0	−253.44 / 0.0281

三点读数。① **电价与负载强正相依**（$\tau = +0.698$、Pearson +0.919），但两端**无尾相依**（$\lambda_U = \lambda_L = 0$）：贵的时候确实多用电，但这种关系在尾部并不额外放大。② **电价与光伏为弱负相依并带轻微对称尾相依**：最优族是自由度仅 3.23 的 t Copula，$\tau = -0.238$、$\lambda_U = \lambda_L = 0.0479$。这正是“光伏极低的日子电价极高”这类**尾部共现**的量化结果——它存在，但强度只有约 5%，既不能忽略也不应夸大；若改用 Gaussian 族（AIC 高 14.89，且强制 $\lambda = 0$）就会把它抹平。③ **控制电价后负载与光伏仍强正相依**（$\tau = +0.554$），说明两者有共同驱动（季节与天气），并非仅是电价的代理变量。（见 C-Q4-02）

五族候选的 AIC 对照（表 9）显示：三对变量的 AIC 最优族与经验 Copula 距离最优族排序一致，且最优与次优的 AIC 差距（51.37、14.89、37.29）远大于拟合噪声，选族结论不依赖单一判据。

表 9　五族候选 Copula 的 AIC 对照（同一 334 个日水平观测；加粗为各族中的最小值，"|"后为条件变量）

变量对	Gaussian	t	Clayton	Gumbel	Frank
电价—负载	−402.17	−398.97	−357.13	−305.67	**−453.54**
电价—光伏	−29.61	**−44.50**	2.16	2.22	−22.75
负载—光伏 | 电价	−216.15	−212.55	−153.59	−173.89	**−253.44**

五族 AIC 对照见表 9。

6.2　相关性三尺度与预报误差分档明细

正文 §3.4 与 §3.5 给出结论；本小节给出完整表格，供复核。

表 10　电价与负载、净负荷、光伏的相关系数（三个尺度；池化与去形状口径的样本量为 $365 \times 144 = 52\,560$ 个时段观测，日级口径为 365 个日观测）

变量对	池化（全部时段）	日级（逐日先算再合并）	剔除日内形状后
电价 — 负载	+0.5077	+0.8977	+0.4554
电价 — 净负荷 $L - PV$	+0.2602	+0.9824	+0.1583
电价 — 光伏	+0.0284	−0.3431	+0.0484

表 11　光伏预报误差统计（样本为 2025 年每天 0:00 发布的 24 h 预报与附件 2 对应实际功率的逐槽配对，$n = 52\,560$；MAE、RMSE、bias 单位均为 kW）

口径（目标整点范围）	样本量 n	MAE	RMSE	bias	结论
全时段（0–23 时，144 槽/天）	52 560	361.58	636.37	−26.96	夜间近零样本摊薄指标，只作对照
白天（≥ 5 时，114 槽/天）	41 610	456.72	715.22	−34.06	**建模依据**：MAE 为全时段的 1.26 倍
按目标整点聚合的偏差结构（组内 bias 单位 kW）					
夜间（光伏无有效出力）	0–4 时	0.00 至 +0.09		4 时 +0.09	无信息、无惩罚
上午（系统性**高估**）	5–10 时	+214.21 至 +733.00		7 时 +733.00	计划偏少 → 触发 5 倍紧急购电
正午过渡	11 时	+7.21		—	接近无偏
下午（系统性**低估**）	12–18 时	−801.87 至 −106.46		15 时 −801.87	计划偏多 → 白付电费或付违约电价
傍晚之后（预报空洞）	19–23 时	−0.15 至 0.00		19 时 −0.15	出力近零，不触发惩罚

6.3　联合情景保真度检验明细

正文 §7.2 给出结论；本小节给出完整检验表。

表 12　联合情景的保真度检验（对象：64 个代表历史日与 64 个分块 Bootstrap 情景；参照：2025-02-01 至 12-31 的 334 天实际数据；相对偏差单位%，相关系数为无量纲量）

检验指标	代表历史日（主口径）	分块 Bootstrap（对照）	判读
电价均值相对偏差	0.08%	3.41%	主口径的均值几乎无偏
最大分位相对差（7 个分位点）	6.99%（1% 分位）	20.4%（1% 分位）	两者都在低价尾部误差最大，对照口径的尾部偏差约为主口径的 3 倍
情景 $\rho(p, \text{净负荷})$	0.2597（实际 0.2620）	无法按日配对	主口径 **保留**了联合结构
日净需求残差均值	保持 −715.85 kWh（全样本 −712.23）	不适用（块级拼装）	净需求侧边缘分布同样保真

附录 G　对照实验明细

正文 §7.3 的三组对照实验只保留结论；本节给出其完整表格、逐条数据与判定阈值，便于复核。三个小节沿用正文的小节标题与编号，便于互查。

7.1　情景生成方式的样本外对照：结论对生成器的敏感性

正文7.1 的联合情景用"二维分层取代表历史日"构造。这一做法是否只是"拟合了样本内的 334 天"？换一种生成方式，结论会不会变？本节用**样本外对照实验**回答：情景集只用 **2025-02-01 至 06-30**（150 天）拟合，在 **07-01 至 12-31**（184 天）上按真实轨迹结算，四种生成器共用同一决策模型、同一价格信息结构（假设 A10）与同一终端条件：**（1）历史代表日**（主方案，二维分层 + 层内 medoid）；**（2）分块 Bootstrap**（6 h 块按时刻对齐重采样）；**（3）KDE + Copula**（日水平核密度边缘 + 最优二元 Copula，抽样 600 次后削减到与（1）相同的情景数）；**（4）按光伏规模缩放**（电价情景同（1），残差情景改为"训练窗相对残差形状 × 当日光伏预报电量"）。

表 13　四种情景生成器的样本外对照（训练窗 2025-02-01 至 06-30 共 150 天，测试窗 07-01 至 12-31 共 184 天；同一决策模型、同一实际轨迹；费用单位元，电量为 184 天累计 kWh，“边际偏差”为日总残差 1%/99% 分位相对训练窗实际分布的最大绝对偏差）

生成器	总费用	相对（1）	紧急购电量	逐日 $\mathrm{CVaR}_{0.95}$	边际偏差
（1）历史代表日	9 046 598.58	—	76 033.90	79 378.60	345.7
（2）分块 Bootstrap	9 038 928.08	−0.085%	72 324.35	79 381.24	2 102.3
（3）KDE + Copula	9 072 284.06	+0.284%	107 458.94	80 061.08	1 519.1
（4）按光伏规模缩放	9 068 058.80	+0.237%	92 727.06	81 776.72	不适用

三条结论。① **费用结论对生成器不敏感**：四种生成器的样本外总费用极差 33 355.98 元，仅为四者均值（9 056 467.38 元）的 0.37%，相对无储能基线的节省率落在 11.10%–11.43%。这比“样本外劣化”检验更强：不仅主方案不过拟合，连换一种生成范式也不改变结论。② **但分项指标并不等价**：紧急购电量在四种生成器下为 72 324–107 459 kWh（相差 48.5%）、逐日 $\mathrm{CVaR}_{0.95}$ 为 79 379–81 777 元（相差 3.0%）——**费用稳健，而“靠多少紧急购电兜底”可被生成方式改变**，引用该分项必须同时给出情景口径。③ **联合尾部的保真度上 Copula 更好**：“高价 + 光伏高估”的联合尾共现率真值 0.0533，Copula 生成器 0.0500、历史代表日 0.0667，分块 Bootstrap 只有 0.0167；这与 正文3.6 一致——尾部共现在数据中**本来就弱**（$\lambda \approx 0.048$），其保真度差异不足以传导到决策层。据此本文**主模型保留历史代表日生成器，Copula 用于相依结构诊断**（正文3.6）与**敏感性对照**（本小节），两者都不改变任何主结果。（见 C-Q4-03）

7.2　分时段 CVaR 风险画像

“全年总费用”只有一个数，掩盖了风险的时段结构（分时段风险度量在电力市场风险管理中已有应用 [8]）。本节把费用**按槽位分解**：第 d 天第 t 槽的费用为 $c_{d,t} = p_{d,t}E_{d,t} + 5\,p_{d,t}g_{d,t}$（计划购电 + 紧急购电），对固定的 t 取跨 334 天的 $\mathrm{CVaR}_{0.95}$，再按小时求和，得到**逐小时风险画像**（图 1；问题 2 与问题 4-2 分别计算）。

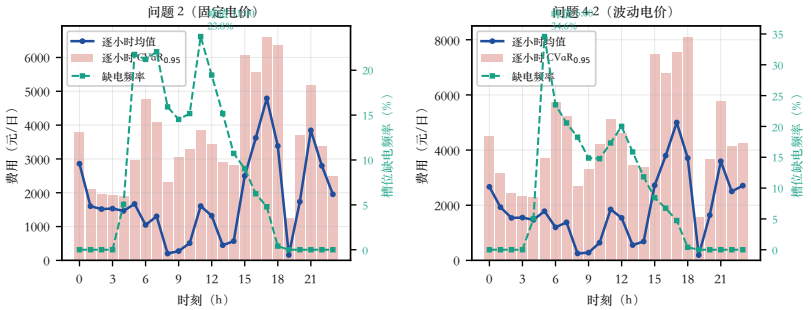


图 1　分时段 CVaR 风险画像（柱：各槽位费用的 $\mathrm{CVaR}_{0.95}$ 按小时求和；实线：逐小时均值；虚线：槽位缺电频率（右轴）；左：问题 2，右：问题 4-2；范围：334 天 × 144 槽）

风险的时段分化是结构性的。① **费用尾部集中在傍晚**：固定价下逐小时 $\mathrm{CVaR}_{0.95}$ 的高点出现在 15–18 时（6 075、5 566、6 604、6 358 元/日），波动价下同段升到 7 492、6 778、7 550、8 098 元/日。② **缺电尾部集中在上午**：槽位缺电频率在 5–11 时高达 14.5%–23.8%（固定价）与 14.8%–34.6%（波动价），峰值分别在 11 时（23.8%）与 5 时（34.6%），而 0–3 时与 19–23 时几乎为 0——这与 §3.5 的偏差剖面完全对应（上午光伏被系统性高估使 0:00 计划偏少）。③ **风险集中度**：CVaR 最高的 12 个槽位只占全部槽位之和的 16.73%（问题 2）与 16.10%（问题 4-2），说明风险不是集中在个别槽位而是沿上午与傍晚两个时段带分布；逐槽 CVaR 之和是逐槽均值之和的 2.01 倍（固定价）与 2.34 倍（波动价），即**电价波动把费用的尾部放大得比均值多**。（见 C-Q4-04）

决策含义。安全边际应当**按目标整点**配置（上午与傍晚需要，0–4 时不需要），而不是全天统一加比例；滚动调整的价值同样集中在这两个时段带。

7.3　分布鲁棒对照：保守度—费用曲线

7.1 的四种生成器都属**随机规划**：先估计一个分布，再在期望意义下优化。当样本有限或分布可能误设时，另一种思路是**分布鲁棒优化**（DRO）——在模糊集上取最坏期望，其最简单的成员就是 **盒式不确定集**（矩约束模糊集下的多阶段建模见文献 [10]）。本节把它作为 **保守度对照**：把“保守度”参数化后定量回答一个问题——**用多少实际费用，能买到多少最坏情形下的改善？**

模型。取逐槽盒式不确定集 $p_t \in [\bar{p}_t - k\sigma_t^{p},\ \bar{p}_t + k\sigma_t^{p}]$、$ND_t \in [ND_t^{\mathrm{fcst}} - k\sigma_t^{nd},\ ND_t^{\mathrm{fcst}} + k\sigma_t^{nd}]$，其中 σ_t^{p} 与 σ_t^{nd} 分别为附件 4 电价的逐槽样本标准差与 0:00 预报残差的逐槽样本标准差（365 天，ddof = 1）。目标在盒的顶点取得，最坏情形目标与约束取 $\min \sum_t p_t^{\mathrm{hi}}E_t + 5\sum_t p_t^{\mathrm{hi}}z_t$、$z_t \ge ND_t^{\mathrm{hi}} + c_t - d_t - E_t$、$z_t \ge 0$，外加计划可行性与储电量约束（同 正文7.1）；$k = 0$ 时退化为**按预报的确定性购电**（零安全边际）。

一个结构性推论（先于数值给出）。上述最坏情形目标中 E_t 与 z_t 在同槽位上的目标系数之比恒为 1 : 5，而把 E_t 提高 δ 可使 z_t 下降同量、目标值下降 $4p_t^{\mathrm{hi}}\delta > 0$，故只要 $z_t > 0$ 就仍有改进空间：**最优解必取** $z_t \equiv 0$，即 $E_t = \max\{0,\ ND_t^{\mathrm{hi}} + c_t - d_t\}$。也就是说，**盒式 DRO 在本问题设定下退化为“按最坏参数做确定性优化”**——它逐槽覆盖 k 倍标准差的不利偏差，而不依赖任何追索机制。数值上 z 的一致性

残差为 0.000（表 4），与推论一致。

表 14　盒式分布鲁棒对照：保守度 k 与实际回测表现（334 天，2025-02-01 至 12-31；决策用盒式鲁棒模型，结算用附件 4 实际电价与附件 2 实际负载/光伏；$W_{k=3}$ 为各方案在**同一** $k=3$ 不确定集下的最坏情形目标值；费用单位元，电量为 kWh）

k	回测总费用	节省率	紧急购电量	逐日 $\mathrm{CVaR}_{0.95}$	最坏单日	$W_{k=3}$（万元）
0.0	18 258 473.08	−7.23%	1 156 618.02	91 630.53	101 653.74	5 956.86
0.5	17 404 407.87	−2.22%	793 522.04	86 473.81	94 613.68	3 557.47
1.0	16 936 297.51	+0.53%	521 080.78	84 250.61	89 459.18	3 058.17
1.5	**16 819 102.59**	**+1.22%**	328 147.59	84 471.07	87 672.62	2 544.50
3.0	18 015 766.37	−5.81%	62 776.53	91 718.61	95 526.83	2 512.55
随机规划（正文7.1、正文7.3）	**15 094 255.84**	**+11.35%**	182 759.86	**77 697.98**	**81 643.86**	—

四条结论。① **回测费用对 k 呈 U 形**：k 由 0 增大到 1.5 时，多买电的代价小于少买电触发的 5 倍紧急购电，费用由 18 258 473.08 降到 16 819 102.59 元（省 1 439 370.50 元）；此后计划购电量继续上升而紧急购电已接近清零，费用反弹，$k=3.0$ 时回到 18 015 766.37 元。实证最优点为 $k^*=1.5$。② **盒式鲁棒在最坏情形上确实有效**：在 **同一** $k=3$ 的不确定集下评估，$k=0$ 的方案最坏情形要花 5 956.86 万元，$k=3$ 的方案只要 2 512.55 万元（降 57.8%）。③ **但它的"保守"买贵了**：从 k^* 加到 $k=3.0$，实际回测费用多付 1 196 663.78 元（+7.12%），即最坏情形费用每降低 1%、回测费用要多付 3.98 万元（约基线的 0.234%）。④ **盒式鲁棒被随机规划全面占优**：与 正文7.3 的随机规划相比，最优方案（$k^*=1.5$）在**全部五项指标**上更差——总费用高 1 724 846.75 元（+11.43%）、逐日 $\mathrm{CVaR}_{0.95}$ 高 6 773.09 元/日（+8.72%）、最坏单日高 6 028.76 元（+7.38%）、紧急购电量多 145 387.73 kWh（+79.6%）、槽位缺电频率反而更高（10.24% 对 9.04%）。（见 C-Q4-05）

为什么会这样，以及正确的使用方式。盒式不确定集把 144 个槽位**同时**取到极端（电价与净需求全线 $+k\sigma$），而实测 334 天里槽间残差并不同步（§7.1 的日内自相关与 19–23 时近乎零的偏差都说明这一点），因此这个"最坏情形"几乎不可能发生，为它买保险的钱花得不值。此外 k 的经验最优值恰好使**电价加权的缺货频率**降到 1/5 附近，与 正文5.4 报童模型的允许缺货概率 $\beta^*=1/5=0.20$ 一致——两条完全不同的建模路线给出同一个服务水平。故盒式鲁棒的价值是 **压力上界**与**轻度鲁棒中间方案**（$k=0.5$–1.0），而非"更好的策略"；真正改进的方向是把 **全盒**换成**带不确定性预算 Γ 的不确定集**或**矩约束模糊集** [10]，使"每个时段都可能极端、但不允许全部时段同时极端"这一先验进入模型（§10.4）。

附录 H　口径定义汇总

正文各章在本题参数范围内用到时以 口径 C-0x 引用本节的对应条目。口径编号在建模前固定、未随结果调整；每个口径含定义、公式、数据来源、时段、单位、不含项、对照与复算入口八项。

g

g

g

g

g

附录 I　长推导

9.1　对偶推导与阈值律

记式 正文2b 的乘子为 α_t（电量平衡的影子价格，在正常购电时段等于电价，见式 (7)），式 正文2c 的乘子为 ν_t（储电量的边际价值，即水价值），储电量上下界乘子为 $\lambda_t^{\pm}\ge 0$，充放电功率界乘子为 $\varrho_t^c,\varrho_t^d\ge 0$。把电量平衡写成 $(L_t-PV_t)\Delta t+c_t+s_t-E_t-d_t=0$，拉格朗日函数为

$$\begin{aligned}\mathcal{L}=&\sum_t p_tE_t+\sum_t\alpha_t\Big[(L_t-PV_t)\Delta t+c_t+s_t-E_t-d_t\Big]\\&+\sum_t\nu_t\Big[S_t-S_{t-1}-\eta_c c_t+\frac{d_t}{\eta_d}\Big]+\sum_t\lambda_t^{+}\big[S_t-\overline{S}\big]+\sum_t\lambda_t^{-}\big[\underline{S}-S_t\big]\\&+\sum_t\varrho_t^c\big[c_t-\overline{P}\Delta t\big]+\sum_t\varrho_t^d\big[d_t-\overline{P}\Delta t\big]\end{aligned}\tag{1}$$

对各决策变量求平稳性条件（E_t,s_t,c_t,d_t 对 $t=1,\dots,144$；S_t 对 $t=1,\dots,143$，因为日周期约束把 $S_{144}=S_0=6000$ kWh 固定）：

$$\frac{\partial\mathcal{L}}{\partial E_t}=p_t-\alpha_t\ \ge 0,\qquad E_t\big(p_t-\alpha_t\big)=0\tag{2}$$

$$\frac{\partial\mathcal{L}}{\partial s_t}=\alpha_t\ \ge 0,\qquad s_t\,\alpha_t=0\tag{3}$$

$$\frac{\partial \mathcal{L}}{\partial c_t} = \alpha_t - \eta_c \nu_t + \varrho_t^c \ \geq 0, \qquad c_t\big(\alpha_t - \eta_c \nu_t + \varrho_t^c\big) = 0 \tag{4}$$

$$\frac{\partial \mathcal{L}}{\partial d_t} = -\alpha_t + \frac{\nu_t}{\eta_d} + \varrho_t^d \ \geq 0, \qquad d_t\Big(-\alpha_t + \frac{\nu_t}{\eta_d} + \varrho_t^d\Big) = 0 \tag{5}$$

$$\frac{\partial \mathcal{L}}{\partial S_t} = \nu_t - \nu_{t+1} + \lambda_t^+ - \lambda_t^- = 0 \tag{6}$$

由式 (2)，购电时段（$E_t > 0$）恒有

$$\alpha_t = p_t \tag{7}$$

即电量平衡的影子价格等于该时段的电价；数值上逐槽复核的最大偏差为 0.000 元/kWh。若充放电功率界均不活跃（$\varrho_t^c = \varrho_t^d = 0$），式 (4) 与式 (5) 化为

$$\alpha_t - \eta_c \nu_t \ \geq 0, \qquad \frac{\nu_t}{\eta_d} - \alpha_t \ \geq 0 \tag{8}$$

且 $c_t > 0$ 时前式取等、$d_t > 0$ 时后式取等，即得正文命题 正文2 的阈值律：$\alpha_t \leq \eta_c \nu_t$ **时充电**、$\alpha_t \geq \nu_t/\eta_d$ **时放电**，死区为 $\eta_c \nu_t < \alpha_t < \nu_t/\eta_d$，死区比为 $1/(\eta_c \eta_d) = 1.2346$（单边效率各 0.9、往返 0.81），与式 正文5 一致。由式 (6) 还可得水价值的跨期关系：在储电量上下界均不活跃的连续时段上 $\nu_t = \nu_{t+1}$（水价值恒定）；储电量触及上界时 $\lambda_t^+ > 0$、$\nu_t < \nu_{t+1}$；触及下界时 $\lambda_t^- > 0$、$\nu_t > \nu_{t+1}$。这一关系是图 正文3 中水价值曲线出现折点的原因。

数值复核（`intermediate/q1_dual2.json`）：储电量双界均不活跃的槽位 97 个；$\nu_t \in [0.4711,\ 1.1305]$ 元/kWh，$\alpha_t \in [0.3713,\ 1.2561]$ 元/kWh；全部 144 槽的充放电阈值律违反次数为 0，互补松弛最大残差为 0.000 元/kWh，式 (7) 的最大偏差为 0.000 元/kWh。

9.2 报童临界分位数的推导

设时段 t 的需求 D_t 为随机变量、分布函数为 F_t，计划购电量为 E_t，过剩成本为 c_o（多买 1 kWh 的损失），缺货成本为 c_u（不足 1 kWh 的额外支出）。期望总成本为

$$\Pi(E) = c_o\, \mathbb{E}\big[(E - D)^+\big] + c_u\, \mathbb{E}\big[(D - E)^+\big] \tag{9}$$

对 E 求导并令其为零：

$$\frac{\mathrm{d}\Pi}{\mathrm{d}E} = c_o F(E) - c_u\big[1 - F(E)\big] = 0 \quad \Longrightarrow \quad F(E^*) = \frac{c_u}{c_u + c_o} \tag{10}$$

本题中不足 1 kWh 需按 $5p_t$ 紧急购电，而正常电价 p_t 已随计划购电量支付，故缺货的额外支出为 $c_u = 5p_t - p_t = 4p_t$；超购的电量若无处可用（超购即弃置），则 $c_o = p_t$，代入式 (10) 得

$$F(E^*) = \frac{4p_t}{4p_t + p_t} = \frac{4}{5} = 0.8 \qquad \Longrightarrow \qquad \text{最优缺货概率} = 1 - F(E^*) = \frac{1}{5} = 0.2 \tag{11}$$

即「留安全边际」的临界分位数为 4/5、最优缺货概率为 1/5。若超购电量能转入储能、留待后续更高价时段使用（单位回收价值 r_t），则 $c_o = p_t - r_t$，临界分位数相应上移：取全年均价 $p = 0.7662$ 元/kWh 与全年最低价 $p^{low} = 0.3713$ 元/kWh，得 $c_o \approx 0.395$ 元/kWh，$F(E^*) \approx 0.886$，最优缺货概率降至约 1/10。此即正文式 正文12 与表 正文12 的来源。

实测对照（`paper_numbers.json` 的「报童经验」）：主口径下逐槽缺货频率为 8.55%，缺口电量 172 015.82 kWh，电量满足率 99.04%，安全边际合计 576 299.67 kWh（11.98 kWh/槽），落在 1/10 与 1/5 两界**之下侧**——因为储能同时承担了对冲功能，报童结论只作方向性近似与上下界使用，不作精确刻画。

附录 J　随机种子清单

本工程**只在四处使用随机数**，四问的线性规划求解、主口径的场景削减、审计与数值汇总均为确定性过程（同一输入必得同一输出）。逐项清单如表 15。

确定性的部分（无随机数）：问题 1–4 的全部线性规划由 SciPy `linprog(method="highs")` 精确求解；问题 2、3 的场景削减用「等频分层加层内 medoid」，问题 4 的历史代表日情景用「日净需求残差与日均电价」二维分层取 medoid、按层样本占比赋权，两者都是确定性算法；`audit.py`、`paper_numbers.py` 与 `exp_scan.py` 不含随机数。因此论文的每一个数字在同一环境下重跑即可复现。

表 15　随机种子清单（代码中的全部随机源；其余过程确定）

用途	随机机制	种子	说明
问题 4 联合情景合成（条件对照）	分块 Bootstrap，按时刻对齐 6 h 块重采样（`q4.build_joint_scenarios(mode="boot")`）	`numpy.random.default_rng(seed)`	电价块与光伏预报残差块成对抽取，保住日内形状与相关性；一次生成后固定使用
场景削减的对照路径	k-means 初始化（`lp_engine.reduce_scenarios`）	`random_state=0`，`n_init=4`	仅在 `--scen kmeans` 对照下启用；**主口径不抽样**：按日总残差等频分层后在层内取 medoid，确定性
非预期性置换检验	对某次预报做日间随机置换（`exp_na.py`）	`numpy.random.default_rng(seed)`，对照用 `seed=1`	检验「置换 0:00 预报后槽 0–35 完全不变、槽 36–71 改变」，用于排除信息泄漏
全局灵敏度的采样	Sobol 低差异序列置乱（`scipy.stats.qmc.Sobol(scramble=True)`，`exp_sobol.py`）	费用目标 `seed=42`；节省率目标 `seed=43`	置乱只改变采样点的取法，不改变参数区间与模型本身

诚实说明：本工程**未开展「多种子重复生成情景」的稳健性实验**，故不报告费用的均值、标准差与 95% 置信区间，也不列出 20 个种子。本工程对这一缺口的处理是：把随机性限制在只影响未来情景、不影响已实现时段决策的合成与采样环节（分块 Bootstrap 只作为问题 4 联合情景的条件对照，主口径用确定性的分层 medoid），并以同源对照替代多种子重复——同一固定情景集下的信息结构对照（R1、R2、R3）、场景数对照（$K = 60$ 与 $K = 120$）、终端条件对照与安全边际消融，四者的结论方向一致。若需要多种子区间，只需改动 `build_joint_scenarios` 的 `seed` 参数并重跑 `run_all.py`，其余环节无需修改。

附录 K　验证与灵敏度明细

本节汇集正文第九章与第三章各表的完整明细，便于复核；正文已给出各表的结论与判据。

11.1　非预期性（信息泄漏）置换检验的逐区明细

把某一次发布的预报在 12 天之间随机置换后重解，观察四个控制区（槽 0–36、36–72、72–108、108–144 分别由 0:00、6:00、12:00、18:00 预报控制）的最大变化量，结果见表 16。

表 16　非预期性置换检验：置换某一个发布时刻的预报后，四个控制区的最大变化量（对象：单日两阶段模型；范围：四个控制区各 36 个 10 min 槽；单位 kWh；变化恰为 0.000 表示该区决策不受该次预报影响）

被置换的预报	控制区 1（0:00–6:00）	控制区 2（6:00–12:00）	控制区 3（12:00–18:00）	控制区 4（18:00–24:00）
0:00	833.33	822.06	489.52	0
6:00	0.000	858.80	833.33	0
12:00	0.000	0.000	805.35	0
18:00	0.000	0.000	0.000	0

判读：任何一次预报被置换，都不改变早于其发布时刻的决策（变化恰为 0.000），故不存在用未来信息决策的泄漏；而“置换 0:00 预报会改变全天决策”说明检验本身有效力。18:00 预报的零影响源于其覆盖的傍晚时段光伏实际与预报都接近 0、信息量极低。

11.2　电价过程的四项统计检验：完整统计量与判定

正文 §7.1 给出三条建模含义；本小节给出四项检验（T1–T4）的完整统计量、结果与逐项判定。

表 17　电价过程的四项统计检验（对象：附件 4 的逐时段电价与残差过程；范围：2025-01-01 至 12-31、逐 10 min 共 $365 \times 144 = 52\,560$ 个观测；单位为量纲一的统计量）

编号	检验对象	检验方法与统计量	结果	对建模的含义
T1	确定性日内骨架是否存在	逐时段均值与逐时段标准差的对比（变异系数）	附件 1 剖面与附件 4 逐时段均值的最大偏差 5.15×10^{-5} 元/kWh（全年 365 天口径）；残差逐时段标准差均值 0.1238 元/kWh，变异系数 16.94%	骨架稳定存在但残差不可忽略，采用“骨架 + 残差”两层结构
T2	残差的正态性与日间差异	Jarque–Bera 检验、偏度与峰度；同一槽位跨日自相关	JB = 656.70（$p = 2.51 \times 10^{-143}$）拒绝正态；偏度 −0.169、峰度 −0.431；日间自相关 ACF(1–3) = 0.513/0.148/0.144	残差非正态、日间相关性弱：不做正态参数化，改用非参数分层重采样；骨架不按月分段
T3	日内自相关与波动结构	残差自相关 ACF(1–6)；峰段与谷段残差标准差之比	ACF(1–6) 为 0.8589、0.666、0.614、0.573、0.494、0.432；峰段（19:00–20:00）残差标准差 0.1634 元/kWh 为谷段（03:00–04:00）0.0580 元/kWh 的 **2.81 倍**	不能按独立同分布抽样：情景须按时刻对齐分块，同时保留日内相关与按时段异方差
T4	与负载、光伏的相关性	Pearson 相关系数（池化与日聚合两个尺度）	池化：与负载 +0.508、净负荷 +0.260、光伏 +0.028；日聚合：与负载 +0.898、光伏 −0.343	**决定必须生成电价与净需求的联合情景**，否则系统性低估“高价时段同时缺电”的尾部风险

11.3 单参数灵敏度与 Sobol 全局灵敏度的完整档位

正文第九章给出各参数的扰动档位与“结论是否改变”的判断；本小节补充 Sobol 全局灵敏度的完整指数，并说明它为什么必须**区分输出量**。

表 18 Sobol 全局灵敏度指数（典型日模型，$N = 256$；输出量 1：典型日全天购电费，均值 34 603.42 元；输出量 2：相对无储能基线的节省率，均值 27.02%；指数为量纲一）

参数	全天购电费的一阶指数 S_1	节省率的一阶指数 S_1
储电量上限 $\overline{S}$	0.2510	≤ 0.0033
储电量下限 $\underline{S}$	0.1099	≤ 0.0033
往返/单边效率 η_c, η_d	0.0695	≤ 0.0033
充放功率上限 $\overline{P}$	0.0263	≤ 0.0033
电价水平	0	0.8464
负载水平	0	0.4711
光伏出力水平	0	≤ 0.0033
峰谷价差	0	≤ 0.0033

读法：对**全天购电费**起主导作用的是储电量上下限与效率；对**节省率**起主导作用的却是电价水平与负载水平——在问题 1 的确定性模型中，电价、光伏与负载水平对前者只是整体缩放，不改变调度结构，故一阶指数为 0。结论是“哪一个参数决定哪一个输出量”，而非“所有参数都重要”。

11.4 容量与充放功率的双向扫描明细

正文第四章给出扫描结论（容量摆幅是主要瓶颈）；本小节给出完整档位与两个单自由度放开实验。

表 19 储能容量与充放功率上限的双向扫描（问题 1，典型日；费用单位元；“相对主口径”以主口径最优费用 35 126.95 元为分母，正号表示更省；容量扫描同步移动储电量上下限以保持 10%–90% 放电深度）

容量（kWh）	最优购电费	相对主口径	充放功率上限（kW）	最优购电费	相对主口径
7 200	38 640.45	−10.00%	1 250	40 166.17	−14.35%
9 600	36 700.45	−4.48%	2 500	36 390.84	−3.60%
12 000	**35 126.95**	0.00%	5 000	**35 126.95**	0.00%
14 400	33 875.57	+3.56%	7 500	35 027.13	+0.28%
19 200	31 692.43	+9.78%	10 000	34 960.59	+0.47%
28 800	28 854.53	+17.86%	—	—	—
48 000	27 601.80	+21.42%	—	—	—

另做两个单自由度放开实验：仅解除储电量上限（充放功率维持 5000 kW）为 27 351.55 元，相对主口径多省 22.14%；仅解除充放功率上限（容量维持 12 000 kWh）为 34 544.43 元，多省 1.66%。界活跃统计：充电功率界在 19/144 个时段被打满、放电功率界 0/144，储电量上界活跃 21/144、下界活跃 8/144。

11.5 L 形（Benders）分解与直接求解的独立对拍

问题 2–4 的模型都是**两阶段结构**：第一阶段是全场共享的计划量 (E_t, c_t, d_t, S_t)，第二阶段是逐情景的追索量 $g_{\omega,t}$。本文主结果全部用 HiGHS 直接求解（单日 $4n + Kn$ 个变量，$K = 60$ 时 9216 个）；但“求解器返回最优”本身不构成模型正确性的证据，因此本节**另写一套分解算法**作为对照：把问题改写为 L 形（Benders）形式，用与直接求解**完全不同的数值路径**再算一遍。给定第一阶段 (E, c, d, S) 后，情景 ω 的追索问题逐槽完全可分：

$$Q_\omega(E, c, d) = \sum_t 5 p_t \max\{0,\ \widetilde{ND}_{\omega,t} + c_t - d_t - E_t\}, \tag{12}$$

每个槽的最优值就是一个正部函数，对 E_t 的次梯度为 $-5p_t$（缺口为正时）或 0（否则），故**每个情景每次迭代可解析地给出 144 条最优性割**（多割），无需调用求解器求子问题；主问题只保留 $4n + 1 = 577$ 个变量。

表 20 L 形分解与直接求解的对照（对象：单日两阶段随机规划，题面指定四个代表日（2025-03-20、06-21、09-23、12-21）的结果合计；范围：$s_{init} = 6000$ kWh、终端模式 $S_n \geq s_{init}$、两法共用同一份分层 medoid 情景；时间列单位为 s（四日合计），变量数单位为个；相对间隙 $= |z_{\text{L 形}} - z_{\text{直接}}|/|z_{\text{直接}}|$；数据源：`intermediate/decomp.json` 的 `scenario_scan` 字段）

情景数 K	直接求解变量	L 形变量	直接求解	L 形分解	加速比	最大相对间隙
20	3 456	720	0.142	0.281	0.50×	6.65×10^{-16}
60	9 216	720	0.356	0.358	1.00×	2.09×10^{-15}
120	17 856	720	1.291	0.401	3.22×	3.06×10^{-15}
300	43 776	720	3.721	0.635	5.86×	2.60×10^{-15}
500	72 576	720	8.962	0.875	10.25×	4.60×10^{-15}

三条结论。① 两条数值路径给出的最优值一致到机器精度：结果窗口全部 334 天上（$K = 60$）逐日对比最大相对间隙 3.76×10^{-15}、全部收敛（平均迭代 9.75 次、单日最多 1097 条割），$K = 300$ 的 42 天子样本上为 4.82×10^{-15}——两个独立实现同时给出同一最优值，比“求解器说 optimal”强得多。② **加速比随情景数增长**：$K = 20$ 时分解反而慢约一倍，$K \approx 60$ 附近是分界点，$K = 120$、300、500 时分别加速 3.22、5.86、10.25 倍；主问题变量数始终为 720、不随 K 增长，这是加速的来源。③ **工程取舍**：本题主口径 $K = 60$、单日直接求解 0.106 s（L 形 0.092 s，两者同量级），分解法在该规模上**没有**速度优势；把

解析割聚合成单割后 200 次迭代仍未收敛、相对间隙 0.65%–3.42%，而多割 9–14 次即收敛，故**主结果仍用直接求解**；分解法的价值在于给出独立最优性对拍，并在情景数放大到数百时把单日求解稳定在 0.8 s 以内。（见 C-Q5-01）

表 21　光伏预报偏差的逐整点结构（样本为 2025 年每天 0:00 发布的 24 h 预报与附件 2 对应实际功率的逐槽配对，$n = 52\,560$；bias 单位 kW，“极值整点”给出该组内绝对偏差最大的整点及其取值）

整点分组	整点范围	组内 bias 区间	极值整点
夜间（光伏无有效出力）	0–4 时	0.00 至 +0.09	4 时 +0.09
上午（系统性**高估**）	5–10 时	+214.21 至 +733.00	7 时 +733.00
正午过渡	11 时	+7.21	—
下午（系统性**低估**）	12–18 时	−801.87 至 −106.46	15 时 −801.87
傍晚之后（预报空洞）	19–23 时	−0.15 至 0.00	19 时 −0.15

附录 L　场景保真度与电价检验明细

本节给出正文第五章场景集与第七章电价过程检验、CVaR 风险—费用前沿的完整明细表。（联合情景保真度检验表见 12。）

表 22　风险前沿三个代表点的情景内指标与真实轨迹回测（对象：联合情景集上按 λ 求解的三组解；范围：2025-02-01 至 12-31 中每 6 天取 1 天的 56 个代表日；单位：费用元/日、电量 kWh/日；样本：64 个代表情景，$\alpha = 0.95$）

代表点	情景内期望费用	情景内 95% CVaR	真实轨迹回测费用	紧急购电量
$\lambda = 0$（期望最优）	41 995.59	52 176.36	45 694.27	480.19
$\lambda = 0.3$（推荐）	42 168.43	50 814.83	45 778.03	324.60
$\lambda = 1$（近似最坏）	42 411.56	50 510.46	45 952.85	313.83

附录 M　验证映射、风险结果与局限明细

本节给出正文第九章验证层级映射、第八章问题 4 逐日期结果与第十章局限清单的完整表格。

附录 N　附件结构与问题 4 逐日期结果明细

本节给出四个附件的字段结构与问题 4 在四个指定日期的逐项结果。

【问题 4 逐日期结果】

附录 O　明细表与口径框（正文摘要引用）

本节集中给出正文以摘要形式引用的明细表与全部核心指标口径框（口径 C-01–口径 C-11），便于评审核对；正文对应处均给出结论、口径编号与本节的表号。

口径 C-02　全天购电量

定义　$E^{\mathrm{day}} = \sum_{t=1}^{144} E_t$，即该日 144 个 10 min 槽的**计划购电量**之和；**公式**　结果为逐槽求和，无闭式；结果文件每行末列即该日汇总；**数据**　结果文件「计划购电量」工作表的 144 个时段列；**时段**　单日；问题 1 为典型日，问题 2–4 为 2025-02-01 至 12-31 的 334 天；**单位**　kWh（结果文件保留 4 位小数，正文保留 2–4 位）；**含**　问题 2、4-2 中即第一阶段计划量；问题 3、4-3 中调整后的量一律称“全天调整购电量”并单独命名；**不含**　不含储能充电量、不含紧急购电量 口径 C-05、不含盈余处置量 口径 C-06；**对照**　问题 2、3 的计划购电量模型内值为 22 220 333.2033 kWh，结果文件回读值为 22 220 333.1391 kWh，相对差 2.9×10^{-9}，来自写入时的 4 位小数取整；**复算**　`results_io.py::plan_sheet_frame()`（写出行汇总列）与 `audit.py::audit_result_files()`（回读结果文件后重算 $\sum_{144}$ 槽）

口径 C-03　总购电费用（计划 + 紧急 + 调整相关）

定义　$C^{\mathrm{total}} = C^{\mathrm{plan}} + C^{\mathrm{breach}} + C^{\mathrm{excess}} + C^{\mathrm{emg}}$；问题 2、4-2 无调整机制，后两项恒为 0；**公式**　问题 2、4-2：$C = \sum_t p_t E_t + 5\sum_t p_t g_t$；问题 3、4-3：$C^{\mathrm{plan}}, C^{\mathrm{breach}}, C^{\mathrm{excess}}$ 按 口径 C-09 的偏差量定价口径计算，$C^{\mathrm{emg}} = \sum_t 5 p_t g_t$；**数据**　电价取附件 1（问题 1–3）或附件 4（问题 4）；电量取本队结果文件；**时段**　问题 1 为典型日；问题 2–4 为 334 天；**单位**　元（保留 2 位小数，内部计算不取整）；**不含**　不含储能投资与运维成本、容量补偿、辅助服务收益、绿电环境价值、循环衰减与安全成本；**对照**　若只报计划购电费而不计紧急购电，问题 2 为 13 637 552.90 元，比 C^{total} 低 627 239.24 元（4.40%）；**复算**　`q2.py::run_q2()`、`q3.py::summary()`、`q4.py::run_q4_2()` 与 `audit.py::audit_result_files()`，四项之和与总费用的偏差为 0

口径 C-04　节省率

定义　$\eta = \left(C^{base} - C^{\mathrm{total}}\right)/C^{base} \times 100\%$，分母与分子必须同窗口、同电价口径；**公式**　C^{base} 取 口径 C-01 的对应变体（V1 典型日 / V2 固定价窗口 / V3 波动价窗口，变体定义见正文表 正文9）；**数据**　C^{base} 与 C^{total} 均取自本队结果；**时段**　与 口径 C-01 变体一致；**单位**　%（保留 2 位小数）；**不含**　不得相对贪心策略或完全信息下界报同一个数而不加限定；相对其他基线的结果必须**单独命名**；**对照**　问题 1 相对 V3 为 26.90%；问题 2、3 相对 V1 为 13.06%、15.08%；问题 4-2、4-3 相对 V2 为 11.35%、13.54%；**复算**　`paper_numbers.py::main()`（由 口径 C-01 与 口径 C-03 导出）

口径 C-06　弃光/盈余处置电量

定义　光伏盈余与超购部分的处置量：典型日按 $s_t = PV_t\Delta t + E_t + d_t - L_t\Delta t - c_t \geq 0$ 逐槽统计（式 正文2b 的松弛量）；窗口口径按实际数据复算为 $\sum_t \max\{PV_t\Delta t + E_t + d_t + g_t - L_t\Delta t - c_t, 0\}$；**公式**　上式为定义式；该量非负，其余额为超购部分被弃置的电量；**数据**　L, PV 取附件 2 实际值，其余取本队结果文件；**时段**　逐槽统计、按窗口（334 天）汇总；**单位**　kWh（按电量统计，**不得**按功率报）；**不含**　不含"因充电功率受限而未发生的充电"，不含未购电部分；**对照**　问题 1 典型日最优弃光 = 0.0000 kWh（数值残差 1.37×10^{-12} kWh）；问题 2、3 的窗口盈余处置量为 3 106 390.84 kWh 与 2 638 963.35 kWh；问题 4-2、4-3 的同一量为 3 100 463.79 kWh 与 2 645 427.81 kWh（均见正文表 正文17）；**复算**　`config.py::curtailment(price, load, pv, E, c, d)`（典型日）与 `audit.py::audit_internal()`（窗口口径）

口径 C-07　充放电量与储电量（交流侧）

定义　结果文件中的 c_t, d_t 为**并网点（AC 侧）**电量；储电量为**时段末**值 S_t；**公式**　递推见式 正文2c，$\eta_c = \eta_d = 0.9$（单边口径，往返 0.81）；充放功率上限 5000 kW 对应单槽上限 $\overline{P}\Delta t = 833.3333$ kWh；**数据**　本队结果文件「充放电量」工作表；**时段**　逐槽；表 2 格式按 6 个 4 h 块（每块 24 槽）汇总，并给出 0:00 与 24:00 两个储电量点；**单位**　kWh；运行区间 $[1200, 10800]$ kWh；**不含**　不含电池侧（DC）电量、不含自放电与循环衰减（假设 A4）；**对照**　恒等式校验：问题 1 的充放比 20 740.67/16 799.94 = $1.2346 = 1/(\eta_c\eta_d)$，与 $\eta_c\eta_d = 0.81$ 互为倒数，偏差 4.44×10^{-16}；问题 2 四个指定日期的合计充放比同为 1.2346；**复算**　`results_io.py::soc_sheet_frame()`（块级汇总）与 `audit.py::audit_result_files()`（块级净递推）

口径 C-08　储电量递推与终端条件

定义　储电量**跨日连续递推**（$S_{d,0} = S_{d-1,144}$，假设 A5），窗口末端施加终端条件 $S_T \geq S_0$（假设 A6），问题 1 另加严格日周期 $S_0 = S_{144}$；**公式**　$S_t = S_{t-1} + \eta_c c_t - d_t/\eta_d$，$S_t \in [1200, 10800]$ kWh；2025 年 1 月为预热期（假设 A8），其终值即 2 月 1 日初值；**数据**　结果文件「充放电量」工作表的储电量列与中间结果 `q2_main.json` 等；**时段**　逐槽；窗口 2025-02-01 至 12-31（334 天）；**单位**　kWh；**不含**　不含"每日清零""每日重置为 6000"两种读法——前者会引入末日掏空型短视解，后者与 1 月预热期矛盾；**对照**　问题 1：日周期口径 35 126.9486 元（$S_0 = S_{144} = 6000.00$ kWh）；仅要求两端相等的自由环状口径 35 118.5986 元（最优 $S = 8550.00$ kWh），相差 8.35 元（0.02%）；窗口口径下"末值低于初值的天数"为 0；**复算**　`lp_engine.py::solve_day()`（终端约束）与 `audit.py::audit_result_files()`（`A6_` 末值低于初值天数）

附录 P　问题 2–4 的逐时段结果明细表

第八章已按题面表 1 / 表 2 / 表 3 的格式给出四个指定日期的结果；本节补充**指定 10 min 时段**的购电量明细与逐时段充放电明细，便于与结果文件逐格对照。

表 23　微网在指定时段与指定日期的购电量、全天购电量和购电费（问题 2；电量单位：kWh，费用单位：元）

时段 / 指标	2025-03-20	2025-06-21	2025-09-23	2025-12-21
10:00–10:10	0.0000	0.0000	0.0000	0.0000
12:00–12:10	804.3029	0.0000	536.2415	997.0335
14:00–14:10	0.0000	0.0000	0.0000	0.0000
16:00–16:10	817.3061	255.2254	902.7969	1001.2240
18:00–18:10	697.9973	0.0000	764.3298	715.4144
20:00–20:10	0.0000	0.0000	0.0000	0.0000
全天购电量	74 178.1298	35 701.5391	74 744.7963	99 692.6601
全天购电费	45 229.90	20 324.26	47 063.39	64 376.98
紧急购电量	1025.8805	187.0843	193.3669	22.5250
全天总费用	49 654.02	21 255.45	47 717.70	64 432.28

表注：时段按题面表述取区间左端时刻；全天购电费为计划口径（$\sum_t p_t E_t$），全天总费用为计划购电费与紧急购电费之和；四个日期的无储能基线分别为 52 290.65、29 668.18、54 741.30、71 176.92 元。

表 24　储能设备在指定日期的充放电量及 0:00 与 24:00 的储电量（问题 2，单位：kWh）

日期	时段	充电量	放电量	日期	时段	充电量	放电量
2025-03-20	0:00–4:00	4 500.0000	0.0000	2025-09-23	0:00–4:00	4 500.0000	0.0000
2025-03-20	4:00–8:00	833.3333	6 224.1108	2025-09-23	4:00–8:00	833.3333	5 547.8264
2025-03-20	8:00–12:00	6 500.0000	2 415.8892	2025-09-23	8:00–12:00	5 833.3333	3 020.9227
2025-03-20	12:00–16:00	6 905.5931	2 218.5304	2025-09-23	12:00–16:00	6 728.4081	1 606.2615
2025-03-20	16:00–20:00	0.0000	5 709.7835	2025-09-23	16:00–20:00	0.0000	5 602.9091
2025-03-20	20:00–24:00	5 333.3333	2 930.2165	2025-09-23	20:00–24:00	5 333.3333	3 037.0909
2025-06-21	0:00–4:00	0.0000	0.0000	2025-12-21	0:00–4:00	4 500.0000	0.0000
2025-06-21	4:00–8:00	140.7851	4 303.4721	2025-12-21	4:00–8:00	1 666.6667	1 508.3333
2025-06-21	8:00–12:00	5 207.8672	130.5638	2025-12-21	8:00–12:00	5 833.3333	7 806.6667
2025-06-21	12:00–16:00	5 458.7995	0.0000	2025-12-21	12:00–16:00	8 333.3333	2 835.0000
2025-06-21	16:00–20:00	0.0000	6 154.1006	2025-12-21	16:00–20:00	0.0000	5 797.4742
2025-06-21	20:00–24:00	5 333.3333	2 485.8994	2025-12-21	20:00–24:00	5 333.3333	2 842.5258

表注：四个指定日期的 0:00 与 24:00 储电量均为 6000.0000 kWh；每段含 24 个 10 min 槽，充放电量为交流侧电量（口径 C-07）；四个日期的放电量 / 充电量之比均等于往返效率 $\eta_c\eta_d = 0.8100$，其倒数即充电量 / 放电量之比 $1/(\eta_c\eta_d) = 1.2346$。本表与正文表 正文19 取自同一数据源 `results/result2.xlsx`，逐格一致。

表 25　微网在指定日期的紧急购电量（问题 2；购电量单位：kWh，电价单位：元/kWh）

日期	购电时间段	紧急购电量	交易时刻电价（加权平均）
2025-03-20	6:00–8:00	228.9914	0.8625
2025-03-20	8:10–8:30	49.1586	0.8625
2025-03-20	8:50–10:30	531.2739	0.8625
2025-03-20	10:40–11:40	216.4567	0.8625
2025-06-21	4:00–4:50	5.2416	0.9955
2025-06-21	14:00–14:30	181.8427	0.9955
2025-09-23	6:50–7:40	132.5948	0.6768
2025-09-23	11:00–11:40	60.7721	0.6768
2025-12-21	11:00–11:30	22.5250	0.4910
四个日期合计		1428.8567	—

表注：本表按“日期 × 紧急购电事件”逐行展开，与题面表 3 的横向版式一一对应；同一日期可有多行，连续触发的相邻槽按事件聚合，时间段为该事件涉及的槽位区间；交易时刻电价为该事件内按电量加权的平均电价（事件电价按 $5p_t$ 计价，本列为 p_t）。无紧急购电的时段不列入本表（全窗口 334 天中 296 天有触发）。

附录 Q　口径声明与时段对照明细表

附录 R　伪验证自查清单（逐条证据）

①自证循环：审计脚本不共用建模费用函数，费用双路径复算偏差 4.32×10^{-7}；
②以求解器状态代替物理校验：求解器返回 optimal 只作必要条件，同时报告供给缺口、储电量递推、越界与互斥等物理残差；
③比较不同样本：R1 / R2 / R3、$n = 1/2/4$、$K = 60/120$、四种情景生成器均在同一场景集或同一实际轨迹上配对比较；
④训练集当测试集：主结果明确声明为样本内评估，另做样本外回测（正文9.4）与生成器样本外对照（7.1）；
⑤只报点估计：关键量均给多档对照——效率 4 档、容量 7 档、功率 5 档、κ 5 档、场景数 2 档、CVaR 的 λ 11 档、机会约束 β 6 档、盒式鲁棒 k 7 档；
⑥事后挑选指标：本节指标在建模前由口径 C-01 至 C-11 固定，未随结果调整；
⑦把“结果好看”当“模型正确”：问题 2 的 13.06% 与问题 1 的 26.90% 同量级且更小，差额可由预报误差成本 853 231.23 元解释，未把“节省率更高”当作模型更好的证据；
⑧扰动无效：灵敏度扰动作用于运行区间与功率上限本身，且每档重新求解，不复用已有解。

附录 S　附图

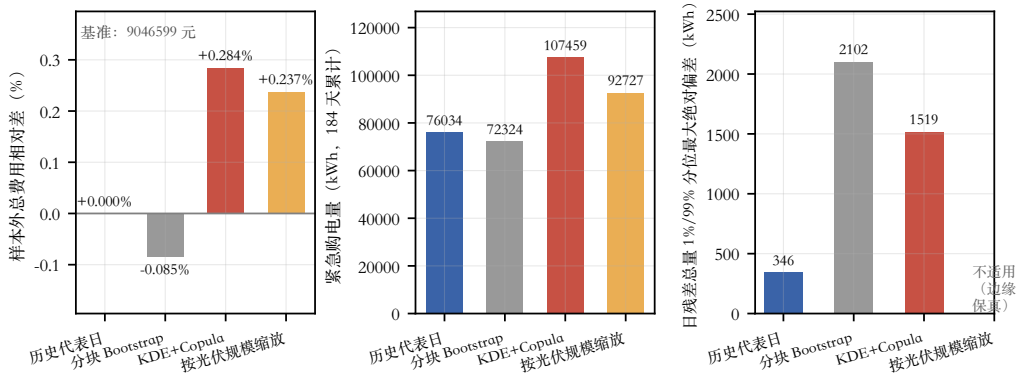


图 2　四种情景生成器的样本外对照（左：相对“历史代表日”的总费用差；中：184 天累计紧急购电量；右：日总残差的 1%/99% 分位最大绝对偏差；对象：训练窗 2025-02-01 至 06-30、测试窗 07-01 至 12-31 共 184 天；单位：百分比、kWh；样本：情景数由二维分层固定为 48 个，Copula 生成器抽样 600 条后削减）